HR MANAGEMENT · SOURCE ARCHIVE

Complete Frontend and Backend Source Code

A printable, searchable listing of the project's text-based source and configuration files.

Frontend files: 32

Backend files: 52

Total included files: 84

Exclusions: .env secrets, uploaded PDF/DOCX documents, and generated node_modules dependencies.

Contents

Each entry links to the complete text of that source file. Use the PDF reader's search to find a filename, function, or code string.

Frontend (32 files)

- Backend/authMiddleware.js
- Backend/config/email.js
- Backend/controllers/attendanceController.js
- Backend/controllers/auditLogController.js
- Backend/controllers/authController.js
- Backend/controllers/candidateController.js
- Backend/controllers/dashboardController.js
- Backend/controllers/departmentController.js
- Backend/controllers/employeeController.js
- Backend/controllers/interviewController.js
- Backend/controllers/jobController.js
- Backend/controllers/leaveController.js
- Backend/controllers/notificationController.js
- Backend/controllers/payrollController.js
- Backend/controllers/performanceController.js
- Backend/controllers/userController.js
- Backend/middleware/authMiddleware.js
- Backend/middleware/roleMiddleware.js
- Backend/middleware/uploadMiddleware.js
- Backend/models/Attendance.js
- Backend/models/AuditLog.js
- Backend/models/Candidate.js
- Backend/models/Department.js
- Backend/models/Employee.js
- Backend/models/Interview.js
- Backend/models/Job.js
- Backend/models/Leave.js
- Backend/models/Notification.js
- Backend/models/Payroll.js
- Backend/models/Performance.js
- Backend/models/User.js
- Backend/package-lock.json

Backend (52 files)

- Backend/package.json
- Backend/resetAdminPassword.js
- Backend/resetEmployeePassword.js
- Backend/routes/attendanceRoutes.js
- Backend/routes/auditLogRoutes.js
- Backend/routes/authRoutes.js
- Backend/routes/candidateRoutes.js
- Backend/routes/dashboardRoutes.js
- Backend/routes/departmentRoutes.js
- Backend/routes/employeeRoutes.js
- Backend/routes/interviewRoutes.js
- Backend/routes/jobRoutes.js
- Backend/routes/leaveRoutes.js
- Backend/routes/notificationRoutes.js
- Backend/routes/payrollRoutes.js
- Backend/routes/performanceRoutes.js
- Backend/routes/userRoutes.js
- Backend/server.js
- Backend/services/notificationService.js
- Backend/testNotification.js
- Frontend/attendance.html
- Frontend/candidates.html
- Frontend/CSS/candidates.css
- Frontend/CSS/dashboard.css
- Frontend/CSS/departments.css
- Frontend/CSS/interviews.css
- Frontend/CSS/notifications.css
- Frontend/CSS/setting.css
- Frontend/CSS/style.css
- Frontend/dashboard.html
- Frontend/departments.html
- Frontend/employees.html
- Frontend/forgot-password.html
- Frontend/index.html
- Frontend/interviews.html
- Frontend/jobs.html
- Frontend/JS/attendance.js
- Frontend/JS/auth.js
- Frontend/JS/dashboard.js
- Frontend/JS/departments.js
- Frontend/JS/employees.js
- Frontend/JS/interviews.js
- Frontend/JS/jobs.js
- Frontend/JS/notifications.js
- Frontend/JS/payroll.js
- Frontend/JS/theme.js
- Frontend/leaves.html
- Frontend/login.html
- Frontend/notifications.html
- Frontend/payroll.html
- Frontend/performance.html
- Frontend/settings.html

Backend/authMiddleware.js

```
const jwt = require("jsonwebtoken");

const authMiddleware = (req, res, next) => {
    try {
        const authHeader = req.headers.authorization;

        if (!authHeader || !authHeader.startsWith("Bearer ")) {
            return res.status(401).json({
                message: "Access denied. No token provided."
            });
        }

        const token = authHeader.split(" ")[1];

        const decoded = jwt.verify(
            token,
            process.env.JWT_SECRET
        );

        req.user = decoded;

        next();

    } catch (error) {
        return res.status(401).json({
            message: "Invalid or expired token"
        });
    }
};

module.exports = authMiddleware;
```

Backend/config/email.js

```
require("dotenv").config();

const nodemailer = require("nodemailer");

const transporter = nodemailer.createTransport({
    service: "gmail",
    auth: {
        user: process.env.EMAIL_USER,
        pass: process.env.EMAIL_PASS
    }
});

const sendOTPEmail = async (email, otp) => {
    await transporter.sendMail({
        from: `"HRGenius" <${process.env.EMAIL_USER}>`,
        to: email,
        subject: "HRGenius Password Reset OTP",
        html: `
            <div style="font-family: Arial, sans-serif; max-width: 600px; margin: auto; padding: 30px; border: 1px solid #ddd; border-radius: 12px;">
                <h2 style="color: #2563eb;">HRGenius</h2>

                <h3>Password Reset Request</h3>

                <p>You requested to reset your HRGenius account password.</p>

                <p>Your OTP is:</p>

                <div style="font-size: 32px; font-weight: bold; letter-spacing: 8px; color: #2563eb; margin: 20px 0;">
                    ${otp}
                </div>

                <p>This OTP will expire in <b>10 minutes</b>.</p>

                <p>If you did not request this password reset, you can safely ignore this email.</p>

                <hr>

                <p style="color: #777; font-size: 13px;">
                    This is an automated email from HRGenius.
                </p>
            </div>
        `
    });
};

module.exports = sendOTPEmail;
```

Backend/controllers/attendanceController.js

```
const Attendance = require("../models/Attendance");
const Employee = require("../models/Employee");
const User = require("../models/User");

const {
    createNotifications
} = require("../services/notificationService");

const getLoggedInEmployee = async (userId) => {
    const user = await User.findById(userId);

    if (!user) {
        return null;
    }

    const employee = await Employee.findOne({
        email: user.email
    });

    return employee;
};

// CHECK IN
const checkIn = async (req, res) => {
    try {
        let { employeeId } = req.body;

        // Employee can only check in himself
        if (req.user.role === "employee") {
            const employee = await getLoggedInEmployee(req.user.id);

            if (!employee) {
                return res.status(404).json({
                    message: "Employee profile not found"
                });
            }

            employeeId = employee._id;
        }

        if (!employeeId) {
            return res.status(400).json({
                message: "Employee ID is required"
            });
        }

        const employee = await Employee.findById(employeeId);

        if (!employee) {
            return res.status(404).json({
                message: "Employee not found"
            });
        }

        const today = new Date();

        const startOfDay = new Date(today);
        startOfDay.setHours(0, 0, 0, 0);

        const endOfDay = new Date(today);
        endOfDay.setHours(23, 59, 59, 999);

        const existingAttendance = await Attendance.findOne({
            employee: employeeId,
            date: {
                $gte: startOfDay,
                $lte: endOfDay
            }
        });

        if (existingAttendance) {
            return res.status(400).json({
                message: "Attendance already marked for today"
            });
        }

        const attendance = await Attendance.create({
            employee: employeeId,
            date: today,
            checkIn: today,
            status: "Present"
        });

        const adminHrUsers = await User.find({
            role: {
                $in: ["admin", "hr"]
            }
        });

        const notifications = adminHrUsers.map((user) => {
            return {
                userId: user._id,
                title: "Attendance Marked",
                message: `${employee.name} marked attendance as Present.`,
                type: "Attendance",
                priority: "Normal",
                actionUrl: "/attendance.html",
                relatedId: attendance._id,
                relatedModel: "Attendance"
            };
        });
```

```
        await createNotifications(notifications);

        res.status(201).json({
            message: "Check-in successful",
            attendance
        });

    } catch (error) {
        console.log("Check-In Error:", error);

        res.status(500).json({
            message: "Server error",
            error: error.message
        });
    }
};

// CHECK OUT
const checkOut = async (req, res) => {
    try {
        let { employeeId } = req.body;

        // Employee can only check out himself
        if (req.user.role === "employee") {
            const employee = await getLoggedInEmployee(req.user.id);

            if (!employee) {
                return res.status(404).json({
                    message: "Employee profile not found"
                });
            }

            employeeId = employee._id;
        }

        if (!employeeId) {
            return res.status(400).json({
                message: "Employee ID is required"
            });
        }

        const employee = await Employee.findById(employeeId);

        if (!employee) {
            return res.status(404).json({
                message: "Employee not found"
            });
        }

        const today = new Date();

        const startOfDay = new Date(today);
        startOfDay.setHours(0, 0, 0, 0);

        const endOfDay = new Date(today);
        endOfDay.setHours(23, 59, 59, 999);

        const attendance = await Attendance.findOne({
            employee: employeeId,
            date: {
                $gte: startOfDay,
                $lte: endOfDay
            }
        });

        if (!attendance) {
            return res.status(404).json({
                message: "No check-in record found for today"
            });
        }

        if (attendance.checkOut) {
            return res.status(400).json({
                message: "Employee already checked out"
            });
        }

        attendance.checkOut = today;

        await attendance.save();

        const adminHrUsers = await User.find({
            role: {
                $in: ["admin", "hr"]
            }
        });

        const notifications = adminHrUsers.map((user) => {
            return {
                userId: user._id,
                title: "Attendance Checkout",
                message: `${employee.name} checked out from attendance.`,
                type: "Attendance",
                priority: "Normal",
                actionUrl: "/attendance.html",
                relatedId: attendance._id,
                relatedModel: "Attendance"
            };
        });

        await createNotifications(notifications);

        res.status(200).json({
            message: "Check-out successful",
            attendance
```

```
        });

    } catch (error) {
        console.log("Check-Out Error:", error);

        res.status(500).json({
            message: "Server error",
            error: error.message
        });
    }
};

// ADMIN / HR - GET ALL ATTENDANCE
const getAttendance = async (req, res) => {
    try {
        const attendance = await Attendance.find()
            .populate(
                "employee",
                "name email department designation"
            )
            .sort({
                date: -1
            });

        res.status(200).json({
            count: attendance.length,
            attendance
        });

    } catch (error) {
        console.log("Get Attendance Error:", error);

        res.status(500).json({
            message: "Server error",
            error: error.message
        });
    }
};

// EMPLOYEE - GET OWN ATTENDANCE
const getMyAttendance = async (req, res) => {
    try {
        const employee = await getLoggedInEmployee(req.user.id);

        if (!employee) {
            return res.status(404).json({
                message: "Employee profile not found"
            });
        }

        const attendance = await Attendance.find({
            employee: employee._id
        })
            .populate(
                "employee",
                "name email department designation"
            )
            .sort({
                date: -1
            });

        res.status(200).json({
            count: attendance.length,
            attendance
        });

    } catch (error) {
        console.log("Get My Attendance Error:", error);

        res.status(500).json({
            message: "Server error",
            error: error.message
        });
    }
};

// ADMIN / HR - GET SPECIFIC EMPLOYEE ATTENDANCE
const getEmployeeAttendance = async (req, res) => {
    try {
        const attendance = await Attendance.find({
            employee: req.params.employeeId
        })
            .populate(
                "employee",
                "name email department designation"
            )
            .sort({
                date: -1
            });

        res.status(200).json({
            count: attendance.length,
            attendance
        });

    } catch (error) {
        console.log(
            "Get Employee Attendance Error:",
            error
        );

        res.status(500).json({
            message: "Server error",
            error: error.message
        });
```

```
    }
};

module.exports = {
    checkIn,
    checkOut,
    getAttendance,
    getMyAttendance,
    getEmployeeAttendance
};
```

Backend/controllers/auditLogController.js

```
const AuditLog = require("../models/AuditLog");

const createAuditLog = async (req, res) => {
    try {
        const {
            action,
            module,
            description
        } = req.body;

        const auditLog = await AuditLog.create({
            userId: req.user.id,
            action,
            module,
            description,
            ipAddress: req.ip
        });

        res.status(201).json({
            message: "Audit log created successfully",
            auditLog
        });
    } catch (error) {
        res.status(500).json({
            message: "Error creating audit log",
            error: error.message
        });
    }
};

const getAllAuditLogs = async (req, res) => {
    try {
        const auditLogs = await AuditLog.find()
            .populate("userId", "name email role")
            .sort({ createdAt: -1 });

        res.status(200).json({
            count: auditLogs.length,
            auditLogs
        });
    } catch (error) {
        res.status(500).json({
            message: "Error fetching audit logs",
            error: error.message
        });
    }
};

module.exports = {
    createAuditLog,
    getAllAuditLogs
};
```

Backend/controllers/authController.js

```
const User = require("../models/User");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const crypto = require("crypto");
const sendOTPEmail = require("../config/email");


const registerUser = async (req, res) => {
    try {
        const { name, email, password, role } = req.body;

        if (!name || !email || !password) {
            return res.status(400).json({
                message: "Name, email and password are required"
            });
        }

        const existingUser = await User.findOne({ email });

        if (existingUser) {
            return res.status(400).json({
                message: "User already exists"
            });
        }

        const hashedPassword = await bcrypt.hash(password, 10);

        const user = await User.create({
            name,
            email,
            password: hashedPassword,
            role: role || "employee"
        });

        res.status(201).json({
            message: "User registered successfully",
            user: {
                id: user._id,
                name: user.name,
                email: user.email,
                role: user.role
            }
        });

    } catch (error) {
        res.status(500).json({
            message: "Server error",
            error: error.message
        });
    }
};


const loginUser = async (req, res) => {
    try {
        const { email, password } = req.body;

        if (!email || !password) {
            return res.status(400).json({
                message: "Email and password are required"
            });
        }

        const user = await User.findOne({ email });

        if (!user) {
            return res.status(401).json({
                message: "Invalid email or password"
            });
        }

        const isPasswordValid = await bcrypt.compare(
            password,
            user.password
        );

        if (!isPasswordValid) {
            return res.status(401).json({
                message: "Invalid email or password"
            });
        }

        const token = jwt.sign(
            {
                id: user._id,
                role: user.role
            },
            process.env.JWT_SECRET,
            {
                expiresIn: "1d"
            }
        );

        res.status(200).json({
            message: "Login successful",
            token: token,
            user: {
                id: user._id,
                name: user.name,
                email: user.email,
                role: user.role
            }
```

```
        });

    } catch (error) {
        res.status(500).json({
            message: "Server error",
            error: error.message
        });
    }
};


const forgotPassword = async (req, res) => {
    try {
        const { email } = req.body;

        if (!email) {
            return res.status(400).json({
                message: "Email is required"
            });
        }

        const user = await User.findOne({
            email: email.toLowerCase()
        });

        if (!user) {
            return res.status(404).json({
                message: "No account found with this email"
            });
        }

        const otp = Math.floor(
            100000 + Math.random() * 900000
        ).toString();

        user.resetOTP = crypto
            .createHash("sha256")
            .update(otp)
            .digest("hex");

        user.resetOTPExpires = Date.now() + 10 * 60 * 1000;

        await user.save();

        await sendOTPEmail(user.email, otp);

        res.status(200).json({
            message: "OTP sent successfully to your email"
        });

    } catch (error) {
        console.error(error);

        res.status(500).json({
            message: "Failed to send OTP"
        });
    }
};


const verifyResetOTP = async (req, res) => {
    try {
        const { email, otp } = req.body;

        if (!email || !otp) {
            return res.status(400).json({
                message: "Email and OTP are required"
            });
        }

        const hashedOTP = crypto
            .createHash("sha256")
            .update(otp)
            .digest("hex");

        const user = await User.findOne({
            email: email.toLowerCase(),
            resetOTP: hashedOTP,
            resetOTPExpires: { $gt: Date.now() }
        });

        if (!user) {
            return res.status(400).json({
                message: "Invalid or expired OTP"
            });
        }

        res.status(200).json({
            message: "OTP verified successfully"
        });

    } catch (error) {
        console.error(error);

        res.status(500).json({
            message: "OTP verification failed"
        });
    }
};


const resetPassword = async (req, res) => {
    try {
        const { email, otp, newPassword } = req.body;
```

```
        if (!email || !otp || !newPassword) {
            return res.status(400).json({
                message: "Email, OTP and new password are required"
            });
        }

        if (newPassword.length < 6) {
            return res.status(400).json({
                message: "Password must be at least 6 characters"
            });
        }

        const hashedOTP = crypto
            .createHash("sha256")
            .update(otp)
            .digest("hex");

        const user = await User.findOne({
            email: email.toLowerCase(),
            resetOTP: hashedOTP,
            resetOTPExpires: { $gt: Date.now() }
        });

        if (!user) {
            return res.status(400).json({
                message: "Invalid or expired OTP"
            });
        }

        const hashedPassword = await bcrypt.hash(
            newPassword,
            10
        );

        user.password = hashedPassword;
        user.resetOTP = null;
        user.resetOTPExpires = null;

        await user.save();

        res.status(200).json({
            message: "Password reset successfully"
        });

    } catch (error) {
        console.error(error);

        res.status(500).json({
            message: "Password reset failed"
        });
    }
};


module.exports = {
    registerUser,
    loginUser,
    forgotPassword,
    verifyResetOTP,
    resetPassword
};
```

Backend/controllers/candidateController.js

```
const Candidate = require("../models/Candidate");
const User = require("../models/User");

const {
    createNotifications
} = require("../services/notificationService");

const fs = require("fs");
const path = require("path");


// CREATE CANDIDATE
const createCandidate = async (req, res) => {
    try {

        const {
            name,
            email,
            phone,
            skills,
            experience,
            education,
            appliedJob
        } = req.body;


        if (
            !name ||
            !email ||
            !phone ||
            !skills ||
            !experience ||
            !education ||
            !appliedJob
        ) {
            return res.status(400).json({
                message: "All candidate fields are required"
            });
        }


        if (!req.file) {
            return res.status(400).json({
                message: "Resume file is required"
            });
        }


        const resume =
            `/uploads/${req.file.filename}`;


        const candidate =
            await Candidate.create({
                name,
                email,
                phone,
                resume,
                skills,
                experience,
                education,
                appliedJob
            });


        const populatedCandidate =
            await Candidate
                .findById(candidate._id)
                .populate(
                    "appliedJob",
                    "title department location"
                );


        // FIND ADMIN AND HR USERS
        const hrUsers =
            await User.find({
                role: {
                    $in: ["admin", "hr"]
                }
            }).select("_id");


        // CREATE RECRUITMENT NOTIFICATIONS
        if (hrUsers.length > 0) {

            await createNotifications(
                hrUsers.map(user => ({
                    userId: user._id,

                    title: "New Candidate Applied",

                    message:
                        `${candidate.name} has applied for a job.`,

                    type: "Recruitment",

                    priority: "Normal",

                    actionUrl: "/candidates.html",

                    relatedId: candidate._id,
```

```
                    relatedModel: "Candidate"
                }))
            );

        }


        res.status(201).json({
            message: "Candidate created successfully",
            candidate: populatedCandidate
        });


    } catch (error) {

        console.error(
            "Create Candidate Error:",
            error
        );


        res.status(500).json({
            message: "Error creating candidate",
            error: error.message
        });

    }
};



// GET ALL CANDIDATES
const getAllCandidates = async (req, res) => {
    try {

        const candidates =
            await Candidate
                .find()
                .populate(
                    "appliedJob",
                    "title department location"
                )
                .sort({
                    createdAt: -1
                });


        res.status(200).json({
            count: candidates.length,
            candidates
        });


    } catch (error) {

        console.error(
            "Get Candidates Error:",
            error
        );


        res.status(500).json({
            message: "Error fetching candidates",
            error: error.message
        });

    }
};



// GET CANDIDATE BY ID
const getCandidateById = async (req, res) => {
    try {

        const candidate =
            await Candidate
                .findById(req.params.id)
                .populate(
                    "appliedJob",
                    "title department location"
                );


        if (!candidate) {
            return res.status(404).json({
                message: "Candidate not found"
            });
        }


        res.status(200).json({
            candidate
        });


    } catch (error) {

        console.error(
            "Get Candidate Error:",
            error
        );
```

```
        res.status(500).json({
            message: "Error fetching candidate",
            error: error.message
        });

    }
};



// UPDATE CANDIDATE STATUS
const updateCandidateStatus = async (req, res) => {
    try {

        const {
            status
        } = req.body;


        const allowedStatuses = [
            "Applied",
            "Shortlisted",
            "Interview",
            "Selected",
            "Rejected"
        ];


        if (
            !allowedStatuses.includes(status)
        ) {
            return res.status(400).json({
                message: "Invalid candidate status"
            });
        }


        const candidate =
            await Candidate
                .findByIdAndUpdate(
                    req.params.id,
                    {
                        status
                    },
                    {
                        new: true,
                        runValidators: true
                    }
                )
                .populate(
                    "appliedJob",
                    "title department location"
                );


        if (!candidate) {
            return res.status(404).json({
                message: "Candidate not found"
            });
        }


        // FIND ADMIN AND HR USERS
        const hrUsers =
            await User.find({
                role: {
                    $in: ["admin", "hr"]
                }
            }).select("_id");


        // CREATE STATUS UPDATE NOTIFICATIONS
        if (hrUsers.length > 0) {

            await createNotifications(
                hrUsers.map(user => ({
                    userId: user._id,

                    title: "Candidate Status Updated",

                    message:
                        `${candidate.name}'s status changed to ${status}.`,

                    type: "Recruitment",

                    priority:
                        status === "Selected"
                            ? "High"
                            : "Normal",

                    actionUrl: "/candidates.html",

                    relatedId: candidate._id,

                    relatedModel: "Candidate"
                }))
            );

        }


        res.status(200).json({
            message:
                "Candidate status updated successfully",
```

```
            candidate
        });


    } catch (error) {

        console.error(
            "Update Candidate Status Error:",
            error
        );


        res.status(500).json({
            message:
                "Error updating candidate status",

            error: error.message
        });

    }
};



// DELETE CANDIDATE
const deleteCandidate = async (req, res) => {
    try {

        const candidate =
            await Candidate.findById(
                req.params.id
            );


        if (!candidate) {
            return res.status(404).json({
                message: "Candidate not found"
            });
        }


        if (candidate.resume) {

            const cleanResumePath =
                candidate.resume.replace(
                    /^[\/\\]+/,
                    ""
                );


            const resumePath =
                path.join(
                    __dirname,
                    "..",
                    cleanResumePath
                );


            if (
                fs.existsSync(resumePath)
            ) {

                fs.unlinkSync(
                    resumePath
                );

            }

        }


        await Candidate.findByIdAndDelete(
            req.params.id
        );


        res.status(200).json({
            message:
                "Candidate deleted successfully"
        });


    } catch (error) {

        console.error(
            "Delete Candidate Error:",
            error
        );


        res.status(500).json({
            message:
                "Error deleting candidate",

            error: error.message
        });

    }
};



module.exports = {
    createCandidate,
    getAllCandidates,
```

```
    getCandidateById,
    updateCandidateStatus,
    deleteCandidate
};
```

Backend/controllers/dashboardController.js

```
const Employee = require("../models/Employee");
const Job = require("../models/Job");
const Candidate = require("../models/Candidate");
const Interview = require("../models/Interview");
const Leave = require("../models/Leave");
const Payroll = require("../models/Payroll");
const Attendance = require("../models/Attendance");
const Department = require("../models/Department");
const AuditLog = require("../models/AuditLog");
const Notification = require("../models/Notification");


// ========================
// DASHBOARD STATS
// ========================

exports.getDashboardStats = async (req, res) => {

    try {

        console.log("Loading Dashboard Statistics...");


        // ========================
        // CURRENT DATE
        // ========================

        const now = new Date();

        const currentYear =
            now.getFullYear();

        const currentMonth =
            now.getMonth();


        // ========================
        // TODAY
        // ========================

        const todayYear =
            now.getFullYear();

        const todayMonth =
            now.getMonth();

        const todayDate =
            now.getDate();


        // ========================
        // FETCH DATA
        // ========================

        const [
            employees,
            jobs,
            candidates,
            interviews,
            leaves,
            payrolls,
            attendances,
            departments,
            unreadNotifications,
            recentEmployees,
            recentCandidates,
            recentInterviews,
            recentLeaves,
            recentPayrolls,
            recentAuditLogs
        ] = await Promise.all([

            Employee
                .find()
                .select(
                    "name department designation salary joiningDate createdAt"
                ),

            Job
                .find()
                .select(
                    "title status createdAt"
                ),

            Candidate
                .find()
                .populate(
                    "appliedJob",
                    "title"
                )
                .select(
                    "name status appliedJob createdAt"
                ),

            Interview
                .find()
                .sort({
                    createdAt: -1
                }),

            Leave
                .find()
```

```
        .sort({
            createdAt: -1
        }),

    Payroll
        .find()
        .populate(
            "employeeId",
            "name"
        )
        .sort({
            createdAt: -1
        }),

    // Get ALL attendance records
    Attendance
        .find()
        .sort({
            date: -1
        }),

    Department
        .find()
        .populate(
            "head",
            "name"
        ),

    Notification
        .countDocuments({
            read: false
        }),

    Employee
        .find()
        .sort({
            createdAt: -1
        })
        .limit(5)
        .select(
            "name department designation createdAt"
        ),

    Candidate
        .find()
        .sort({
            createdAt: -1
        })
        .limit(5)
        .select(
            "name status createdAt"
        ),

    Interview
        .find()
        .sort({
            createdAt: -1
        })
        .limit(5),

    Leave
        .find()
        .sort({
            createdAt: -1
        })
        .limit(5)
        .select(
            "status leaveType createdAt"
        ),

    Payroll
        .find()
        .sort({
            createdAt: -1
        })
        .limit(5)
        .select(
            "netSalary paymentStatus month createdAt"
        ),

    AuditLog
        .find()
        .populate(
            "userId",
            "name"
        )
        .sort({
            createdAt: -1
        })
        .limit(5)
        .select(
            "userId action module description createdAt"
        )

]);


// ========================
// BASIC COUNTS
// ========================

const totalEmployees =
    employees.length;

const totalJobs =
```

```
    jobs.length;

const totalCandidates =
    candidates.length;

const totalInterviews =
    interviews.length;

const totalPayrollRecords =
    payrolls.length;


// =========================
// LEAVES
// =========================

const pendingLeaves =
    leaves.filter(
        function (leave) {

            return (
                String(
                    leave.status || ""
                )
                    .toLowerCase() ===
                "pending"
            );

        }
    ).length;


const approvedLeaves =
    leaves.filter(
        function (leave) {

            return (
                String(
                    leave.status || ""
                )
                    .toLowerCase() ===
                "approved"
            );

        }
    ).length;


const rejectedLeaves =
    leaves.filter(
        function (leave) {

            return (
                String(
                    leave.status || ""
                )
                    .toLowerCase() ===
                "rejected"
            );

        }
    ).length;


// =========================
// PAYROLL
// =========================

const totalPayroll =
    payrolls.reduce(
        function (sum, payroll) {

            return (
                sum +
                Number(
                    payroll.netSalary || 0
                )
            );

        },
        0
    );


const paidPayroll =
    payrolls
        .filter(
            function (payroll) {

                return (
                    payroll.paymentStatus ===
                    "Paid"
                );

            }
        )
        .reduce(
            function (sum, payroll) {

                return (
                    sum +
                    Number(
                        payroll.netSalary || 0
                    )
                );
```

```
            },
            0
        );


const pendingPayrollAmount =
    payrolls
        .filter(
            function (payroll) {

                return (
                    payroll.paymentStatus ===
                    "Pending"
                );

            }
        )
        .reduce(
            function (sum, payroll) {

                return (
                    sum +
                    Number(
                        payroll.netSalary || 0
                    )
                );

            },
            0
        );


const paidPayrollCount =
    payrolls.filter(
        function (payroll) {

            return (
                payroll.paymentStatus ===
                "Paid"
            );

        }
    ).length;


const pendingPayrollCount =
    payrolls.filter(
        function (payroll) {

            return (
                payroll.paymentStatus ===
                "Pending"
            );

        }
    ).length;


// ========================
// ATTENDANCE - CURRENT MONTH
// ========================

const monthAttendances =
    attendances.filter(
        function (attendance) {

            if (!attendance.date) {
                return false;
            }

            const attendanceDate =
                new Date(
                    attendance.date
                );

            return (
                attendanceDate.getFullYear() ===
                    currentYear &&
                attendanceDate.getMonth() ===
                    currentMonth
            );

        }
    );


const presentThisMonth =
    monthAttendances.filter(
        function (attendance) {

            return (
                String(
                    attendance.status || ""
                )
                    .toLowerCase() ===
                "present"
            );

        }
    ).length;


const absentThisMonth =
    monthAttendances.filter(
```

```
        function (attendance) {

            return (
                String(
                    attendance.status || ""
                )
                    .toLowerCase() ===
                "absent"
            );

        }
    ).length;


const leaveThisMonth =
    monthAttendances.filter(
        function (attendance) {

            return (
                String(
                    attendance.status || ""
                )
                    .toLowerCase() ===
                "leave"
            );

        }
    ).length;


const attendanceTotal =
    presentThisMonth +
    absentThisMonth;


const attendanceRate =
    attendanceTotal > 0
        ? Math.round(
            (
                presentThisMonth /
                attendanceTotal
            ) * 100
        )
        : 0;


// =========================
// ATTENDANCE - TODAY
// =========================

const todayAttendances =
    attendances.filter(
        function (attendance) {

            if (!attendance.date) {
                return false;
            }

            const attendanceDate =
                new Date(
                    attendance.date
                );

            return (
                attendanceDate.getFullYear() ===
                    todayYear &&
                attendanceDate.getMonth() ===
                    todayMonth &&
                attendanceDate.getDate() ===
                    todayDate
            );

        }
    );


const presentToday =
    todayAttendances.filter(
        function (attendance) {

            return (
                String(
                    attendance.status || ""
                )
                    .toLowerCase() ===
                "present"
            );

        }
    ).length;


const absentToday =
    todayAttendances.filter(
        function (attendance) {

            return (
                String(
                    attendance.status || ""
                )
                    .toLowerCase() ===
                "absent"
            );

        }
```

```
    ).length;


const leaveToday =
    todayAttendances.filter(
        function (attendance) {

            return (
                String(
                    attendance.status || ""
                )
                    .toLowerCase() ===
                "leave"
            );

        }
    ).length;


console.log(
    "Attendance Current Month:",
    {
        total: monthAttendances.length,
        present: presentThisMonth,
        absent: absentThisMonth,
        leave: leaveThisMonth,
        rate: attendanceRate
    }
);


console.log(
    "Attendance Today:",
    {
        total: todayAttendances.length,
        present: presentToday,
        absent: absentToday,
        leave: leaveToday
    }
);


// =========================
// RECRUITMENT
// =========================

const appliedCandidates =
    candidates.filter(
        function (candidate) {

            return (
                String(
                    candidate.status || ""
                )
                    .toLowerCase() ===
                "applied"
            );

        }
    ).length;


const shortlistedCandidates =
    candidates.filter(
        function (candidate) {

            return (
                String(
                    candidate.status || ""
                )
                    .toLowerCase() ===
                "shortlisted"
            );

        }
    ).length;


const interviewCandidates =
    candidates.filter(
        function (candidate) {

            return (
                String(
                    candidate.status || ""
                )
                    .toLowerCase() ===
                "interview"
            );

        }
    ).length;


const selectedCandidates =
    candidates.filter(
        function (candidate) {

            return (
                String(
                    candidate.status || ""
                )
                    .toLowerCase() ===
                "selected"
            );
```

```
        }
    ).length;


const rejectedCandidates =
    candidates.filter(
        function (candidate) {

            return (
                String(
                    candidate.status || ""
                )
                    .toLowerCase() ===
                "rejected"
            );

        }
    ).length;


const candidateStatus = {

    applied:
        appliedCandidates,

    shortlisted:
        shortlistedCandidates,

    interview:
        interviewCandidates,

    selected:
        selectedCandidates,

    rejected:
        rejectedCandidates

};


// ========================
// INTERVIEW STATUS
// ========================

const interviewStatus = {

    scheduled:
        interviews.filter(
            function (interview) {

                return (
                    String(
                        interview.status || ""
                    )
                        .toLowerCase() ===
                    "scheduled"
                );

            }
        ).length,

    completed:
        interviews.filter(
            function (interview) {

                return (
                    String(
                        interview.status || ""
                    )
                        .toLowerCase() ===
                    "completed"
                );

            }
        ).length,

    cancelled:
        interviews.filter(
            function (interview) {

                return (
                    String(
                        interview.status || ""
                    )
                        .toLowerCase() ===
                    "cancelled"
                );

            }
        ).length

};


// ========================
// DEPARTMENT ANALYTICS
// ========================

const departmentMap = {};


employees.forEach(
    function (employee) {
```

```
        const departmentName =
            employee.department ||
            "Other";


        if (
            !departmentMap[
                departmentName
            ]
        ) {

            departmentMap[
                departmentName
            ] = {

                name:
                    departmentName,

                employees:
                    0,

                totalSalary:
                    0

            };

        }


        departmentMap[
            departmentName
        ].employees += 1;


        departmentMap[
            departmentName
        ].totalSalary +=
            Number(
                employee.salary || 0
            );

    }
);


const departmentAnalytics =
    Object.values(
        departmentMap
    );


// =========================
// DEPARTMENT DETAILS
// =========================

const departmentDetails =
    departments.map(
        function (department) {

            const departmentEmployees =
                employees.filter(
                    function (employee) {

                        return (
                            employee.department ===
                            department.name
                        );

                    }
                );


            const departmentSalary =
                departmentEmployees.reduce(
                    function (
                        sum,
                        employee
                    ) {

                        return (
                            sum +
                            Number(
                                employee.salary ||
                                0
                            )
                        );

                    },
                    0
                );


            return {

                id:
                    department._id,

                name:
                    department.name,

                head:
                    department.head
                        ? department.head.name
                        : "Not Assigned",
```

```
                description:
                    department.description ||
                    "",

                status:
                    department.status ||
                    "Active",

                employeeCount:
                    departmentEmployees.length,

                totalSalary:
                    departmentSalary

            };

        }
    );


// ========================
// RECENT ACTIVITY
// ========================

const recentActivity = [];


recentEmployees.forEach(
    function (employee) {

        recentActivity.push({

            type:
                "employee",

            title:
                "New Employee Added",

            description:
                `${employee.name} joined ${employee.department || "the company"}`,

            date:
                employee.createdAt

        });

    }
);


recentCandidates.forEach(
    function (candidate) {

        recentActivity.push({

            type:
                "candidate",

            title:
                "Candidate Updated",

            description:
                `${candidate.name} - ${candidate.status}`,

            date:
                candidate.createdAt

        });

    }
);


recentInterviews.forEach(
    function (interview) {

        recentActivity.push({

            type:
                "interview",

            title:
                "Interview Activity",

            description:
                `Interview status: ${interview.status || "Scheduled"}`,

            date:
                interview.createdAt

        });

    }
);


recentLeaves.forEach(
    function (leave) {

        recentActivity.push({

            type:
                "leave",

            title:
```

```
                "Leave Request",

            description:
                `Leave request is ${leave.status || "Pending"}`,

            date:
                leave.createdAt

        });

    }
);


recentPayrolls.forEach(
    function (payroll) {

        recentActivity.push({

            type:
                "payroll",

            title:
                "Payroll Activity",

            description:
                `Payroll ${payroll.paymentStatus || "created"} - ₹${Number(
                    payroll.netSalary || 0
                ).toLocaleString("en-IN")}`,

            date:
                payroll.createdAt

        });

    }
);


recentAuditLogs.forEach(
    function (log) {

        recentActivity.push({

            type:
                "audit",

            title:
                log.action ||
                "System Activity",

            description:
                log.description ||
                log.module ||
                "HR system activity",

            date:
                log.createdAt

        });

    }
);


recentActivity.sort(
    function (a, b) {

        return (
            new Date(b.date) -
            new Date(a.date)
        );

    }
);


const finalRecentActivity =
    recentActivity.slice(
        0,
        10
    );


// ========================
// TOTAL SALARY
// ========================

const totalSalary =
    employees.reduce(
        function (sum, employee) {

            return (
                sum +
                Number(
                    employee.salary || 0
                )
            );

        },
        0
    );


// ========================
```

```
// FINAL RESPONSE
// ========================

res.status(200).json({

    success:
        true,

    totalEmployees:
        totalEmployees,

    totalJobs:
        totalJobs,

    totalCandidates:
        totalCandidates,

    totalInterviews:
        totalInterviews,

    pendingLeaves:
        pendingLeaves,

    totalPayroll:
        totalPayroll,

    totalPayrollRecords:
        totalPayrollRecords,

    unreadNotifications:
        unreadNotifications,

    totalSalary:
        totalSalary,

    candidateStatus:
        candidateStatus,

    interviewStatus:
        interviewStatus,

    recentActivity:
        finalRecentActivity,


    // ========================
    // ANALYTICS
    // ========================

    analytics: {

        // ========================
        // DEPARTMENTS
        // ========================

        departments:
            departmentAnalytics,

        departmentDetails:
            departmentDetails,


        // ========================
        // ATTENDANCE
        // ========================

        attendance: {

            present:
                presentThisMonth,

            absent:
                absentThisMonth,

            leave:
                leaveThisMonth,

            total:
                attendanceTotal,

            rate:
                attendanceRate,

            presentToday:
                presentToday,

            absentToday:
                absentToday,

            leaveToday:
                leaveToday,

            presentThisMonth:
                presentThisMonth,

            absentThisMonth:
                absentThisMonth,

            leaveThisMonth:
                leaveThisMonth

        },


        // ========================
```

```
            // LEAVES
            // ========================

            leaves: {

                total:
                    leaves.length,

                pending:
                    pendingLeaves,

                approved:
                    approvedLeaves,

                rejected:
                    rejectedLeaves

            },


            // ========================
            // PAYROLL
            // ========================

            payroll: {

                totalRecords:
                    payrolls.length,

                totalAmount:
                    totalPayroll,

                paid:
                    paidPayroll,

                pending:
                    pendingPayrollAmount,

                paidCount:
                    paidPayrollCount,

                pendingCount:
                    pendingPayrollCount,

                paidAmount:
                    paidPayroll,

                pendingAmount:
                    pendingPayrollAmount

            },


            // ========================
            // RECRUITMENT
            // ========================

            recruitment: {

                totalCandidates:
                    candidates.length,

                totalJobs:
                    jobs.length,

                totalInterviews:
                    interviews.length,

                applied:
                    appliedCandidates,

                shortlisted:
                    shortlistedCandidates,

                interview:
                    interviewCandidates,

                selected:
                    selectedCandidates,

                rejected:
                    rejectedCandidates

            }

        }

    });


}

catch (error) {

    console.error(
        "Dashboard Stats Error:",
        error
    );


    res.status(500).json({

        success:
            false,
```

```
            message:
                "Failed to load dashboard statistics",

            error:
                error.message

        });

    }

};
```

Backend/controllers/departmentController.js

```
const Department = require("../models/Department");
const Employee = require("../models/Employee");

const createDepartment = async (req, res) => {
    try {
        const {
            name,
            description
        } = req.body;

        if (!name) {
            return res.status(400).json({
                message: "Department name is required"
            });
        }

        const existingDepartment =
            await Department.findOne({
                name: name.trim()
            });

        if (existingDepartment) {
            return res.status(400).json({
                message: "Department already exists"
            });
        }

        const department =
            await Department.create({
                name: name.trim(),
                description: description || "",
                status: "Active",
                head: null
            });

        res.status(201).json({
            message:
                "Department created successfully",
            department
        });

    } catch (error) {
        console.error(
            "Create Department Error:",
            error
        );

        res.status(500).json({
            message:
                "Error creating department",
            error: error.message
        });
    }
};


const getAllDepartments = async (req, res) => {
    try {
        const departments =
            await Department.find()
                .populate(
                    "head",
                    "name email designation department"
                )
                .sort({
                    createdAt: -1
                });

        const departmentsWithStats =
            await Promise.all(
                departments.map(
                    async department => {

                        const employees =
                            await Employee.find({
                                department:
                                    department.name
                            });

                        const employeeCount =
                            employees.length;

                        const totalSalary =
                            employees.reduce(
                                (
                                    total,
                                    employee
                                ) => {
                                    return (
                                        total +
                                        Number(
                                            employee.salary || 0
                                        )
                                    );
                                },
                                0
                            );

                        return {
                            ...department.toObject(),
                            employeeCount,
                            totalSalary
```

```
                        };
                    }
                )
            );

        res.status(200).json({
            count:
                departmentsWithStats.length,
            departments:
                departmentsWithStats
        });

    } catch (error) {
        console.error(
            "Get Departments Error:",
            error
        );

        res.status(500).json({
            message:
                "Error fetching departments",
            error: error.message
        });
    }
};


const getDepartmentById = async (
    req,
    res
) => {
    try {

        const department =
            await Department.findById(
                req.params.id
            ).populate(
                "head",
                "name email designation department"
            );

        if (!department) {
            return res.status(404).json({
                message:
                    "Department not found"
            });
        }

        const employees =
            await Employee.find({
                department:
                    department.name
            });

        const employeeCount =
            employees.length;

        const totalSalary =
            employees.reduce(
                (
                    total,
                    employee
                ) => {
                    return (
                        total +
                        Number(
                            employee.salary || 0
                        )
                    );
                },
                0
            );

        res.status(200).json({
            department,
            employeeCount,
            totalSalary,
            employees
        });

    } catch (error) {
        console.error(
            "Get Department Error:",
            error
        );

        res.status(500).json({
            message:
                "Error fetching department",
            error: error.message
        });
    }
};


const updateDepartment = async (
    req,
    res
) => {
    try {

        const {
            name,
            description,
            status
```

```
        } = req.body;

        const department =
            await Department.findById(
                req.params.id
            );

        if (!department) {
            return res.status(404).json({
                message:
                    "Department not found"
            });
        }

        const oldName =
            department.name;

        if (name) {

            const newName =
                name.trim();

            const existingDepartment =
                await Department.findOne({
                    name: newName,
                    _id: {
                        $ne:
                            req.params.id
                    }
                });

            if (existingDepartment) {
                return res.status(400).json({
                    message:
                        "Department name already exists"
                });
            }

            department.name =
                newName;

            if (oldName !== newName) {

                await Employee.updateMany(
                    {
                        department:
                            oldName
                    },
                    {
                        $set: {
                            department:
                                newName
                        }
                    }
                );
            }
        }

        if (description !== undefined) {
            department.description =
                description;
        }

        if (status) {
            department.status =
                status;
        }

        await department.save();

        res.status(200).json({
            message:
                "Department updated successfully",
            department
        });

    } catch (error) {
        console.error(
            "Update Department Error:",
            error
        );

        res.status(500).json({
            message:
                "Error updating department",
            error: error.message
        });
    }
};


const changeDepartmentHead = async (
    req,
    res
) => {
    try {

        const {
            head
        } = req.body;

        const department =
            await Department.findById(
                req.params.id
            );
```

```
        if (!department) {
            return res.status(404).json({
                message:
                    "Department not found"
            });
        }

        if (!head) {

            department.head = null;

            await department.save();

            return res.status(200).json({
                message:
                    "Department head removed successfully",
                department
            });
        }

        const employee =
            await Employee.findById(head);

        if (!employee) {
            return res.status(404).json({
                message:
                    "Employee not found"
            });
        }

        if (
            employee.department !==
            department.name
        ) {
            return res.status(400).json({
                message:
                    "Department head must belong to this department"
            });
        }

        department.head =
            employee._id;

        await department.save();

        const updatedDepartment =
            await Department.findById(
                department._id
            ).populate(
                "head",
                "name email designation department"
            );

        res.status(200).json({
            message:
                "Department head updated successfully",
            department:
                updatedDepartment
        });

    } catch (error) {
        console.error(
            "Change Department Head Error:",
            error
        );

        res.status(500).json({
            message:
                "Error changing department head",
            error: error.message
        });
    }
};


const deleteDepartment = async (
    req,
    res
) => {
    try {

        const department =
            await Department.findById(
                req.params.id
            );

        if (!department) {
            return res.status(404).json({
                message:
                    "Department not found"
            });
        }

        const employeeCount =
            await Employee.countDocuments({
                department:
                    department.name
            });

        if (employeeCount > 0) {
            return res.status(400).json({
                message:
                    "Cannot delete department because employees are assigned to it"
            });
```

```
        }

        await Department.findByIdAndDelete(
            req.params.id
        );

        res.status(200).json({
            message:
                "Department deleted successfully"
        });

    } catch (error) {
        console.error(
            "Delete Department Error:",
            error
        );

        res.status(500).json({
            message:
                "Error deleting department",
            error: error.message
        });
    }
};


module.exports = {
    createDepartment,
    getAllDepartments,
    getDepartmentById,
    updateDepartment,
    changeDepartmentHead,
    deleteDepartment
};
```

Backend/controllers/employeeController.js

```
const Employee = require("../models/Employee");
const User = require("../models/User");

const {
    createNotifications
} = require("../services/notificationService");


/* ========================
   CREATE EMPLOYEE
======================== */

const createEmployee = async (req, res) => {

    try {

        const {
            name,
            email,
            phone,
            department,
            designation,
            salary,
            joiningDate,
            address
        } = req.body;


        if (
            !name ||
            !email ||
            !phone ||
            !department ||
            !designation ||
            !salary ||
            !joiningDate ||
            !address
        ) {

            return res.status(400).json({
                message:
                    "All employee fields are required"
            });

        }


        const existingEmployee =
            await Employee.findOne({
                email
            });


        if (existingEmployee) {

            return res.status(400).json({
                message:
                    "Employee with this email already exists"
            });

        }


        const employee =
            await Employee.create({

                name,
                email,
                phone,
                department,
                designation,
                salary,
                joiningDate,
                address

            });


        console.log(
            "Employee Created:",
            employee.name
        );


        /* ========================
           FIND HR + ADMIN USERS
        ======================== */

        const hrUsers =
            await User.find({

                role: {
                    $in: [
                        "admin",
                        "hr"
                    ]
                }

            }).select("_id");


        console.log(
```

```
            "Admin/HR Users Found:",
            hrUsers.length
        );


        /* ========================
           CREATE NOTIFICATIONS
        ======================== */

        if (hrUsers.length > 0) {

            const notifications =
                hrUsers.map(user => ({

                    userId:
                        user._id,

                    title:
                        "New Employee Added",

                    message:
                        `${employee.name} has been added to the ${employee.department} department.`,

                    type:
                        "Employee",

                    priority:
                        "Normal",

                    actionUrl:
                        "/employees.html",

                    relatedId:
                        employee._id,

                    relatedModel:
                        "Employee"

                }));


            console.log(
                "Creating Employee Notifications:",
                notifications.length
            );


            const createdNotifications =
                await createNotifications(
                    notifications
                );


            console.log(
                "Employee Notifications Created:",
                createdNotifications.length
            );

        } else {

            console.log(
                "No Admin or HR users found"
            );

        }


        res.status(201).json({

            message:
                "Employee created successfully",

            employee

        });


    } catch (error) {

        console.error(
            "Create Employee Error:",
            error
        );


        res.status(500).json({

            message:
                "Server error",

            error:
                error.message

        });

    }

};


/* ========================
   GET ALL EMPLOYEES
======================== */

const getEmployees = async (req, res) => {
```

```
    try {

        const employees =
            await Employee.find()
                .sort({
                    createdAt: -1
                });


        res.status(200).json({

            count:
                employees.length,

            employees

        });


    } catch (error) {

        console.error(
            "Get Employees Error:",
            error
        );


        res.status(500).json({

            message:
                "Server error",

            error:
                error.message

        });

    }

};


/* ========================
   GET EMPLOYEE BY ID
======================== */

const getEmployeeById = async (req, res) => {

    try {

        const employeeId =
            req.params.id;


        if (
            !employeeId.match(
                /^[0-9a-fA-F]{24}$/
            )
        ) {

            return res.status(400).json({

                message:
                    "Invalid employee ID"

            });

        }


        const employee =
            await Employee.findById(
                employeeId
            );


        if (!employee) {

            return res.status(404).json({

                message:
                    "Employee not found"

            });

        }


        res.status(200).json({

            employee

        });


    } catch (error) {

        console.error(
            "Get Employee Error:",
            error
        );


        res.status(500).json({
```

```
            message:
                "Server error",

            error:
                error.message

        });

    }

};


/* ========================
   UPDATE EMPLOYEE
======================== */

const updateEmployee = async (req, res) => {

    try {

        const employeeId =
            req.params.id;


        if (
            !employeeId.match(
                /^[0-9a-fA-F]{24}$/
            )
        ) {

            return res.status(400).json({

                message:
                    "Invalid employee ID"

            });

        }


        const employee =
            await Employee.findByIdAndUpdate(

                employeeId,

                req.body,

                {
                    new: true,
                    runValidators: true
                }

            );


        if (!employee) {

            return res.status(404).json({

                message:
                    "Employee not found"

            });

        }


        res.status(200).json({

            message:
                "Employee updated successfully",

            employee

        });


    } catch (error) {

        console.error(
            "Update Employee Error:",
            error
        );


        res.status(500).json({

            message:
                "Server error",

            error:
                error.message

        });

    }

};


/* ========================
   DELETE EMPLOYEE
======================== */
```

```
const deleteEmployee = async (req, res) => {

    try {

        const employeeId =
            req.params.id;


        if (
            !employeeId.match(
                /^[0-9a-fA-F]{24}$/
            )
        ) {

            return res.status(400).json({

                message:
                    "Invalid employee ID"

            });

        }


        const employee =
            await Employee.findByIdAndDelete(
                employeeId
            );


        if (!employee) {

            return res.status(404).json({

                message:
                    "Employee not found"

            });

        }


        res.status(200).json({

            message:
                "Employee deleted successfully"

        });


    } catch (error) {

        console.error(
            "Delete Employee Error:",
            error
        );


        res.status(500).json({

            message:
                "Server error",

            error:
                error.message

        });

    }

};


/* ========================
   EXPORT
======================== */

module.exports = {

    createEmployee,
    getEmployees,
    getEmployeeById,
    updateEmployee,
    deleteEmployee

};
```

Backend/controllers/interviewController.js

```
const Interview = require("../models/Interview");

const createInterview = async (req, res) => {
    try {
        const {
            candidateId,
            jobId,
            interviewer,
            interviewDate,
            mode,
            meetingLink,
            location
        } = req.body;

        if (
            !candidateId ||
            !jobId ||
            !interviewer ||
            !interviewDate ||
            !mode
        ) {
            return res.status(400).json({
                message: "All required interview fields are required"
            });
        }

        if (mode === "Online" && !meetingLink) {
            return res.status(400).json({
                message: "Meeting link is required for online interview"
            });
        }

        if (mode === "Offline" && !location) {
            return res.status(400).json({
                message: "Location is required for offline interview"
            });
        }

        const interview = await Interview.create({
            candidateId,
            jobId,
            interviewer,
            interviewDate,
            mode,
            meetingLink,
            location
        });

        res.status(201).json({
            message: "Interview scheduled successfully",
            interview
        });

    } catch (error) {

        console.error(
            "Create Interview Error:",
            error
        );

        res.status(500).json({
            message: "Error scheduling interview",
            error: error.message
        });
    }
};

const getAllInterviews = async (req, res) => {
    try {
        const interviews = await Interview.find()
            .populate("candidateId")
            .populate("jobId");

        res.status(200).json({
            count: interviews.length,
            interviews
        });

    } catch (error) {

        res.status(500).json({
            message: "Error fetching interviews",
            error: error.message
        });
    }
};

const getInterviewById = async (req, res) => {
    try {
        const interview = await Interview.findById(req.params.id)
            .populate("candidateId")
            .populate("jobId");

        if (!interview) {
            return res.status(404).json({
                message: "Interview not found"
            });
        }

        res.status(200).json({
            interview
        });
```

```
    } catch (error) {

        res.status(500).json({
            message: "Error fetching interview",
            error: error.message
        });
    }
};

const updateInterviewStatus = async (req, res) => {
    try {
        const { status } = req.body;

        const interview = await Interview.findByIdAndUpdate(
            req.params.id,
            { status },
            {
                new: true,
                runValidators: true
            }
        );

        if (!interview) {
            return res.status(404).json({
                message: "Interview not found"
            });
        }

        res.status(200).json({
            message: "Interview status updated successfully",
            interview
        });

    } catch (error) {

        res.status(500).json({
            message: "Error updating interview status",
            error: error.message
        });
    }
};

const updateInterviewResult = async (req, res) => {
    try {
        const { result, feedback } = req.body;

        const interview = await Interview.findByIdAndUpdate(
            req.params.id,
            {
                result,
                feedback
            },
            {
                new: true,
                runValidators: true
            }
        );

        if (!interview) {
            return res.status(404).json({
                message: "Interview not found"
            });
        }

        res.status(200).json({
            message: "Interview result updated successfully",
            interview
        });

    } catch (error) {

        res.status(500).json({
            message: "Error updating interview result",
            error: error.message
        });
    }
};

const deleteInterview = async (req, res) => {
    try {
        const interview = await Interview.findByIdAndDelete(
            req.params.id
        );

        if (!interview) {
            return res.status(404).json({
                message: "Interview not found"
            });
        }

        res.status(200).json({
            message: "Interview deleted successfully"
        });

    } catch (error) {

        res.status(500).json({
            message: "Error deleting interview",
            error: error.message
        });
    }
};

module.exports = {
```

```
    createInterview,
    getAllInterviews,
    getInterviewById,
    updateInterviewStatus,
    updateInterviewResult,
    deleteInterview
};
```

Backend/controllers/jobController.js

```
const Job = require("../models/Job");


/* ========================
   CREATE JOB
======================== */

const createJob = async (req, res) => {

    try {

        const {
            title,
            department,
            location,
            jobType,
            salary,
            description,
            requirements,
            status
        } = req.body;


        if (
            !title ||
            !department ||
            !location ||
            !jobType ||
            !salary ||
            !description ||
            !requirements
        ) {

            return res.status(400).json({
                message:
                    "All job fields are required"
            });

        }


        const job =
            await Job.create({

                title,
                department,
                location,
                jobType,
                salary,
                description,
                requirements,
                status:
                    status || "Open",

                postedBy:
                    req.user
                        ? req.user.id
                        : undefined

            });


        res.status(201).json({

            message:
                "Job created successfully",

            job

        });


    } catch (error) {

        console.error(
            "Create Job Error:",
            error
        );


        res.status(500).json({

            message:
                "Server error",

            error:
                error.message

        });

    }

};


/* ========================
   GET ALL JOBS
======================== */

const getJobs = async (req, res) => {

    try {
```

```
        const jobs =
            await Job.find()
                .populate(
                    "postedBy",
                    "name email"
                )
                .sort({
                    createdAt: -1
                });


        res.status(200).json({

            count:
                jobs.length,

            jobs

        });


    } catch (error) {

        console.error(
            "Get Jobs Error:",
            error
        );


        res.status(500).json({

            message:
                "Server error",

            error:
                error.message

        });

    }

};


/* ========================
   GET JOB BY ID
======================== */

const getJobById = async (req, res) => {

    try {

        const jobId =
            req.params.id;


        if (
            !jobId.match(
                /^[0-9a-fA-F]{24}$/
            )
        ) {

            return res.status(400).json({

                message:
                    "Invalid job ID"

            });

        }


        const job =
            await Job.findById(
                jobId
            ).populate(
                "postedBy",
                "name email"
            );


        if (!job) {

            return res.status(404).json({

                message:
                    "Job not found"

            });

        }


        res.status(200).json({

            job

        });


    } catch (error) {

        console.error(
            "Get Job Error:",
```

```
            error
        );


        res.status(500).json({

            message:
                "Server error",

            error:
                error.message

        });

    }

};


/* ========================
   UPDATE JOB
======================== */

const updateJob = async (req, res) => {

    try {

        const jobId =
            req.params.id;


        if (
            !jobId.match(
                /^[0-9a-fA-F]{24}$/
            )
        ) {

            return res.status(400).json({

                message:
                    "Invalid job ID"

            });

        }


        const job =
            await Job.findByIdAndUpdate(

                jobId,

                req.body,

                {
                    new: true,
                    runValidators: true
                }

            );


        if (!job) {

            return res.status(404).json({

                message:
                    "Job not found"

            });

        }


        res.status(200).json({

            message:
                "Job updated successfully",

            job

        });


    } catch (error) {

        console.error(
            "Update Job Error:",
            error
        );


        res.status(500).json({

            message:
                "Server error",

            error:
                error.message

        });

    }

};
```

```
/* ========================
   DELETE JOB
======================== */

const deleteJob = async (req, res) => {

    try {

        const jobId =
            req.params.id;


        if (
            !jobId.match(
                /^[0-9a-fA-F]{24}$/
            )
        ) {

            return res.status(400).json({

                message:
                    "Invalid job ID"

            });

        }


        const job =
            await Job.findByIdAndDelete(
                jobId
            );


        if (!job) {

            return res.status(404).json({

                message:
                    "Job not found"

            });

        }


        res.status(200).json({

            message:
                "Job deleted successfully"

        });


    } catch (error) {

        console.error(
            "Delete Job Error:",
            error
        );


        res.status(500).json({

            message:
                "Server error",

            error:
                error.message

        });

    }

};


module.exports = {

    createJob,
    getJobs,
    getJobById,
    updateJob,
    deleteJob

};
```

Backend/controllers/leaveController.js

```
const Leave = require("../models/Leave");
const Employee = require("../models/Employee");
const User = require("../models/User");

const {
    createNotifications
} = require("../services/notificationService");


// Employee + Admin: Create Leave
const createLeave = async (req, res) => {
    try {

        console.log("REQ USER:", req.user);

        const {
            employeeId,
            leaveType,
            startDate,
            endDate,
            reason
        } = req.body;

        if (!leaveType || !startDate || !endDate || !reason) {
            return res.status(400).json({
                message: "All leave fields are required"
            });
        }

        const user = await User.findById(req.user.id);

        console.log("DB USER:", user);

        if (!user) {
            return res.status(404).json({
                message: "User not found"
            });
        }

        let finalEmployeeId = employeeId;


        // Employee login
        if (user.role === "employee") {

            const employee = await Employee.findOne({
                email: user.email
            });

            console.log("EMPLOYEE RECORD:", employee);

            if (!employee) {
                return res.status(404).json({
                    message: "Employee record not found for this user"
                });
            }

            finalEmployeeId = employee._id;
        }


        // Admin must provide employeeId
        if (!finalEmployeeId) {
            return res.status(400).json({
                message: "Employee ID is required"
            });
        }


        const leave = await Leave.create({
            employeeId: finalEmployeeId,
            leaveType,
            startDate,
            endDate,
            reason,
            status: "Pending"
        });


        console.log(
            "Leave Created:",
            leave._id
        );


        // Find employee details
        const employee =
            await Employee.findById(
                finalEmployeeId
            );


        // Notify Admin + HR
        const hrUsers =
            await User.find({
                role: {
                    $in: [
                        "admin",
                        "hr"
                    ]
                }
            }).select("_id");
```

```
        console.log(
            "Admin/HR Users Found:",
            hrUsers.length
        );


        if (
            employee &&
            hrUsers.length > 0
        ) {

            const notifications =
                hrUsers.map(user => ({

                    userId:
                        user._id,

                    title:
                        "New Leave Request",

                    message:
                        `${employee.name} has applied for ${leaveType} leave.`,

                    type:
                        "Leave",

                    priority:
                        "Normal",

                    actionUrl:
                        "/leaves.html",

                    relatedId:
                        leave._id,

                    relatedModel:
                        "Leave"

                }));


            console.log(
                "Creating Leave Notifications:",
                notifications.length
            );


            const createdNotifications =
                await createNotifications(
                    notifications
                );


            console.log(
                "Leave Notifications Created:",
                createdNotifications.length
            );
        }


        res.status(201).json({
            message: "Leave applied successfully",
            leave
        });

    } catch (error) {

        console.log(
            "Create Leave Error:",
            error.message
        );

        res.status(500).json({
            message: "Error creating leave",
            error: error.message
        });
    }
};


// Admin: Get All Leaves
const getAllLeaves = async (req, res) => {
    try {

        const leaves = await Leave.find()
            .populate(
                "employeeId",
                "name email department designation"
            )
            .sort({ createdAt: -1 });

        res.status(200).json({
            count: leaves.length,
            leaves
        });

    } catch (error) {

        console.log(
            "Get All Leaves Error:",
            error.message
        );
```

```
        res.status(500).json({
            message: "Error fetching leaves",
            error: error.message
        });
    }
};


// Employee: Get Own Leaves
const getMyLeaves = async (req, res) => {
    try {

        console.log(
            "MY LEAVES USER:",
            req.user
        );

        const user =
            await User.findById(
                req.user.id
            );

        if (!user) {
            return res.status(404).json({
                message: "User not found"
            });
        }

        const employee =
            await Employee.findOne({
                email: user.email
            });

        console.log(
            "MY EMPLOYEE RECORD:",
            employee
        );

        if (!employee) {
            return res.status(404).json({
                message:
                    "Employee record not found for this user"
            });
        }

        const leaves =
            await Leave.find({
                employeeId:
                    employee._id
            })
                .populate(
                    "employeeId",
                    "name email department designation"
                )
                .sort({
                    createdAt: -1
                });

        res.status(200).json({
            count: leaves.length,
            leaves
        });

    } catch (error) {

        console.log(
            "Get My Leaves Error:",
            error.message
        );

        res.status(500).json({
            message:
                "Error fetching your leaves",
            error: error.message
        });
    }
};


// Admin: Approve Leave
const approveLeave = async (req, res) => {
    try {

        const leave =
            await Leave.findByIdAndUpdate(
                req.params.id,
                {
                    status: "Approved"
                },
                {
                    new: true,
                    runValidators: true
                }
            );

        if (!leave) {
            return res.status(404).json({
                message:
                    "Leave not found"
            });
        }


        // Find employee
        const employee =
```

```
            await Employee.findById(
                leave.employeeId
            );


        if (employee) {

            // Find employee user
            const employeeUser =
                await User.findOne({
                    email:
                        employee.email
                });


            if (employeeUser) {

                await createNotifications(
                    [
                        {
                            userId:
                                employeeUser._id,

                            title:
                                "Leave Approved",

                            message:
                                `Your ${leave.leaveType} leave from ${leave.startDate} to ${leave.endDate} has been approved.`,

                            type:
                                "Leave",

                            priority:
                                "Normal",

                            actionUrl:
                                "/leaves.html",

                            relatedId:
                                leave._id,

                            relatedModel:
                                "Leave"
                        }
                    ]
                );

            }

        }


        res.status(200).json({
            message:
                "Leave approved successfully",
            leave
        });

    } catch (error) {

        console.log(
            "Approve Leave Error:",
            error.message
        );

        res.status(500).json({
            message:
                "Error approving leave",
            error: error.message
        });
    }
};


// Admin: Reject Leave
const rejectLeave = async (req, res) => {
    try {

        const leave =
            await Leave.findByIdAndUpdate(
                req.params.id,
                {
                    status: "Rejected"
                },
                {
                    new: true,
                    runValidators: true
                }
            );

        if (!leave) {
            return res.status(404).json({
                message:
                    "Leave not found"
            });
        }


        // Find employee
        const employee =
            await Employee.findById(
                leave.employeeId
            );
```

```
        if (employee) {

            // Find employee user
            const employeeUser =
                await User.findOne({
                    email:
                        employee.email
                });


            if (employeeUser) {

                await createNotifications(
                    [
                        {
                            userId:
                                employeeUser._id,

                            title:
                                "Leave Rejected",

                            message:
                                `Your ${leave.leaveType} leave from ${leave.startDate} to ${leave.endDate} has been rejected.`,

                            type:
                                "Leave",

                            priority:
                                "High",

                            actionUrl:
                                "/leaves.html",

                            relatedId:
                                leave._id,

                            relatedModel:
                                "Leave"
                        }
                    ]
                );

            }

        }


        res.status(200).json({
            message:
                "Leave rejected successfully",
            leave
        });

    } catch (error) {

        console.log(
            "Reject Leave Error:",
            error.message
        );

        res.status(500).json({
            message:
                "Error rejecting leave",
            error: error.message
        });
    }
};


// Admin: Employee-wise Leaves
const getEmployeeLeaves = async (req, res) => {
    try {

        const leaves =
            await Leave.find({
                employeeId:
                    req.params.employeeId
            })
                .populate(
                    "employeeId",
                    "name email department designation"
                )
                .sort({
                    createdAt: -1
                });

        res.status(200).json({
            count: leaves.length,
            leaves
        });

    } catch (error) {

        console.log(
            "Employee Leaves Error:",
            error.message
        );

        res.status(500).json({
            message:
                "Error fetching employee leaves",
            error: error.message
        });
    }
};
```

```
module.exports = {

    createLeave,
    getAllLeaves,
    getMyLeaves,
    approveLeave,
    rejectLeave,
    getEmployeeLeaves

};
```

Backend/controllers/notificationController.js

```
const Notification = require("../models/Notification");


/* ========================
   GET NOTIFICATIONS
======================== */

const getNotifications = async (req, res) => {

    try {

        const {
            page = 1,
            limit = 100,
            search = "",
            status = "all",
            type = "all"
        } = req.query;


        const filter = {

            userId: req.user.id

        };


        if (search.trim()) {

            filter.$or = [

                {
                    title: {
                        $regex: search,
                        $options: "i"
                    }
                },

                {
                    message: {
                        $regex: search,
                        $options: "i"
                    }
                }

            ];

        }


        if (status === "read") {

            filter.isRead = true;

        }


        if (status === "unread") {

            filter.isRead = false;

        }


        if (type !== "all") {

            filter.type = type;

        }


        const skip =
            (Number(page) - 1) *
            Number(limit);


        const notifications =
            await Notification
                .find(filter)
                .sort({
                    createdAt: -1
                })
                .skip(skip)
                .limit(Number(limit));


        const total =
            await Notification.countDocuments(
                filter
            );


        const unreadCount =
            await Notification.countDocuments({

                userId: req.user.id,

                isRead: false

            });
```

```
        res.status(200).json({

            success: true,

            notifications,

            pagination: {

                page: Number(page),

                limit: Number(limit),

                total,

                totalPages:
                    Math.ceil(
                        total /
                        Number(limit)
                    )

            },

            unreadCount

        });


    } catch (error) {

        console.log(
            "Get Notifications Error:",
            error
        );


        res.status(500).json({

            success: false,

            message:
                "Failed to fetch notifications"

        });

    }

};


/* ========================
   GET UNREAD COUNT
======================== */

const getUnreadCount = async (
    req,
    res
) => {

    try {

        const count =
            await Notification.countDocuments({

                userId: req.user.id,

                isRead: false

            });


        res.status(200).json({

            success: true,

            unreadCount: count

        });


    } catch (error) {

        console.log(
            "Unread Count Error:",
            error
        );


        res.status(500).json({

            success: false,

            message:
                "Failed to get unread count"

        });

    }

};


/* ========================
   GET BY ID
======================== */
```

```
const getNotificationById = async (
    req,
    res
) => {

    try {

        const notification =
            await Notification.findOne({

                _id: req.params.id,

                userId: req.user.id

            });


        if (!notification) {

            return res.status(404).json({

                success: false,

                message:
                    "Notification not found"

            });

        }


        res.status(200).json({

            success: true,

            notification

        });


    } catch (error) {

        console.log(
            "Get Notification Error:",
            error
        );


        res.status(500).json({

            success: false,

            message:
                "Failed to fetch notification"

        });

    }

};


/* ========================
   MARK AS READ
======================== */

const markAsRead = async (
    req,
    res
) => {

    try {

        const notification =
            await Notification.findOne({

                _id: req.params.id,

                userId: req.user.id

            });


        if (!notification) {

            return res.status(404).json({

                success: false,

                message:
                    "Notification not found"

            });

        }


        notification.isRead = true;

        notification.readAt =
            new Date();


        await notification.save();
```

```
        res.status(200).json({

            success: true,

            message:
                "Notification marked as read",

            notification

        });


    } catch (error) {

        console.log(
            "Mark Read Error:",
            error
        );


        res.status(500).json({

            success: false,

            message:
                "Failed to mark notification as read"

        });

    }

};


/* ========================
   MARK AS UNREAD
======================== */

const markAsUnread = async (
    req,
    res
) => {

    try {

        const notification =
            await Notification.findOne({

                _id: req.params.id,

                userId: req.user.id

            });


        if (!notification) {

            return res.status(404).json({

                success: false,

                message:
                    "Notification not found"

            });

        }


        notification.isRead = false;

        notification.readAt = null;


        await notification.save();


        res.status(200).json({

            success: true,

            message:
                "Notification marked as unread",

            notification

        });


    } catch (error) {

        console.log(
            "Mark Unread Error:",
            error
        );


        res.status(500).json({

            success: false,

            message:
                "Failed to mark notification as unread"
```

```
        });

    }

};


/* ========================
   MARK ALL AS READ
======================== */

const markAllAsRead = async (
    req,
    res
) => {

    try {

        const result =
            await Notification.updateMany(

                {
                    userId: req.user.id,

                    isRead: false
                },

                {
                    $set: {

                        isRead: true,

                        readAt:
                            new Date()

                    }

                }

            );


        res.status(200).json({

            success: true,

            message:
                "All notifications marked as read",

            modifiedCount:
                result.modifiedCount

        });


    } catch (error) {

        console.log(
            "Mark All Read Error:",
            error
        );


        res.status(500).json({

            success: false,

            message:
                "Failed to mark all notifications as read"

        });

    }

};


/* ========================
   DELETE NOTIFICATION
======================== */

const deleteNotification = async (
    req,
    res
) => {

    try {

        const notification =
            await Notification.findOneAndDelete({

                _id: req.params.id,

                userId: req.user.id

            });


        if (!notification) {

            return res.status(404).json({

                success: false,

                message:
```

```
                    "Notification not found"

            });

        }


        res.status(200).json({

            success: true,

            message:
                "Notification deleted successfully"

        });


    } catch (error) {

        console.log(
            "Delete Notification Error:",
            error
        );


        res.status(500).json({

            success: false,

            message:
                "Failed to delete notification"

        });

    }

};


/* ========================
   DELETE READ
======================== */

const deleteReadNotifications =
    async (
        req,
        res
    ) => {

        try {

            const result =
                await Notification.deleteMany({

                    userId:
                        req.user.id,

                    isRead: true

                });


            res.status(200).json({

                success: true,

                message:
                    "Read notifications deleted successfully",

                deletedCount:
                    result.deletedCount

            });


        } catch (error) {

            console.log(
                "Delete Read Notifications Error:",
                error
            );


            res.status(500).json({

                success: false,

                message:
                    "Failed to delete read notifications"

            });

        }

    };


module.exports = {

    getNotifications,

    getUnreadCount,

    getNotificationById,
```

```
    markAsRead,
    markAsUnread,
    markAllAsRead,
    deleteNotification,
    deleteReadNotifications
};
```

Backend/controllers/payrollController.js

```
const Payroll = require("../models/Payroll");
const Employee = require("../models/Employee");
const User = require("../models/User");

const {
    createNotifications
} = require("../services/notificationService");


// Create Payroll
const createPayroll = async (req, res) => {
    try {

        const {
            employeeId,
            month,
            basicSalary,
            allowances,
            deductions
        } = req.body;


        const netSalary =
            Number(basicSalary) +
            Number(allowances || 0) -
            Number(deductions || 0);


        const payroll =
            await Payroll.create({
                employeeId,
                month,
                basicSalary,
                allowances,
                deductions,
                netSalary
            });


        console.log(
            "Payroll Created:",
            payroll._id
        );


        // Find employee
        const employee =
            await Employee.findById(
                employeeId
            );


        // Find employee user
        if (employee) {

            const employeeUser =
                await User.findOne({
                    email: employee.email
                });


            if (employeeUser) {

                await createNotifications([
                    {
                        userId:
                            employeeUser._id,

                        title:
                            "Payroll Generated",

                        message:
                            `Your payroll for ${month} has been generated. Net salary: ₹${netSalary}.`,

                        type:
                            "Payroll",

                        priority:
                            "Normal",

                        actionUrl:
                            "/payroll.html",

                        relatedId:
                            payroll._id,

                        relatedModel:
                            "Payroll"
                    }
                ]);


                console.log(
                    "Payroll Notification Created"
                );

            } else {

                console.log(
                    "Employee user not found:",
                    employee.email
                );
```

```
            }

        }


        res.status(201).json({
            message:
                "Payroll created successfully",
            payroll
        });


    } catch (error) {

        console.log(
            "Create Payroll Error:",
            error.message
        );

        res.status(500).json({
            message:
                "Error creating payroll",
            error: error.message
        });
    }
};


// Get All Payrolls
const getAllPayrolls = async (req, res) => {
    try {

        const payrolls =
            await Payroll.find()
                .populate("employeeId");


        res.status(200).json({
            count:
                payrolls.length,
            payrolls
        });


    } catch (error) {

        res.status(500).json({
            message:
                "Error fetching payrolls",
            error: error.message
        });
    }
};


// Get Employee Payroll
const getEmployeePayroll = async (req, res) => {
    try {

        const payrolls =
            await Payroll.find({
                employeeId:
                    req.params.employeeId
            }).populate("employeeId");


        res.status(200).json({
            count:
                payrolls.length,
            payrolls
        });


    } catch (error) {

        res.status(500).json({
            message:
                "Error fetching employee payroll",
            error: error.message
        });
    }
};


// Update Payment Status
const updatePaymentStatus = async (req, res) => {
    try {

        const {
            paymentStatus
        } = req.body;


        const existingPayroll =
            await Payroll.findById(
                req.params.id
            );


        if (!existingPayroll) {

            return res.status(404).json({
                message:
                    "Payroll not found"
```

```
        });

    }


    const previousStatus =
        existingPayroll.paymentStatus;


    const payroll =
        await Payroll.findByIdAndUpdate(
            req.params.id,
            {
                paymentStatus,

                paymentDate:
                    paymentStatus === "Paid"
                        ? new Date()
                        : null
            },
            {
                new: true
            }
        );


    // Notify employee when salary is paid
    if (
        paymentStatus === "Paid" &&
        previousStatus !== "Paid"
    ) {

        const employee =
            await Employee.findById(
                payroll.employeeId
            );


        if (employee) {

            const employeeUser =
                await User.findOne({
                    email:
                        employee.email
                });


            if (employeeUser) {

                await createNotifications([
                    {
                        userId:
                            employeeUser._id,

                        title:
                            "Salary Paid",

                        message:
                            `Your salary for ${payroll.month} has been paid. Amount: ₹${payroll.netSalary}.`,

                        type:
                            "Payroll",

                        priority:
                            "High",

                        actionUrl:
                            "/payroll.html",

                        relatedId:
                            payroll._id,

                        relatedModel:
                            "Payroll"
                    }
                ]);


                console.log(
                    "Salary Paid Notification Created"
                );

            }

        }

    }


    res.status(200).json({
        message:
            "Payment status updated successfully",
        payroll
    });


} catch (error) {

    console.log(
        "Update Payment Status Error:",
        error.message
    );

    res.status(500).json({
        message:
```

```
                "Error updating payment status",
            error: error.message
        });
    }
};


module.exports = {
    createPayroll,
    getAllPayrolls,
    getEmployeePayroll,
    updatePaymentStatus
};
```

Backend/controllers/performanceController.js

```
const Performance = require("../models/Performance");

const createPerformance = async (req, res) => {
    try {
        const {
            employeeId,
            reviewPeriod,
            goals,
            achievements,
            rating,
            feedback
        } = req.body;

        const performance = await Performance.create({
            employeeId,
            reviewPeriod,
            goals,
            achievements,
            rating,
            feedback
        });

        res.status(201).json({
            message: "Performance review created successfully",
            performance
        });
    } catch (error) {
        res.status(500).json({
            message: "Error creating performance review",
            error: error.message
        });
    }
};

const getAllPerformances = async (req, res) => {
    try {
        const performances = await Performance.find()
            .populate("employeeId");

        res.status(200).json({
            count: performances.length,
            performances
        });
    } catch (error) {
        res.status(500).json({
            message: "Error fetching performance reviews",
            error: error.message
        });
    }
};

const getEmployeePerformance = async (req, res) => {
    try {
        const performances = await Performance.find({
            employeeId: req.params.employeeId
        }).populate("employeeId");

        res.status(200).json({
            count: performances.length,
            performances
        });
    } catch (error) {
        res.status(500).json({
            message: "Error fetching employee performance",
            error: error.message
        });
    }
};

const updatePerformanceStatus = async (req, res) => {
    try {
        const { status } = req.body;

        const performance = await Performance.findByIdAndUpdate(
            req.params.id,
            { status },
            { new: true }
        );

        if (!performance) {
            return res.status(404).json({
                message: "Performance review not found"
            });
        }

        res.status(200).json({
            message: "Performance status updated successfully",
            performance
        });
    } catch (error) {
        res.status(500).json({
            message: "Error updating performance status",
            error: error.message
        });
    }
};

module.exports = {
    createPerformance,
    getAllPerformances,
    getEmployeePerformance,
```

```
    updatePerformanceStatus
};
```

Backend/controllers/userController.js

```
const User = require("../models/User");
const bcrypt = require("bcryptjs");


// Get My Profile

const getMyProfile = async (req, res) => {

    try {

        const user = await User.findById(req.user.id)
            .select("-password");

        if (!user) {

            return res.status(404).json({
                message: "User not found"
            });

        }

        res.status(200).json({
            user
        });

    } catch (error) {

        console.log("Profile Error:", error.message);

        res.status(500).json({
            message: "Error fetching profile",
            error: error.message
        });

    }

};



// Update My Profile

const updateMyProfile = async (req, res) => {

    try {

        const {
            name,
            email
        } = req.body;


        if (!name || !email) {

            return res.status(400).json({
                message: "Name and email are required"
            });

        }


        const existingUser = await User.findOne({
            email: email.toLowerCase(),
            _id: { $ne: req.user.id }
        });


        if (existingUser) {

            return res.status(400).json({
                message: "Email already exists"
            });

        }


        const user = await User.findByIdAndUpdate(

            req.user.id,

            {
                name: name.trim(),
                email: email.toLowerCase().trim()
            },

            {
                new: true,
                runValidators: true
            }

        ).select("-password");


        if (!user) {

            return res.status(404).json({
                message: "User not found"
            });

        }
```

```
        res.status(200).json({

            message: "Profile updated successfully",

            user

        });


    } catch (error) {

        console.log(
            "Update Profile Error:",
            error.message
        );

        res.status(500).json({

            message: "Error updating profile",

            error: error.message

        });

    }

};


// Change Password

const changePassword = async (req, res) => {

    try {

        const {
            currentPassword,
            newPassword
        } = req.body;


        if (!currentPassword || !newPassword) {

            return res.status(400).json({

                message:
                    "Current password and new password are required"

            });

        }


        if (newPassword.length < 6) {

            return res.status(400).json({

                message:
                    "New password must be at least 6 characters"

            });

        }


        const user = await User.findById(
            req.user.id
        );


        if (!user) {

            return res.status(404).json({

                message:
                    "User not found"

            });

        }


        const isPasswordCorrect =
            await bcrypt.compare(
                currentPassword,
                user.password
            );


        if (!isPasswordCorrect) {

            return res.status(400).json({

                message:
                    "Current password is incorrect"

            });

        }


        const hashedPassword =
            await bcrypt.hash(
```

```
                newPassword,
                10
            );


        user.password =
            hashedPassword;


        await user.save();


        res.status(200).json({

            message:
                "Password changed successfully"

        });


    } catch (error) {

        console.log(
            "Change Password Error:",
            error.message
        );

        res.status(500).json({

            message:
                "Error changing password",

            error:
                error.message

        });

    }

};



module.exports = {

    getMyProfile,

    updateMyProfile,

    changePassword

};
```

Backend/middleware/authMiddleware.js

```
const jwt = require("jsonwebtoken");

const authMiddleware = (req, res, next) => {
    try {
        const authHeader = req.headers.authorization;

        if (!authHeader || !authHeader.startsWith("Bearer ")) {
            return res.status(401).json({
                message: "Access denied. No token provided."
            });
        }

        const token = authHeader.split(" ")[1];

        const decoded = jwt.verify(
            token,
            process.env.JWT_SECRET
        );

        req.user = decoded;

        next();

    } catch (error) {
        return res.status(401).json({
            message: "Invalid or expired token"
        });
    }
};

module.exports = authMiddleware;
```

Backend/middleware/roleMiddleware.js

```
const roleMiddleware = (...allowedRoles) => {
    return (req, res, next) => {

        if (!req.user) {
            return res.status(401).json({
                message: "Unauthorized"
            });
        }

        if (!allowedRoles.includes(req.user.role)) {
            return res.status(403).json({
                message: "Access denied. You do not have permission."
            });
        }

        next();
    };
};

module.exports = roleMiddleware;
```

Backend/middleware/uploadMiddleware.js

```
const multer = require("multer");
const path = require("path");
const fs = require("fs");

const uploadFolder = path.join(
    __dirname,
    "../uploads"
);

if (!fs.existsSync(uploadFolder)) {
    fs.mkdirSync(uploadFolder, {
        recursive: true
    });
}

const storage = multer.diskStorage({

    destination: function (
        req,
        file,
        cb
    ) {
        cb(
            null,
            uploadFolder
        );
    },

    filename: function (
        req,
        file,
        cb
    ) {

        const extension =
            path.extname(
                file.originalname
            ).toLowerCase();

        const fileName =
            Date.now() +
            "-" +
            Math.round(
                Math.random() * 1000000000
            ) +
            extension;

        cb(
            null,
            fileName
        );
    }
});

const fileFilter = function (
    req,
    file,
    cb
) {

    const allowedExtensions = [
        ".pdf",
        ".doc",
        ".docx"
    ];

    const extension =
        path.extname(
            file.originalname
        ).toLowerCase();

    if (
        allowedExtensions.includes(
            extension
        )
    ) {
        cb(
            null,
            true
        );
    } else {
        cb(
            new Error(
                "Only PDF, DOC and DOCX files are allowed"
            ),
            false
        );
    }
};

const upload = multer({

    storage: storage,

    fileFilter: fileFilter,

    limits: {
        fileSize:
            5 * 1024 * 1024
    }
});
```

```
module.exports = upload;
```

Backend/models/Attendance.js

```
const mongoose = require("mongoose");

const attendanceSchema = new mongoose.Schema(
    {
        employee: {
            type: mongoose.Schema.Types.ObjectId,
            ref: "Employee",
            required: true
        },

        date: {
            type: Date,
            required: true
        },

        checkIn: {
            type: Date
        },

        checkOut: {
            type: Date
        },

        status: {
            type: String,
            enum: ["Present", "Absent", "Leave"],
            default: "Present"
        }
    },
    {
        timestamps: true
    }
);

module.exports = mongoose.model("Attendance", attendanceSchema);
```

Backend/models/AuditLog.js

```
const mongoose = require("mongoose");

const auditLogSchema = new mongoose.Schema(
    {
        userId: {
            type: mongoose.Schema.Types.ObjectId,
            ref: "User",
            required: true
        },

        action: {
            type: String,
            required: true
        },

        module: {
            type: String,
            required: true
        },

        description: {
            type: String,
            required: true
        },

        ipAddress: {
            type: String
        }
    },
    {
        timestamps: true
    }
);

module.exports = mongoose.model("AuditLog", auditLogSchema);
```

Backend/models/Candidate.js

```
const mongoose = require("mongoose");

const candidateSchema = new mongoose.Schema(
    {
        name: {
            type: String,
            required: true,
            trim: true
        },

        email: {
            type: String,
            required: true,
            trim: true,
            lowercase: true
        },

        phone: {
            type: String,
            required: true,
            trim: true
        },

        resume: {
            type: String,
            default: ""
        },

        skills: {
            type: String,
            required: true,
            trim: true
        },

        experience: {
            type: String,
            required: true,
            trim: true
        },

        education: {
            type: String,
            required: true,
            trim: true
        },

        appliedJob: {
            type: mongoose.Schema.Types.ObjectId,
            ref: "Job",
            required: true
        },

        status: {
            type: String,
            enum: [
                "Applied",
                "Shortlisted",
                "Interview",
                "Selected",
                "Rejected"
            ],
            default: "Applied"
        }
    },
    {
        timestamps: true
    }
);

module.exports =
    mongoose.model("Candidate", candidateSchema);
```

Backend/models/Department.js

```
const mongoose = require("mongoose");

const departmentSchema = new mongoose.Schema(
    {
        name: {
            type: String,
            required: true,
            unique: true,
            trim: true
        },

        head: {
            type: mongoose.Schema.Types.ObjectId,
            ref: "Employee",
            default: null
        },

        description: {
            type: String,
            trim: true,
            default: ""
        },

        status: {
            type: String,
            enum: ["Active", "Inactive"],
            default: "Active"
        }
    },
    {
        timestamps: true
    }
);

module.exports =
    mongoose.model(
        "Department",
        departmentSchema
    );
```

Backend/models/Employee.js

```
const mongoose = require("mongoose");

const employeeSchema = new mongoose.Schema(
    {
        name: {
            type: String,
            required: true,
            trim: true
        },

        email: {
            type: String,
            required: true,
            unique: true,
            trim: true,
            lowercase: true
        },

        phone: {
            type: String,
            required: true,
            trim: true
        },

        department: {
            type: String,
            required: true,
            trim: true
        },

        designation: {
            type: String,
            required: true,
            trim: true
        },

        salary: {
            type: Number,
            required: true
        },

        joiningDate: {
            type: Date,
            required: true
        },

        address: {
            type: String,
            required: true,
            trim: true
        }
    },
    {
        timestamps: true
    }
);

module.exports = mongoose.model("Employee", employeeSchema);
```

Backend/models/Interview.js

```
const mongoose = require("mongoose");

const interviewSchema = new mongoose.Schema(
    {
        candidateId: {
            type: mongoose.Schema.Types.ObjectId,
            ref: "Candidate",
            required: true
        },

        jobId: {
            type: mongoose.Schema.Types.ObjectId,
            ref: "Job",
            required: true
        },

        interviewer: {
            type: String,
            required: true
        },

        interviewDate: {
            type: Date,
            required: true
        },

        mode: {
            type: String,
            enum: ["Online", "Offline"],
            required: true
        },

        meetingLink: {
            type: String
        },

        location: {
            type: String
        },

        status: {
            type: String,
            enum: [
                "Scheduled",
                "Completed",
                "Cancelled"
            ],
            default: "Scheduled"
        },

        feedback: {
            type: String
        },

        result: {
            type: String,
            enum: [
                "Pending",
                "Selected",
                "Rejected"
            ],
            default: "Pending"
        }
    },
    {
        timestamps: true
    }
);

module.exports = mongoose.model("Interview", interviewSchema);
```

Backend/models/Job.js

```
const mongoose = require("mongoose");

const jobSchema = new mongoose.Schema(
    {
        title: {
            type: String,
            required: true,
            trim: true
        },

        department: {
            type: String,
            required: true,
            trim: true
        },

        location: {
            type: String,
            required: true,
            trim: true
        },

        jobType: {
            type: String,
            enum: [
                "Full Time",
                "Part Time",
                "Internship",
                "Contract"
            ],
            default: "Full Time"
        },

        salary: {
            type: Number,
            required: true
        },

        description: {
            type: String,
            required: true,
            trim: true
        },

        requirements: {
            type: String,
            required: true,
            trim: true
        },

        status: {
            type: String,
            enum: [
                "Open",
                "Closed"
            ],
            default: "Open"
        },

        postedBy: {
            type: mongoose.Schema.Types.ObjectId,
            ref: "User",
            required: false
        }
    },
    {
        timestamps: true
    }
);

module.exports =
    mongoose.model("Job", jobSchema);
```

Backend/models/Leave.js

```
const mongoose = require("mongoose");

const leaveSchema = new mongoose.Schema(
    {
        employeeId: {
            type: mongoose.Schema.Types.ObjectId,
            ref: "Employee",
            required: true
        },

        leaveType: {
            type: String,
            enum: ["Casual", "Sick", "Earned", "Unpaid"],
            required: true
        },

        startDate: {
            type: Date,
            required: true
        },

        endDate: {
            type: Date,
            required: true
        },

        reason: {
            type: String,
            required: true
        },

        status: {
            type: String,
            enum: ["Pending", "Approved", "Rejected"],
            default: "Pending"
        }
    },
    {
        timestamps: true
    }
);

module.exports = mongoose.model("Leave", leaveSchema);
```

Backend/models/Notification.js

```
const mongoose = require("mongoose");

const notificationSchema = new mongoose.Schema(
    {
        userId: {
            type: mongoose.Schema.Types.ObjectId,
            ref: "User",
            required: true
        },

        title: {
            type: String,
            required: true,
            trim: true
        },

        message: {
            type: String,
            required: true,
            trim: true
        },

        type: {
            type: String,
            enum: [
                "Recruitment",
                "Interview",
                "Leave",
                "Payroll",
                "Employee",
                "Department",
                "Attendance",
                "Performance",
                "System"
            ],
            default: "System"
        },

        priority: {
            type: String,
            enum: [
                "Low",
                "Normal",
                "High",
                "Urgent"
            ],
            default: "Normal"
        },

        isRead: {
            type: Boolean,
            default: false
        },

        actionUrl: {
            type: String,
            default: ""
        },

        relatedId: {
            type: mongoose.Schema.Types.ObjectId,
            default: null
        },

        relatedModel: {
            type: String,
            default: ""
        },

        readAt: {
            type: Date,
            default: null
        }
    },
    {
        timestamps: true
    }
);

notificationSchema.index({
    userId: 1,
    isRead: 1,
    createdAt: -1
});

notificationSchema.index({
    type: 1,
    createdAt: -1
});

module.exports = mongoose.model(
    "Notification",
    notificationSchema
);
```

Backend/models/Payroll.js

```
const mongoose = require("mongoose");

const payrollSchema = new mongoose.Schema(
    {
        employeeId: {
            type: mongoose.Schema.Types.ObjectId,
            ref: "Employee",
            required: true
        },

        month: {
            type: String,
            required: true
        },

        basicSalary: {
            type: Number,
            required: true
        },

        allowances: {
            type: Number,
            default: 0
        },

        deductions: {
            type: Number,
            default: 0
        },

        netSalary: {
            type: Number,
            required: true
        },

        paymentStatus: {
            type: String,
            enum: ["Pending", "Paid"],
            default: "Pending"
        },

        paymentDate: {
            type: Date
        }
    },
    {
        timestamps: true
    }
);

module.exports = mongoose.model("Payroll", payrollSchema);
```

Backend/models/Performance.js

```
const mongoose = require("mongoose");

const performanceSchema = new mongoose.Schema(
    {
        employeeId: {
            type: mongoose.Schema.Types.ObjectId,
            ref: "Employee",
            required: true
        },

        reviewPeriod: {
            type: String,
            required: true
        },

        goals: {
            type: String,
            required: true
        },

        achievements: {
            type: String,
            required: true
        },

        rating: {
            type: Number,
            min: 1,
            max: 5,
            required: true
        },

        feedback: {
            type: String,
            required: true
        },

        status: {
            type: String,
            enum: ["Pending", "Reviewed"],
            default: "Pending"
        }
    },
    {
        timestamps: true
    }
);

module.exports = mongoose.model("Performance", performanceSchema);
```

Backend/models/User.js

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
    {
        name: {
            type: String,
            required: true,
            trim: true
        },

        email: {
            type: String,
            required: true,
            unique: true,
            lowercase: true,
            trim: true
        },

        password: {
            type: String,
            required: true
        },

        role: {
            type: String,
            enum: ["admin", "hr", "employee", "candidate"],
            default: "employee"
        },

        resetOTP: {
            type: String,
            default: null
        },

        resetOTPExpires: {
            type: Date,
            default: null
        }
    },
    {
        timestamps: true
    }
);

module.exports = mongoose.model("User", userSchema);
```

Backend/package-lock.json


```
{
  "name": "backend",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "backend",
      "version": "1.0.0",
      "license": "ISC",
      "dependencies": {
        "bcryptjs": "^3.0.3",
        "cors": "^2.8.6",
        "dotenv": "^18.0.4",
        "express": "^5.2.1",
        "jsonwebtoken": "^9.0.3",
        "mongoose": "^9.10.2",
        "multer": "^2.4.0",
        "nodemailer": "^10.0.13"
      },
      "devDependencies": {
        "nodemon": "^3.1.14"
      }
    },
    "node_modules/@mongodb-js/saslprep": {
      "version": "1.5.4",
      "resolved": "https://registry.npmjs.org/@mongodb-js/saslprep/-/saslprep-1.5.4.tgz",
      "integrity": "sha512-05UC0jQsjKAOuXQ0H9Ud9vUTJpZIg+n/FinpR30tI5I8pY2inTfPOZ5OF/cg3Ce/N9MoD1xhRCeOsJtuTbFYlw==",
      "license": "MIT",
      "dependencies": {
        "sparse-bitfield": "^3.0.3"
      }
    },
    "node_modules/@standard-schema/spec": {
      "version": "1.1.0",
      "resolved": "https://registry.npmjs.org/@standard-schema/spec/-/spec-1.1.0.tgz",
      "integrity": "sha512-l2aFy5jALhniG5HgqrD6jXLi/rUWrKvqN/qJx6yoJsgKhblVd+iqqU4RCXavm/jPityDo5TCvKMnpjKnOriy0w==",
      "license": "MIT"
    },
    "node_modules/@types/webidl-conversions": {
      "version": "7.0.3",
      "resolved": "https://registry.npmjs.org/@types/webidl-conversions/-/webidl-conversions-7.0.3.tgz",
      "integrity": "sha512-CiJJvcRtIgzadHCYXw7dqEnMNRjhGZlYK05Mj9OyktqV8uVT8fD2BFOB7S1uwBE3Kj2Z+4UyPmFw/Ixgw/LAlA==",
      "license": "MIT"
    },
    "node_modules/@types/whatwg-url": {
      "version": "13.0.0",
      "resolved": "https://registry.npmjs.org/@types/whatwg-url/-/whatwg-url-13.0.0.tgz",
      "integrity": "sha512-N8WXpbE6Wgri7KUSvrmQcqrMllKZ9uxkYWMt+mCSGwNc0Hsw9VQTW7ApqI4XNrx6/SaM2QQJCzMPDEXE058s+Q==",
      "license": "MIT",
      "dependencies": {
        "@types/webidl-conversions": "*"
      }
    },
    "node_modules/accepts": {
      "version": "2.0.0",
      "resolved": "https://registry.npmjs.org/accepts/-/accepts-2.0.0.tgz",
      "integrity": "sha512-5cvg6CtKwfgdmVqY1WIiXKc3Q1bkRqGLi+2W/6ao+6Y7gu/RCwRuAhGEzh5B4KlszSuTLgZYuqFqo5bImjNKng==",
      "license": "MIT",
      "dependencies": {
        "mime-types": "^3.0.0",
        "negotiator": "^1.0.0"
      },
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/anymatch": {
      "version": "3.1.3",
      "resolved": "https://registry.npmjs.org/anymatch/-/anymatch-3.1.3.tgz",
      "integrity": "sha512-KMReFUr0B4t+D+OBkjR3KYqvocp2XaSzO55UcB6mgQMd3KbcE+mWTyvVV7D/zsdEbNnV6acZUutkiHQXvTr1Rw==",
      "dev": true,
      "license": "ISC",
      "dependencies": {
        "normalize-path": "^3.0.0",
        "picomatch": "^2.0.4"
      },
      "engines": {
        "node": ">= 8"
      }
    },
    "node_modules/append-field": {
      "version": "1.0.0",
      "resolved": "https://registry.npmjs.org/append-field/-/append-field-1.0.0.tgz",
      "integrity": "sha512-klpgFSWLW1ZEs8svjfb7g4qWY0YS5imI82dTg+QahUvJ8YqAY0P10Uk8tTyh9ZGuYEZEMaeJYCF5BFuX552hsw==",
      "license": "MIT"
    },
    "node_modules/balanced-match": {
      "version": "4.0.4",
      "resolved": "https://registry.npmjs.org/balanced-match/-/balanced-match-4.0.4.tgz",
      "integrity": "sha512-BLrgEcRTwX2o6gGxGOCNyMvGSp35YofuYzw9h1IMTRmKqttAZZVU67bdb9Pr2vUHA8+j3i2tJfjO6C6+4myGTA==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": "18 || 20 || >=22"
      }
    },
    "node_modules/bcryptjs": {
      "version": "3.0.3",
      "resolved": "https://registry.npmjs.org/bcryptjs/-/bcryptjs-3.0.3.tgz",
      "integrity": "sha512-GlF5wPWnSa/X5LKM1o0wz0suXIINz1iHRLvTS+sLyi7XPbe5ycmYI3DlZqVGZZtDgl4DmasFg7gOB3JYbphV5g==",
      "license": "BSD-3-Clause",
```

```
  "bin": {
    "bcrypt": "bin/bcrypt"
  }
},
"node_modules/binary-extensions": {
  "version": "2.3.0",
  "resolved": "https://registry.npmjs.org/binary-extensions/-/binary-extensions-2.3.0.tgz",
  "integrity": "sha512-Ceh+7ox5qe7LJuLHoY0feh3pHuUDHAcRUeyL2VYghZwfpkNIy/+8Ocg0a3UuSoYzavmylwuLWQOf3hl0jjMMIw==",
  "dev": true,
  "license": "MIT",
  "engines": {
    "node": ">=8"
  },
  "funding": {
    "url": "https://github.com/sponsors/sindresorhus"
  }
},
"node_modules/body-parser": {
  "version": "2.3.0",
  "resolved": "https://registry.npmjs.org/body-parser/-/body-parser-2.3.0.tgz",
  "integrity": "sha512-2cGmJupaNgg+QUwVLAucDuWuoMZ6EX9iHDRswZ5lsNYEmwPaRknMPCLZz07yTzVq/83p4o/wzbDZbBrTvGGTIw==",
  "license": "MIT",
  "dependencies": {
    "bytes": "^3.1.2",
    "content-type": "^2.0.0",
    "debug": "^4.4.3",
    "http-errors": "^2.0.1",
    "iconv-lite": "^0.7.2",
    "on-finished": "^2.4.1",
    "qs": "^6.15.2",
    "raw-body": "^3.0.2",
    "type-is": "^2.1.0"
  },
  "engines": {
    "node": ">=18"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/express"
  }
},
"node_modules/body-parser/node_modules/content-type": {
  "version": "2.1.0",
  "resolved": "https://registry.npmjs.org/content-type/-/content-type-2.1.0.tgz",
  "integrity": "sha512-mj7UPXE0jaqaOsukNZRUEfEi2AcL7C/vwmwcHV0O97eO1E1pxBZuyjlZrx5seTaNBg1U6+o35wpa35Qfcc+7ag==",
  "license": "MIT",
  "engines": {
    "node": ">=18"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/express"
  }
},
"node_modules/brace-expansion": {
  "version": "5.0.12",
  "resolved": "https://registry.npmjs.org/brace-expansion/-/brace-expansion-5.0.12.tgz",
  "integrity": "sha512-YovQ3rzhaLMIrDjNDMkNS01tea93qhEhG5xy8f6+R0l+dw3Ki+5sCoIoI942iuLZTHWogWktgwVDhU09iNEimQ==",
  "dev": true,
  "license": "MIT",
  "dependencies": {
    "balanced-match": "^4.0.2"
  },
  "engines": {
    "node": "20 || >=22"
  }
},
"node_modules/braces": {
  "version": "3.0.3",
  "resolved": "https://registry.npmjs.org/braces/-/braces-3.0.3.tgz",
  "integrity": "sha512-yQbXgO/OSZVD2IsiLlro+7Hf6Q18EJrKSEsdoMzKePKXct3gvD8oLcOQdIzGupr5Fj+EDe8gO/lxc1BzfMpxvA==",
  "dev": true,
  "license": "MIT",
  "dependencies": {
    "fill-range": "^7.1.1"
  },
  "engines": {
    "node": ">=8"
  }
},
"node_modules/bson": {
  "version": "7.3.3",
  "resolved": "https://registry.npmjs.org/bson/-/bson-7.3.3.tgz",
  "integrity": "sha512-oz3LwE3qGEhTCSyhzQRLY3Bj1NLOsjFMf6tcmY5J8M+3R7EVcmoBfSHWvjUK3yXCLDUsGpIuB3aQL4nDZ1EV/g==",
  "license": "Apache-2.0",
  "engines": {
    "node": ">=20.19.0"
  }
},
"node_modules/buffer-equal-constant-time": {
  "version": "1.0.1",
  "resolved": "https://registry.npmjs.org/buffer-equal-constant-time/-/buffer-equal-constant-time-1.0.1.tgz",
  "integrity": "sha512-zRpUiDwd/xk6ADqPMATG8vc9VPrkck7T07OIx0gnjmJAnHnTVXNQG3vfvWNuiZIkwu9KrKdA1iJKfsfTVxE6NA==",
  "license": "BSD-3-Clause"
},
"node_modules/busboy": {
  "version": "1.6.0",
  "resolved": "https://registry.npmjs.org/busboy/-/busboy-1.6.0.tgz",
  "integrity": "sha512-8SFQbg/0hQ9xy3UNTB0YEnsNBbWfhf7RtnzpL7TkBiTBRfrQ9Fxcnz7VJsleJpyp6rVLvXiuORqjlHi5q+PYuA==",
  "dependencies": {
    "streamsearch": "^1.1.0"
  },
  "engines": {
    "node": ">=10.16.0"
  }
```

```
},
"node_modules/bytes": {
  "version": "3.1.2",
  "resolved": "https://registry.npmjs.org/bytes/-/bytes-3.1.2.tgz",
  "integrity": "sha512-/Nf7TyzTx6S3yRJObOAV7956r8cr2+Oj8AC5dt8wSP3BQAoeX58NoHyCU8P8zGkNXStjTSi6fzO6F0pBdcYbEg==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.8"
  }
},
"node_modules/call-bind-apply-helpers": {
  "version": "1.0.2",
  "resolved": "https://registry.npmjs.org/call-bind-apply-helpers/-/call-bind-apply-helpers-1.0.2.tgz",
  "integrity": "sha512-Sp1ablJ0ivDkSzjcaJdxEunN5/XvksFJ2sMBFfq6x0ryhQV/2b/KwFe21cMpmHtPOSij8K99/wSfoEuTObmuMQ==",
  "license": "MIT",
  "dependencies": {
    "es-errors": "^1.3.0",
    "function-bind": "^1.1.2"
  },
  "engines": {
    "node": ">= 0.4"
  }
},
"node_modules/call-bound": {
  "version": "1.0.4",
  "resolved": "https://registry.npmjs.org/call-bound/-/call-bound-1.0.4.tgz",
  "integrity": "sha512-+ys997U96po4Kx/ABpBCqhA9EuxJaQWDQg7295H4hBphv3IZg0boBKuwYpt4YXp6MZ5AmZQnU/tyMTlRpaSejg==",
  "license": "MIT",
  "dependencies": {
    "call-bind-apply-helpers": "^1.0.2",
    "get-intrinsic": "^1.3.0"
  },
  "engines": {
    "node": ">= 0.4"
  },
  "funding": {
    "url": "https://github.com/sponsors/ljharb"
  }
},
"node_modules/chokidar": {
  "version": "3.6.0",
  "resolved": "https://registry.npmjs.org/chokidar/-/chokidar-3.6.0.tgz",
  "integrity": "sha512-7VT13fmjotKpGipCW9JEQAusEPE+Ei8nl6/g4FBAmIm0GOOLMua9NDDo/DWp0ZAxCr3cPq5ZpBqmPAQgDda2Pw==",
  "dev": true,
  "license": "MIT",
  "dependencies": {
    "anymatch": "~3.1.2",
    "braces": "~3.0.2",
    "glob-parent": "~5.1.2",
    "is-binary-path": "~2.1.0",
    "is-glob": "~4.0.1",
    "normalize-path": "~3.0.0",
    "readdirp": "~3.6.0"
  },
  "engines": {
    "node": ">= 8.10.0"
  },
  "funding": {
    "url": "https://paulmillr.com/funding/"
  },
  "optionalDependencies": {
    "fsevents": "~2.3.2"
  }
},
"node_modules/content-disposition": {
  "version": "1.1.0",
  "resolved": "https://registry.npmjs.org/content-disposition/-/content-disposition-1.1.0.tgz",
  "integrity": "sha512-5jRCH9Z/+DRP7rkvY83B+yGIGX96OYdJmzngqnw2SBSxqCFPd0w2km3s5iawpGX8krnwSGmF0FW5Nhr0Hfai3g==",
  "license": "MIT",
  "engines": {
    "node": ">=18"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/express"
  }
},
"node_modules/content-type": {
  "version": "1.0.5",
  "resolved": "https://registry.npmjs.org/content-type/-/content-type-1.0.5.tgz",
  "integrity": "sha512-nTjqfcBFEipKdXCv4YDQWCfmcLZKm81ldF0pAopTvyrFGVbcR6P/VAAd5G7N+0tTr8QqiU0tFadD6FK4NtJwOA==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.6"
  }
},
"node_modules/cookie": {
  "version": "0.7.2",
  "resolved": "https://registry.npmjs.org/cookie/-/cookie-0.7.2.tgz",
  "integrity": "sha512-yki5XnKuf750l50uGTllt6kKILY4nQ1eNIQatoXEByZ5dWgnKqbnqmTrBE5B4N7lrMJKQ2ytWMiTO2o0v6Ew/w==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.6"
  }
},
"node_modules/cookie-signature": {
  "version": "1.2.2",
  "resolved": "https://registry.npmjs.org/cookie-signature/-/cookie-signature-1.2.2.tgz",
  "integrity": "sha512-D76uU73ulSXrD1UXF4KE2TMxVVwhsnCgfAyTg9k8P6KGZjlXKrOLe4dJQKI3Bxi5wjesZoFXJWElNWBjPZMbhg==",
  "license": "MIT",
  "engines": {
    "node": ">=6.6.0"
  }
},
"node_modules/cors": {
```

```
    "version": "2.8.6",
    "resolved": "https://registry.npmjs.org/cors/-/cors-2.8.6.tgz",
    "integrity": "sha512-tJtZBBHA6vjIAaF6EnIaq6laBBP9aq/Y3ouVJjEfoHbRBcHBAHYcMh/w8LDrk2PvIMMq8gmopa5D4V8RmbrxGw==",
    "license": "MIT",
    "dependencies": {
      "object-assign": "^4",
      "vary": "^1"
    },
    "engines": {
      "node": ">= 0.10"
    },
    "funding": {
      "type": "opencollective",
      "url": "https://opencollective.com/express"
    }
  },
  "node_modules/debug": {
    "version": "4.4.3",
    "resolved": "https://registry.npmjs.org/debug/-/debug-4.4.3.tgz",
    "integrity": "sha512-RGwwWnwQvkVfavKVt22FGLw+xYSdzARwm0ru6DhTVA3umU5hZc28V3kO4stgYryrTlLpuvgI9GiijltAjNbcqA==",
    "license": "MIT",
    "dependencies": {
      "ms": "^2.1.3"
    },
    "engines": {
      "node": ">=6.0"
    },
    "peerDependenciesMeta": {
      "supports-color": {
        "optional": true
      }
    }
  },
  "node_modules/depd": {
    "version": "2.0.0",
    "resolved": "https://registry.npmjs.org/depd/-/depd-2.0.0.tgz",
    "integrity": "sha512-g7nH6P6dyDioJogAAGprGpCtVImJhpPk/roCzdb3fIh61/s/nPsfR6onyMwkCAR/OlC3yBC0lESvUoQEAssIrw==",
    "license": "MIT",
    "engines": {
      "node": ">= 0.8"
    }
  },
  "node_modules/dotenv": {
    "version": "18.0.4",
    "resolved": "https://registry.npmjs.org/dotenv/-/dotenv-18.0.4.tgz",
    "integrity": "sha512-jGt9uI5BIxpVO15kvy+iEFqNAwxYClKWnkptLjSNPnFLC003Imd0ofr3aEVsbpHdSBdSsbcApNIlz4nbyGMwdA==",
    "license": "BSD-2-Clause",
    "bin": {
      "dotenv": "dist/index.cjs"
    },
    "engines": {
      "node": ">=12"
    },
    "funding": {
      "url": "https://dotenvx.com"
    }
  },
  "node_modules/dunder-proto": {
    "version": "1.0.1",
    "resolved": "https://registry.npmjs.org/dunder-proto/-/dunder-proto-1.0.1.tgz",
    "integrity": "sha512-KIN/nDJBQRcXw0MLVhZE9iQHmG68qAVIBg9CqmUYjmQIhgij9U5MFvrqkUL5FbtyyzZuOeOt0zdeRe4UY7ct+A==",
    "license": "MIT",
    "dependencies": {
      "call-bind-apply-helpers": "^1.0.1",
      "es-errors": "^1.3.0",
      "gopd": "^1.2.0"
    },
    "engines": {
      "node": ">= 0.4"
    }
  },
  "node_modules/ecdsa-sig-formatter": {
    "version": "1.0.11",
    "resolved": "https://registry.npmjs.org/ecdsa-sig-formatter/-/ecdsa-sig-formatter-1.0.11.tgz",
    "integrity": "sha512-nagl3RYrbNv6kQkeJIpt6NJZy8twLB/2vtz6yN9Z4vRKHN4/QZJIEbqohALSgwKdnksuY3k5Addp5lg8sVoVcQ==",
    "license": "Apache-2.0",
    "dependencies": {
      "safe-buffer": "^5.0.1"
    }
  },
  "node_modules/ee-first": {
    "version": "1.1.1",
    "resolved": "https://registry.npmjs.org/ee-first/-/ee-first-1.1.1.tgz",
    "integrity": "sha512-WMwm9LhRUo+WUaRN+vRuETqG89IgZphVSNkdFgeb6sS/E4OrDIN7t48CAewSHXc6C8lefD8KKfr5vY61brQlow==",
    "license": "MIT"
  },
  "node_modules/encodeurl": {
    "version": "2.0.0",
    "resolved": "https://registry.npmjs.org/encodeurl/-/encodeurl-2.0.0.tgz",
    "integrity": "sha512-Q0n9HRi4m6JuGIV1eFlmvJB7ZEVxu93IrMyiMsGC0lrMJMWzRgx6WGquyfQgZVb31vhGgXnfmPNNXmxnOkRBrg==",
    "license": "MIT",
    "engines": {
      "node": ">= 0.8"
    }
  },
  "node_modules/es-define-property": {
    "version": "1.0.1",
    "resolved": "https://registry.npmjs.org/es-define-property/-/es-define-property-1.0.1.tgz",
    "integrity": "sha512-e3nRfgfUZ4rNGL232gUgX06QNyyez04KdjFrF+LTRoOXmrOgFKDg4BCdsjW8EnT69eqdYGmRpJwiPVYNrCaW3g==",
    "license": "MIT",
    "engines": {
      "node": ">= 0.4"
    }
  },
  "node_modules/es-errors": {
```

```
    "version": "1.3.0",
    "resolved": "https://registry.npmjs.org/es-errors/-/es-errors-1.3.0.tgz",
    "integrity": "sha512-Zf5H2Kxt2xjTvbJvP2ZWLEICxA6j+hAmMzIlypy4xcBg1vKVnx89Wy0GbS+kf5cwCVFFzdCFh2XSCFNULS6csw==",
    "license": "MIT",
    "engines": {
      "node": ">= 0.4"
    }
  },
  "node_modules/es-object-atoms": {
    "version": "1.1.2",
    "resolved": "https://registry.npmjs.org/es-object-atoms/-/es-object-atoms-1.1.2.tgz",
    "integrity": "sha512-HWcBoN6NileqtSydK2FqHbS/LoDd2pqrnQHLyJzBj4kOp/ky2MWMN694xOfkK8/SnUsW2DH7EfyVlydKCsm1Zw==",
    "license": "MIT",
    "dependencies": {
      "es-errors": "^1.3.0"
    },
    "engines": {
      "node": ">= 0.4"
    }
  },
  "node_modules/escape-html": {
    "version": "1.0.3",
    "resolved": "https://registry.npmjs.org/escape-html/-/escape-html-1.0.3.tgz",
    "integrity": "sha512-NiSupZ4OeuGwr68lGIeym/ksIZMJodUGOSCZ/FSnTxcrekbvqrgdUxlJOMpijaKZVjAJrWrGs/6Jy8OMuyj9ow==",
    "license": "MIT"
  },
  "node_modules/etag": {
    "version": "1.8.1",
    "resolved": "https://registry.npmjs.org/etag/-/etag-1.8.1.tgz",
    "integrity": "sha512-aIL5Fx7mawVa300al2BnEE4iNvo1qETxLrPI/o05L7z6go7fCw1J6EQmbK4FmJ2AS7kgVF/KEZWufBfdClMcPg==",
    "license": "MIT",
    "engines": {
      "node": ">= 0.6"
    }
  },
  "node_modules/express": {
    "version": "5.2.1",
    "resolved": "https://registry.npmjs.org/express/-/express-5.2.1.tgz",
    "integrity": "sha512-hIS4idWWai69NezIdRt2xFVofaF4j+6INOpJlVOLDO8zXGpUVEVzIYk12UUi2JzjEzWL3IOAxcTubgz9Po0yXw==",
    "license": "MIT",
    "dependencies": {
      "accepts": "^2.0.0",
      "body-parser": "^2.2.1",
      "content-disposition": "^1.0.0",
      "content-type": "^1.0.5",
      "cookie": "^0.7.1",
      "cookie-signature": "^1.2.1",
      "debug": "^4.4.0",
      "depd": "^2.0.0",
      "encodeurl": "^2.0.0",
      "escape-html": "^1.0.3",
      "etag": "^1.8.1",
      "finalhandler": "^2.1.0",
      "fresh": "^2.0.0",
      "http-errors": "^2.0.0",
      "merge-descriptors": "^2.0.0",
      "mime-types": "^3.0.0",
      "on-finished": "^2.4.1",
      "once": "^1.4.0",
      "parseurl": "^1.3.3",
      "proxy-addr": "^2.0.7",
      "qs": "^6.14.0",
      "range-parser": "^1.2.1",
      "router": "^2.2.0",
      "send": "^1.1.0",
      "serve-static": "^2.2.0",
      "statuses": "^2.0.1",
      "type-is": "^2.0.1",
      "vary": "^1.1.2"
    },
    "engines": {
      "node": ">= 18"
    },
    "funding": {
      "type": "opencollective",
      "url": "https://opencollective.com/express"
    }
  },
  "node_modules/fill-range": {
    "version": "7.1.1",
    "resolved": "https://registry.npmjs.org/fill-range/-/fill-range-7.1.1.tgz",
    "integrity": "sha512-YsGpe3WHLK8ZYi4tWDg2Jy3ebRz2rXowDxnld4bkQB00cc/1Zw9AWnC0i9ztDJitivtQvaI9KaLyKrc+hBW0yg==",
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "to-regex-range": "^5.0.1"
    },
    "engines": {
      "node": ">=8"
    }
  },
  "node_modules/finalhandler": {
    "version": "2.1.1",
    "resolved": "https://registry.npmjs.org/finalhandler/-/finalhandler-2.1.1.tgz",
    "integrity": "sha512-S8KoZgRZN+a5rNwqTxlZZePjT/4cnm0ROV70LedRHZ0p8u9fRID0hJUZQpkKLzro8LfmC8sx23bY6tVNxv8pQA==",
    "license": "MIT",
    "dependencies": {
      "debug": "^4.4.0",
      "encodeurl": "^2.0.0",
      "escape-html": "^1.0.3",
      "on-finished": "^2.4.1",
      "parseurl": "^1.3.3",
      "statuses": "^2.0.1"
    },
    "engines": {
```

```
    "node": ">= 18.0.0"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/express"
  }
},
"node_modules/forwarded": {
  "version": "0.2.0",
  "resolved": "https://registry.npmjs.org/forwarded/-/forwarded-0.2.0.tgz",
  "integrity": "sha512-buRG0fpBtRHSTCOASe6hD258tEubFoRLb4ZNA6NxMVHNw2gOcwHo9wyablzMzOA5z9xA9L1KNjk/Nt6MT9aYow==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.6"
  }
},
"node_modules/fresh": {
  "version": "2.0.0",
  "resolved": "https://registry.npmjs.org/fresh/-/fresh-2.0.0.tgz",
  "integrity": "sha512-Rx/WycZ60HOaqLKAi6cHRKKI7zxWbJ31MhntmtwMoaTeF7XFH9hhBp8vITaMidfljRQ6eYWCKkaTK+ykVJHP2A==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.8"
  }
},
"node_modules/fsevents": {
  "version": "2.3.3",
  "resolved": "https://registry.npmjs.org/fsevents/-/fsevents-2.3.3.tgz",
  "integrity": "sha512-5xoDfX+fL7faATnagmWPpbFtwh/R77WmMMqqHGS65C3vvB0YHrgF+B1YmZ3441tMj5n63k0212XNoJwzlhffQw==",
  "dev": true,
  "hasInstallScript": true,
  "license": "MIT",
  "optional": true,
  "os": [
    "darwin"
  ],
  "engines": {
    "node": "^8.16.0 || ^10.6.0 || >=11.0.0"
  }
},
"node_modules/function-bind": {
  "version": "1.1.2",
  "resolved": "https://registry.npmjs.org/function-bind/-/function-bind-1.1.2.tgz",
  "integrity": "sha512-7XHNxH7qX9xG5mIwxkhumTox/MIRNcOgDrxWsMt2pAr23WHp6MrRlN7FBSFpCpr+oVO0F744iUgR82nJMfG2SA==",
  "license": "MIT",
  "funding": {
    "url": "https://github.com/sponsors/ljharb"
  }
},
"node_modules/get-intrinsic": {
  "version": "1.3.0",
  "resolved": "https://registry.npmjs.org/get-intrinsic/-/get-intrinsic-1.3.0.tgz",
  "integrity": "sha512-9fSjSaos/fRIVIp+xSJlE6lfwhES7LNtKaCBIamHsjr2na1BiABJPo0mOjjz8GJDURarmCPGqaiVg5mfjb98CQ==",
  "license": "MIT",
  "dependencies": {
    "call-bind-apply-helpers": "^1.0.2",
    "es-define-property": "^1.0.1",
    "es-errors": "^1.3.0",
    "es-object-atoms": "^1.1.1",
    "function-bind": "^1.1.2",
    "get-proto": "^1.0.1",
    "gopd": "^1.2.0",
    "has-symbols": "^1.1.0",
    "hasown": "^2.0.2",
    "math-intrinsics": "^1.1.0"
  },
  "engines": {
    "node": ">= 0.4"
  },
  "funding": {
    "url": "https://github.com/sponsors/ljharb"
  }
},
"node_modules/get-proto": {
  "version": "1.0.1",
  "resolved": "https://registry.npmjs.org/get-proto/-/get-proto-1.0.1.tgz",
  "integrity": "sha512-sTSfBjoXBp89JvIKIefqw7U2CCebsc74kiY6awiGogKtoSGbgjYE/G/+l9sF3MWFPNc9IcoOC4ODfKHfxFmp0g==",
  "license": "MIT",
  "dependencies": {
    "dunder-proto": "^1.0.1",
    "es-object-atoms": "^1.0.0"
  },
  "engines": {
    "node": ">= 0.4"
  }
},
"node_modules/glob-parent": {
  "version": "5.1.2",
  "resolved": "https://registry.npmjs.org/glob-parent/-/glob-parent-5.1.2.tgz",
  "integrity": "sha512-AOIgSQCepiJYwP3ARnGx+5VnTu2HBYdzbGP45eLw1vr3zB3vZLeyed1sC9hnbcOc9/SrMyM5RPQrkGz4aS9Zow==",
  "dev": true,
  "license": "ISC",
  "dependencies": {
    "is-glob": "^4.0.1"
  },
  "engines": {
    "node": ">= 6"
  }
},
"node_modules/gopd": {
  "version": "1.2.0",
  "resolved": "https://registry.npmjs.org/gopd/-/gopd-1.2.0.tgz",
  "integrity": "sha512-ZUKRh6/kUFoAiTAtTYPZJ3hw9wNxx+BIBOijnlG9PnrJsCcSjs1wyyD6vJpaYtgnzDrKYRSqf3OO6Rfa93xsRg==",
  "license": "MIT",
  "engines": {
```

```
    "node": ">= 0.4"
  },
  "funding": {
    "url": "https://github.com/sponsors/ljharb"
  }
},
"node_modules/has-flag": {
  "version": "3.0.0",
  "resolved": "https://registry.npmjs.org/has-flag/-/has-flag-3.0.0.tgz",
  "integrity": "sha512-sKJf1+ceQBr4SMkvQnBDNDtf4TXpVhVGateu0t918bl30FnbE2m4vNLX+VWe/dpjlb+HugGYzW7uQXH98HPEYw==",
  "dev": true,
  "license": "MIT",
  "engines": {
    "node": ">=4"
  }
},
"node_modules/has-symbols": {
  "version": "1.1.0",
  "resolved": "https://registry.npmjs.org/has-symbols/-/has-symbols-1.1.0.tgz",
  "integrity": "sha512-1cDNdwJ2Jaohmb3sg4OmKaMBwuC48sYni5HUw2DvsC8LjGTLK9h+eb1X6RyuOHe4hT0ULCW68iomhjUoKUqlPQ==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.4"
  },
  "funding": {
    "url": "https://github.com/sponsors/ljharb"
  }
},
"node_modules/hasown": {
  "version": "2.0.4",
  "resolved": "https://registry.npmjs.org/hasown/-/hasown-2.0.4.tgz",
  "integrity": "sha512-T2UbfbBEF32wiepXIsMlTW9+dDYC6wMh/t/vYA4tuOMKqWz/n3vr1NFSxQiyP+zk2mXsoMA/i/7qV6LKut1t1A==",
  "license": "MIT",
  "dependencies": {
    "function-bind": "^1.1.2"
  },
  "engines": {
    "node": ">= 0.4"
  }
},
"node_modules/http-errors": {
  "version": "2.0.1",
  "resolved": "https://registry.npmjs.org/http-errors/-/http-errors-2.0.1.tgz",
  "integrity": "sha512-4FbRdAX+bSdmo4AUFuS0WNiPz8NgFt+r8ThgNWmlrjQjt1Q7ZR9+zTlce2859x4KSXrwIsaeTqDoKQmtP8pLmQ==",
  "license": "MIT",
  "dependencies": {
    "depd": "~2.0.0",
    "inherits": "~2.0.4",
    "setprototypeof": "~1.2.0",
    "statuses": "~2.0.2",
    "toidentifier": "~1.0.1"
  },
  "engines": {
    "node": ">= 0.8"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/express"
  }
},
"node_modules/iconv-lite": {
  "version": "0.7.3",
  "resolved": "https://registry.npmjs.org/iconv-lite/-/iconv-lite-0.7.3.tgz",
  "integrity": "sha512-IKXpvIzjnC9XTAUbVBcMfGS0EPaIXtW6v+zr+RRp+hqULEpo0owZax6wyRwPOJbWbzjYspQwusTsfVr0ifh4uQ==",
  "license": "MIT",
  "dependencies": {
    "safer-buffer": ">= 2.1.2 < 3.0.0"
  },
  "engines": {
    "node": ">=0.10.0"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/express"
  }
},
"node_modules/ignore-by-default": {
  "version": "1.0.1",
  "resolved": "https://registry.npmjs.org/ignore-by-default/-/ignore-by-default-1.0.1.tgz",
  "integrity": "sha512-Ius2VYcGNk7T90CppJqcIkS5ooHUZyIQK+ClZfMfMNFEF9VSE73Fq+906u/CWu92x4gzZMWOwfFYckPObzdEbA==",
  "dev": true,
  "license": "ISC"
},
"node_modules/inherits": {
  "version": "2.0.4",
  "resolved": "https://registry.npmjs.org/inherits/-/inherits-2.0.4.tgz",
  "integrity": "sha512-k/vGaX4/Yla3WzyMCvTQOXYeIHvqOKtnqBduzTHpzpQZzAskKMhZ2K+EnBiSM9zGSoIFeMpXKxa4dYeZIQqewQ==",
  "license": "ISC"
},
"node_modules/ipaddr.js": {
  "version": "1.9.1",
  "resolved": "https://registry.npmjs.org/ipaddr.js/-/ipaddr.js-1.9.1.tgz",
  "integrity": "sha512-0KI/607xoxSToH7GjN1FfSbLoU0+btTicjsQSWQlh/hZykN8KpmMf7uYwPW3R+akZ6R/w18ZlXSHBYXiYUPO3g==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.10"
  }
},
"node_modules/is-binary-path": {
  "version": "2.1.0",
  "resolved": "https://registry.npmjs.org/is-binary-path/-/is-binary-path-2.1.0.tgz",
  "integrity": "sha512-ZMERYes6pDydyuGidse7OsHxtbI7WVeUEozgR/g7rd0xUimYNlvZRE/K2MgZTjWy725IfelLeVcEM97mmtRGXw==",
  "dev": true,
  "license": "MIT",
  "dependencies": {
```

```
    "binary-extensions": "^2.0.0"
  },
  "engines": {
    "node": ">=8"
  }
},
"node_modules/is-extglob": {
  "version": "2.1.1",
  "resolved": "https://registry.npmjs.org/is-extglob/-/is-extglob-2.1.1.tgz",
  "integrity": "sha512-SbKbANkN603Vi4jEZv49LeVJMn4yGwsbzZworEoyEiutsN3nJYdbO36zfhGJ6QEDpOZIFkDtnq5JRxmvl3jsoQ==",
  "dev": true,
  "license": "MIT",
  "engines": {
    "node": ">=0.10.0"
  }
},
"node_modules/is-glob": {
  "version": "4.0.3",
  "resolved": "https://registry.npmjs.org/is-glob/-/is-glob-4.0.3.tgz",
  "integrity": "sha512-xelSayHH36ZgE7ZWhli7pW34hNbNl8Ojv5KVmkJD4hBdD3th8Tfk9vYasLM+mXWOZhFkgZfxhLSnrwRr4elSSg==",
  "dev": true,
  "license": "MIT",
  "dependencies": {
    "is-extglob": "^2.1.1"
  },
  "engines": {
    "node": ">=0.10.0"
  }
},
"node_modules/is-number": {
  "version": "7.0.0",
  "resolved": "https://registry.npmjs.org/is-number/-/is-number-7.0.0.tgz",
  "integrity": "sha512-41Cifkg6e8TylSpdtTpeLVMqvSBEVzTttHvERD741+pnZ8ANv0004MRL43QKPDlK9cGvNp6NZWZUBlbGXYxxng==",
  "dev": true,
  "license": "MIT",
  "engines": {
    "node": ">=0.12.0"
  }
},
"node_modules/is-promise": {
  "version": "4.0.0",
  "resolved": "https://registry.npmjs.org/is-promise/-/is-promise-4.0.0.tgz",
  "integrity": "sha512-hvpoI6korhJMnej285dSg6nu1+e6uxs7zG3BYAm5byqDsgJNWwxzM6z6iZiAgQR4TJ30JmBTOwqZUw3WlyH3AQ==",
  "license": "MIT"
},
"node_modules/jsonwebtoken": {
  "version": "9.0.3",
  "resolved": "https://registry.npmjs.org/jsonwebtoken/-/jsonwebtoken-9.0.3.tgz",
  "integrity": "sha512-MT/xP0CrubFRNLNKvxJ2BYfy53Zkm++5bX9dtuPbqAeQpTVe0MQTFhao8+Cp//EmJp244xt6Drw/GVEGCUj40g==",
  "license": "MIT",
  "dependencies": {
    "jws": "^4.0.1",
    "lodash.includes": "^4.3.0",
    "lodash.isboolean": "^3.0.3",
    "lodash.isinteger": "^4.0.4",
    "lodash.isnumber": "^3.0.3",
    "lodash.isplainobject": "^4.0.6",
    "lodash.isstring": "^4.0.1",
    "lodash.once": "^4.0.0",
    "ms": "^2.1.1",
    "semver": "^7.5.4"
  },
  "engines": {
    "node": ">=12",
    "npm": ">=6"
  }
},
"node_modules/jwa": {
  "version": "2.0.1",
  "resolved": "https://registry.npmjs.org/jwa/-/jwa-2.0.1.tgz",
  "integrity": "sha512-hRF04fqJIP8Abbkq5NKGN0Bbr3JxlQ+qhZufXVr0DvujKy93ZCbXZMHDL4EOtodSbCWxOqR8MS1tXA5hwqCXDg==",
  "license": "MIT",
  "dependencies": {
    "buffer-equal-constant-time": "^1.0.1",
    "ecdsa-sig-formatter": "1.0.11",
    "safe-buffer": "^5.0.1"
  }
},
"node_modules/jws": {
  "version": "4.0.1",
  "resolved": "https://registry.npmjs.org/jws/-/jws-4.0.1.tgz",
  "integrity": "sha512-EKI/M/yqPncGUUh44xz0PxSidXFr/+r0pA70+gIYhjv+et7yxM+s29Y+VGDkovRofQem0fs7Uvf4+YmAdyRduA==",
  "license": "MIT",
  "dependencies": {
    "jwa": "^2.0.1",
    "safe-buffer": "^5.0.1"
  }
},
"node_modules/kareem": {
  "version": "3.4.0",
  "resolved": "https://registry.npmjs.org/kareem/-/kareem-3.4.0.tgz",
  "integrity": "sha512-JAKhnR4S027v/8949QXNInRskgETCg9rFUMi1VjhGbjX+XltCAAEFcXykkSocNkom1kc66EMJfjWvhj87BDQKg==",
  "license": "Apache-2.0",
  "engines": {
    "node": ">=18.0.0"
  }
},
"node_modules/lodash.includes": {
  "version": "4.3.0",
  "resolved": "https://registry.npmjs.org/lodash.includes/-/lodash.includes-4.3.0.tgz",
  "integrity": "sha512-W3Bx6mdkRTGtlJISOvVD/lbqjTlPPUDTMnlXZFnVwi9NKJ6tiAk6LVdlhZMm17VZisqhKcgzpO5Wz91PCt5b0w==",
  "license": "MIT"
},
"node_modules/lodash.isboolean": {
  "version": "3.0.3",
```

```
  "resolved": "https://registry.npmjs.org/lodash.isboolean/-/lodash.isboolean-3.0.3.tgz",
  "integrity": "sha512-Bz5mupy2SVbPHURB98VAcw+aHh4vRV5IPNhILUCsOzRmsTmSQ17jIuqopAentWoehktxGd9e/hbIXq980/1QJg==",
  "license": "MIT"
},
"node_modules/lodash.isinteger": {
  "version": "4.0.4",
  "resolved": "https://registry.npmjs.org/lodash.isinteger/-/lodash.isinteger-4.0.4.tgz",
  "integrity": "sha512-DBwtEWN2caHQ9/imiNeEA5ys1JoRtRfY3d7V9wkqtbycnAmTvRRmbHKDV4a0EYc678/dia0jrte4tjYwVBaZUA==",
  "license": "MIT"
},
"node_modules/lodash.isnumber": {
  "version": "3.0.3",
  "resolved": "https://registry.npmjs.org/lodash.isnumber/-/lodash.isnumber-3.0.3.tgz",
  "integrity": "sha512-QYqzpfwO3/CWf3XP+Z+tkQsfaLL/EnUlXWVkIk5FUPc4sBdTehEqZONuyRt2P67PXAk+NXmTBcc97zw9t1FQrw==",
  "license": "MIT"
},
"node_modules/lodash.isplainobject": {
  "version": "4.0.6",
  "resolved": "https://registry.npmjs.org/lodash.isplainobject/-/lodash.isplainobject-4.0.6.tgz",
  "integrity": "sha512-oSXzaWypCMHkPC3NvBEaPHf0KsA5mvPrOPgQWDsbg8n7orZ290M0BmC/jgRZ4vcJ6DTAhjrsSYgdsW/F+MFOBA==",
  "license": "MIT"
},
"node_modules/lodash.isstring": {
  "version": "4.0.1",
  "resolved": "https://registry.npmjs.org/lodash.isstring/-/lodash.isstring-4.0.1.tgz",
  "integrity": "sha512-0wJxfxH1wgO3GrbuP+dTTk7op+6L41QCXbGINEmD+ny/G/eCqGzxyCsh7159S+mgDDcoarnBw6PC1PS5+wUGgw==",
  "license": "MIT"
},
"node_modules/lodash.once": {
  "version": "4.1.1",
  "resolved": "https://registry.npmjs.org/lodash.once/-/lodash.once-4.1.1.tgz",
  "integrity": "sha512-Sb487aTOCr9drQVL8pIxOzVhafOjZN9UU54hiN8PU3uAiSV7lx1yYNpbNmex2PK6dSJoNTSJUUswT651yww3Mg==",
  "license": "MIT"
},
"node_modules/math-intrinsics": {
  "version": "1.1.0",
  "resolved": "https://registry.npmjs.org/math-intrinsics/-/math-intrinsics-1.1.0.tgz",
  "integrity": "sha512-/IXtbwEk5HTPyEwyKX6hGkYXxM9nbj64B+ilVJnC/R6B0pH5G4V3b0pVbL7DBj4tkhBAppbQUlf6F6Xl9LHu1g==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.4"
  }
},
"node_modules/media-typer": {
  "version": "1.1.1",
  "resolved": "https://registry.npmjs.org/media-typer/-/media-typer-1.1.1.tgz",
  "integrity": "sha512-yz3xRaG20c6/BOzvYoDaGtPmGscs7YivItZEEqe6GbwNfHuxu9YNmvnEkMzKldAGY4/80pRcQRZSEnhquk9XuQ==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.8"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/express"
  }
},
"node_modules/memory-pager": {
  "version": "1.5.0",
  "resolved": "https://registry.npmjs.org/memory-pager/-/memory-pager-1.5.0.tgz",
  "integrity": "sha512-ZS4Bp4r/Zoeq6+NLJpP+0Zzm0pR8whtGPf1XExKLJBAczGMnSi3It14OiNCStjQjM6NU1okjQGSxgEZN8eBYKg==",
  "license": "MIT"
},
"node_modules/merge-descriptors": {
  "version": "2.0.0",
  "resolved": "https://registry.npmjs.org/merge-descriptors/-/merge-descriptors-2.0.0.tgz",
  "integrity": "sha512-Snk314V5ayFLhp3fkUREub6WtjBfPdCPY1Ln8/8munuLuiYhsABgBVWsozAG+MWMbVEvcdcpbi9R7ww22l9Q3g==",
  "license": "MIT",
  "engines": {
    "node": ">=18"
  },
  "funding": {
    "url": "https://github.com/sponsors/sindresorhus"
  }
},
"node_modules/mime-db": {
  "version": "1.54.0",
  "resolved": "https://registry.npmjs.org/mime-db/-/mime-db-1.54.0.tgz",
  "integrity": "sha512-aU5EJuIN2WDemCcAp2vFBfp/m4EAhWJnUNSSw0ixs7/kXbd6Pg64EmwJkNdFhB8aWt1sH2CTXrLxo/iAGV3oPQ==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.6"
  }
},
"node_modules/mime-types": {
  "version": "3.0.2",
  "resolved": "https://registry.npmjs.org/mime-types/-/mime-types-3.0.2.tgz",
  "integrity": "sha512-Lbgzdk0h4juoQ9fCKXW4by0UJqj+nOOrI9MJ1sSj4nI8aI2eo1qmvQEie4VD1glsS250n15LsWsYtCugiStS5A==",
  "license": "MIT",
  "dependencies": {
    "mime-db": "^1.54.0"
  },
  "engines": {
    "node": ">=18"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/express"
  }
},
"node_modules/minimatch": {
  "version": "10.2.6",
  "resolved": "https://registry.npmjs.org/minimatch/-/minimatch-10.2.6.tgz",
  "integrity": "sha512-vpLQEs+VLCr1nU0BXS07maYoFwlDAH0gngQuuttxIwutDFEMHq2blX+8vpgxDdK3J1PwjCJiep77OitTZ4Ll1A==",
  "dev": true,
  "license": "BlueOak-1.0.0",
```

```
    "dependencies": {
      "brace-expansion": "^5.0.8"
    },
    "engines": {
      "node": "18 || 20 || >=22"
    },
    "funding": {
      "url": "https://github.com/sponsors/isaacs"
    }
  },
  "node_modules/mongodb": {
    "version": "7.6.0",
    "resolved": "https://registry.npmjs.org/mongodb/-/mongodb-7.6.0.tgz",
    "integrity": "sha512-WbZ6OCjYw2c53LOjfkQa+reXr7kIiOVpXXglnASFuiMtif0BvsMwHe3ClJHLm1/r7wJFwaasfFtD6iYIktB01g==",
    "license": "Apache-2.0",
    "dependencies": {
      "@mongodb-js/saslprep": "^1.4.11",
      "bson": "^7.2.0",
      "mongodb-connection-string-url": "^7.0.1"
    },
    "engines": {
      "node": ">=20.19.0"
    },
    "peerDependencies": {
      "@aws-sdk/credential-providers": "^3.806.0",
      "@mongodb-js/zstd": "^7.0.0",
      "gcp-metadata": "^7.0.1",
      "kerberos": "^7.0.0",
      "mongodb-client-encryption": "^7.2.0",
      "snappy": "^7.3.2",
      "socks": "^2.8.6"
    },
    "peerDependenciesMeta": {
      "@aws-sdk/credential-providers": {
        "optional": true
      },
      "@mongodb-js/zstd": {
        "optional": true
      },
      "gcp-metadata": {
        "optional": true
      },
      "kerberos": {
        "optional": true
      },
      "mongodb-client-encryption": {
        "optional": true
      },
      "snappy": {
        "optional": true
      },
      "socks": {
        "optional": true
      }
    }
  },
  "node_modules/mongodb-connection-string-url": {
    "version": "7.0.2",
    "resolved": "https://registry.npmjs.org/mongodb-connection-string-url/-/mongodb-connection-string-url-7.0.2.tgz",
    "integrity": "sha512-ZoS07RoFqpKYQwAk59qmrx8+jJHNHU30UjlU96QktiGn1ltvDr+vCznLX5DiUBLEpMAHatHNWV1nM/74ul66kA==",
    "license": "Apache-2.0",
    "dependencies": {
      "@types/whatwg-url": "^13.0.0",
      "whatwg-url": "^14.1.0"
    },
    "engines": {
      "node": ">=20.19.0"
    }
  },
  "node_modules/mongoose": {
    "version": "9.10.2",
    "resolved": "https://registry.npmjs.org/mongoose/-/mongoose-9.10.2.tgz",
    "integrity": "sha512-oFcFL3dsX5tDzMwBD+GGoDqKIj6RNG3bu3oqlvcUIgPulHP14/6N7c07j3s0XgbrlT7U/lbiXOD89hmPtEXnfQ==",
    "license": "MIT",
    "dependencies": {
      "@standard-schema/spec": "^1.1.0",
      "kareem": "3.4.0",
      "mongodb": "~7.6",
      "mpath": "0.9.0",
      "mquery": "6.0.0",
      "ms": "2.1.3",
      "sift": "17.1.3"
    },
    "engines": {
      "node": ">=20.19.0"
    },
    "funding": {
      "type": "opencollective",
      "url": "https://opencollective.com/mongoose"
    }
  },
  "node_modules/mpath": {
    "version": "0.9.0",
    "resolved": "https://registry.npmjs.org/mpath/-/mpath-0.9.0.tgz",
    "integrity": "sha512-ikJRQTk8hw5DEoFVxHG1Gn9T/xcjtdnOKIU1JTmGjZZlg9LST2mBLmcX3/ICIbgJydT2GOc15RnNy5mHmzfSew==",
    "license": "MIT",
    "engines": {
      "node": ">=4.0.0"
    }
  },
  "node_modules/mquery": {
    "version": "6.0.0",
    "resolved": "https://registry.npmjs.org/mquery/-/mquery-6.0.0.tgz",
    "integrity": "sha512-b2KQNsmgtkscfeDgkYMcWGn9vZI9YoXh802VDEwE6qc50zxBFQ0Oo8ROkawbPAsXCY1/Z1yp0MagqsZStPWJjw==",
    "license": "MIT",
```

```
  "engines": {
    "node": ">=20.19.0"
  }
},
"node_modules/ms": {
  "version": "2.1.3",
  "resolved": "https://registry.npmjs.org/ms/-/ms-2.1.3.tgz",
  "integrity": "sha512-6FlzubTLZG3J2a/NVCAleEhjzq5oxgHyaCU9yYXvcLsvoVaHJq/s5xXI6/XXP6tz7R9xAOtHnSO/tXtF3WRTlA==",
  "license": "MIT"
},
"node_modules/multer": {
  "version": "2.4.0",
  "resolved": "https://registry.npmjs.org/multer/-/multer-2.4.0.tgz",
  "integrity": "sha512-7dqa0ZcFfzbefdTuIkzOSMvZWC0J7FLqBOjJUZvDCXShIURWKxAyTT1wHhnE5q19c7jOJf43IYYKjBmZVZmvhg==",
  "license": "MIT",
  "dependencies": {
    "append-field": "^1.0.0",
    "busboy": "^1.6.0",
    "type-is": "^1.6.18"
  },
  "engines": {
    "node": ">= 10.16.0"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/express"
  }
},
"node_modules/multer/node_modules/media-typer": {
  "version": "0.3.0",
  "resolved": "https://registry.npmjs.org/media-typer/-/media-typer-0.3.0.tgz",
  "integrity": "sha512-dq+qelQ9akHpcOl/gUVRTxVIOkAJ1wR3QAvb4RsVjS8oVoFjDGTc679wJYmUmknUF5HwMLOgb5O+a3KxfWapPQ==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.6"
  }
},
"node_modules/multer/node_modules/mime-db": {
  "version": "1.52.0",
  "resolved": "https://registry.npmjs.org/mime-db/-/mime-db-1.52.0.tgz",
  "integrity": "sha512-sPU4uV7dYlvtWJxwwxHD0PuihVNiE7TyAbQ5SWxDCB9mUYvOgroQOwYQQOKPJ8CIbE+1ETVlOoK1UC2nU3gYvg==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.6"
  }
},
"node_modules/multer/node_modules/mime-types": {
  "version": "2.1.35",
  "resolved": "https://registry.npmjs.org/mime-types/-/mime-types-2.1.35.tgz",
  "integrity": "sha512-ZDY+bPm5zTTF+YpCrAU9nK0UgICYPT0QtT1NZWFv4s++TNkcgVaT0g6+4R2uI4MjQjzysHB1zxuWL50hzaeXiw==",
  "license": "MIT",
  "dependencies": {
    "mime-db": "1.52.0"
  },
  "engines": {
    "node": ">= 0.6"
  }
},
"node_modules/multer/node_modules/type-is": {
  "version": "1.6.18",
  "resolved": "https://registry.npmjs.org/type-is/-/type-is-1.6.18.tgz",
  "integrity": "sha512-TkRKr9sUTxEH8MdfuCSP7VizJyzRNMjj2J2do2Jr3Kym598JVdEksuzPQCnlFPW4ky9Q+iA+ma9BGm06XQBy8g==",
  "license": "MIT",
  "dependencies": {
    "media-typer": "0.3.0",
    "mime-types": "~2.1.24"
  },
  "engines": {
    "node": ">= 0.6"
  }
},
"node_modules/negotiator": {
  "version": "1.1.0",
  "resolved": "https://registry.npmjs.org/negotiator/-/negotiator-1.1.0.tgz",
  "integrity": "sha512-NMPBRMJgiQHjbd8phG3Vebdx4kZ1H121rbl5IkMqeOsahptB9BKo/d7oJ3zTXqTgagn2bWlNSXkh0QUGM31RYg==",
  "license": "MIT",
  "dependencies": {
    "content-type": "^2.1.0"
  },
  "engines": {
    "node": ">=18"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/express"
  }
},
"node_modules/negotiator/node_modules/content-type": {
  "version": "2.1.0",
  "resolved": "https://registry.npmjs.org/content-type/-/content-type-2.1.0.tgz",
  "integrity": "sha512-mj7UPXE0jaqaOsukNZRUEfEi2AcL7C/vwmwcHV0O97eO1E1pxBZuyjlZrx5seTaNBg1U6+o35wpa35Qfcc+7ag==",
  "license": "MIT",
  "engines": {
    "node": ">=18"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/express"
  }
},
"node_modules/nodemailer": {
  "version": "10.0.13",
  "resolved": "https://registry.npmjs.org/nodemailer/-/nodemailer-10.0.13.tgz",
  "integrity": "sha512-SzG86OlvcW/NNhUFC6uROMwRTL4n7MswfQqC/T8mhkmnY1YVa23zUEMYi4ijSeXSl9GLz9ZeTJDUatEDuY5FeQ==",
  "license": "MIT-0",
```

```
    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/nodemon": {
    "version": "3.1.14",
    "resolved": "https://registry.npmjs.org/nodemon/-/nodemon-3.1.14.tgz",
    "integrity": "sha512-jakjZi93UtB3jHMWsXL68FXSAosbLfY0In5gtKq3niLSkrWznrVBzXFNOEMJUfc9+Ke7SHWoAZsiMkNP3vq6Jw==",
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "chokidar": "^3.5.2",
      "debug": "^4",
      "ignore-by-default": "^1.0.1",
      "minimatch": "^10.2.1",
      "pstree.remy": "^1.1.8",
      "semver": "^7.5.3",
      "simple-update-notifier": "^2.0.0",
      "supports-color": "^5.5.0",
      "touch": "^3.1.0",
      "undefsafe": "^2.0.5"
    },
    "bin": {
      "nodemon": "bin/nodemon.js"
    },
    "engines": {
      "node": ">=10"
    },
    "funding": {
      "type": "opencollective",
      "url": "https://opencollective.com/nodemon"
    }
  },
  "node_modules/normalize-path": {
    "version": "3.0.0",
    "resolved": "https://registry.npmjs.org/normalize-path/-/normalize-path-3.0.0.tgz",
    "integrity": "sha512-6eZs5Ls3WtCisHWp9S2GUy8dqkpGi4BVSz3GaqiE6ezub0512ESztXUwUB6C6IKbQkY2Pnb/mD4WYojCRwcwLA==",
    "dev": true,
    "license": "MIT",
    "engines": {
      "node": ">=0.10.0"
    }
  },
  "node_modules/object-assign": {
    "version": "4.1.1",
    "resolved": "https://registry.npmjs.org/object-assign/-/object-assign-4.1.1.tgz",
    "integrity": "sha512-rJgTQnkUnH1sFw8yT6VSU3zD3sWmu6sZhIseY8VX+GRu3P6F7Fu+JNDoXfklElbLJSnc3FUQHVe4cU5hj+BcUg==",
    "license": "MIT",
    "engines": {
      "node": ">=0.10.0"
    }
  },
  "node_modules/object-inspect": {
    "version": "1.13.4",
    "resolved": "https://registry.npmjs.org/object-inspect/-/object-inspect-1.13.4.tgz",
    "integrity": "sha512-W67iLl4J2EXEGTbfeHCffrjDfitvLANg0UlX3wFUUSTx92KXRFegMHUVgSqE+wvhAbi4WqjGg9czysTV2Epbew==",
    "license": "MIT",
    "engines": {
      "node": ">= 0.4"
    },
    "funding": {
      "url": "https://github.com/sponsors/ljharb"
    }
  },
  "node_modules/on-finished": {
    "version": "2.4.1",
    "resolved": "https://registry.npmjs.org/on-finished/-/on-finished-2.4.1.tgz",
    "integrity": "sha512-oVlzkg3ENAhCk2zdv7IJwd/QUD4z2RxRwpkcGY8psCVcCYZNq4wYnVWALHM+brtuJjePWiYF/ClmuDr8Ch5+kg==",
    "license": "MIT",
    "dependencies": {
      "ee-first": "1.1.1"
    },
    "engines": {
      "node": ">= 0.8"
    }
  },
  "node_modules/once": {
    "version": "1.4.0",
    "resolved": "https://registry.npmjs.org/once/-/once-1.4.0.tgz",
    "integrity": "sha512-lNaJgI+2Q5URQBkccEKHTQOPaXdUxnZZElQTZY0MFUAuaEqe1E+Nyvgdz/aIyNi6Z9MzO5dv1H8n58/GELp3+w==",
    "license": "ISC",
    "dependencies": {
      "wrappy": "1"
    }
  },
  "node_modules/parseurl": {
    "version": "1.3.3",
    "resolved": "https://registry.npmjs.org/parseurl/-/parseurl-1.3.3.tgz",
    "integrity": "sha512-CiyeOxFT/JZyN5m0z9PfXw4SCBJ6Sygz1Dpl0wqjlhDEGGBP1GnsUVEL0p63hoG1fcj3fHynXi9NYO4nWOL+qQ==",
    "license": "MIT",
    "engines": {
      "node": ">= 0.8"
    }
  },
  "node_modules/path-to-regexp": {
    "version": "8.4.2",
    "resolved": "https://registry.npmjs.org/path-to-regexp/-/path-to-regexp-8.4.2.tgz",
    "integrity": "sha512-qRcuIdP69NPm4qbACK+aDogI5CBDMi1jKe0ry5rSQJz8JVLsC7jV8XpiJjGRLLol3N+R5ihGYcrPLTno6pAdBA==",
    "license": "MIT",
    "funding": {
      "type": "opencollective",
      "url": "https://opencollective.com/express"
    }
  },
  "node_modules/picomatch": {
```

```
    "version": "2.3.2",
    "resolved": "https://registry.npmjs.org/picomatch/-/picomatch-2.3.2.tgz",
    "integrity": "sha512-V7+vQEJ06Z+c5tSye8S+nHUfI51xoXIXjHQ99cQtKUkQqqO1kO/KCJUfZXuB47h/YBlDhah2H3hdUGXn8ie0oA==",
    "dev": true,
    "license": "MIT",
    "engines": {
      "node": ">=8.6"
    },
    "funding": {
      "url": "https://github.com/sponsors/jonschlinkert"
    }
  },
  "node_modules/proxy-addr": {
    "version": "2.0.8",
    "resolved": "https://registry.npmjs.org/proxy-addr/-/proxy-addr-2.0.8.tgz",
    "integrity": "sha512-5nnx0yGyVUcY6t9RnWcARWtwT9F1D8O9rt08htPvnd49W1IgZtmLkhu9WfMzQj1cFxjHIO6connUNVW5k7AVyQ==",
    "license": "MIT",
    "dependencies": {
      "forwarded": "0.2.0",
      "ipaddr.js": "1.9.1"
    },
    "engines": {
      "node": ">= 0.10"
    },
    "funding": {
      "type": "opencollective",
      "url": "https://opencollective.com/express"
    }
  },
  "node_modules/pstree.remy": {
    "version": "1.1.8",
    "resolved": "https://registry.npmjs.org/pstree.remy/-/pstree.remy-1.1.8.tgz",
    "integrity": "sha512-77DZwxQmxKnu3aR542U+X8FypNzbfJ+C5XQDk3uWjWxn6151aIMGthWYRXTqT1E5oJvg+ljaa2OJi+VfvCOQ8w==",
    "dev": true,
    "license": "MIT"
  },
  "node_modules/punycode": {
    "version": "2.3.1",
    "resolved": "https://registry.npmjs.org/punycode/-/punycode-2.3.1.tgz",
    "integrity": "sha512-vYt7UD1U9Wg6138shLtLOvdAu+8DsC/ilFtEVHcH+wydcSpNE20AfSOduf6MkRFahL5FY7X1oU7nKVZFtfq8Fg==",
    "license": "MIT",
    "engines": {
      "node": ">=6"
    }
  },
  "node_modules/qs": {
    "version": "6.16.0",
    "resolved": "https://registry.npmjs.org/qs/-/qs-6.16.0.tgz",
    "integrity": "sha512-h6fhOIaRrID2CbEY2fqs+7t+UXZo+MLAnU5gRIq85uFtdiUPCdsApMlHhXogKVM4HM2DVbIjGNTTYH2OcmP1vA==",
    "license": "BSD-3-Clause",
    "dependencies": {
      "es-define-property": "^1.0.1",
      "side-channel": "^1.1.1"
    },
    "engines": {
      "node": ">=0.6"
    },
    "funding": {
      "url": "https://github.com/sponsors/ljharb"
    }
  },
  "node_modules/range-parser": {
    "version": "1.3.0",
    "resolved": "https://registry.npmjs.org/range-parser/-/range-parser-1.3.0.tgz",
    "integrity": "sha512-hek2mFQpPuI4E1BBKrSto+BU3e3x4xuarsbiwr3+lf7p44juvFMV0XFWQAP3xUyqXA4RrXLIoaSUGbSt056ZMw==",
    "license": "MIT",
    "engines": {
      "node": ">= 0.6"
    },
    "funding": {
      "type": "opencollective",
      "url": "https://opencollective.com/express"
    }
  },
  "node_modules/raw-body": {
    "version": "3.0.2",
    "resolved": "https://registry.npmjs.org/raw-body/-/raw-body-3.0.2.tgz",
    "integrity": "sha512-K5zQjDllxWkf7Z5xJdV0/B0WTNqx6vxG70zJE4N0kBs4LovmEYWJzQGxC9bS9RAKu3bgM40lrd5zoLJ12MQ5BA==",
    "license": "MIT",
    "dependencies": {
      "bytes": "~3.1.2",
      "http-errors": "~2.0.1",
      "iconv-lite": "~0.7.0",
      "unpipe": "~1.0.0"
    },
    "engines": {
      "node": ">= 0.10"
    }
  },
  "node_modules/readdirp": {
    "version": "3.6.0",
    "resolved": "https://registry.npmjs.org/readdirp/-/readdirp-3.6.0.tgz",
    "integrity": "sha512-hOS089on8RduqdbhvQ5Z37A0ESjsqz6qnRcffsMU3495FuTdqSm+7bhJ29JvIOsBDEEnan5DPu9t3To9VRlMzA==",
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "picomatch": "^2.2.1"
    },
    "engines": {
      "node": ">=8.10.0"
    }
  },
  "node_modules/router": {
    "version": "2.2.0",
    "resolved": "https://registry.npmjs.org/router/-/router-2.2.0.tgz",
```

```
    "integrity": "sha512-nLTrUKm2UyiL7rlhapu/Zl45FwNgkZGaCpZbIHajDYgwlJCOzLSk+cIPAnsEqV955GjILJnKbdQC1nVPz+gAYQ==",
    "license": "MIT",
    "dependencies": {
      "debug": "^4.4.0",
      "depd": "^2.0.0",
      "is-promise": "^4.0.0",
      "parseurl": "^1.3.3",
      "path-to-regexp": "^8.0.0"
    },
    "engines": {
      "node": ">= 18"
    }
  },
  "node_modules/safe-buffer": {
    "version": "5.2.1",
    "resolved": "https://registry.npmjs.org/safe-buffer/-/safe-buffer-5.2.1.tgz",
    "integrity": "sha512-rp3So07KcdmmKbGvgaNxQSJr7bGVSVk5S9Eq1F+ppbRo70+YeaDxkw5Dd8NPN+GD6bjnYm2VuPuCXmpuYvmCXQ==",
    "funding": [
      {
        "type": "github",
        "url": "https://github.com/sponsors/feross"
      },
      {
        "type": "patreon",
        "url": "https://www.patreon.com/feross"
      },
      {
        "type": "consulting",
        "url": "https://feross.org/support"
      }
    ],
    "license": "MIT"
  },
  "node_modules/safer-buffer": {
    "version": "2.1.2",
    "resolved": "https://registry.npmjs.org/safer-buffer/-/safer-buffer-2.1.2.tgz",
    "integrity": "sha512-YZo3K82SD7Riyi0E1EQPojLz7kpepnSQI9IyPbHHg1XXXevb5dJI7tpyN2ADxGcQbHG7vcyRHk0cbwqcQriUtg==",
    "license": "MIT"
  },
  "node_modules/semver": {
    "version": "7.8.5",
    "resolved": "https://registry.npmjs.org/semver/-/semver-7.8.5.tgz",
    "integrity": "sha512-Y7/KDsb8LjooZpwaqGyulO6DQlksgCncchHGk+sZIY4SBvUocMBEFH5Ur1fI4dV+Jvl0w6cjvucaIi40puRioA==",
    "license": "ISC",
    "bin": {
      "semver": "bin/semver.js"
    },
    "engines": {
      "node": ">=10"
    }
  },
  "node_modules/send": {
    "version": "1.2.1",
    "resolved": "https://registry.npmjs.org/send/-/send-1.2.1.tgz",
    "integrity": "sha512-1gnZf7DFcoIcajTjTwjwuDjzuz4PPcY2StKPlsGAQ1+YH20IRVrBaXSWmdjowTJ6u8Rc01PoYOGHXfP1mYcZNQ==",
    "license": "MIT",
    "dependencies": {
      "debug": "^4.4.3",
      "encodeurl": "^2.0.0",
      "escape-html": "^1.0.3",
      "etag": "^1.8.1",
      "fresh": "^2.0.0",
      "http-errors": "^2.0.1",
      "mime-types": "^3.0.2",
      "ms": "^2.1.3",
      "on-finished": "^2.4.1",
      "range-parser": "^1.2.1",
      "statuses": "^2.0.2"
    },
    "engines": {
      "node": ">= 18"
    },
    "funding": {
      "type": "opencollective",
      "url": "https://opencollective.com/express"
    }
  },
  "node_modules/serve-static": {
    "version": "2.2.1",
    "resolved": "https://registry.npmjs.org/serve-static/-/serve-static-2.2.1.tgz",
    "integrity": "sha512-xRXBn0pPqQTVQiC8wyQrKs2MOlX24zQ0POGaj0kultvoOCstBQM5yvOhAVSUwOMjQtTvsPWoNCHfPGwaaQJhTw==",
    "license": "MIT",
    "dependencies": {
      "encodeurl": "^2.0.0",
      "escape-html": "^1.0.3",
      "parseurl": "^1.3.3",
      "send": "^1.2.0"
    },
    "engines": {
      "node": ">= 18"
    },
    "funding": {
      "type": "opencollective",
      "url": "https://opencollective.com/express"
    }
  },
  "node_modules/setprototypeof": {
    "version": "1.2.0",
    "resolved": "https://registry.npmjs.org/setprototypeof/-/setprototypeof-1.2.0.tgz",
    "integrity": "sha512-E5LDX7Wrp85Kil5bhZv46j8jOeboKq5JMmYM3gVGdGH8xFpPWXUMsNrlODCrkoxMEeNi/XZIwuRvY4XNwYMJpw==",
    "license": "ISC"
  },
  "node_modules/side-channel": {
    "version": "1.1.1",
    "resolved": "https://registry.npmjs.org/side-channel/-/side-channel-1.1.1.tgz",
```

```
    "integrity": "sha512-6x6dK6zJdpTzF4sQeNYxwtvBzf6Eg4GtlesS94HOvTudUeyK2WXAaIfmDgsyslYrRBeFIlsi54AYsFGUuhmvrQ==",
    "license": "MIT",
    "dependencies": {
      "es-errors": "^1.3.0",
      "object-inspect": "^1.13.4",
      "side-channel-list": "^1.0.1",
      "side-channel-map": "^1.0.1",
      "side-channel-weakmap": "^1.0.2"
    },
    "engines": {
      "node": ">= 0.4"
    },
    "funding": {
      "url": "https://github.com/sponsors/ljharb"
    }
  },
  "node_modules/side-channel-list": {
    "version": "1.0.1",
    "resolved": "https://registry.npmjs.org/side-channel-list/-/side-channel-list-1.0.1.tgz",
    "integrity": "sha512-mjn/0bi/oUURjc5Xl7IaWi/OJJJumuoJFQJfDDyO46+hBWsfaVM65TBHq2eoZBhzl9EchxOijpkbRC8SVBQU0w==",
    "license": "MIT",
    "dependencies": {
      "es-errors": "^1.3.0",
      "object-inspect": "^1.13.4"
    },
    "engines": {
      "node": ">= 0.4"
    },
    "funding": {
      "url": "https://github.com/sponsors/ljharb"
    }
  },
  "node_modules/side-channel-map": {
    "version": "1.0.1",
    "resolved": "https://registry.npmjs.org/side-channel-map/-/side-channel-map-1.0.1.tgz",
    "integrity": "sha512-VCjCNfgMsby3tTdo02nbjtM/ewra6jPHmpThenkTYh8pG9ucZ/1P8So4u4FGBek/BjpOVsDCMoLA/iuBKIFXRA==",
    "license": "MIT",
    "dependencies": {
      "call-bound": "^1.0.2",
      "es-errors": "^1.3.0",
      "get-intrinsic": "^1.2.5",
      "object-inspect": "^1.13.3"
    },
    "engines": {
      "node": ">= 0.4"
    },
    "funding": {
      "url": "https://github.com/sponsors/ljharb"
    }
  },
  "node_modules/side-channel-weakmap": {
    "version": "1.0.2",
    "resolved": "https://registry.npmjs.org/side-channel-weakmap/-/side-channel-weakmap-1.0.2.tgz",
    "integrity": "sha512-WPS/HvHQTYnHisLo9McqBHOJk2FkHO/tlpvldyrnem4aeQp4hai3gythswg6p01oSoTl58rcpiFAjF2br2Ak2A==",
    "license": "MIT",
    "dependencies": {
      "call-bound": "^1.0.2",
      "es-errors": "^1.3.0",
      "get-intrinsic": "^1.2.5",
      "object-inspect": "^1.13.3",
      "side-channel-map": "^1.0.1"
    },
    "engines": {
      "node": ">= 0.4"
    },
    "funding": {
      "url": "https://github.com/sponsors/ljharb"
    }
  },
  "node_modules/sift": {
    "version": "17.1.3",
    "resolved": "https://registry.npmjs.org/sift/-/sift-17.1.3.tgz",
    "integrity": "sha512-Rtlj66/b0ICeFzYTuNvX/EF1igRbbnGSvEyT79McoZa/DeGhMyC5pWKOEsZKnpkqtSeovd5FL/bjHWC3CIIvCQ==",
    "license": "MIT"
  },
  "node_modules/simple-update-notifier": {
    "version": "2.0.0",
    "resolved": "https://registry.npmjs.org/simple-update-notifier/-/simple-update-notifier-2.0.0.tgz",
    "integrity": "sha512-a2B9Y0KlNXl9u/vsW6sTIu9vGEpfKu2wRV6l1H3XEas/0gUIzGzBoP/IouTcUQbm9JWZLH3COxyn03TYlFax6w==",
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "semver": "^7.5.3"
    },
    "engines": {
      "node": ">=10"
    }
  },
  "node_modules/sparse-bitfield": {
    "version": "3.0.3",
    "resolved": "https://registry.npmjs.org/sparse-bitfield/-/sparse-bitfield-3.0.3.tgz",
    "integrity": "sha512-kvzhi7vqKTfkh0PZU+2D2PIllw2ymqJKujUcyPMd9Y75Nv4nPbGJZXNhxsgdQab2BmlDct1YnfQCguEvHr7VsQ==",
    "license": "MIT",
    "dependencies": {
      "memory-pager": "^1.0.2"
    }
  },
  "node_modules/statuses": {
    "version": "2.0.2",
    "resolved": "https://registry.npmjs.org/statuses/-/statuses-2.0.2.tgz",
    "integrity": "sha512-DvEy55V3DB7uknRo+4iOGT5fP1slR8wQohVdknigZPMpMstaKJQWhwiYBACJE3Ul2pTnATihhBYnRhZQHGBiRw==",
    "license": "MIT",
    "engines": {
      "node": ">= 0.8"
    }
```

```
},
"node_modules/streamsearch": {
  "version": "1.1.0",
  "resolved": "https://registry.npmjs.org/streamsearch/-/streamsearch-1.1.0.tgz",
  "integrity": "sha512-Mcc5wHehp9aXz1ax6bZUyY5afg9u2rv5cqQI3mRrYkGC8rW2hM02jWuwjtL++LS5qinSyhj2QfLyNsuc+VsExg==",
  "engines": {
    "node": ">=10.0.0"
  }
},
"node_modules/supports-color": {
  "version": "5.5.0",
  "resolved": "https://registry.npmjs.org/supports-color/-/supports-color-5.5.0.tgz",
  "integrity": "sha512-QjVjwdXIt408MIiAqCX4oUKsgU2EqAGzs2Ppkm4aQYbjm+ZEWEcW4SfFNTr4uMNZma0ey4f5lgLrkB0aX0QMow==",
  "dev": true,
  "license": "MIT",
  "dependencies": {
    "has-flag": "^3.0.0"
  },
  "engines": {
    "node": ">=4"
  }
},
"node_modules/to-regex-range": {
  "version": "5.0.1",
  "resolved": "https://registry.npmjs.org/to-regex-range/-/to-regex-range-5.0.1.tgz",
  "integrity": "sha512-65P7iz6X5yEr1cwcgvQxbbIw7Uk3gOy5dIdtZ4rDveLqhrdJP+Li/Hx6tyK0NEb+2GCyneCMJiGqrADCSNk8sQ==",
  "dev": true,
  "license": "MIT",
  "dependencies": {
    "is-number": "^7.0.0"
  },
  "engines": {
    "node": ">=8.0"
  }
},
"node_modules/toidentifier": {
  "version": "1.0.1",
  "resolved": "https://registry.npmjs.org/toidentifier/-/toidentifier-1.0.1.tgz",
  "integrity": "sha512-o5sSPKEkg/DIQNmH43V0/uerLrpzVedkUh8tGNvaeXpfpuwjKenlSox/2O/BTlZUtEe+JG7s5YhEz608PlAHRA==",
  "license": "MIT",
  "engines": {
    "node": ">=0.6"
  }
},
"node_modules/touch": {
  "version": "3.1.1",
  "resolved": "https://registry.npmjs.org/touch/-/touch-3.1.1.tgz",
  "integrity": "sha512-r0eojU4bI8MnHr8c5bNo7lJDdI2qXlWWJk6a9EAFG7vbhTjElYhBVS3/miuE0uOuoLdb8Mc/rVfsmm6eo5o9GA==",
  "dev": true,
  "license": "ISC",
  "bin": {
    "nodetouch": "bin/nodetouch.js"
  }
},
"node_modules/tr46": {
  "version": "5.1.1",
  "resolved": "https://registry.npmjs.org/tr46/-/tr46-5.1.1.tgz",
  "integrity": "sha512-hdF5ZgjTqgAntKkklYw0R03MG2x/bSzTtkxmIRw/sTNV8YXsCJ1tfLAX23lhxhHJlEf3CRCOCGGWw3vI3GaSPw==",
  "license": "MIT",
  "dependencies": {
    "punycode": "^2.3.1"
  },
  "engines": {
    "node": ">=18"
  }
},
"node_modules/type-is": {
  "version": "2.1.0",
  "resolved": "https://registry.npmjs.org/type-is/-/type-is-2.1.0.tgz",
  "integrity": "sha512-faYHw0anBbc/kWF3zFTEnxSFOAGUX9GFbOBthvDdLsIlEoWOFOtS0zgCiQYwIskL9iGXZL3kAXD8OoZ4GmMATA==",
  "license": "MIT",
  "dependencies": {
    "content-type": "^2.0.0",
    "media-typer": "^1.1.0",
    "mime-types": "^3.0.0"
  },
  "engines": {
    "node": ">= 18"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/express"
  }
},
"node_modules/type-is/node_modules/content-type": {
  "version": "2.1.0",
  "resolved": "https://registry.npmjs.org/content-type/-/content-type-2.1.0.tgz",
  "integrity": "sha512-mj7UPXE0jaqaOsukNZRUEfEi2AcL7C/vwmwcHV0O97eO1E1pxBZuyjlZrx5seTaNBg1U6+o35wpa35Qfcc+7ag==",
  "license": "MIT",
  "engines": {
    "node": ">=18"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/express"
  }
},
"node_modules/undefsafe": {
  "version": "2.0.5",
  "resolved": "https://registry.npmjs.org/undefsafe/-/undefsafe-2.0.5.tgz",
  "integrity": "sha512-WxONCrssBM8TSPRqN5EmsjVrsv4A8X12J4ArBiiayv3DyyG3ZlIg6yysuuSYdZsVz3TKcTg2fd//Ujd4CHV1iA==",
  "dev": true,
  "license": "MIT"
},
"node_modules/unpipe": {
```

```
      "version": "1.0.0",
      "resolved": "https://registry.npmjs.org/unpipe/-/unpipe-1.0.0.tgz",
      "integrity": "sha512-pjy2bYhSsufwWlKwPc+l3cN7+wuJlK6uz0YdJEOlQDbl6jo/YlPi4mb8agUkVC8BF7V8NuzeyPNqRksA3hztKQ==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.8"
      }
    },
    "node_modules/vary": {
      "version": "1.1.2",
      "resolved": "https://registry.npmjs.org/vary/-/vary-1.1.2.tgz",
      "integrity": "sha512-BNGbWLfd0eUPabhkXUVm0j8uuvREyTh5ovRa/dyow/BqAbZJyC+5fU+IzQOzmAKzYqYRAISoRhdQr3eIZ/PXqg==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.8"
      }
    },
    "node_modules/webidl-conversions": {
      "version": "7.0.0",
      "resolved": "https://registry.npmjs.org/webidl-conversions/-/webidl-conversions-7.0.0.tgz",
      "integrity": "sha512-VwddBukDzu71offAQR975unBIGqfKZpM+8ZX6ySk8nYhVoo5CYaZyzt3YBvYtRtO+aoGlqxPg/B87NGVZ/fu6g==",
      "license": "BSD-2-Clause",
      "engines": {
        "node": ">=12"
      }
    },
    "node_modules/whatwg-url": {
      "version": "14.2.0",
      "resolved": "https://registry.npmjs.org/whatwg-url/-/whatwg-url-14.2.0.tgz",
      "integrity": "sha512-De72GdQZzNTUBBChsXueQUnPKDkg/5A5zp7pFDuQAj5UFoENpiACU0wlCvzpAGnTkj++ihpKwKyYewn/XNUbKw==",
      "license": "MIT",
      "dependencies": {
        "tr46": "^5.1.0",
        "webidl-conversions": "^7.0.0"
      },
      "engines": {
        "node": ">=18"
      }
    },
    "node_modules/wrappy": {
      "version": "1.0.2",
      "resolved": "https://registry.npmjs.org/wrappy/-/wrappy-1.0.2.tgz",
      "integrity": "sha512-l4Sp/DRseor9wL6EvV2+TuQn63dMkPjZ/sp9XkghTEbV9KlPS1xUsZ3u7/IQO4wxtcFB4bgpQPRcR3QCvezPcQ==",
      "license": "ISC"
    }
  }
}
```

Backend/package.json

```
{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "cors": "^2.8.6",
    "dotenv": "^18.0.4",
    "express": "^5.2.1",
    "jsonwebtoken": "^9.0.3",
    "mongoose": "^9.10.2",
    "multer": "^2.4.0",
    "nodemailer": "^10.0.13"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

Backend/resetAdminPassword.js

```
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");
require("dotenv").config();

const User = require("./models/User");

const resetPassword = async () => {
    try {
        await mongoose.connect(process.env.MONGO_URI);

        const hashedPassword = await bcrypt.hash("Admin@123", 10);

        const user = await User.findOneAndUpdate(
            { email: "admin@example.com" },
            { password: hashedPassword },
            { new: true }
        );

        if (!user) {
            console.log("Admin user not found");
        } else {
            console.log("Admin password reset successfully");
            console.log("Email:", user.email);
            console.log("Password: Admin@123");
        }

        await mongoose.disconnect();

    } catch (error) {
        console.log("Error:", error.message);
    }
};

resetPassword();
```

Backend/resetEmployeePassword.js

```
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");
require("dotenv").config();

const User = require("./models/User");

const resetPassword = async () => {
    try {
        await mongoose.connect(process.env.MONGO_URI);

        const hashedPassword = await bcrypt.hash("Employee@123", 10);

        const user = await User.findOneAndUpdate(
            { email: "durgesh@example.com" },
            { password: hashedPassword },
            { new: true }
        );

        if (!user) {
            console.log("Employee user not found");
        } else {
            console.log("Employee password reset successfully");
            console.log("Email:", user.email);
            console.log("Password: Employee@123");
        }

        await mongoose.disconnect();

    } catch (error) {
        console.log("Error:", error.message);
    }
};

resetPassword();
```

Backend/routes/attendanceRoutes.js

```
const express = require("express");

const {
    checkIn,
    checkOut,
    getAttendance,
    getMyAttendance,
    getEmployeeAttendance
} = require("../controllers/attendanceController");

const authMiddleware = require("../middleware/authMiddleware");

const router = express.Router();

// CHECK-IN
// Admin, HR and Employee
router.post(
    "/check-in",
    authMiddleware,
    checkIn
);

// CHECK-OUT
// Admin, HR and Employee
router.post(
    "/check-out",
    authMiddleware,
    checkOut
);

// GET ALL ATTENDANCE
// Admin and HR only
router.get(
    "/",
    authMiddleware,
    (req, res, next) => {

        if (
            req.user.role !== "admin" &&
            req.user.role !== "hr"
        ) {
            return res.status(403).json({
                message: "Access denied. Admin or HR only."
            });
        }

        next();
    },
    getAttendance
);

// GET MY ATTENDANCE
// Employee only
router.get(
    "/my",
    authMiddleware,
    (req, res, next) => {

        if (req.user.role !== "employee") {
            return res.status(403).json({
                message: "This endpoint is for employees only."
            });
        }

        next();
    },
    getMyAttendance
);

// GET SPECIFIC EMPLOYEE ATTENDANCE
// Admin and HR only
router.get(
    "/employee/:employeeId",
    authMiddleware,
    (req, res, next) => {

        if (
            req.user.role !== "admin" &&
            req.user.role !== "hr"
        ) {
            return res.status(403).json({
                message: "Access denied. Admin or HR only."
            });
        }

        next();
    },
    getEmployeeAttendance
);

module.exports = router;
```

Backend/routes/auditLogRoutes.js

```
const express = require("express");

const {
    createAuditLog,
    getAllAuditLogs
} = require("../controllers/auditLogController");

const authMiddleware = require("../middleware/authMiddleware");
const roleMiddleware = require("../middleware/roleMiddleware");

const router = express.Router();

router.post(
    "/create",
    authMiddleware,
    roleMiddleware("admin"),
    createAuditLog
);

router.get(
    "/",
    authMiddleware,
    roleMiddleware("admin"),
    getAllAuditLogs
);

module.exports = router;
```

Backend/routes/authRoutes.js

```
const express = require("express");

const {
    registerUser,
    loginUser,
    forgotPassword,
    verifyResetOTP,
    resetPassword
} = require("../controllers/authController");

const router = express.Router();

router.post("/register", registerUser);

router.post("/login", loginUser);

router.post("/forgot-password", forgotPassword);

router.post("/verify-reset-otp", verifyResetOTP);

router.post("/reset-password", resetPassword);

module.exports = router;
```

Backend/routes/candidateRoutes.js

```
const express = require("express");

const {
    createCandidate,
    getAllCandidates,
    getCandidateById,
    updateCandidateStatus,
    deleteCandidate
} = require("../controllers/candidateController");

const authMiddleware =
    require("../middleware/authMiddleware");

const roleMiddleware =
    require("../middleware/roleMiddleware");

const upload =
    require("../middleware/uploadMiddleware");

const router =
    express.Router();


router.post(
    "/",
    authMiddleware,
    roleMiddleware(
        "admin",
        "hr"
    ),
    upload.single("resume"),
    createCandidate
);


router.get(
    "/",
    authMiddleware,
    roleMiddleware(
        "admin",
        "hr",
        "employee"
    ),
    getAllCandidates
);


router.get(
    "/:id",
    authMiddleware,
    roleMiddleware(
        "admin",
        "hr",
        "employee"
    ),
    getCandidateById
);


router.put(
    "/:id/status",
    authMiddleware,
    roleMiddleware(
        "admin",
        "hr"
    ),
    updateCandidateStatus
);


router.delete(
    "/:id",
    authMiddleware,
    roleMiddleware(
        "admin"
    ),
    deleteCandidate
);


module.exports = router;
```

Backend/routes/dashboardRoutes.js

```
const express = require("express");

const {
    getDashboardStats
} = require("../controllers/dashboardController");

const authMiddleware = require("../middleware/authMiddleware");
const roleMiddleware = require("../middleware/roleMiddleware");

const router = express.Router();

router.get(
    "/stats",
    authMiddleware,
    roleMiddleware("admin", "employee"),
    getDashboardStats
);

module.exports = router;
```

Backend/routes/departmentRoutes.js

```
const express = require("express");

const {
    createDepartment,
    getAllDepartments,
    getDepartmentById,
    updateDepartment,
    changeDepartmentHead,
    deleteDepartment
} = require("../controllers/departmentController");

const authMiddleware =
    require("../middleware/authMiddleware");

const roleMiddleware =
    require("../middleware/roleMiddleware");

const router =
    express.Router();


router.post(
    "/",
    authMiddleware,
    roleMiddleware("admin", "hr"),
    createDepartment
);


router.get(
    "/",
    authMiddleware,
    roleMiddleware(
        "admin",
        "hr",
        "employee"
    ),
    getAllDepartments
);


router.get(
    "/:id",
    authMiddleware,
    roleMiddleware(
        "admin",
        "hr",
        "employee"
    ),
    getDepartmentById
);


router.put(
    "/:id",
    authMiddleware,
    roleMiddleware("admin", "hr"),
    updateDepartment
);


router.put(
    "/:id/head",
    authMiddleware,
    roleMiddleware("admin", "hr"),
    changeDepartmentHead
);


router.delete(
    "/:id",
    authMiddleware,
    roleMiddleware("admin"),
    deleteDepartment
);


module.exports = router;
```

Backend/routes/employeeRoutes.js

```
const express = require("express");

const {
    createEmployee,
    getEmployees,
    getEmployeeById,
    updateEmployee,
    deleteEmployee
} = require("../controllers/employeeController");

const authMiddleware =
    require("../middleware/authMiddleware");

const roleMiddleware =
    require("../middleware/roleMiddleware");

const router =
    express.Router();


/* ========================
   CREATE EMPLOYEE
   ADMIN ONLY
======================== */

router.post(
    "/",
    authMiddleware,
    roleMiddleware("admin"),
    createEmployee
);


/* ========================
   GET ALL EMPLOYEES
   ADMIN + HR + EMPLOYEE
======================== */

router.get(
    "/",
    authMiddleware,
    roleMiddleware(
        "admin",
        "hr",
        "employee"
    ),
    getEmployees
);


/* ========================
   GET SINGLE EMPLOYEE
   ADMIN + HR + EMPLOYEE
======================== */

router.get(
    "/:id",
    authMiddleware,
    roleMiddleware(
        "admin",
        "hr",
        "employee"
    ),
    getEmployeeById
);


/* ========================
   UPDATE EMPLOYEE
   ADMIN ONLY
======================== */

router.put(
    "/:id",
    authMiddleware,
    roleMiddleware("admin"),
    updateEmployee
);


/* ========================
   DELETE EMPLOYEE
   ADMIN ONLY
======================== */

router.delete(
    "/:id",
    authMiddleware,
    roleMiddleware("admin"),
    deleteEmployee
);


module.exports = router;
```

Backend/routes/interviewRoutes.js

```
const express = require("express");

const {
    createInterview,
    getAllInterviews,
    getInterviewById,
    updateInterviewStatus,
    updateInterviewResult,
    deleteInterview
} = require("../controllers/interviewController");

const authMiddleware =
    require("../middleware/authMiddleware");

const roleMiddleware =
    require("../middleware/roleMiddleware");

const router = express.Router();

router.post(
    "/",
    authMiddleware,
    roleMiddleware("admin", "hr"),
    createInterview
);

router.get(
    "/",
    authMiddleware,
    roleMiddleware("admin", "hr", "employee"),
    getAllInterviews
);

router.get(
    "/:id",
    authMiddleware,
    roleMiddleware("admin", "hr", "employee"),
    getInterviewById
);

router.put(
    "/:id/status",
    authMiddleware,
    roleMiddleware("admin", "hr"),
    updateInterviewStatus
);

router.put(
    "/:id/result",
    authMiddleware,
    roleMiddleware("admin", "hr"),
    updateInterviewResult
);

router.delete(
    "/:id",
    authMiddleware,
    roleMiddleware("admin"),
    deleteInterview
);

module.exports = router;
```

Backend/routes/jobRoutes.js

```
const express = require("express");

const {
    createJob,
    getJobs,
    getJobById,
    updateJob,
    deleteJob
} = require("../controllers/jobController");

const authMiddleware =
    require("../middleware/authMiddleware");

const roleMiddleware =
    require("../middleware/roleMiddleware");

const router =
    express.Router();


/* ========================
   CREATE JOB
   ADMIN + HR
======================== */

router.post(
    "/",
    authMiddleware,
    roleMiddleware(
        "admin",
        "hr"
    ),
    createJob
);


/* ========================
   GET ALL JOBS
   ADMIN + HR + EMPLOYEE
======================== */

router.get(
    "/",
    authMiddleware,
    roleMiddleware(
        "admin",
        "hr",
        "employee"
    ),
    getJobs
);


/* ========================
   GET SINGLE JOB
======================== */

router.get(
    "/:id",
    authMiddleware,
    roleMiddleware(
        "admin",
        "hr",
        "employee"
    ),
    getJobById
);


/* ========================
   UPDATE JOB
   ADMIN + HR
======================== */

router.put(
    "/:id",
    authMiddleware,
    roleMiddleware(
        "admin",
        "hr"
    ),
    updateJob
);


/* ========================
   DELETE JOB
   ADMIN ONLY
======================== */

router.delete(
    "/:id",
    authMiddleware,
    roleMiddleware("admin"),
    deleteJob
);


module.exports = router;
```

Backend/routes/leaveRoutes.js

```
const express = require("express");

const router = express.Router();

const authMiddleware = require("../middleware/authMiddleware");
const roleMiddleware = require("../middleware/roleMiddleware");

const {
    createLeave,
    getAllLeaves,
    getMyLeaves,
    approveLeave,
    rejectLeave,
    getEmployeeLeaves
} = require("../controllers/leaveController");


// Employee + Admin: Apply Leave
router.post(
    "/create",
    authMiddleware,
    roleMiddleware("employee", "admin"),
    createLeave
);


// Employee: View Own Leaves
router.get(
    "/my",
    authMiddleware,
    roleMiddleware("employee"),
    getMyLeaves
);


// Admin: View All Leaves
router.get(
    "/",
    authMiddleware,
    roleMiddleware("admin"),
    getAllLeaves
);


// Admin: Approve Leave
router.put(
    "/:id/approve",
    authMiddleware,
    roleMiddleware("admin"),
    approveLeave
);


// Admin: Reject Leave
router.put(
    "/:id/reject",
    authMiddleware,
    roleMiddleware("admin"),
    rejectLeave
);


// Admin: View Employee-wise Leaves
router.get(
    "/employee/:employeeId",
    authMiddleware,
    roleMiddleware("admin"),
    getEmployeeLeaves
);


module.exports = router;
```

Backend/routes/notificationRoutes.js

```
const express = require("express");

const {

    getNotifications,

    getUnreadCount,

    getNotificationById,

    markAsRead,

    markAsUnread,

    markAllAsRead,

    deleteNotification,

    deleteReadNotifications

} = require(
    "../controllers/notificationController"
);


const authMiddleware =
    require(
        "../middleware/authMiddleware"
    );


const router =
    express.Router();


/* ========================
   GET
======================== */

router.get(
    "/",
    authMiddleware,
    getNotifications
);


router.get(
    "/unread-count",
    authMiddleware,
    getUnreadCount
);


/* ========================
   IMPORTANT:
   read-all MUST COME BEFORE /:id
======================== */

router.put(
    "/read-all",
    authMiddleware,
    markAllAsRead
);


router.put(
    "/:id/read",
    authMiddleware,
    markAsRead
);


router.put(
    "/:id/unread",
    authMiddleware,
    markAsUnread
);


/* ========================
   DELETE READ
   MUST COME BEFORE /:id
======================== */

router.delete(
    "/read",
    authMiddleware,
    deleteReadNotifications
);


router.delete(
    "/:id",
    authMiddleware,
    deleteNotification
);


/* ========================
   GET SINGLE
======================== */
```

```
router.get(
    "/:id",
    authMiddleware,
    getNotificationById
);


module.exports = router;
```

Backend/routes/payrollRoutes.js

```
const express = require("express");

const {
    createPayroll,
    getAllPayrolls,
    getEmployeePayroll,
    updatePaymentStatus
} = require("../controllers/payrollController");

const authMiddleware = require("../middleware/authMiddleware");
const roleMiddleware = require("../middleware/roleMiddleware");

const router = express.Router();

// Create payroll - Admin only
router.post(
    "/create",
    authMiddleware,
    roleMiddleware("admin"),
    createPayroll
);

// Get all payrolls - Admin only
router.get(
    "/",
    authMiddleware,
    roleMiddleware("admin"),
    getAllPayrolls
);

// Get employee payroll - Admin only
router.get(
    "/employee/:employeeId",
    authMiddleware,
    roleMiddleware("admin"),
    getEmployeePayroll
);

// Update payment status - Admin only
router.put(
    "/:id/status",
    authMiddleware,
    roleMiddleware("admin"),
    updatePaymentStatus
);

module.exports = router;
```

Backend/routes/performanceRoutes.js

```
const express = require("express");

const {
    createPerformance,
    getAllPerformances,
    getEmployeePerformance,
    updatePerformanceStatus
} = require("../controllers/performanceController");

const authMiddleware = require("../middleware/authMiddleware");
const roleMiddleware = require("../middleware/roleMiddleware");

const router = express.Router();

router.post(
    "/create",
    authMiddleware,
    roleMiddleware("admin"),
    createPerformance
);

router.get(
    "/",
    authMiddleware,
    roleMiddleware("admin"),
    getAllPerformances
);

router.get(
    "/employee/:employeeId",
    authMiddleware,
    roleMiddleware("admin"),
    getEmployeePerformance
);

router.put(
    "/:id/status",
    authMiddleware,
    roleMiddleware("admin"),
    updatePerformanceStatus
);

module.exports = router;
```

Backend/routes/userRoutes.js

```
const express = require("express");

const router = express.Router();


const authMiddleware =
    require("../middleware/authMiddleware");


const {
    getMyProfile,
    updateMyProfile,
    changePassword
} = require("../controllers/userController");



// Get Profile

router.get(
    "/profile",
    authMiddleware,
    getMyProfile
);



// Update Profile

router.put(
    "/profile",
    authMiddleware,
    updateMyProfile
);



// Change Password

router.put(
    "/change-password",
    authMiddleware,
    changePassword
);


module.exports = router;
```

Backend/server.js

```
const express = require("express");
const cors = require("cors");
const mongoose = require("mongoose");
const path = require("path");

require("dotenv").config();


const authRoutes =
    require("./routes/authRoutes");

const employeeRoutes =
    require("./routes/employeeRoutes");

const attendanceRoutes =
    require("./routes/attendanceRoutes");

const leaveRoutes =
    require("./routes/leaveRoutes");

const payrollRoutes =
    require("./routes/payrollRoutes");

const performanceRoutes =
    require("./routes/performanceRoutes");

const jobRoutes =
    require("./routes/jobRoutes");

const candidateRoutes =
    require("./routes/candidateRoutes");

const interviewRoutes =
    require("./routes/interviewRoutes");

const dashboardRoutes =
    require("./routes/dashboardRoutes");

const departmentRoutes =
    require("./routes/departmentRoutes");

const auditLogRoutes =
    require("./routes/auditLogRoutes");

const notificationRoutes =
    require("./routes/notificationRoutes");

const userRoutes =
    require("./routes/userRoutes");


const authMiddleware =
    require("./middleware/authMiddleware");


const app = express();


/* ========================
   MIDDLEWARE
======================== */

app.use(cors());

app.use(express.json());

app.use(
    express.urlencoded({
        extended: true
    })
);


/* ========================
   UPLOADS
======================== */

app.use(
    "/uploads",
    express.static(
        path.join(
            __dirname,
            "uploads"
        )
    )
);


/* ========================
   API ROUTES
======================== */

app.use(
    "/api/auth",
    authRoutes
);

app.use(
    "/api/employees",
    employeeRoutes
);
```

```
app.use(
    "/api/attendance",
    attendanceRoutes
);

app.use(
    "/api/leaves",
    leaveRoutes
);

app.use(
    "/api/payroll",
    payrollRoutes
);

app.use(
    "/api/performance",
    performanceRoutes
);

app.use(
    "/api/jobs",
    jobRoutes
);

app.use(
    "/api/candidates",
    candidateRoutes
);

app.use(
    "/api/interviews",
    interviewRoutes
);

app.use(
    "/api/dashboard",
    dashboardRoutes
);

app.use(
    "/api/departments",
    departmentRoutes
);

app.use(
    "/api/audit-logs",
    auditLogRoutes
);

app.use(
    "/api/notifications",
    notificationRoutes
);

app.use(
    "/api/users",
    userRoutes
);


/* ========================
   ROOT ROUTE
======================== */

app.get(
    "/",
    (req, res) => {

        res.send(
            "HRGenius Backend is Running!"
        );

    }
);


/* ========================
   PROTECTED TEST ROUTE
======================== */

app.get(
    "/api/protected",

    authMiddleware,

    (req, res) => {

        res.status(200).json({

            message:
                "You have access to protected API",

            user:
                req.user

        });

    }
);


/* ========================
   MONGODB
======================== */
```

```
mongoose
    .connect(
        process.env.MONGO_URI
    )

    .then(() => {

        console.log(
            "MongoDB Connected Successfully"
        );

    })

    .catch(
        (error) => {

            console.log(
                "MongoDB Connection Error:",
                error
            );

        }
    );


/* =========================
   SERVER
========================= */

const PORT =
    process.env.PORT || 5000;


app.listen(
    PORT,
    () => {

        console.log(
            `Server running on http://localhost:${PORT}`
        );

    }
);
```

Backend/services/notificationService.js

```
const Notification =
    require("../models/Notification");


/* ========================
   CREATE ONE
======================== */

const createNotification = async ({
    userId,
    title,
    message,
    type = "System",
    priority = "Normal",
    actionUrl = "",
    relatedId = null,
    relatedModel = ""
}) => {

    try {

        const notification =
            await Notification.create({

                userId,

                title,

                message,

                type,

                priority,

                actionUrl,

                relatedId,

                relatedModel

            });


        console.log(
            "Notification Created:",
            notification.title
        );


        return notification;


    } catch (error) {

        console.error(
            "Create Notification Error:",
            error
        );


        return null;

    }

};


/* ========================
   CREATE MANY
======================== */

const createNotifications =
    async (notifications) => {

        try {

            if (
                !Array.isArray(
                    notifications
                ) ||
                notifications.length === 0
            ) {

                console.log(
                    "No notifications to create"
                );

                return [];

            }


            console.log(
                "Creating Notifications:",
                notifications.length
            );


            const createdNotifications =
                await Notification.insertMany(
                    notifications
                );
```

```
            console.log(
                "Notifications Created:",
                createdNotifications.length
            );


            return createdNotifications;


        } catch (error) {

            console.error(
                "Create Notifications Error:",
                error
            );


            return [];

        }

    };


module.exports = {

    createNotification,

    createNotifications

};
```

Backend/testNotification.js

```
const mongoose = require("mongoose");
require("dotenv").config();

const Notification = require("./models/Notification");
const {
    createNotification
} = require("./services/notificationService");

const testNotification = async () => {

    try {

        await mongoose.connect(process.env.MONGO_URI);

        console.log("MongoDB Connected");

        const notification = await createNotification({

            userId: "6abb4ecce8ed40e277e74bb2",

            title: "Welcome to HRGenius",

            message:
                "This is a test notification from HRGenius.",

            type: "System",

            priority: "Normal",

            actionUrl: "/notifications.html"

        });

        console.log(
            "Notification Created:",
            notification
        );

    } catch (error) {

        console.log(
            "Test Notification Error:",
            error
        );

    } finally {

        await mongoose.connection.close();

        console.log("MongoDB Connection Closed");
    }
};

testNotification();
```

Frontend/attendance.html

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>HRGenius - Attendance</title>

    <link rel="stylesheet"
          href="CSS/style.css">

    <style>

        /* =========================
           NAVBAR
        ========================= */

        .navbar {
            height: 65px;

            background: var(--primary);
            color: #ffffff;

            display: flex;
            align-items: center;
            justify-content: space-between;

            padding: 0 30px;
        }

        .navbar h2 {
            font-size: 24px;
            color: #ffffff;
        }

        .nav-buttons {
            display: flex;
            align-items: center;
            gap: 10px;
        }

        .nav-btn {
            background: #ffffff;
            color: var(--primary);

            border: none;

            padding: 9px 16px;

            border-radius: 6px;

            cursor: pointer;

            font-weight: bold;
        }

        .nav-btn:hover {
            background: #e5e7eb;
        }


        /* =========================
           CONTAINER
        ========================= */

        .container {
            padding: 30px;
        }

        .heading {
            margin-bottom: 25px;
        }

        .heading h1 {
            margin-bottom: 8px;
            color: var(--text);
        }

        .heading p {
            color: var(--muted);
        }


        /* =========================
           SUMMARY CARDS
        ========================= */

        .cards {
            display: grid;

            grid-template-columns:
                repeat(3, 1fr);

            gap: 20px;

            margin-bottom: 30px;
        }
```

```
.card {
    background: var(--card);

    color: var(--text);

    padding: 22px;

    border-radius: 10px;

    border: 1px solid var(--border);

    box-shadow: var(--shadow);
}

.card h3 {
    color: var(--muted);

    margin-bottom: 12px;
}

.card .number {
    font-size: 30px;

    font-weight: bold;

    color: var(--primary);
}


/* ========================
   MESSAGE
======================== */

#message {
    display: none;

    padding: 12px;

    border-radius: 6px;

    margin-bottom: 20px;
}


/* ========================
   FORM
======================== */

.form-box {
    background: var(--card);

    color: var(--text);

    padding: 25px;

    border-radius: 10px;

    border: 1px solid var(--border);

    box-shadow: var(--shadow);

    margin-bottom: 30px;
}

.form-box h2 {
    margin-bottom: 20px;

    color: var(--text);
}

.form-grid {
    display: grid;

    grid-template-columns:
        repeat(3, 1fr);

    gap: 15px;
}

.form-group {
    display: flex;

    flex-direction: column;
}

.form-group label {
    margin-bottom: 6px;

    font-weight: bold;

    color: var(--text);
}

.form-group input,
.form-group select {
    padding: 11px;

    border: 1px solid var(--border);

    border-radius: 6px;

    font-size: 14px;

    background: var(--input);
```

```
    color: var(--text);
}

.form-group input:focus,
.form-group select:focus {
    outline: none;

    border-color: var(--primary);
}


/* ========================
   FORM BUTTONS
======================== */

.form-actions {
    margin-top: 20px;

    display: flex;

    gap: 10px;
}

.action-btn {
    border: none;

    padding: 11px 22px;

    border-radius: 6px;

    cursor: pointer;

    color: white;

    font-weight: bold;
}

.check-in-btn {
    background: #16a34a;
}

.check-in-btn:hover {
    background: #15803d;
}

.check-out-btn {
    background: #dc2626;
}

.check-out-btn:hover {
    background: #b91c1c;
}


/* ========================
   TABLE
======================== */

.table-box {
    background: var(--card);

    color: var(--text);

    padding: 25px;

    border-radius: 10px;

    border: 1px solid var(--border);

    box-shadow: var(--shadow);

    overflow-x: auto;
}

.table-box h2 {
    margin-bottom: 20px;

    color: var(--text);
}

table {
    width: 100%;

    border-collapse: collapse;

    min-width: 800px;

    background: var(--card);
}

th,
td {
    padding: 12px;

    border-bottom:
        1px solid var(--border);

    text-align: left;
}

th {
    background: var(--hover);

    color: var(--text);
}
```

```
        td {
            color: var(--text);
        }

        tr:hover {
            background: var(--hover);
        }


        /* ========================
           STATUS
        ======================== */

        .status {
            padding: 5px 10px;

            border-radius: 20px;

            font-size: 12px;

            font-weight: bold;
        }

        .present {
            background:
                rgba(22, 163, 74, 0.12);

            color: var(--success);
        }

        .absent {
            background:
                rgba(220, 38, 38, 0.12);

            color: var(--danger);
        }


        /* ========================
           NO DATA
        ======================== */

        .no-data {
            text-align: center;

            padding: 20px;

            color: var(--muted);
        }


        /* ========================
           RESPONSIVE
        ======================== */

        @media (max-width: 800px) {

            .cards {
                grid-template-columns: 1fr;
            }

            .form-grid {
                grid-template-columns: 1fr;
            }

            .container {
                padding: 15px;
            }

            .navbar {
                padding: 0 15px;
            }

            .nav-buttons {
                gap: 5px;
            }

            .nav-btn {
                padding: 8px 10px;
            }

        }

    </style>

</head>


<body>


    <!-- ========================
         NAVBAR
    ======================== -->

    <div class="navbar">

        <h2>HRGenius</h2>


        <div class="nav-buttons">


            <button
```

```
            class="nav-btn"
            onclick="window.location.href='dashboard.html'">

            Dashboard

        </button>


        <button
            id="themeToggle"
            class="theme-toggle"
            onclick="toggleTheme()">

            🌙 Dark Mode

        </button>


        <button
            class="nav-btn"
            onclick="logout()">

            Logout

        </button>


    </div>

</div>


<!-- ========================
     MAIN CONTAINER
======================== -->

<div class="container">


    <!-- ========================
         HEADING
    ======================== -->

    <div class="heading">

        <h1>
            Attendance Management
        </h1>

        <p>
            Manage employee attendance
            and check-in/check-out
        </p>

    </div>


    <!-- ========================
         MESSAGE
    ======================== -->

    <div id="message"></div>


    <!-- ========================
         SUMMARY CARDS
    ======================== -->

    <div class="cards">


        <div class="card">

            <h3>
                Total Attendance
            </h3>

            <div
                class="number"
                id="totalAttendance">

                0

            </div>

        </div>


        <div class="card">

            <h3>
                Present Today
            </h3>

            <div
                class="number"
                id="presentToday">

                0

            </div>

        </div>
```

```
    <div class="card">

        <h3>
            Attendance Records
        </h3>

        <div
            class="number"
            id="attendanceRecords">

            0

        </div>

    </div>


</div>


<!-- ========================
     MARK ATTENDANCE
======================== -->

<div class="form-box">

    <h2>
        Mark Attendance
    </h2>


    <div class="form-grid">


        <!-- EMPLOYEE ID -->

        <div class="form-group">

            <label for="employeeId">
                Employee ID
            </label>

            <input
                type="text"
                id="employeeId"
                placeholder="Enter Employee ID">

        </div>


        <!-- DATE -->

        <div class="form-group">

            <label for="attendanceDate">
                Date
            </label>

            <input
                type="date"
                id="attendanceDate">

        </div>


        <!-- STATUS -->

        <div class="form-group">

            <label for="status">
                Status
            </label>

            <select id="status">

                <option value="Present">
                    Present
                </option>

                <option value="Absent">
                    Absent
                </option>

            </select>

        </div>


    </div>


    <!-- ========================
         BUTTONS
    ======================== -->

    <div class="form-actions">


        <button
            type="button"
            class="action-btn check-in-btn"
            onclick="checkIn()">

            Check In
```

```
            </button>


            <button
                type="button"
                class="action-btn check-out-btn"
                onclick="checkOut()">

                Check Out

            </button>


        </div>


    </div>


    <!-- =========================
         ATTENDANCE TABLE
    ========================== -->

    <div class="table-box">

        <h2>
            Attendance Records
        </h2>


        <table>

            <thead>

                <tr>

                    <th>
                        Employee
                    </th>

                    <th>
                        Email
                    </th>

                    <th>
                        Department
                    </th>

                    <th>
                        Date
                    </th>

                    <th>
                        Status
                    </th>

                    <th>
                        Check In
                    </th>

                    <th>
                        Check Out
                    </th>

                </tr>

            </thead>


            <tbody id="attendanceTableBody">

                <tr>

                    <td
                        colspan="7"
                        class="no-data">

                        Loading attendance
                        records...

                    </td>

                </tr>

            </tbody>

        </table>

    </div>


</div>


<!-- =========================
     JAVASCRIPT FILES
========================== -->

<script src="JS/auth.js"></script>

<script src="JS/theme.js"></script>

<script src="JS/attendance.js"></script>
```

```
</body>
</html>
```

Frontend/candidates.html

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta
        name="viewport"
        content="width=device-width, initial-scale=1.0"
    >

    <title>HRGenius - Candidates</title>

    <link
        rel="stylesheet"
        href="CSS/style.css"
    >
    <link rel="stylesheet" href="CSS/candidates.css">

    <style>

        .candidate-page {
            padding: 30px;
        }

        .candidate-header {
            display: flex;
            justify-content: space-between;
            align-items: center;
            margin-bottom: 25px;
            gap: 20px;
            flex-wrap: wrap;
        }

        .candidate-header h1 {
            margin: 0;
        }

        .candidate-header p {
            margin-top: 6px;
            opacity: 0.7;
        }

        .stats-container {
            display: grid;
            grid-template-columns: repeat(3, 1fr);
            gap: 20px;
            margin-bottom: 30px;
        }

        .stat-card {
            padding: 22px;
            border-radius: 12px;
            background: var(--card-bg, #ffffff);
            border: 1px solid var(--border-color, #ddd);
        }

        .stat-card h3 {
            margin: 0;
            font-size: 15px;
            opacity: 0.7;
        }

        .stat-card .number {
            font-size: 30px;
            font-weight: bold;
            margin-top: 10px;
        }

        .form-card {
            padding: 25px;
            border-radius: 12px;
            background: var(--card-bg, #ffffff);
            border: 1px solid var(--border-color, #ddd);
            margin-bottom: 30px;
        }

        .form-card h2 {
            margin-top: 0;
        }

        .form-grid {
            display: grid;
            grid-template-columns: repeat(2, 1fr);
            gap: 18px;
        }

        .form-group {
            display: flex;
            flex-direction: column;
            gap: 7px;
        }

        .form-group.full {
            grid-column: 1 / -1;
        }

        .form-group label {
            font-weight: 600;
        }
```

```
.form-group input,
.form-group select,
.form-group textarea {
    padding: 12px;
    border-radius: 8px;
    border: 1px solid #ccc;
    font-size: 15px;
}

.file-info {
    font-size: 13px;
    opacity: 0.7;
}

.message {
    display: none;
    padding: 12px;
    border-radius: 8px;
    margin-bottom: 15px;
}

.message.success {
    display: block;
    background: #dcfce7;
    color: #166534;
}

.message.error {
    display: block;
    background: #fee2e2;
    color: #991b1b;
}

.table-card {
    padding: 25px;
    border-radius: 12px;
    background: var(--card-bg, #ffffff);
    border: 1px solid var(--border-color, #ddd);
    overflow-x: auto;
}

.table-card h2 {
    margin-top: 0;
}

table {
    width: 100%;
    border-collapse: collapse;
    min-width: 1150px;
}

th,
td {
    padding: 13px;
    text-align: left;
    border-bottom: 1px solid #ddd;
}

th {
    font-weight: 700;
}

.resume-actions {
    display: flex;
    flex-direction: column;
    gap: 7px;
    min-width: 110px;
}

.resume-actions a {
    display: inline-block;
    padding: 7px 10px;
    border-radius: 6px;
    text-decoration: none;
    text-align: center;
}

.view-resume {
    background: #2563eb;
    color: white;
}

.download-resume {
    background: #16a34a;
    color: white;
}

.status-select {
    padding: 7px;
    border-radius: 6px;
}

.delete-btn {
    background: #dc2626;
    color: white;
    border: none;
    padding: 8px 12px;
    border-radius: 6px;
    cursor: pointer;
}

.no-data {
    text-align: center;
    padding: 25px;
```

```
        }

        .loading {
            text-align: center;
            padding: 25px;
        }

        @media (max-width: 768px) {

            .candidate-page {
                padding: 15px;
            }

            .stats-container {
                grid-template-columns: 1fr;
            }

            .form-grid {
                grid-template-columns: 1fr;
            }

            .form-group.full {
                grid-column: auto;
            }

        }

    </style>

</head>


<body>

    <header>

        <h2>
            HRGenius
        </h2>

        <nav>

            <a href="dashboard.html">
                Dashboard
            </a>

            <a href="jobs.html">
                Jobs
            </a>

            <a href="candidates.html">
                Candidates
            </a>

            <button
                type="button"
                id="themeToggle"
                onclick="toggleTheme()"
            >
                🌙 Dark Mode
            </button>

            <button
                type="button"
                onclick="logout()"
            >
                Logout
            </button>

        </nav>

    </header>


    <main class="candidate-page">

        <div class="candidate-header">

            <div>

                <h1>
                    Candidate Management
                </h1>

                <p>
                    Manage job applicants and recruitment status
                </p>

            </div>

        </div>


        <div class="stats-container">

            <div class="stat-card">

                <h3>
                    Total Candidates
                </h3>

                <div
                    class="number"
                    id="totalCandidates"
                >
```

```
            0
        </div>

    </div>


    <div class="stat-card">

        <h3>
            Applied
        </h3>

        <div
            class="number"
            id="appliedCandidates"
        >
            0
        </div>

    </div>


    <div class="stat-card">

        <h3>
            Shortlisted
        </h3>

        <div
            class="number"
            id="shortlistedCandidates"
        >
            0
        </div>

    </div>


    <div class="stat-card">

        <h3>
            Interview
        </h3>

        <div
            class="number"
            id="interviewCandidates"
        >
            0
        </div>

    </div>


    <div class="stat-card">

        <h3>
            Selected
        </h3>

        <div
            class="number"
            id="selectedCandidates"
        >
            0
        </div>

    </div>


    <div class="stat-card">

        <h3>
            Rejected
        </h3>

        <div
            class="number"
            id="rejectedCandidates"
        >
            0
        </div>

    </div>

</div>


<div
    id="message"
    class="message"
></div>


<div class="form-card">

    <h2>
        Add New Candidate
    </h2>


    <form id="candidateForm">

        <div class="form-grid">
```

```
<div class="form-group">

    <label for="name">
        Candidate Name
    </label>

    <input
        type="text"
        id="name"
        required
    >

</div>


<div class="form-group">

    <label for="email">
        Email
    </label>

    <input
        type="email"
        id="email"
        required
    >

</div>


<div class="form-group">

    <label for="phone">
        Phone
    </label>

    <input
        type="text"
        id="phone"
        required
    >

</div>


<div class="form-group">

    <label for="appliedJob">
        Applied Job
    </label>

    <select
        id="appliedJob"
        required
    >

        <option value="">
            Loading Jobs...
        </option>

    </select>

</div>


<div class="form-group">

    <label for="skills">
        Skills
    </label>

    <input
        type="text"
        id="skills"
        placeholder="JavaScript, Node.js, MongoDB"
        required
    >

</div>


<div class="form-group">

    <label for="experience">
        Experience
    </label>

    <input
        type="text"
        id="experience"
        placeholder="Fresher / 1 Year"
        required
    >

</div>


<div class="form-group full">

    <label for="education">
        Education
    </label>

    <input
```

```
                    type="text"
                    id="education"
                    placeholder="B.Tech CSE"
                    required
                >

            </div>


            <div class="form-group full">

                <label for="resume">
                    Resume
                </label>

                <input
                    type="file"
                    id="resume"
                    name="resume"
                    accept=".pdf,.doc,.docx"
                    required
                >

                <span class="file-info">
                    Allowed: PDF, DOC, DOCX |
                    Maximum size: 5MB
                </span>

            </div>


            <div class="form-group full">

                <button
                    type="submit"
                    id="addCandidateBtn"
                >
                    Add Candidate
                </button>

            </div>

        </div>

    </form>

</div>


<div class="table-card">

    <h2>
        Candidate List
    </h2>


    <table>

        <thead>

            <tr>

                <th>Name</th>

                <th>Email</th>

                <th>Phone</th>

                <th>Skills</th>

                <th>Experience</th>

                <th>Education</th>

                <th>Applied Job</th>

                <th>Resume</th>

                <th>Status</th>

                <th>Action</th>

            </tr>

        </thead>


        <tbody id="candidateTableBody">

            <tr>

                <td
                    colspan="10"
                    class="loading"
                >
                    Loading candidates...
                </td>

            </tr>

        </tbody>

    </table>

</div>
```

```
</main>


<script src="JS/auth.js"></script>

<script src="JS/theme.js"></script>


<script>

    console.log(
        "Candidates page JavaScript started"
    );


    const CANDIDATE_API =
        "http://localhost:5000/api";


    const candidateToken =
        localStorage.getItem(
            "token"
        );


    let candidates = [];


    const candidateForm =
        document.getElementById(
            "candidateForm"
        );


    const messageBox =
        document.getElementById(
            "message"
        );


    function showMessage(
        text,
        type
    ) {

        messageBox.textContent =
            text;

        messageBox.className =
            `message ${type}`;

        setTimeout(
            function () {

                messageBox.className =
                    "message";

            },
            4000
        );
    }


    function escapeHTML(
        value
    ) {

        if (
            value === null ||
            value === undefined
        ) {
            return "";
        }


        return String(value)

            .replace(
                /&/g,
                "&amp;"
            )

            .replace(
                /</g,
                "&lt;"
            )

            .replace(
                />/g,
                "&gt;"
            )

            .replace(
                /"/g,
                "&quot;"
            )

            .replace(
                /'/g,
                "&#039;"
            );
    }
```

```
function getResumeURL(
    resume
) {

    if (!resume) {
        return "";
    }


    if (
        resume.startsWith(
            "http://"
        ) ||
        resume.startsWith(
            "https://"
        )
    ) {

        return resume;
    }


    let cleanPath =
        resume
            .replace(
                /\\/g,
                "/"
            );


    if (
        !cleanPath.startsWith("/")
    ) {

        cleanPath =
            "/" +
            cleanPath;

    }


    return (
        "http://localhost:5000" +
        cleanPath
    );
}


async function loadJobs() {

    console.log(
        "Loading jobs..."
    );


    const jobSelect =
        document.getElementById(
            "appliedJob"
        );


    try {

        const response =
            await fetch(
                `${CANDIDATE_API}/jobs`,
                {
                    method: "GET",

                    headers: {
                        "Authorization":
                            `Bearer ${candidateToken}`
                    }
                }
            );


        const data =
            await response.json();


        console.log(
            "Jobs Status:",
            response.status
        );


        console.log(
            "Jobs Response:",
            data
        );


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to load jobs"
            );
        }


        jobSelect.innerHTML =
            `
            <option value="">
```

```
                Select Job
            </option>
            `;


        const jobs =
            Array.isArray(
                data.jobs
            )
                ? data.jobs
                : [];


        jobs.forEach(
            function (job) {

                const option =
                    document.createElement(
                        "option"
                    );


                option.value =
                    job._id;


                option.textContent =
                    `${job.title} - ${job.department}`;


                jobSelect.appendChild(
                    option
                );

            }
        );


        if (
            jobs.length === 0
        ) {

            jobSelect.innerHTML =
                `
                <option value="">
                    No jobs available
                </option>
                `;

        }


    } catch (error) {

        console.error(
            "Load Jobs Error:",
            error
        );


        jobSelect.innerHTML =
            `
            <option value="">
                Unable to load jobs
            </option>
            `;

    }
}


async function loadCandidates() {

    console.log(
        "Loading candidates..."
    );


    const table =
        document.getElementById(
            "candidateTableBody"
        );


    try {

        table.innerHTML =
            `
            <tr>

                <td
                    colspan="10"
                    class="loading"
                >
                    Loading candidates...
                </td>

            </tr>
            `;


        const response =
            await fetch(
                `${CANDIDATE_API}/candidates`,
                {
```

```
                method: "GET",

                headers: {
                    "Authorization":
                        `Bearer ${candidateToken}`
                }
            }
        );


        const data =
            await response.json();


        console.log(
            "Candidates API Status:",
            response.status
        );


        console.log(
            "Candidates API Response:",
            data
        );


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to load candidates"
            );
        }


        candidates =
            Array.isArray(
                data.candidates
            )
                ? data.candidates
                : [];


        updateStats();

        displayCandidates();


    } catch (error) {

        console.error(
            "Load Candidates Error:",
            error
        );


        table.innerHTML =
            `

            <tr>

                <td
                    colspan="10"
                    class="no-data"
                >
                    ${escapeHTML(
                        error.message
                    )}
                </td>

            </tr>
            `;

    }
}


function updateStats() {

    document.getElementById(
        "totalCandidates"
    ).textContent =
        candidates.length;


    document.getElementById(
        "appliedCandidates"
    ).textContent =
        candidates.filter(
            function (candidate) {

                return (
                    candidate.status ===
                    "Applied"
                );

            }
        ).length;


    document.getElementById(
        "shortlistedCandidates"
    ).textContent =
        candidates.filter(
            function (candidate) {
```

```
                return (
                    candidate.status ===
                    "Shortlisted"
                );

            }
        ).length;


    document.getElementById(
        "interviewCandidates"
    ).textContent =
        candidates.filter(
            function (candidate) {

                return (
                    candidate.status ===
                    "Interview"
                );

            }
        ).length;


    document.getElementById(
        "selectedCandidates"
    ).textContent =
        candidates.filter(
            function (candidate) {

                return (
                    candidate.status ===
                    "Selected"
                );

            }
        ).length;


    document.getElementById(
        "rejectedCandidates"
    ).textContent =
        candidates.filter(
            function (candidate) {

                return (
                    candidate.status ===
                    "Rejected"
                );

            }
        ).length;

}


function displayCandidates() {

    const table =
        document.getElementById(
            "candidateTableBody"
        );


    if (
        candidates.length === 0
    ) {

        table.innerHTML =
            `

            <tr>

                <td
                    colspan="10"
                    class="no-data"
                >
                    No candidates found
                </td>

            </tr>
            `;

        return;
    }


    table.innerHTML =
        candidates
            .map(
                function (candidate) {

                    let resumeHTML =
                        "No Resume";


                    if (
                        candidate.resume
                    ) {

                        const resumeURL =
                            getResumeURL(
                                candidate.resume
                            );
```

```
    const fileName =
        candidate.resume
            .replace(
                /\\/g,
                "/"
            )
            .split("/")
            .pop();


    resumeHTML =
        `
        <div class="resume-actions">

            <a
                href="${resumeURL}"
                target="_blank"
                rel="noopener noreferrer"
                class="view-resume"
            >
                View Resume
            </a>

            <a
                href="${resumeURL}"
                download
                class="download-resume"
            >
                Download
            </a>

            <small>
                ${escapeHTML(
                    fileName
                )}
            </small>

        </div>
        `;

}


let jobTitle =
    "N/A";


if (
    candidate.appliedJob
) {

    jobTitle =
        candidate
            .appliedJob
            .title ||
        "N/A";

}


return `
<tr>

    <td>
        ${escapeHTML(
            candidate.name
        )}
    </td>

    <td>
        ${escapeHTML(
            candidate.email
        )}
    </td>

    <td>
        ${escapeHTML(
            candidate.phone
        )}
    </td>

    <td>
        ${escapeHTML(
            candidate.skills
        )}
    </td>

    <td>
        ${escapeHTML(
            candidate.experience
        )}
    </td>

    <td>
        ${escapeHTML(
            candidate.education
        )}
    </td>

    <td>
        ${escapeHTML(
            jobTitle
        )}
    </td>
```

```
                        <td>
                            ${resumeHTML}
                        </td>

                        <td>

                            <select
                                class="status-select"
                                onchange="updateCandidateStatus(
                                    '${candidate._id}',
                                    this.value
                                )"
                            >

                                <option
                                    value="Applied"
                                    ${
                                        candidate.status ===
                                        "Applied"
                                            ? "selected"
                                            : ""
                                    }
                                >
                                    Applied
                                </option>

                                <option
                                    value="Shortlisted"
                                    ${
                                        candidate.status ===
                                        "Shortlisted"
                                            ? "selected"
                                            : ""
                                    }
                                >
                                    Shortlisted
                                </option>

                                <option
                                    value="Interview"
                                    ${
                                        candidate.status ===
                                        "Interview"
                                            ? "selected"
                                            : ""
                                    }
                                >
                                    Interview
                                </option>

                                <option
                                    value="Selected"
                                    ${
                                        candidate.status ===
                                        "Selected"
                                            ? "selected"
                                            : ""
                                    }
                                >
                                    Selected
                                </option>

                                <option
                                    value="Rejected"
                                    ${
                                        candidate.status ===
                                        "Rejected"
                                            ? "selected"
                                            : ""
                                    }
                                >
                                    Rejected
                                </option>

                            </select>

                        </td>

                        <td>

                            <button
                                type="button"
                                class="delete-btn"
                                onclick="deleteCandidate(
                                    '${candidate._id}'
                                )"
                            >
                                Delete
                            </button>

                        </td>

                    </tr>
                    `;

                }
            )
            .join("");

}


candidateForm.addEventListener(
    "submit",
    async function (event) {
```

```
event.preventDefault();


const addButton =
    document.getElementById(
        "addCandidateBtn"
    );


try {

    addButton.disabled =
        true;


    addButton.textContent =
        "Uploading...";


    const resumeInput =
        document.getElementById(
            "resume"
        );


    const resume =
        resumeInput.files[0];


    if (!resume) {

        throw new Error(
            "Please select a resume"
        );

    }


    if (
        resume.size >
        5 * 1024 * 1024
    ) {

        throw new Error(
            "Resume size must be less than 5MB"
        );

    }


    const appliedJob =
        document.getElementById(
            "appliedJob"
        ).value;


    if (!appliedJob) {

        throw new Error(
            "Please select a job"
        );

    }


    const formData =
        new FormData();


    formData.append(
        "name",
        document.getElementById(
            "name"
        ).value.trim()
    );


    formData.append(
        "email",
        document.getElementById(
            "email"
        ).value.trim()
    );


    formData.append(
        "phone",
        document.getElementById(
            "phone"
        ).value.trim()
    );


    formData.append(
        "skills",
        document.getElementById(
            "skills"
        ).value.trim()
    );


    formData.append(
        "experience",
        document.getElementById(
```

```
            "experience"
        ).value.trim()
    );

    formData.append(
        "education",
        document.getElementById(
            "education"
        ).value.trim()
    );

    formData.append(
        "appliedJob",
        appliedJob
    );

    formData.append(
        "resume",
        resume
    );

    console.log(
        "Resume selected:",
        resume.name
    );

    console.log(
        "Resume size:",
        resume.size
    );

    console.log(
        "Sending candidate..."
    );

    const response =
        await fetch(
            `${CANDIDATE_API}/candidates`,
            {
                method:
                    "POST",

                headers: {
                    "Authorization":
                        `Bearer ${candidateToken}`
                },

                body:
                    formData
            }
        );

    const data =
        await response.json();

    console.log(
        "Candidate API Status:",
        response.status
    );

    console.log(
        "Candidate API Response:",
        data
    );

    if (!response.ok) {

        throw new Error(
            data.message ||
            "Failed to create candidate"
        );

    }

    showMessage(
        "Candidate added successfully",
        "success"
    );

    candidateForm.reset();

    await loadCandidates();

} catch (error) {

    console.error(
        "Candidate Upload Error:",
        error
    );
```

```
            showMessage(
                error.message,
                "error"
            );


        } finally {

            addButton.disabled =
                false;


            addButton.textContent =
                "Add Candidate";

        }

    }
);


async function updateCandidateStatus(
    id,
    status
) {

    try {

        const response =
            await fetch(
                `${CANDIDATE_API}/candidates/${id}/status`,
                {
                    method:
                        "PUT",

                    headers: {

                        "Authorization":
                            `Bearer ${candidateToken}`,

                        "Content-Type":
                            "application/json"

                    },

                    body:
                        JSON.stringify({
                            status:
                                status
                        })

                }
            );


        const data =
            await response.json();


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to update status"
            );

        }


        showMessage(
            "Candidate status updated successfully",
            "success"
        );


        await loadCandidates();


    } catch (error) {

        console.error(
            "Status Update Error:",
            error
        );


        showMessage(
            error.message,
            "error"
        );


        await loadCandidates();

    }
}


async function deleteCandidate(
    id
) {

    const confirmDelete =
        confirm(
```

```
            "Are you sure you want to delete this candidate?"
        );

    if (!confirmDelete) {
        return;
    }

    try {

        const response =
            await fetch(
                `${CANDIDATE_API}/candidates/${id}`,
                {
                    method:
                        "DELETE",

                    headers: {
                        "Authorization":
                            `Bearer ${candidateToken}`
                    }
                }
            );


        const data =
            await response.json();


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to delete candidate"
            );

        }


        showMessage(
            "Candidate deleted successfully",
            "success"
        );


        await loadCandidates();


    } catch (error) {

        console.error(
            "Delete Candidate Error:",
            error
        );


        showMessage(
            error.message,
            "error"
        );

    }
}


function logout() {

    localStorage.removeItem(
        "token"
    );

    localStorage.removeItem(
        "user"
    );


    window.location.href =
        "login.html";
}


document.addEventListener(
    "DOMContentLoaded",
    async function () {

        console.log(
            "DOM loaded"
        );


        if (!candidateToken) {

            window.location.href =
                "login.html";

            return;

        }


        await loadJobs();

        await loadCandidates();
```

```
            }
        );

    </script>

</body>

</html>
```

Frontend/CSS/candidates.css

```
/* ====================================================
   HRGENIUS CANDIDATES PAGE
==================================================== */

.candidate-page {
    width: 100%;
    max-width: 1400px;
    margin: 0 auto;
    padding: 35px;
}


/* ====================================================
   HEADER
==================================================== */

header {
    width: 100%;
    min-height: 70px;

    display: flex;
    align-items: center;
    justify-content: space-between;

    padding: 15px 35px;

    background: var(--topbar);
    color: var(--text);

    border-bottom: 1px solid var(--border);

    box-shadow: var(--shadow);

    position: relative;
    z-index: 10;
}

header h2 {
    margin: 0;

    color: var(--text);

    font-size: 25px;
    font-weight: 700;
}

header nav {
    display: flex;
    align-items: center;

    gap: 8px;

    flex-wrap: wrap;
}

header nav a {
    color: var(--text);

    text-decoration: none;

    padding: 9px 12px;

    border-radius: 7px;

    font-size: 14px;
    font-weight: 600;

    transition: 0.2s ease;
}

header nav a:hover {
    color: var(--primary);

    background: var(--hover);
}

header nav button {
    border: 1px solid var(--border);

    border-radius: 7px;

    padding: 9px 14px;

    background: var(--card);
    color: var(--text);

    cursor: pointer;

    font-size: 14px;
    font-weight: 600;

    transition: 0.2s ease;
}

header nav button:hover {
    background: var(--hover);

    border-color: var(--primary);
}


/* ====================================================
```

```
   PAGE HEADER
========================================================= */

.candidate-header {
    margin-bottom: 28px;
}

.candidate-header h1 {
    margin: 0 0 8px 0;

    color: var(--text);

    font-size: 36px;
    font-weight: 700;
}

.candidate-header p {
    margin: 0;

    color: var(--muted);

    font-size: 16px;
}


/* =========================================================
   STAT CARDS
========================================================= */

.stats-container {
    display: grid;

    grid-template-columns: repeat(3, minmax(0, 1fr));

    gap: 22px;

    margin-bottom: 30px;
}

.stat-card {
    min-height: 125px;

    padding: 25px;

    background: var(--card);

    color: var(--text);

    border: 1px solid var(--border);

    border-radius: 14px;

    box-shadow: var(--shadow);

    transition:
        transform 0.2s ease,
        border-color 0.2s ease;
}

.stat-card:hover {
    transform: translateY(-2px);

    border-color: var(--primary);
}

.stat-card h3 {
    margin: 0;

    color: var(--muted);

    font-size: 15px;
    font-weight: 600;
}

.stat-card .number {
    margin-top: 12px;

    color: var(--text);

    font-size: 34px;
    font-weight: 700;
}


/* =========================================================
   FORM CARD
========================================================= */

.form-card {
    width: 100%;

    padding: 30px;

    margin-bottom: 30px;

    background: var(--card);

    color: var(--text);

    border: 1px solid var(--border);

    border-radius: 14px;

    box-shadow: var(--shadow);
}
```

```
.form-card h2 {
    margin: 0 0 25px 0;

    color: var(--text);

    font-size: 25px;
}

.form-grid {
    display: grid;

    grid-template-columns: repeat(2, minmax(0, 1fr));

    gap: 20px;
}

.form-group {
    display: flex;

    flex-direction: column;

    gap: 8px;
}

.form-group.full {
    grid-column: 1 / -1;
}

.form-group label {
    color: var(--text);

    font-size: 14px;

    font-weight: 600;
}


/* =====================================================
   INPUTS
===================================================== */

.form-group input,
.form-group select,
.form-group textarea {
    width: 100%;

    padding: 13px 14px;

    background: var(--input);

    color: var(--text);

    border: 1px solid var(--border);

    border-radius: 8px;

    outline: none;

    font-family: inherit;

    font-size: 15px;

    transition:
        border-color 0.2s ease,
        box-shadow 0.2s ease;
}

.form-group input:focus,
.form-group select:focus,
.form-group textarea:focus {
    border-color: var(--primary);

    box-shadow:
        0 0 0 3px rgba(37, 99, 235, 0.12);
}

.form-group input::placeholder,
.form-group textarea::placeholder {
    color: var(--muted);
}

.form-group textarea {
    min-height: 100px;

    resize: vertical;
}

.form-group input[type="file"] {
    padding: 10px;

    cursor: pointer;
}


/* =====================================================
   FILE INFO
===================================================== */

.file-info {
    color: var(--muted);

    font-size: 13px;
}
```

```
/* ======================================================
   SUBMIT BUTTON
====================================================== */

.submit-btn {
    margin-top: 25px;

    padding: 12px 22px;

    background: var(--primary);

    color: #ffffff;

    border: none;

    border-radius: 8px;

    cursor: pointer;

    font-size: 15px;

    font-weight: 600;

    transition: 0.2s ease;
}

.submit-btn:hover {
    background: var(--primary-hover);
}


/* ======================================================
   MESSAGE
====================================================== */

.message {
    display: none;

    padding: 12px 15px;

    margin-bottom: 18px;

    border-radius: 8px;

    font-size: 14px;
}

.message.success {
    display: block;

    background: rgba(22, 163, 74, 0.12);

    color: var(--success);

    border: 1px solid rgba(22, 163, 74, 0.25);
}

.message.error {
    display: block;

    background: rgba(220, 38, 38, 0.12);

    color: var(--danger);

    border: 1px solid rgba(220, 38, 38, 0.25);
}


/* ======================================================
   TABLE CARD
====================================================== */

.table-card {
    width: 100%;

    padding: 30px;

    background: var(--card);

    color: var(--text);

    border: 1px solid var(--border);

    border-radius: 14px;

    box-shadow: var(--shadow);

    overflow-x: auto;
}

.table-card h2 {
    margin: 0 0 22px 0;

    color: var(--text);

    font-size: 25px;
}


/* ======================================================
   TABLE
====================================================== */

.table-card table {
```

```
    width: 100%;

    min-width: 1100px;

    border-collapse: collapse;

    background: var(--card);

    color: var(--text);
}

.table-card thead {
    background: var(--hover);
}

.table-card th {
    padding: 14px;

    text-align: left;

    color: var(--text);

    font-size: 14px;
    font-weight: 700;

    border-bottom: 1px solid var(--border);
}

.table-card td {
    padding: 14px;

    color: var(--text);

    font-size: 14px;

    border-bottom: 1px solid var(--border);
}

.table-card tbody tr {
    background: var(--card);

    transition: 0.2s ease;
}

.table-card tbody tr:hover {
    background: var(--hover);
}


/* ====================================================
   RESUME
==================================================== */

.resume-actions {
    display: flex;

    flex-direction: column;

    gap: 7px;

    min-width: 110px;
}

.resume-actions a {
    display: block;

    padding: 7px 10px;

    border-radius: 6px;

    text-align: center;

    text-decoration: none;

    font-size: 13px;
    font-weight: 600;
}

.view-resume {
    background: #2563eb;

    color: #ffffff;
}

.view-resume:hover {
    background: #1d4ed8;

    color: #ffffff;
}

.download-resume {
    background: #16a34a;

    color: #ffffff;
}

.download-resume:hover {
    background: #15803d;

    color: #ffffff;
}


/* ====================================================
   STATUS
```

```
==================================================== */

.status-select {
    padding: 7px 9px;

    background: var(--input);

    color: var(--text);

    border: 1px solid var(--border);

    border-radius: 6px;

    cursor: pointer;
}


/* ====================================================
   DELETE
==================================================== */

.delete-btn {
    padding: 8px 12px;

    background: var(--danger);

    color: #ffffff;

    border: none;

    border-radius: 6px;

    cursor: pointer;

    font-weight: 600;
}

.delete-btn:hover {
    opacity: 0.85;
}


/* ====================================================
   STATUS COLORS
==================================================== */

.status-applied {
    display: inline-block;

    padding: 5px 10px;

    border-radius: 20px;

    background: rgba(37, 99, 235, 0.12);

    color: var(--primary);
}

.status-shortlisted {
    display: inline-block;

    padding: 5px 10px;

    border-radius: 20px;

    background: rgba(217, 119, 6, 0.12);

    color: var(--warning);
}

.status-interview {
    display: inline-block;

    padding: 5px 10px;

    border-radius: 20px;

    background: rgba(124, 58, 237, 0.12);

    color: #8b5cf6;
}

.status-selected {
    display: inline-block;

    padding: 5px 10px;

    border-radius: 20px;

    background: rgba(22, 163, 74, 0.12);

    color: var(--success);
}

.status-rejected {
    display: inline-block;

    padding: 5px 10px;

    border-radius: 20px;

    background: rgba(220, 38, 38, 0.12);

    color: var(--danger);
}
```

```
/* ==================================================
   LOADING / NO DATA
================================================== */

.loading,
.no-data {
    padding: 30px;

    text-align: center;

    color: var(--muted);
}


/* ==================================================
   DARK MODE FORCE
================================================== */

html[data-theme="dark"] .stat-card,
html[data-theme="dark"] .form-card,
html[data-theme="dark"] .table-card {
    background: var(--card);

    color: var(--text);

    border-color: var(--border);
}

html[data-theme="dark"] .stat-card h3 {
    color: var(--muted);
}

html[data-theme="dark"] .stat-card .number {
    color: var(--text);
}


/* ==================================================
   RESPONSIVE
================================================== */

@media (max-width: 1000px) {

    .candidate-page {
        padding: 25px;
    }

    .stats-container {
        grid-template-columns: repeat(2, minmax(0, 1fr));
    }

    header {
        padding: 15px 25px;
    }
}


@media (max-width: 768px) {

    header {
        flex-direction: column;

        align-items: flex-start;

        gap: 12px;

        padding: 15px 20px;
    }

    header nav {
        width: 100%;
    }

    .candidate-page {
        padding: 20px;
    }

    .candidate-header h1 {
        font-size: 30px;
    }

    .stats-container {
        grid-template-columns: 1fr;
    }

    .form-grid {
        grid-template-columns: 1fr;
    }

    .form-group.full {
        grid-column: auto;
    }
}


@media (max-width: 500px) {

    .candidate-page {
        padding: 15px;
    }

    .candidate-header h1 {
        font-size: 26px;
```

```
    }

    .stat-card {
        padding: 20px;
    }

    .stat-card .number {
        font-size: 30px;
    }

    .form-card,
    .table-card {
        padding: 20px;
    }

    header h2 {
        font-size: 22px;
    }
}
```

Frontend/CSS/dashboard.css

```
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: var(--bg);
    color: var(--text);
}


/* ========================
   LOADER
======================== */

#pageLoader {
    position: fixed;
    inset: 0;
    z-index: 99999;
    background: var(--bg);
    display: flex;
    align-items: center;
    justify-content: center;
}

.loader-box {
    text-align: center;
}

.loader-box h3 {
    margin: 18px 0 5px;
}

.loader-box p {
    color: var(--muted);
    margin: 0;
}

.loader {
    width: 42px;
    height: 42px;
    border: 4px solid var(--border);
    border-top-color: var(--primary);
    border-radius: 50%;
    animation: spin 0.8s linear infinite;
    margin: auto;
}

@keyframes spin {
    to {
        transform: rotate(360deg);
    }
}


/* ========================
   SIDEBAR
======================== */

.sidebar {
    width: 255px;
    height: 100vh;
    position: fixed;
    left: 0;
    top: 0;
    z-index: 1000;
    background: var(--card);
    border-right: 1px solid var(--border);
    display: flex;
    flex-direction: column;
    padding: 20px 14px;
}

.logo {
    display: flex;
    align-items: center;
    gap: 12px;
    padding: 8px 12px 25px;
}

.logo-icon {
    width: 42px;
    height: 42px;
    border-radius: 12px;
    background: var(--primary);
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 22px;
    font-weight: bold;
}

.logo h2 {
    margin: 0;
    font-size: 20px;
}

.logo span {
    color: var(--muted);
```

```
    font-size: 11px;
}

.sidebar-menu {
    flex: 1;
    overflow-y: auto;
}

.menu-title {
    color: var(--muted);
    font-size: 10px;
    font-weight: 700;
    letter-spacing: 1px;
    padding: 14px 12px 7px;
    margin: 0;
}

.side-link {
    width: 100%;
    display: flex;
    align-items: center;
    gap: 12px;
    min-height: 43px;
    padding: 10px 13px;
    margin: 3px 0;
    border: 0;
    border-radius: 9px;
    background: transparent;
    color: var(--text);
    text-decoration: none;
    font-size: 14px;
    cursor: pointer;
    transition: 0.2s;
    text-align: left;
}

.side-link span:first-child {
    width: 22px;
    text-align: center;
}

.side-link:hover {
    background: var(--input);
    color: var(--primary);
}

.side-link.active {
    background: var(--primary);
    color: white;
    box-shadow: 0 5px 14px rgba(37, 99, 235, 0.2);
}

.nav-badge {
    margin-left: auto;
    min-width: 20px;
    padding: 2px 6px;
    border-radius: 10px;
    background: #ef4444;
    color: white;
    text-align: center;
    font-size: 10px;
}

.sidebar-bottom {
    border-top: 1px solid var(--border);
    padding-top: 12px;
}

.logout-link {
    color: #ef4444;
}


/* ========================
   MAIN
======================== */

.main-content {
    margin-left: 255px;
    min-height: 100vh;
}


/* ========================
   TOPBAR
======================== */

.topbar {
    height: 72px;
    position: sticky;
    top: 0;
    z-index: 900;
    background: var(--card);
    border-bottom: 1px solid var(--border);
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 0 30px;
}

.search-box {
    width: 360px;
    height: 40px;
    display: flex;
    align-items: center;
```

```
    gap: 10px;
    background: var(--input);
    border: 1px solid var(--border);
    border-radius: 9px;
    padding: 0 13px;
}

.search-box input {
    flex: 1;
    border: none;
    outline: none;
    background: transparent;
    color: var(--text);
    font-size: 13px;
}

.search-box input::placeholder {
    color: var(--muted);
}

.topbar-right {
    display: flex;
    align-items: center;
    gap: 15px;
}

.theme-toggle {
    border: 1px solid var(--border);
    background: var(--input);
    color: var(--text);
    padding: 9px 12px;
    border-radius: 8px;
    cursor: pointer;
}

.notification-btn {
    width: 40px;
    height: 40px;
    border: 1px solid var(--border);
    border-radius: 9px;
    background: var(--input);
    cursor: pointer;
    position: relative;
    font-size: 17px;
}

.notification-dot {
    width: 7px;
    height: 7px;
    background: #ef4444;
    border-radius: 50%;
    position: absolute;
    right: 8px;
    top: 7px;
    display: none;
}

.profile {
    display: flex;
    align-items: center;
    gap: 10px;
}

.profile-avatar {
    width: 40px;
    height: 40px;
    border-radius: 50%;
    background: var(--primary);
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    font-weight: bold;
}

.profile-info {
    display: flex;
    flex-direction: column;
}

.profile-info strong {
    font-size: 13px;
}

.profile-info span {
    color: var(--muted);
    font-size: 11px;
    text-transform: capitalize;
}

.mobile-logo {
    display: none;
}


/* ========================
   DASHBOARD
======================== */

.dashboard-container {
    padding: 28px;
    max-width: 1600px;
    margin: auto;
}
```

```
/* ========================
   WELCOME
======================== */

.welcome-section {
    background: linear-gradient(
        135deg,
        var(--primary),
        #4f46e5
    );
    color: white;
    border-radius: 16px;
    padding: 27px 30px;
    display: flex;
    align-items: center;
    justify-content: space-between;
    margin-bottom: 22px;
    box-shadow: 0 12px 30px rgba(37, 99, 235, 0.16);
}

.welcome-badge {
    display: inline-flex;
    gap: 7px;
    align-items: center;
    padding: 6px 11px;
    background: rgba(255, 255, 255, 0.14);
    border: 1px solid rgba(255, 255, 255, 0.2);
    border-radius: 20px;
    font-size: 11px;
    margin-bottom: 12px;
}

.welcome-section h1 {
    margin: 0 0 7px;
    color: white;
    font-size: 27px;
}

.welcome-section p {
    margin: 0;
    color: white;
    opacity: 0.92;
    font-size: 14px;
}

.date-box {
    display: flex;
    align-items: center;
    gap: 12px;
    background: rgba(255, 255, 255, 0.12);
    border: 1px solid rgba(255, 255, 255, 0.2);
    padding: 12px 16px;
    border-radius: 11px;
    min-width: 170px;
}

.date-icon {
    font-size: 23px;
}

.date-box div {
    display: flex;
    flex-direction: column;
}

.date-box small {
    opacity: 0.75;
    font-size: 9px;
}

.date-box strong {
    font-size: 13px;
    margin-top: 3px;
    color: white;
}


/* ========================
   STATS
======================== */

.stats-grid {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 17px;
    margin-bottom: 20px;
}

.stat-card {
    background: var(--card);
    border: 1px solid var(--border);
    border-radius: 13px;
    padding: 20px;
    transition: 0.2s;
}

.stat-card:hover {
    transform: translateY(-3px);
    box-shadow: var(--shadow);
}

.stat-top {
    display: flex;
    align-items: center;
```

```
    gap: 10px;
}

.stat-icon {
    width: 38px;
    height: 38px;
    border-radius: 10px;
    display: flex;
    align-items: center;
    justify-content: center;
}

.employees-icon {
    background: rgba(37, 99, 235, 0.12);
}

.jobs-icon {
    background: rgba(16, 185, 129, 0.12);
}

.candidates-icon {
    background: rgba(245, 158, 11, 0.12);
}

.payroll-icon {
    background: rgba(139, 92, 246, 0.12);
}

.stat-label {
    color: var(--muted);
    font-size: 13px;
}

.stat-number {
    font-size: 28px;
    font-weight: 700;
    margin: 15px 0 10px;
}

.stat-bottom {
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.stat-green {
    color: #16a34a;
    font-size: 11px;
}

.stat-arrow {
    color: var(--muted);
}


/* ========================
   CARDS
======================== */

.content-grid {
    display: grid;
    grid-template-columns: 1.15fr 1fr;
    gap: 18px;
    margin-bottom: 18px;
}

.bottom-grid {
    display: grid;
    grid-template-columns: 1.15fr 1fr;
    gap: 18px;
    margin-bottom: 18px;
}

.dashboard-card {
    background: var(--card);
    border: 1px solid var(--border);
    border-radius: 13px;
    padding: 21px;
    box-shadow: var(--shadow);
}

.card-header {
    display: flex;
    align-items: center;
    justify-content: space-between;
    margin-bottom: 22px;
}

.card-header h2 {
    margin: 0 0 5px;
    font-size: 17px;
}

.card-header p {
    margin: 0;
    color: var(--muted);
    font-size: 11px;
}

.view-btn {
    text-decoration: none;
    color: var(--primary);
    font-size: 12px;
    font-weight: 600;
}
```

```
/* ========================
   EMPLOYEE
======================== */

.overview-content {
    display: flex;
    align-items: center;
    gap: 35px;
}

.overview-circle {
    width: 150px;
    height: 150px;
    border-radius: 50%;
    background: conic-gradient(
        var(--primary) 0 75%,
        var(--border) 75% 100%
    );
    display: flex;
    align-items: center;
    justify-content: center;
}

.circle-inner {
    width: 112px;
    height: 112px;
    background: var(--card);
    border-radius: 50%;
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
}

.circle-inner strong {
    font-size: 27px;
}

.circle-inner span {
    color: var(--muted);
    font-size: 11px;
    margin-top: 3px;
}

.overview-details {
    flex: 1;
    display: grid;
    gap: 17px;
}

.overview-item {
    display: flex;
    align-items: center;
    gap: 11px;
}

.overview-dot {
    width: 9px;
    height: 9px;
    border-radius: 50%;
}

.overview-dot.blue {
    background: #3b82f6;
}

.overview-dot.green {
    background: #22c55e;
}

.overview-dot.orange {
    background: #f59e0b;
}

.overview-item div {
    display: flex;
    flex-direction: column;
}

.overview-item strong {
    font-size: 15px;
}

.overview-item small {
    color: var(--muted);
    font-size: 10px;
    margin-top: 2px;
}


/* ========================
   RECRUITMENT
======================== */

.recruitment-list {
    display: grid;
    gap: 5px;
}

.recruitment-item {
    display: flex;
    align-items: center;
    justify-content: space-between;
```

```
    font-size: 13px;
    margin-top: 4px;
}

.recruitment-name {
    display: flex;
    align-items: center;
    gap: 8px;
}

.status-dot {
    width: 8px;
    height: 8px;
    border-radius: 50%;
}

.status-dot.applied {
    background: #3b82f6;
}

.status-dot.shortlisted {
    background: #f59e0b;
}

.status-dot.interview {
    background: #8b5cf6;
}

.status-dot.selected {
    background: #22c55e;
}

.progress-line {
    width: 100%;
    height: 5px;
    background: var(--input);
    border-radius: 10px;
    margin: 2px 0 10px;
    overflow: hidden;
}

.progress-bar {
    height: 100%;
    width: 0;
    border-radius: 10px;
    transition: width 0.7s ease;
}

.applied-bar {
    background: #3b82f6;
}

.shortlisted-bar {
    background: #f59e0b;
}

.interview-bar {
    background: #8b5cf6;
}

.selected-bar {
    background: #22c55e;
}


/* ========================
   HR ANALYTICS
======================== */

.analytics-section {
    margin: 24px 0;
}

.analytics-section > .section-header {
    margin-bottom: 15px;
}

.section-header {
    display: flex;
    align-items: center;
    justify-content: space-between;
}

.section-header h2 {
    margin: 0 0 5px;
    font-size: 19px;
}

.section-header p {
    margin: 0;
    color: var(--muted);
    font-size: 12px;
}

.analytics-grid {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 17px;
}

.analytics-card {
    background: var(--card);
    border: 1px solid var(--border);
    border-radius: 13px;
    padding: 20px;
```

```
    min-width: 0;
    transition: 0.2s;
}

.analytics-card:hover {
    transform: translateY(-3px);
    box-shadow: var(--shadow);
}

.analytics-card-header {
    display: flex;
    align-items: center;
    gap: 11px;
}

.analytics-card-header h3 {
    margin: 0 0 4px;
    font-size: 14px;
}

.analytics-card-header span:last-child {
    color: var(--muted);
    font-size: 10px;
}

.analytics-icon {
    width: 40px;
    height: 40px;
    border-radius: 10px;
    background: var(--input);
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 19px;
}

.analytics-value {
    margin: 20px 0 13px;
}

.analytics-value span {
    font-size: 27px;
    font-weight: 700;
}

.analytics-progress {
    width: 100%;
    height: 8px;
    background: var(--input);
    border-radius: 10px;
    overflow: hidden;
    margin-bottom: 15px;
}

.analytics-progress-fill {
    width: 0;
    height: 100%;
    border-radius: 10px;
    background: var(--primary);
    transition: width 0.7s ease;
}

.analytics-details {
    display: flex;
    flex-wrap: wrap;
    gap: 8px 15px;
}

.analytics-details span {
    color: var(--muted);
    font-size: 11px;
}

.analytics-details strong {
    color: var(--text);
    font-size: 12px;
}


/* ========================
   RECRUITMENT MINI LIST
======================== */

.recruitment-mini-list {
    display: grid;
    gap: 8px;
    margin-top: 8px;
}

.recruitment-mini-list div {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 7px 0;
    border-bottom: 1px solid var(--border);
}

.recruitment-mini-list div:last-child {
    border-bottom: none;
}

.recruitment-mini-list span {
    color: var(--muted);
    font-size: 11px;
}
```

```
.recruitment-mini-list strong {
    font-size: 12px;
}


/* ========================
   DEPARTMENT ANALYTICS
======================== */

.department-analytics {
    background: var(--card);
    border: 1px solid var(--border);
    border-radius: 13px;
    padding: 21px;
    margin-top: 18px;
    box-shadow: var(--shadow);
}

.department-analytics .section-header {
    margin-bottom: 18px;
}

.department-list {
    display: grid;
    gap: 12px;
}

.department-item {
    display: grid;
    grid-template-columns: minmax(130px, 1fr) 2fr auto;
    align-items: center;
    gap: 15px;
    padding: 12px 0;
    border-bottom: 1px solid var(--border);
}

.department-item:last-child {
    border-bottom: none;
}

.department-name {
    font-weight: 600;
    font-size: 13px;
}

.department-count {
    color: var(--muted);
    font-size: 11px;
    margin-top: 3px;
}

.department-progress {
    width: 100%;
    height: 7px;
    background: var(--input);
    border-radius: 10px;
    overflow: hidden;
}

.department-progress-fill {
    height: 100%;
    width: 0;
    border-radius: 10px;
    background: var(--primary);
    transition: width 0.7s ease;
}

.department-salary {
    text-align: right;
    white-space: nowrap;
}

.department-salary strong {
    display: block;
    font-size: 12px;
}

.department-salary span {
    color: var(--muted);
    font-size: 10px;
}

.analytics-empty {
    text-align: center;
    padding: 25px 10px;
    color: var(--muted);
    font-size: 12px;
}


/* ========================
   QUICK ACTIONS
======================== */

.quick-actions {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 10px;
}

.quick-action {
    display: flex;
    align-items: center;
    gap: 10px;
```

```
    text-decoration: none;
    color: var(--text);
    border: 1px solid var(--border);
    padding: 12px;
    border-radius: 9px;
    transition: 0.2s;
}

.quick-action:hover {
    border-color: var(--primary);
    background: var(--input);
    transform: translateY(-2px);
}

.quick-icon {
    width: 35px;
    height: 35px;
    display: flex;
    align-items: center;
    justify-content: center;
    background: var(--input);
    border-radius: 8px;
}

.quick-action div {
    flex: 1;
    display: flex;
    flex-direction: column;
}

.quick-action strong {
    font-size: 12px;
}

.quick-action small {
    color: var(--muted);
    font-size: 10px;
    margin-top: 3px;
}


/* ========================
   SUMMARY
======================== */

.summary-list {
    display: grid;
}

.summary-row {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 12px 0;
    border-bottom: 1px solid var(--border);
}

.summary-row:last-child {
    border-bottom: none;
}

.summary-left {
    display: flex;
    align-items: center;
    gap: 10px;
    font-size: 12px;
}

.summary-icon {
    width: 32px;
    height: 32px;
    background: var(--input);
    border-radius: 8px;
    display: flex;
    align-items: center;
    justify-content: center;
}

.summary-row strong {
    font-size: 13px;
}


/* ========================
   ACTIVITY
======================== */

.activity-card {
    margin-bottom: 25px;
}

.activity-list {
    display: grid;
}

.activity-item {
    display: flex;
    align-items: center;
    gap: 13px;
    padding: 14px 0;
    border-bottom: 1px solid var(--border);
}

.activity-item:last-child {
    border-bottom: none;
```

```
}

.activity-icon {
    width: 38px;
    height: 38px;
    border-radius: 9px;
    background: var(--input);
    display: flex;
    align-items: center;
    justify-content: center;
}

.activity-content {
    flex: 1;
    display: flex;
    flex-direction: column;
    gap: 4px;
}

.activity-content strong {
    font-size: 12px;
}

.activity-content span {
    color: var(--muted);
    font-size: 11px;
}

.activity-item small {
    color: var(--muted);
    font-size: 10px;
}


/* ========================
   RESPONSIVE
======================== */

@media (max-width: 1200px) {

    .stats-grid {
        grid-template-columns: repeat(2, 1fr);
    }

    .analytics-grid {
        grid-template-columns: repeat(2, 1fr);
    }

    .content-grid,
    .bottom-grid {
        grid-template-columns: 1fr;
    }

}


@media (max-width: 850px) {

    .sidebar {
        width: 210px;
    }

    .main-content {
        margin-left: 210px;
    }

    .topbar {
        padding: 0 18px;
    }

    .search-box {
        width: 250px;
    }

    .department-item {
        grid-template-columns: 1fr;
        gap: 8px;
    }

    .department-salary {
        text-align: left;
    }

}


@media (max-width: 700px) {

    .sidebar {
        display: none;
    }

    .main-content {
        margin-left: 0;
    }

    .mobile-logo {
        display: flex;
        align-items: center;
        gap: 7px;
    }

    .mobile-logo .logo-icon {
        width: 32px;
        height: 32px;
```

```
    }

    .search-box {
        display: none;
    }

    .profile-info {
        display: none;
    }

    .theme-toggle {
        font-size: 0;
        padding: 9px;
    }

    .theme-toggle::first-letter {
        font-size: 15px;
    }

    .dashboard-container {
        padding: 17px;
    }

    .welcome-section {
        flex-direction: column;
        align-items: flex-start;
        gap: 20px;
        padding: 22px;
    }

    .date-box {
        width: 100%;
    }

    .stats-grid {
        grid-template-columns: 1fr;
    }

    .analytics-grid {
        grid-template-columns: 1fr;
    }

    .overview-content {
        flex-direction: column;
    }

    .quick-actions {
        grid-template-columns: 1fr;
    }

}


@media (max-width: 450px) {

    .topbar {
        height: 62px;
    }

    .topbar-right {
        gap: 7px;
    }

    .notification-btn {
        width: 35px;
        height: 35px;
    }

    .profile-avatar {
        width: 35px;
        height: 35px;
    }

    .welcome-section h1 {
        font-size: 22px;
    }

    .stat-number {
        font-size: 25px;
    }

    .analytics-value span {
        font-size: 24px;
    }

    .dashboard-card,
    .analytics-card,
    .department-analytics {
        padding: 16px;
    }

}
```

Frontend/CSS/departments.css

```
/* =====================================================
   HRGENIUS - DEPARTMENTS PAGE
   COMPLETE PAGE CSS
===================================================== */


/* =====================================================
   BASIC RESET
===================================================== */

html,
body {
    margin: 0;
    padding: 0;
    width: 100%;
    min-height: 100%;
}

* {
    box-sizing: border-box;
}

body {
    background: var(--bg, #f5f7fb);
    color: var(--text, #111827);
    font-family: Arial, Helvetica, sans-serif;
}


/* =====================================================
   TOPBAR
===================================================== */

.topbar {
    width: 100%;
    height: 64px;
    padding: 0 30px;

    display: flex;
    align-items: center;
    justify-content: space-between;

    background: var(--topbar, #ffffff);
    border-bottom: 1px solid var(--border, #d1d5db);

    position: relative;
    z-index: 1000;
}

.brand {
    display: flex;
    align-items: center;
}

.brand h2 {
    margin: 0;
    padding: 0;

    color: var(--text, #111827);
    font-size: 25px;
    font-weight: 800;
    line-height: 1;
}

.topbar-actions {
    display: flex;
    align-items: center;
    gap: 10px;
}


/* =====================================================
   THEME BUTTON
===================================================== */

.theme-btn {
    width: 40px;
    height: 40px;

    display: flex;
    align-items: center;
    justify-content: center;

    padding: 0;

    border: 1px solid var(--border, #d1d5db);
    border-radius: 8px;

    background: var(--input, #ffffff);
    color: var(--text, #111827);

    font-size: 18px;
    cursor: pointer;

    transition: 0.2s ease;
}

.theme-btn:hover {
    background: var(--hover, #f3f4f6);
}
```

```
/* ====================================================
   LOGOUT BUTTON
==================================================== */

.logout-btn {
    min-width: 80px;
    height: 40px;

    padding: 0 15px;

    border: 1px solid var(--border, #d1d5db);
    border-radius: 8px;

    background: var(--input, #ffffff);
    color: var(--text, #111827);

    font-family: inherit;
    font-size: 14px;
    font-weight: 600;

    cursor: pointer;

    transition: 0.2s ease;
}

.logout-btn:hover {
    background: var(--hover, #f3f4f6);
    color: var(--danger, #dc2626);
}


/* ====================================================
   MAIN PAGE
==================================================== */

.department-page {
    width: 100%;
    max-width: 1600px;

    margin: 0 auto;
    padding: 32px;

    min-height: calc(100vh - 64px);
}


/* ====================================================
   PAGE HEADER
==================================================== */

.page-header {
    display: flex;
    align-items: center;
    justify-content: space-between;

    gap: 20px;

    margin-bottom: 30px;
}

.page-title {
    min-width: 0;
}

.page-header h1 {
    margin: 0 0 8px;

    color: var(--text, #111827);

    font-size: 32px;
    font-weight: 750;

    line-height: 1.2;
}

.page-header p {
    margin: 0;

    color: var(--muted, #6b7280);

    font-size: 15px;
    line-height: 1.5;
}


/* ====================================================
   BUTTONS
==================================================== */

.primary-btn,
.secondary-btn,
.danger-btn {
    min-height: 42px;

    padding: 10px 17px;

    border-radius: 8px;

    font-family: inherit;
    font-size: 14px;
    font-weight: 600;

    cursor: pointer;

    transition: 0.2s ease;
```

```
}

.primary-btn {
    border: 1px solid var(--primary, #2563eb);

    background: var(--primary, #2563eb);
    color: #ffffff;
}

.primary-btn:hover {
    background: var(--primary-hover, #1d4ed8);
}

.secondary-btn {
    border: 1px solid var(--border, #d1d5db);

    background: var(--input, #ffffff);
    color: var(--text, #111827);
}

.secondary-btn:hover {
    background: var(--hover, #f3f4f6);
}

.danger-btn {
    border: 1px solid var(--danger, #dc2626);

    background: var(--danger, #dc2626);
    color: #ffffff;
}

.danger-btn:hover {
    opacity: 0.9;
}


/* ====================================================
   STATISTICS
==================================================== */

.stats-grid {
    display: grid;

    grid-template-columns:
        repeat(4, minmax(0, 1fr));

    gap: 20px;

    margin-bottom: 25px;
}

.stat-card {
    min-width: 0;

    display: flex;
    align-items: center;

    gap: 16px;

    padding: 22px;

    background: var(--card, #ffffff);

    border: 1px solid var(--border, #d1d5db);
    border-radius: 12px;

    box-shadow:
        var(--shadow,
        0 8px 25px rgba(0, 0, 0, 0.08));
}

.stat-icon {
    width: 50px;
    height: 50px;

    min-width: 50px;

    display: flex;
    align-items: center;
    justify-content: center;

    border-radius: 10px;

    background: var(--hover, #f3f4f6);

    font-size: 24px;
}

.stat-content {
    min-width: 0;
}

.stat-card span {
    display: block;

    margin-bottom: 6px;

    color: var(--muted, #6b7280);

    font-size: 13px;
}

.stat-card strong {
    display: block;
```

```
    color: var(--text, #111827);

    font-size: 22px;
}


/* ====================================================
   FILTER SECTION
==================================================== */

.filter-section {
    display: flex;

    align-items: flex-end;
    justify-content: space-between;

    gap: 18px;

    margin-bottom: 20px;

    padding: 16px;

    background: var(--card, #ffffff);

    border: 1px solid var(--border, #d1d5db);
    border-radius: 12px;

    box-shadow:
        var(--shadow,
        0 8px 25px rgba(0, 0, 0, 0.08));
}


/* ====================================================
   SEARCH AREA
==================================================== */

.search-area {
    flex: 1;
    min-width: 0;
}

.search-box {
    position: relative;

    width: 100%;
    height: 46px;

    display: flex;
    align-items: center;

    gap: 10px;

    padding: 0 12px;

    background: var(--input, #ffffff);

    border: 1px solid var(--border, #d1d5db);
    border-radius: 9px;

    transition: 0.2s ease;
}

.search-box:focus-within {
    border-color: var(--primary, #2563eb);

    box-shadow:
        0 0 0 3px
        rgba(37, 99, 235, 0.10);
}

.search-icon {
    width: 22px;
    min-width: 22px;

    display: flex;
    align-items: center;
    justify-content: center;

    color: var(--muted, #6b7280);

    font-size: 17px;
}

.search-box input {
    flex: 1;

    width: 100%;
    height: 100%;

    min-width: 0;

    padding: 0;

    border: none;
    outline: none;

    background: transparent;

    color: var(--text, #111827);

    font-family: inherit;
    font-size: 14px;
}
```

```
.search-box input::placeholder {
    color: var(--muted, #6b7280);
}


/* ====================================================
   CLEAR SEARCH
==================================================== */

.clear-search-btn {
    width: 28px;
    height: 28px;

    display: flex;
    align-items: center;
    justify-content: center;

    padding: 0;

    border: none;
    border-radius: 50%;

    background: var(--hover, #f3f4f6);

    color: var(--muted, #6b7280);

    font-size: 19px;

    cursor: pointer;
}

.clear-search-btn:hover {
    background: var(--border, #d1d5db);

    color: var(--text, #111827);
}


/* ====================================================
   SEARCH SHORTCUT
==================================================== */

.search-shortcut {
    display: inline-flex;
    align-items: center;
    justify-content: center;

    height: 25px;

    padding: 0 8px;

    border: 1px solid var(--border, #d1d5db);
    border-radius: 5px;

    background: var(--hover, #f3f4f6);

    color: var(--muted, #6b7280);

    font-size: 10px !important;
    font-weight: 600;

    white-space: nowrap;
}

.search-result-text {
    margin-top: 7px;

    color: var(--muted, #6b7280);

    font-size: 12px;
}


/* ====================================================
   FILTER RIGHT
==================================================== */

.filter-right {
    display: flex;

    align-items: flex-end;

    gap: 10px;

    flex-shrink: 0;
}

.status-filter {
    display: flex;

    flex-direction: column;

    gap: 6px;
}

.status-filter label {
    color: var(--muted, #6b7280);

    font-size: 12px;
    font-weight: 600;
}

.status-filter select {
    width: 155px;
    height: 46px;
```

```
    padding: 0 12px;

    border: 1px solid var(--border, #d1d5db);
    border-radius: 9px;

    background: var(--input, #ffffff);
    color: var(--text, #111827);

    outline: none;

    font-family: inherit;
    font-size: 14px;

    cursor: pointer;
}

.status-filter select:focus {
    border-color: var(--primary, #2563eb);

    box-shadow:
        0 0 0 3px
        rgba(37, 99, 235, 0.10);
}

.reset-btn {
    height: 46px;

    white-space: nowrap;
}


/* ====================================================
   TABLE CARD
==================================================== */

.table-card {
    width: 100%;

    background: var(--card, #ffffff);

    border: 1px solid var(--border, #d1d5db);
    border-radius: 12px;

    overflow: hidden;

    box-shadow:
        var(--shadow,
        0 8px 25px rgba(0, 0, 0, 0.08));
}


/* ====================================================
   TABLE HEADER
==================================================== */

.table-header {
    padding: 20px 22px;

    border-bottom: 1px solid var(--border, #d1d5db);
}

.table-header h2 {
    margin: 0 0 5px;

    color: var(--text, #111827);

    font-size: 20px;
}

.table-header p {
    margin: 0;

    color: var(--muted, #6b7280);

    font-size: 13px;
}


/* ====================================================
   TABLE
==================================================== */

.table-wrapper {
    width: 100%;

    overflow-x: auto;

    -webkit-overflow-scrolling: touch;
}

table {
    width: 100%;

    min-width: 1050px;

    border-collapse: collapse;
}

thead {
    background: var(--hover, #f3f4f6);
}

th {
    padding: 14px 16px;
```

```
    text-align: left;

    color: var(--muted, #6b7280);

    font-size: 12px;
    font-weight: 700;

    text-transform: uppercase;

    white-space: nowrap;
}

td {
    padding: 16px;

    border-top: 1px solid var(--border, #d1d5db);

    color: var(--text, #111827);

    font-size: 14px;

    vertical-align: middle;
}

tbody tr {
    transition: 0.15s ease;
}

tbody tr:hover {
    background: var(--hover, #f3f4f6);
}


/* ======================================================
   SORT BUTTON
====================================================== */

.sort-btn {
    display: inline-flex;
    align-items: center;

    gap: 5px;

    padding: 0;

    border: none;

    background: transparent;

    color: var(--muted, #6b7280);

    font-family: inherit;
    font-size: 12px;
    font-weight: 700;

    text-transform: uppercase;

    cursor: pointer;
}

.sort-btn:hover {
    color: var(--primary, #2563eb);
}

.sort-btn span {
    font-size: 13px;
}


/* ======================================================
   DEPARTMENT CELL
====================================================== */

.department-name {
    margin-bottom: 4px;

    color: var(--text, #111827);

    font-weight: 700;
}

.department-description {
    max-width: 260px;

    overflow: hidden;

    color: var(--muted, #6b7280);

    font-size: 12px;

    white-space: nowrap;

    text-overflow: ellipsis;
}


/* ======================================================
   HEAD
====================================================== */

.head-info {
    display: flex;
    align-items: center;
```

```
    gap: 10px;
}

.avatar {
    width: 36px;
    height: 36px;

    min-width: 36px;

    display: flex;
    align-items: center;
    justify-content: center;

    border-radius: 50%;

    background: var(--primary, #2563eb);

    color: #ffffff;

    font-size: 13px;
    font-weight: 700;
}

.head-name {
    color: var(--text, #111827);

    font-weight: 600;
}

.head-designation {
    margin-top: 2px;

    color: var(--muted, #6b7280);

    font-size: 12px;
}


/* ======================================================
   EMPLOYEE COUNT
====================================================== */

.employee-count-btn {
    display: inline-flex;
    align-items: center;

    gap: 7px;

    padding: 0;

    border: none;

    background: transparent;

    color: var(--primary, #2563eb);

    font-weight: 700;

    cursor: pointer;
}

.employee-count-btn:hover {
    text-decoration: underline;
}

.employee-count-btn span {
    font-size: 12px;
    font-weight: 500;
}


/* ======================================================
   STATUS
====================================================== */

.status-badge {
    display: inline-flex;
    align-items: center;

    padding: 5px 10px;

    border-radius: 20px;

    font-size: 12px;
    font-weight: 700;
}

.status-active {
    background: rgba(22, 163, 74, 0.12);

    color: var(--success, #16a34a);
}

.status-inactive {
    background: rgba(220, 38, 38, 0.12);

    color: var(--danger, #dc2626);
}


/* ======================================================
   ACTIONS
====================================================== */

.action-buttons {
```

```
    display: flex;
    align-items: center;

    gap: 7px;

    flex-wrap: wrap;
}

.action-btn {
    min-width: 34px;
    min-height: 32px;

    padding: 7px 9px;

    border: 1px solid var(--border, #d1d5db);
    border-radius: 6px;

    background: var(--input, #ffffff);

    color: var(--text, #111827);

    font-family: inherit;
    font-size: 12px;

    cursor: pointer;

    transition: 0.2s ease;
}

.action-btn:hover {
    background: var(--hover, #f3f4f6);
}

.action-btn.view:hover,
.action-btn.edit:hover {
    color: var(--primary, #2563eb);
}

.action-btn.head:hover {
    color: var(--warning, #d97706);
}

.action-btn.delete {
    color: var(--danger, #dc2626);
}

.action-btn.delete:hover {
    background: rgba(220, 38, 38, 0.10);
}


/* ======================================================
   EMPTY STATE
====================================================== */

.empty-state {
    padding: 60px 20px !important;

    text-align: center;
}

.empty-state-icon {
    margin-bottom: 12px;

    font-size: 42px;
}

.empty-state h3 {
    margin: 0 0 7px;

    color: var(--text, #111827);
}

.empty-state p {
    margin: 0;

    color: var(--muted, #6b7280);
}


/* ======================================================
   MODALS
====================================================== */

.modal-overlay {
    position: fixed;

    inset: 0;

    z-index: 9999;

    display: none;

    align-items: center;
    justify-content: center;

    padding: 20px;

    background: rgba(0, 0, 0, 0.55);
}

.modal-overlay.show {
    display: flex;
}
```

```
.modal {
    width: min(520px, 100%);

    max-height: 90vh;

    overflow-y: auto;

    padding: 25px;

    background: var(--card, #ffffff);

    border: 1px solid var(--border, #d1d5db);

    border-radius: 14px;

    box-shadow:
        0 20px 50px
        rgba(0, 0, 0, 0.20);
}

.large-modal {
    width: min(950px, 100%);
}

.details-modal {
    width: min(800px, 100%);
}


/* ====================================================
   MODAL HEADER
==================================================== */

.modal-header {
    display: flex;

    align-items: flex-start;

    justify-content: space-between;

    gap: 15px;

    margin-bottom: 24px;
}

.modal-header h2 {
    margin: 0 0 5px;

    color: var(--text, #111827);
}

.modal-header p {
    margin: 0;

    color: var(--muted, #6b7280);

    font-size: 13px;
}

.close-btn {
    width: 36px;
    height: 36px;

    display: flex;
    align-items: center;
    justify-content: center;

    border: none;

    border-radius: 7px;

    background: transparent;

    color: var(--muted, #6b7280);

    font-size: 28px;

    cursor: pointer;
}

.close-btn:hover {
    background: var(--hover, #f3f4f6);

    color: var(--text, #111827);
}


/* ====================================================
   FORM
==================================================== */

.form-group {
    margin-bottom: 18px;
}

.form-group label {
    display: block;

    margin-bottom: 7px;

    color: var(--text, #111827);

    font-size: 14px;
    font-weight: 600;
}
```

```
.form-group input,
.form-group textarea,
.form-group select {
    width: 100%;

    padding: 11px 12px;

    border: 1px solid var(--border, #d1d5db);

    border-radius: 8px;

    outline: none;

    background: var(--input, #ffffff);

    color: var(--text, #111827);

    font-family: inherit;
    font-size: 14px;
}

.form-group textarea {
    resize: vertical;
}

.form-group input:focus,
.form-group textarea:focus,
.form-group select:focus {
    border-color: var(--primary, #2563eb);

    box-shadow:
        0 0 0 3px
        rgba(37, 99, 235, 0.10);
}


/* ====================================================
   MODAL ACTIONS
==================================================== */

.modal-actions {
    display: flex;

    justify-content: flex-end;

    gap: 10px;

    margin-top: 24px;

    flex-wrap: wrap;
}


/* ====================================================
   DELETE MODAL
==================================================== */

.delete-modal {
    max-width: 450px;

    text-align: center;
}

.delete-icon {
    width: 65px;
    height: 65px;

    display: flex;
    align-items: center;
    justify-content: center;

    margin: 0 auto 18px;

    border-radius: 50%;

    background: rgba(220, 38, 38, 0.12);

    font-size: 28px;
}

.delete-modal h2 {
    margin: 0 0 10px;

    color: var(--text, #111827);
}

.delete-modal p {
    margin: 6px 0;

    color: var(--muted, #6b7280);

    line-height: 1.6;
}

.delete-modal strong {
    color: var(--text, #111827);
}

.delete-warning {
    color: var(--danger, #dc2626) !important;

    font-size: 13px;

    font-weight: 600;
```

```
}


/* ====================================================
   HEAD PREVIEW
==================================================== */

.head-preview {
    min-height: 20px;

    padding: 14px;

    border: 1px solid var(--border, #d1d5db);

    border-radius: 8px;

    background: var(--hover, #f3f4f6);

    color: var(--muted, #6b7280);

    font-size: 13px;
}

.head-preview.active {
    color: var(--text, #111827);
}


/* ====================================================
   DEPARTMENT SUMMARY
==================================================== */

.department-summary {
    display: grid;

    grid-template-columns:
        repeat(3, minmax(0, 1fr));

    gap: 12px;

    margin-bottom: 20px;
}

.summary-item {
    padding: 14px;

    background: var(--hover, #f3f4f6);

    border: 1px solid var(--border, #d1d5db);

    border-radius: 8px;
}

.summary-item span {
    display: block;

    margin-bottom: 5px;

    color: var(--muted, #6b7280);

    font-size: 12px;
}

.summary-item strong {
    color: var(--text, #111827);

    font-size: 16px;
}


/* ====================================================
   DETAILS
==================================================== */

.details-main {
    padding: 18px;

    margin-bottom: 20px;

    background: var(--hover, #f3f4f6);

    border: 1px solid var(--border, #d1d5db);

    border-radius: 10px;
}

.details-main h3 {
    margin: 0 0 7px;

    color: var(--text, #111827);

    font-size: 22px;
}

.details-main p {
    margin: 0;

    color: var(--muted, #6b7280);

    line-height: 1.6;
}

.details-grid {
    display: grid;
```

```
    grid-template-columns:
        repeat(3, minmax(0, 1fr));

    gap: 14px;
}

.details-card {
    padding: 17px;

    background: var(--card, #ffffff);

    border: 1px solid var(--border, #d1d5db);

    border-radius: 10px;
}

.details-card span {
    display: block;

    margin-bottom: 8px;

    color: var(--muted, #6b7280);

    font-size: 12px;
}

.details-card strong {
    display: block;

    color: var(--text, #111827);

    font-size: 17px;
}

.details-card small {
    display: block;

    margin-top: 5px;

    color: var(--muted, #6b7280);

    font-size: 12px;
}


/* ====================================================
   TOAST
==================================================== */

.toast-container {
    position: fixed;

    top: 80px;
    right: 25px;

    z-index: 20000;

    width: min(360px, calc(100% - 30px));

    display: flex;
    flex-direction: column;

    gap: 10px;
}

.toast {
    display: flex;
    align-items: flex-start;

    gap: 10px;

    padding: 14px 16px;

    background: var(--card, #ffffff);

    color: var(--text, #111827);

    border: 1px solid var(--border, #d1d5db);

    border-left: 4px solid var(--primary, #2563eb);

    border-radius: 9px;

    box-shadow:
        0 8px 25px
        rgba(0, 0, 0, 0.12);

    animation: toastIn 0.25s ease;
}

.toast.success {
    border-left-color: var(--success, #16a34a);
}

.toast.error {
    border-left-color: var(--danger, #dc2626);
}

.toast.warning {
    border-left-color: var(--warning, #d97706);
}

.toast-icon {
    font-size: 17px;
}
```

```
.toast-content {
    flex: 1;
}

.toast-title {
    margin-bottom: 3px;

    font-size: 13px;

    font-weight: 700;
}

.toast-message {
    color: var(--muted, #6b7280);

    font-size: 12px;

    line-height: 1.5;
}

.toast-close {
    border: none;

    background: transparent;

    color: var(--muted, #6b7280);

    font-size: 17px;

    cursor: pointer;
}

.toast-hide {
    animation: toastOut 0.25s ease forwards;
}


/* ======================================================
   ANIMATIONS
====================================================== */

@keyframes toastIn {

    from {
        opacity: 0;
        transform: translateX(20px);
    }

    to {
        opacity: 1;
        transform: translateX(0);
    }

}

@keyframes toastOut {

    from {
        opacity: 1;
        transform: translateX(0);
    }

    to {
        opacity: 0;
        transform: translateX(20px);
    }

}


/* ======================================================
   TABLET
====================================================== */

@media (max-width: 1100px) {

    .stats-grid {
        grid-template-columns:
            repeat(2, minmax(0, 1fr));
    }

    .details-grid {
        grid-template-columns:
            repeat(2, minmax(0, 1fr));
    }

}


/* ======================================================
   SMALL TABLET
====================================================== */

@media (max-width: 850px) {

    .department-page {
        padding: 25px 20px;
    }

    .filter-section {
        align-items: stretch;

        flex-direction: column;
    }
```

```
    .filter-right {
        width: 100%;
    }

    .status-filter {
        flex: 1;
    }

    .status-filter select {
        width: 100%;
    }

}


/* ======================================================
   MOBILE
====================================================== */

@media (max-width: 650px) {

    .topbar {
        height: auto;

        min-height: 60px;

        padding: 10px 15px;
    }

    .brand h2 {
        font-size: 22px;
    }

    .theme-btn,
    .logout-btn {
        height: 38px;
    }

    .department-page {
        padding: 20px 15px;
    }

    .page-header {
        align-items: flex-start;

        flex-direction: column;
    }

    .page-header h1 {
        font-size: 27px;
    }

    .page-header .primary-btn {
        width: 100%;
    }

    .stats-grid {
        grid-template-columns: 1fr;

        gap: 14px;
    }

    .filter-section {
        padding: 12px;
    }

    .search-shortcut {
        display: none;
    }

    .filter-right {
        flex-direction: column;

        align-items: stretch;
    }

    .status-filter select,
    .reset-btn {
        width: 100%;
    }

    .department-summary {
        grid-template-columns: 1fr;
    }

    .details-grid {
        grid-template-columns: 1fr;
    }

    .modal {
        padding: 20px;
    }

    .modal-actions {
        flex-direction: column;
    }

    .modal-actions button {
        width: 100%;
    }

    .toast-container {
        top: 70px;
```

```
        right: 15px;

        width: calc(100% - 30px);
    }

}


/* ====================================================
   VERY SMALL MOBILE
==================================================== */

@media (max-width: 400px) {

    .brand h2 {
        font-size: 20px;
    }

    .topbar-actions {
        gap: 6px;
    }

    .logout-btn {
        min-width: 70px;

        padding: 0 10px;
    }

    .department-page {
        padding: 18px 12px;
    }

    .page-header h1 {
        font-size: 24px;
    }

    .stat-card {
        padding: 17px;
    }

}


/* ====================================================
   DARK MODE
==================================================== */

html[data-theme="dark"] body {
    background: var(--bg, #0f172a);
}

html[data-theme="dark"] .topbar {
    background: var(--topbar, #111827);
}

html[data-theme="dark"] .search-box:focus-within,
html[data-theme="dark"] .status-filter select:focus,
html[data-theme="dark"] .form-group input:focus,
html[data-theme="dark"] .form-group textarea:focus,
html[data-theme="dark"] .form-group select:focus {
    box-shadow:
        0 0 0 3px
        rgba(59, 130, 246, 0.15);
}

html[data-theme="dark"] .modal-overlay {
    background: rgba(0, 0, 0, 0.72);
}


/* ====================================================
   REDUCED MOTION
==================================================== */

@media (prefers-reduced-motion: reduce) {

    *,
    *::before,
    *::after {
        animation-duration: 0.01ms !important;
        animation-iteration-count: 1 !important;
        transition-duration: 0.01ms !important;
    }

}
```

Frontend/CSS/interviews.css

```
/* ========================
   NAVBAR
======================== */

.topbar {
    width: 100%;
    min-height: 70px;

    display: flex;
    align-items: center;
    justify-content: space-between;

    padding: 0 28px;

    background: var(--topbar);

    border-bottom: 1px solid var(--border);

    box-sizing: border-box;
}


.nav-left {
    display: flex;
    align-items: center;
    gap: 40px;

    min-width: 0;
}


.logo {
    font-size: 24px;
    font-weight: 800;

    color: var(--primary);

    white-space: nowrap;
}


.nav-links {
    display: flex;
    align-items: center;
    gap: 24px;
}


.nav-links a {
    color: var(--muted);

    text-decoration: none;

    font-size: 15px;
    font-weight: 600;

    transition: 0.2s;
}


.nav-links a:hover {
    color: var(--primary);
}


.nav-links a.active {
    color: var(--primary);
}


.nav-actions {
    display: flex;
    align-items: center;
    gap: 10px;
}


.theme-btn {
    width: 42px;
    height: 42px;

    border: 1px solid var(--border);

    border-radius: 8px;

    background: var(--card);

    color: var(--text);

    cursor: pointer;

    font-size: 18px;
}


.theme-btn:hover {
    background: var(--hover);
}


/* ========================
   PAGE
```

```
======================== */

.page-container {
    width: 100%;

    padding: 30px;

    box-sizing: border-box;

    background: var(--bg);

    color: var(--text);

    min-height: calc(100vh - 70px);
}


/* ========================
   PAGE HEADER
======================== */

.page-header {
    margin-bottom: 25px;
}


.page-header h1 {
    margin: 0 0 7px;

    font-size: 30px;
    font-weight: 800;

    color: var(--text);
}


.page-header p {
    margin: 0;

    color: var(--muted);

    font-size: 15px;
}


/* ========================
   STATS
======================== */

.stats-grid {
    display: grid;

    grid-template-columns:
        repeat(4, minmax(0, 1fr));

    gap: 20px;

    margin-bottom: 25px;
}


.stat-card {
    background: var(--card);

    border: 1px solid var(--border);

    border-radius: 12px;

    padding: 22px;

    box-shadow: var(--shadow);
}


.stat-title {
    color: var(--muted);

    font-size: 14px;

    margin-bottom: 10px;
}


.stat-value {
    color: var(--text);

    font-size: 28px;

    font-weight: 800;
}


/* ========================
   FORM CARD
======================== */

.form-card {
    background: var(--card);

    border: 1px solid var(--border);

    border-radius: 12px;

    padding: 25px;
```

```
    margin-bottom: 25px;

    box-shadow: var(--shadow);
}


.section-header {
    margin-bottom: 22px;
}


.section-header h2 {
    margin: 0 0 6px;

    font-size: 21px;

    color: var(--text);
}


.section-header p {
    margin: 0;

    color: var(--muted);

    font-size: 14px;
}


/* ========================
   FORM
======================== */

.form-grid {
    display: grid;

    grid-template-columns:
        repeat(2, minmax(0, 1fr));

    gap: 20px;
}


.form-group {
    display: flex;

    flex-direction: column;

    gap: 8px;
}


.form-group label {
    color: var(--text);

    font-size: 14px;

    font-weight: 600;
}


.form-group input,
.form-group select {
    width: 100%;

    min-height: 44px;

    padding: 10px 12px;

    box-sizing: border-box;

    border: 1px solid var(--border);

    border-radius: 7px;

    background: var(--input);

    color: var(--text);

    font-size: 14px;

    outline: none;
}


.form-group input:focus,
.form-group select:focus {
    border-color: var(--primary);
}


.form-actions {
    display: flex;

    align-items: center;

    gap: 12px;

    margin-top: 24px;
}


/* ========================
   BUTTONS
======================== */
```

```
.primary-btn,
.secondary-btn {
    min-height: 42px;

    padding: 0 18px;

    border-radius: 7px;

    font-size: 14px;

    font-weight: 700;

    cursor: pointer;

    transition: 0.2s;
}


.primary-btn {
    border: 1px solid var(--primary);

    background: var(--primary);

    color: white;
}


.primary-btn:hover {
    background: var(--primary-hover);

    border-color: var(--primary-hover);
}


.secondary-btn {
    border: 1px solid var(--border);

    background: transparent;

    color: var(--text);
}


.secondary-btn:hover {
    background: var(--hover);
}


/* =========================
   TABLE CARD
========================= */

.table-card {
    background: var(--card);

    border: 1px solid var(--border);

    border-radius: 12px;

    padding: 25px;

    box-shadow: var(--shadow);

    overflow: hidden;
}


.table-header {
    display: flex;

    align-items: center;

    justify-content: space-between;

    gap: 20px;

    margin-bottom: 25px;
}


.table-header h2 {
    margin: 0 0 6px;

    color: var(--text);

    font-size: 21px;
}


.table-header p {
    margin: 0;

    color: var(--muted);

    font-size: 14px;
}


/* =========================
   TABLE
========================= */

.table-wrapper {
    width: 100%;
```

```
    overflow-x: auto;
}


table {
    width: 100%;

    border-collapse: collapse;

    min-width: 1100px;
}


thead {
    background: var(--hover);
}


th {
    padding: 17px 18px;

    text-align: left;

    color: var(--muted);

    font-size: 13px;

    font-weight: 800;

    white-space: nowrap;

    border-bottom: 1px solid var(--border);
}


td {
    padding: 18px;

    color: var(--text);

    font-size: 14px;

    border-bottom: 1px solid var(--border);

    vertical-align: middle;
}


tbody tr:last-child td {
    border-bottom: none;
}


tbody tr:hover {
    background: var(--hover);
}


/* ========================
   BADGES
======================== */

.status-badge,
.result-badge {
    display: inline-flex;

    align-items: center;

    justify-content: center;

    min-width: 75px;

    padding: 7px 12px;

    border-radius: 20px;

    font-size: 12px;

    font-weight: 700;

    white-space: nowrap;
}


.status-scheduled {
    background: rgba(37, 99, 235, 0.15);

    color: #60a5fa;
}


.status-completed {
    background: rgba(22, 163, 74, 0.15);

    color: #4ade80;
}


.status-cancelled {
    background: rgba(220, 38, 38, 0.15);

    color: #f87171;
}
```

```
.result-pending {
    background: rgba(217, 119, 6, 0.15);

    color: #fbbf24;
}


.result-selected {
    background: rgba(22, 163, 74, 0.15);

    color: #4ade80;
}


.result-rejected {
    background: rgba(220, 38, 38, 0.15);

    color: #f87171;
}


/* ========================
   MODE
======================== */

.mode-online {
    color: #60a5fa;

    font-weight: 700;
}


.mode-offline {
    color: #c084fc;

    font-weight: 700;
}


/* ========================
   ACTIONS
======================== */

.actions-container {
    display: flex;

    flex-direction: column;

    gap: 8px;

    min-width: 210px;
}


.action-row {
    display: flex;

    flex-wrap: wrap;

    align-items: center;

    gap: 8px;
}


.action-select {
    min-width: 145px;

    height: 40px;

    padding: 0 12px;

    border: 1px solid var(--border);

    border-radius: 7px;

    background: var(--input);

    color: var(--text);

    font-size: 13px;

    outline: none;

    cursor: pointer;
}


.action-select:focus {
    border-color: var(--primary);
}


.action-btn {
    display: inline-flex;

    align-items: center;

    justify-content: center;

    min-height: 38px;

    padding: 0 12px;
```

```
    border-radius: 7px;

    border: 1px solid var(--border);

    background: transparent;

    color: var(--text);

    text-decoration: none;

    font-size: 13px;

    font-weight: 700;

    cursor: pointer;

    transition: 0.2s;
}


.action-btn:hover {
    background: var(--hover);
}


/* ========================
   JOIN MEETING
======================== */

.join-btn {
    background: #16a34a;

    border-color: #16a34a;

    color: white;

    width: fit-content;
}


.join-btn:hover {
    background: #15803d;

    border-color: #15803d;

    color: white;
}


/* ========================
   FEEDBACK
======================== */

.feedback-btn {
    color: #60a5fa;

    border-color: rgba(96, 165, 250, 0.35);
}


.feedback-btn:hover {
    background: rgba(96, 165, 250, 0.12);
}


/* ========================
   DELETE
======================== */

.delete-btn {
    color: #f87171;

    border-color: rgba(248, 113, 113, 0.35);
}


.delete-btn:hover {
    background: rgba(248, 113, 113, 0.12);
}


/* ========================
   EMPTY
======================== */

.empty-state {
    text-align: center;

    padding: 45px 20px;

    color: var(--muted);
}


/* ========================
   RESPONSIVE
======================== */

@media (max-width: 1000px) {

    .nav-left {
        gap: 20px;
    }

    .nav-links {
```

```
        gap: 14px;
    }

    .stats-grid {
        grid-template-columns:
            repeat(2, minmax(0, 1fr));
    }

}


@media (max-width: 750px) {

    .topbar {
        padding: 15px 18px;

        flex-wrap: wrap;

        gap: 15px;
    }


    .nav-left {
        width: 100%;

        justify-content: space-between;
    }


    .nav-links {
        gap: 10px;

        flex-wrap: wrap;
    }


    .page-container {
        padding: 20px 15px;
    }


    .form-grid {
        grid-template-columns: 1fr;
    }


    .table-header {
        align-items: flex-start;

        flex-direction: column;
    }

}


@media (max-width: 500px) {

    .logo {
        font-size: 20px;
    }


    .nav-links a {
        font-size: 13px;
    }


    .stats-grid {
        grid-template-columns: 1fr;
    }


    .form-card,
    .table-card {
        padding: 18px;
    }


    .page-header h1 {
        font-size: 25px;
    }


    .form-actions {
        flex-direction: column;

        align-items: stretch;
    }


    .primary-btn,
    .secondary-btn {
        width: 100%;
    }

}
```

Frontend/CSS/notifications.css

```
/* ========================================
   HRGENIUS NOTIFICATIONS
   ======================================== */

.notifications-page {
    width: 100%;
    min-height: calc(100vh - 70px);
    padding: 30px;
    background: var(--bg);
    color: var(--text);
}


/* ========================================
   TOPBAR
   ======================================== */

.notifications-topbar {
    width: 100%;
    min-height: 68px;
    padding: 0 30px;

    display: flex;
    align-items: center;
    justify-content: space-between;

    background: var(--topbar);
    border-bottom: 1px solid var(--border);

    color: var(--text);
}


.notification-brand h2 {
    margin: 0;
    font-size: 21px;
    color: var(--text);
}


.notification-nav {
    display: flex;
    align-items: center;
    gap: 22px;
}


.notification-nav a {
    color: var(--text);
    text-decoration: none;
    font-size: 14px;
}


.notification-nav a:hover {
    color: var(--primary);
}


.notification-nav a.active {
    color: var(--primary);
    font-weight: 600;
}


.notification-logout {
    padding: 9px 14px;
    border: 1px solid var(--border);
    border-radius: 7px;

    background: var(--card);
    color: var(--text);

    cursor: pointer;
}


.notification-logout:hover {
    background: var(--hover);
}


/* ========================================
   HEADER
   ======================================== */

.notifications-header {
    width: 100%;

    display: flex;
    justify-content: space-between;
    align-items: flex-start;

    gap: 20px;
    margin-bottom: 25px;
}


.notifications-header h1 {
    margin: 0;

    font-size: 30px;
```

```
    font-weight: 700;

    color: var(--text);
}


.notifications-header p {
    margin: 7px 0 0;

    color: var(--muted);

    font-size: 15px;
}


.header-actions {
    display: flex;
    gap: 10px;
}


/* ========================================
   BUTTONS
   ======================================== */

.notification-btn {
    height: 42px;

    padding: 0 17px;

    border: 1px solid var(--border);
    border-radius: 8px;

    background: var(--card);
    color: var(--text);

    cursor: pointer;

    font-size: 14px;
}


.notification-btn:hover {
    background: var(--hover);
}


.notification-btn.primary {
    background: var(--primary);
    border-color: var(--primary);
    color: white;
}


.notification-btn.primary:hover {
    background: var(--primary-hover);
}


/* ========================================
   MESSAGE
   ======================================== */

#message {
    margin-bottom: 20px;
}


/* ========================================
   STAT CARDS
   ======================================== */

.notification-stats {
    display: grid;

    grid-template-columns:
        repeat(3, minmax(0, 1fr));

    gap: 20px;

    margin-bottom: 25px;
}


.notification-stat-card {
    min-height: 110px;

    padding: 20px;

    display: flex;
    align-items: center;

    gap: 16px;

    background: var(--card);

    border: 1px solid var(--border);

    border-radius: 12px;

    box-shadow: var(--shadow);
}


.notification-stat-card.unread-card {
```

```
    border-left: 5px solid var(--primary);
}


.stat-icon {
    width: 58px;
    height: 58px;

    flex-shrink: 0;

    display: flex;
    align-items: center;
    justify-content: center;

    border-radius: 11px;

    background: var(--hover);

    font-size: 27px;
}


.notification-stat-card span {
    display: block;

    margin-bottom: 7px;

    color: var(--muted);

    font-size: 14px;
}


.notification-stat-card strong {
    display: block;

    color: var(--text);

    font-size: 28px;
}


/* ========================================
   MAIN PANEL
   ======================================== */

.notifications-panel {
    width: 100%;

    background: var(--card);

    border: 1px solid var(--border);

    border-radius: 12px;

    box-shadow: var(--shadow);

    overflow: hidden;
}


/* ========================================
   FILTERS
   ======================================== */

.notification-filters {
    width: 100%;

    padding: 20px;

    display: grid;

    grid-template-columns:
        minmax(250px, 1fr)
        200px
        200px
        100px;

    gap: 12px;

    align-items: center;

    border-bottom: 1px solid var(--border);
}


/* SEARCH */

.search-wrapper {
    position: relative;
    width: 100%;
}


.search-wrapper input {
    width: 100%;
    height: 46px;

    padding:
        0 40px
        0 42px;

    border: 1px solid var(--border);

    border-radius: 8px;
```

```
    outline: none;

    background: var(--input);

    color: var(--text);

    font-size: 14px;
}


.search-wrapper input::placeholder {
    color: var(--muted);
}


.search-wrapper input:focus {
    border-color: var(--primary);

    box-shadow:
        0 0 0 3px
        rgba(37, 99, 235, 0.12);
}


.search-icon {
    position: absolute;

    left: 14px;
    top: 50%;

    transform:
        translateY(-50%);

    pointer-events: none;

    font-size: 17px;
}


.clear-search {
    position: absolute;

    right: 8px;
    top: 50%;

    transform:
        translateY(-50%);

    width: 30px;
    height: 30px;

    border: none;

    border-radius: 6px;

    background: transparent;

    color: var(--muted);

    cursor: pointer;

    font-size: 20px;
}


.clear-search:hover {
    background: var(--hover);

    color: var(--text);
}


/* SELECT */

.notification-select {
    width: 100%;
    height: 46px;

    padding: 0 12px;

    border: 1px solid var(--border);

    border-radius: 8px;

    outline: none;

    background: var(--input);

    color: var(--text);

    font-size: 14px;

    cursor: pointer;
}


.notification-select:focus {
    border-color: var(--primary);
}


/* RESET */

.reset-btn {
```

```
    height: 46px;

    padding: 0 14px;

    border: 1px solid var(--border);

    border-radius: 8px;

    background: var(--card);

    color: var(--text);

    cursor: pointer;

    white-space: nowrap;
}


.reset-btn:hover {
    background: var(--hover);
}


/* ========================================
   LIST HEADER
   ======================================== */

.notification-list-header {
    padding: 20px;

    display: flex;

    align-items: center;

    justify-content: space-between;

    gap: 20px;

    border-bottom: 1px solid var(--border);
}


.notification-list-header h2 {
    margin: 0;

    font-size: 20px;

    color: var(--text);
}


.notification-list-header p {
    margin: 5px 0 0;

    color: var(--muted);

    font-size: 13px;
}


.delete-read-btn {
    height: 42px;

    padding: 0 16px;

    border: 1px solid var(--danger);

    border-radius: 8px;

    background: transparent;

    color: var(--danger);

    cursor: pointer;

    font-size: 14px;
}


.delete-read-btn:hover {
    background:
        rgba(220, 38, 38, 0.08);
}


/* ========================================
   NOTIFICATION LIST
   ======================================== */

#notificationList {
    padding: 18px;
}


.notification-item {
    width: 100%;

    display: flex;

    align-items: flex-start;

    gap: 15px;

    padding: 17px;
```

```
    margin-bottom: 12px;

    border: 1px solid var(--border);

    border-radius: 10px;

    background: var(--card);

    transition: 0.2s ease;
}


.notification-item:last-child {
    margin-bottom: 0;
}


.notification-item:hover {
    background: var(--hover);

    border-color: var(--primary);
}


.notification-item.unread {
    border-left: 4px solid var(--primary);
}


.notification-item.read {
    opacity: 0.8;
}


/* ========================================
   NOTIFICATION ICON
   ======================================== */

.notification-icon {
    width: 46px;
    height: 46px;

    min-width: 46px;

    display: flex;

    align-items: center;
    justify-content: center;

    border-radius: 10px;

    background: var(--hover);

    font-size: 21px;
}


/* ========================================
   NOTIFICATION CONTENT
   ======================================== */

.notification-content {
    flex: 1;

    min-width: 0;
}


.notification-title-row {
    display: flex;

    justify-content: space-between;

    gap: 15px;
}


.notification-title {
    margin: 0;

    color: var(--text);

    font-size: 16px;

    font-weight: 650;
}


.notification-time {
    color: var(--muted);

    font-size: 12px;

    white-space: nowrap;
}


.notification-message {
    margin: 7px 0 10px;

    color: var(--muted);

    font-size: 14px;

    line-height: 1.5;
```

```
}


.notification-meta {
    display: flex;

    align-items: center;

    flex-wrap: wrap;

    gap: 7px;
}


.notification-type,
.notification-priority,
.notification-status {
    display: inline-flex;

    align-items: center;

    min-height: 25px;

    padding: 3px 9px;

    border-radius: 20px;

    font-size: 11px;

    font-weight: 600;
}


.notification-type {
    color: var(--primary);

    background:
        rgba(37, 99, 235, 0.10);
}


.notification-priority {
    color: var(--warning);

    background:
        rgba(217, 119, 6, 0.10);
}


.notification-status.unread {
    color: var(--danger);

    background:
        rgba(220, 38, 38, 0.10);
}


.notification-status.read {
    color: var(--success);

    background:
        rgba(22, 163, 74, 0.10);
}


/* ========================================
   ACTIONS
   ======================================== */

.notification-actions {
    display: flex;

    align-items: center;

    gap: 6px;
}


.notification-action-btn {
    width: 36px;
    height: 36px;

    border: 1px solid var(--border);

    border-radius: 7px;

    background: var(--card);

    color: var(--text);

    cursor: pointer;

    display: flex;

    align-items: center;
    justify-content: center;
}


.notification-action-btn:hover {
    background: var(--hover);

    border-color: var(--primary);
}
```

```
.notification-action-btn.delete {
    color: var(--danger);
}


.notification-action-btn.delete:hover {
    border-color: var(--danger);

    background:
        rgba(220, 38, 38, 0.08);
}


/* ========================================
   LOADING
   ======================================== */

.loading-state {
    text-align: center;

    padding: 65px 20px;

    color: var(--muted);
}


.spinner {
    width: 40px;
    height: 40px;

    margin:
        0 auto 15px;

    border:
        3px solid var(--border);

    border-top-color:
        var(--primary);

    border-radius: 50%;

    animation:
        spin 0.8s linear infinite;
}


@keyframes spin {

    to {
        transform:
            rotate(360deg);
    }

}


/* ========================================
   EMPTY
   ======================================== */

.empty-state {
    display: none;

    text-align: center;

    padding: 70px 20px;

    color: var(--muted);
}


.empty-icon {
    font-size: 45px;

    margin-bottom: 12px;
}


.empty-state h3 {
    margin: 0 0 7px;

    color: var(--text);

    font-size: 18px;
}


.empty-state p {
    margin: 0;

    color: var(--muted);
}


/* ========================================
   PAGINATION
   ======================================== */

.pagination {
    display: flex;

    align-items: center;

    justify-content: center;
```

```
    gap: 6px;

    padding: 18px;

    border-top:
        1px solid var(--border);
}


.page-btn {
    min-width: 38px;

    height: 38px;

    padding: 0 10px;

    border: 1px solid var(--border);

    border-radius: 7px;

    background: var(--card);

    color: var(--text);

    cursor: pointer;
}


.page-btn:hover {
    background: var(--hover);
}


.page-btn.active {
    background: var(--primary);

    border-color: var(--primary);

    color: white;
}


.page-btn:disabled {
    opacity: 0.4;

    cursor: not-allowed;
}


/* ========================================
   MODAL
   ======================================== */

.notification-modal {
    position: fixed;

    inset: 0;

    z-index: 9999;

    display: none;

    align-items: center;

    justify-content: center;

    padding: 20px;

    background:
        rgba(0, 0, 0, 0.55);
}


.notification-modal-box {
    width: min(600px, 100%);

    max-height: 90vh;

    overflow-y: auto;

    background: var(--card);

    border:
        1px solid var(--border);

    border-radius: 12px;

    box-shadow: var(--shadow);
}


.modal-header {
    display: flex;

    align-items: center;

    justify-content: space-between;

    padding: 18px 20px;

    border-bottom:
        1px solid var(--border);
}
```

```
.modal-header h2 {
    margin: 0;

    color: var(--text);

    font-size: 19px;
}


.modal-close {
    width: 35px;
    height: 35px;

    border: none;

    border-radius: 7px;

    background: var(--hover);

    color: var(--text);

    cursor: pointer;

    font-size: 21px;
}


.modal-body {
    padding: 20px;
}


.detail-item {
    margin-bottom: 18px;
}


.detail-item span {
    display: block;

    margin-bottom: 5px;

    color: var(--muted);

    font-size: 12px;
}


.detail-item strong,
.detail-item p {
    margin: 0;

    color: var(--text);

    font-size: 14px;

    line-height: 1.5;
}


.detail-grid {
    display: grid;

    grid-template-columns:
        repeat(2, 1fr);

    gap: 20px;
}


.modal-footer {
    display: flex;

    justify-content: flex-end;

    gap: 10px;

    padding: 16px 20px;

    border-top:
        1px solid var(--border);
}


/* ========================================
   RESPONSIVE
   ======================================== */

@media (max-width: 1000px) {

    .notification-filters {
        grid-template-columns:
            1fr 1fr;
    }

    .search-wrapper {
        grid-column: 1 / -1;
    }

}


@media (max-width: 800px) {
```

```
    .notifications-topbar {
        padding: 15px 20px;

        flex-direction: column;

        align-items: flex-start;

        gap: 15px;
    }


    .notification-nav {
        width: 100%;

        flex-wrap: wrap;

        gap: 12px;
    }


    .notifications-page {
        padding: 22px 18px;
    }


    .notifications-header {
        flex-direction: column;
    }


    .header-actions {
        width: 100%;
    }


    .notification-stats {
        grid-template-columns: 1fr;
    }


    .notification-list-header {
        align-items: flex-start;

        flex-direction: column;
    }

}


@media (max-width: 600px) {

    .notifications-page {
        padding: 15px 12px;
    }


    .notifications-header h1 {
        font-size: 25px;
    }


    .header-actions {
        flex-direction: column;
    }


    .notification-btn {
        width: 100%;
    }


    .notification-filters {
        grid-template-columns: 1fr;

        padding: 15px;
    }


    .search-wrapper {
        grid-column: auto;
    }


    .notification-item {
        flex-direction: column;
    }


    .notification-title-row {
        flex-direction: column;

        gap: 5px;
    }


    .notification-actions {
        width: 100%;
    }


    .notification-action-btn {
        flex: 1;
    }
```

```
    .detail-grid {
        grid-template-columns: 1fr;
    }

}
```

Frontend/CSS/setting.css

```
body {
    background: red !important;
}
/* ========================================
   SETTINGS PAGE
======================================== */

.main-content .settings-container {
    padding: 30px;
    background: #f5f7fb;
    min-height: calc(100vh - 70px);
}


/* ========================================
   HEADER
======================================== */

.main-content .settings-header {
    margin-bottom: 25px;
}

.main-content .settings-header h1 {
    margin: 0 0 8px 0;
    font-size: 28px;
    font-weight: 700;
    color: #1f2937;
}

.main-content .settings-header p {
    margin: 0;
    color: #6b7280;
    font-size: 14px;
}


/* ========================================
   GRID
======================================== */

.main-content .settings-grid {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 22px;
}


/* ========================================
   CARD
======================================== */

.main-content .settings-card {
    background: #ffffff;
    border: 1px solid #e5e7eb;
    border-radius: 14px;
    padding: 25px;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.06);
}

.main-content .settings-card.full-width {
    grid-column: 1 / -1;
}

.main-content .settings-card h2 {
    margin: 0 0 8px 0;
    font-size: 19px;
    color: #111827;
}

.main-content .settings-card > p {
    margin: 0 0 22px 0;
    color: #6b7280;
    font-size: 14px;
}


/* ========================================
   FORM
======================================== */

.main-content .form-group {
    margin-bottom: 18px;
}

.main-content .form-group label {
    display: block;
    margin-bottom: 7px;
    font-size: 14px;
    font-weight: 600;
    color: #374151;
}

.main-content .form-group input,
.main-content .form-group select {
    width: 100%;
    box-sizing: border-box;
    padding: 12px 14px;
    border: 1px solid #d1d5db;
    border-radius: 8px;
    background: #ffffff;
    color: #111827;
    font-size: 14px;
```

```
    outline: none;
}

.main-content .form-group input:focus,
.main-content .form-group select:focus {
    border-color: #4f46e5;
    box-shadow: 0 0 0 3px rgba(79, 70, 229, 0.10);
}

.main-content .form-group input[readonly] {
    background: #f3f4f6;
    color: #6b7280;
}


/* ========================================
   BUTTON
======================================== */

.main-content .btn {
    border: none;
    border-radius: 8px;
    padding: 11px 18px;
    font-size: 14px;
    font-weight: 600;
    cursor: pointer;
    transition: 0.2s;
}

.main-content .btn-primary {
    background: #4f46e5;
    color: #ffffff;
}

.main-content .btn-primary:hover {
    background: #4338ca;
    transform: translateY(-1px);
}


/* ========================================
   NOTIFICATION PREFERENCES
======================================== */

.main-content .preference-row {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 20px;
    padding: 16px 0;
    border-bottom: 1px solid #e5e7eb;
}

.main-content .preference-row:last-child {
    border-bottom: none;
}

.main-content .preference-info {
    display: flex;
    flex-direction: column;
    gap: 5px;
}

.main-content .preference-info strong {
    font-size: 14px;
    color: #111827;
}

.main-content .preference-info span {
    font-size: 13px;
    color: #6b7280;
}


/* ========================================
   CHECKBOX
======================================== */

.main-content .preference-row input[type="checkbox"] {
    width: 18px;
    height: 18px;
    cursor: pointer;
}


/* ========================================
   SYSTEM INFORMATION
======================================== */

.main-content .system-info {
    display: grid;
    grid-template-columns: repeat(5, 1fr);
    gap: 15px;
}

.main-content .system-item {
    background: #f8fafc;
    border: 1px solid #e5e7eb;
    border-radius: 10px;
    padding: 16px;
}

.main-content .system-item span {
    display: block;
    font-size: 12px;
```

```
    color: #6b7280;
    margin-bottom: 7px;
}

.main-content .system-item strong {
    display: block;
    font-size: 14px;
    color: #111827;
}


/* ========================================
   RESPONSIVE
======================================== */

@media (max-width: 1000px) {

    .main-content .settings-grid {
        grid-template-columns: 1fr;
    }

    .main-content .settings-card.full-width {
        grid-column: auto;
    }

    .main-content .system-info {
        grid-template-columns: repeat(2, 1fr);
    }

}


@media (max-width: 600px) {

    .main-content .settings-container {
        padding: 18px;
    }

    .main-content .settings-card {
        padding: 18px;
    }

    .main-content .system-info {
        grid-template-columns: 1fr;
    }

}
```

Frontend/CSS/style.css

```
/* ====================================================
   HRGENIUS - GLOBAL STYLE
   Light + Dark Theme
==================================================== */

/* ========================
   GLOBAL VARIABLES
======================== */

:root {
    --bg: #f5f7fb;
    --card: #ffffff;
    --text: #172033;
    --muted: #667085;
    --border: #e5e7eb;
    --hover: #f3f6fa;

    --input: #ffffff;

    --primary: #2563eb;
    --primary-hover: #1d4ed8;

    --success: #16a34a;
    --warning: #d97706;
    --danger: #dc2626;

    --sidebar: #ffffff;
    --topbar: #ffffff;

    --shadow: 0 8px 24px rgba(15, 23, 42, 0.07);
}


/* ========================
   DARK THEME
======================== */

html[data-theme="dark"] {
    --bg: #0f172a;
    --card: #1e293b;
    --text: #f8fafc;
    --muted: #94a3b8;
    --border: #334155;
    --hover: #273449;

    --input: #111827;

    --primary: #60a5fa;
    --primary-hover: #3b82f6;

    --success: #4ade80;
    --warning: #fbbf24;
    --danger: #f87171;

    --sidebar: #111827;
    --topbar: #111827;

    --shadow: 0 8px 24px rgba(0, 0, 0, 0.28);
}


/* ========================
   RESET
======================== */

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}


html {
    min-height: 100%;
    background: var(--bg);
}


body {
    min-height: 100vh;

    font-family:
        Arial,
        Helvetica,
        sans-serif;

    background: var(--bg);
    color: var(--text);

    transition:
        background-color 0.25s ease,
        color 0.25s ease;
}


/* ========================
   LINKS
======================== */

a {
    color: var(--primary);
    text-decoration: none;
```

```
}

a:hover {
    color: var(--primary-hover);
}


/* ========================
   HEADINGS
======================== */

h1,
h2,
h3,
h4,
h5,
h6 {
    color: var(--text);
}


/* ========================
   PARAGRAPHS
======================== */

p {
    color: var(--muted);
}


/* ========================
   SIDEBAR
======================== */

.sidebar {
    background: var(--sidebar);
    color: var(--text);

    border-right: 1px solid var(--border);

    transition:
        background-color 0.25s ease,
        color 0.25s ease,
        border-color 0.25s ease;
}


.logo {
    color: var(--text);
}


.logo h2 {
    color: var(--text);
}


.logo span {
    color: var(--muted);
}


.menu-title {
    color: var(--muted);
}


.side-link {
    background: transparent;
    color: var(--text);

    border: none;

    transition:
        background-color 0.2s ease,
        color 0.2s ease;
}


.side-link:hover {
    background: var(--hover);
    color: var(--primary);
}


.side-link.active {
    background: var(--primary);
    color: #ffffff;
}


.side-link.active:hover {
    background: var(--primary-hover);
    color: #ffffff;
}


.logout-link {
    color: var(--danger);
}


.logout-link:hover {
    background: rgba(220, 38, 38, 0.10);
    color: var(--danger);
```

```
}


/* ========================
   MAIN CONTENT
======================== */

.main-content {
    background: var(--bg);
    color: var(--text);
}


/* ========================
   TOPBAR
======================== */

.topbar {
    background: var(--topbar);
    color: var(--text);

    border-bottom: 1px solid var(--border);

    box-shadow: var(--shadow);
}


.search-box {
    background: var(--input);

    border: 1px solid var(--border);

    color: var(--text);
}


.search-box input {
    background: transparent;
    color: var(--text);

    border: none;
    outline: none;
}


.search-box input::placeholder {
    color: var(--muted);
}


.profile {
    color: var(--text);
}


.profile-info strong {
    color: var(--text);
}


.profile-info span {
    color: var(--muted);
}


/* ========================
   THEME BUTTON
======================== */

.theme-toggle {
    background: var(--card);
    color: var(--text);

    border: 1px solid var(--border);

    cursor: pointer;

    border-radius: 8px;

    padding: 9px 14px;

    font-size: 14px;
    font-weight: 600;

    transition:
        background-color 0.2s ease,
        color 0.2s ease,
        border-color 0.2s ease;
}


.theme-toggle:hover {
    background: var(--hover);
}


/* ========================
   BUTTONS
======================== */

button {
    font-family: inherit;
}
```

```
.btn,
.primary-btn,
.submit-btn,
.add-btn {
    background: var(--primary);
    color: #ffffff;

    border: none;

    cursor: pointer;
}


.btn:hover,
.primary-btn:hover,
.submit-btn:hover,
.add-btn:hover {
    background: var(--primary-hover);
}


/* =========================
   CARDS
========================= */

.card,
.dashboard-card,
.stat-card,
.content-card,
.employee-card,
.form-card,
.section-card,
.panel {
    background: var(--card);

    color: var(--text);

    border: 1px solid var(--border);

    box-shadow: var(--shadow);

    transition:
        background-color 0.25s ease,
        color 0.25s ease,
        border-color 0.25s ease;
}


/* =========================
   CARD TEXT
========================= */

.card h2,
.card h3,
.card h4,
.dashboard-card h2,
.dashboard-card h3,
.dashboard-card h4,
.stat-card h2,
.stat-card h3,
.stat-card h4 {
    color: var(--text);
}


.card p,
.dashboard-card p,
.stat-card p {
    color: var(--muted);
}


/* =========================
   FORM
========================= */

form {
    color: var(--text);
}


label {
    color: var(--text);
}


input,
select,
textarea {
    width: 100%;

    background: var(--input);

    color: var(--text);

    border: 1px solid var(--border);

    outline: none;

    transition:
        background-color 0.2s ease,
        color 0.2s ease,
        border-color 0.2s ease;
}
```

```
input:focus,
select:focus,
textarea:focus {
    border-color: var(--primary);

    box-shadow:
        0 0 0 3px rgba(37, 99, 235, 0.12);
}


input::placeholder,
textarea::placeholder {
    color: var(--muted);
}


/* ========================
   SELECT
======================== */

select {
    cursor: pointer;
}


/* ========================
   TABLE
======================== */

table {
    width: 100%;

    border-collapse: collapse;

    background: var(--card);

    color: var(--text);
}


thead {
    background: var(--hover);
}


th {
    color: var(--text);

    font-weight: 600;

    border-bottom: 1px solid var(--border);
}


td {
    color: var(--text);

    border-bottom: 1px solid var(--border);
}


tbody tr {
    background: var(--card);

    transition:
        background-color 0.2s ease;
}


tbody tr:hover {
    background: var(--hover);
}


/* ========================
   TABLE CONTAINER
======================== */

.table-container {
    background: var(--card);

    border: 1px solid var(--border);

    box-shadow: var(--shadow);

    overflow-x: auto;
}


/* ========================
   STATUS
======================== */

.status {
    border-radius: 20px;
    font-weight: 600;
}


.status-approved,
.status-paid,
.status-active,
.status-reviewed,
.status-selected {
```

```
    color: var(--success);
}


.status-pending {
    color: var(--warning);
}


.status-rejected,
.status-unpaid,
.status-inactive {
    color: var(--danger);
}


/* =========================
   BADGES
========================= */

.badge {
    display: inline-block;

    padding: 5px 10px;

    border-radius: 20px;

    font-size: 12px;

    font-weight: 600;
}


.badge-success {
    background: rgba(22, 163, 74, 0.12);
    color: var(--success);
}


.badge-warning {
    background: rgba(217, 119, 6, 0.12);
    color: var(--warning);
}


.badge-danger {
    background: rgba(220, 38, 38, 0.12);
    color: var(--danger);
}


.badge-primary {
    background: rgba(37, 99, 235, 0.12);
    color: var(--primary);
}


/* =========================
   MODAL
========================= */

.modal,
.modal-content {
    background: var(--card);

    color: var(--text);

    border: 1px solid var(--border);

    box-shadow: var(--shadow);
}


.modal h2,
.modal h3,
.modal h4 {
    color: var(--text);
}


.modal p {
    color: var(--muted);
}


/* =========================
   CLOSE BUTTON
========================= */

.close-btn {
    color: var(--muted);

    background: transparent;

    border: none;

    cursor: pointer;
}


.close-btn:hover {
    color: var(--text);
}
```

```
/* ========================
   NOTIFICATIONS
======================== */

.notification-item {
    background: var(--card);

    color: var(--text);

    border-bottom: 1px solid var(--border);
}


.notification-item:hover {
    background: var(--hover);
}


/* ========================
   QUICK ACTIONS
======================== */

.quick-actions button {
    background: var(--card);

    color: var(--text);

    border: 1px solid var(--border);

    cursor: pointer;
}


.quick-actions button:hover {
    background: var(--hover);

    border-color: var(--primary);
}


.quick-actions strong {
    color: var(--text);
}


.quick-actions small {
    color: var(--muted);
}


/* ========================
   ACTIVITY
======================== */

.activity-item {
    background: var(--card);

    color: var(--text);

    border-bottom: 1px solid var(--border);
}


.activity-item:hover {
    background: var(--hover);
}


.activity-content strong {
    color: var(--text);
}


.activity-content span,
.activity-item small {
    color: var(--muted);
}


/* ========================
   SUMMARY
======================== */

.summary-row {
    color: var(--text);

    border-bottom: 1px solid var(--border);
}


.summary-row span {
    color: var(--text);
}


.summary-row strong {
    color: var(--text);
}


/* ========================
   WELCOME SECTION
======================== */

.welcome-section {
```

```
    color: var(--text);
}


.welcome-section h1 {
    color: var(--text);
}


.welcome-section p {
    color: var(--muted);
}


.date-box {
    background: var(--card);

    color: var(--text);

    border: 1px solid var(--border);

    box-shadow: var(--shadow);
}


.date-box small {
    color: var(--muted);
}


.date-box strong {
    color: var(--text);
}


/* ========================
   EMPLOYEE OVERVIEW
======================== */

.employee-overview {
    background: var(--card);

    color: var(--text);
}


.overview-circle {
    background: var(--hover);

    color: var(--text);
}


.overview-circle strong {
    color: var(--text);
}


/* ========================
   PROGRESS BAR
======================== */

.progress-bar {
    background: var(--border);
}


.progress-fill {
    background: var(--primary);
}


/* ========================
   PAGINATION
======================== */

.pagination button {
    background: var(--card);

    color: var(--text);

    border: 1px solid var(--border);

    cursor: pointer;
}


.pagination button:hover {
    background: var(--hover);
}


.pagination button.active {
    background: var(--primary);

    color: #ffffff;

    border-color: var(--primary);
}


/* ========================
   ALERT
======================== */
```

```
.alert {
    border-radius: 8px;
}


.alert-success {
    background: rgba(22, 163, 74, 0.12);
    color: var(--success);
}


.alert-warning {
    background: rgba(217, 119, 6, 0.12);
    color: var(--warning);
}


.alert-danger {
    background: rgba(220, 38, 38, 0.12);
    color: var(--danger);
}


.alert-info {
    background: rgba(37, 99, 235, 0.12);
    color: var(--primary);
}


/* =========================
   SCROLLBAR
========================= */

::-webkit-scrollbar {
    width: 8px;
    height: 8px;
}


::-webkit-scrollbar-track {
    background: var(--bg);
}


::-webkit-scrollbar-thumb {
    background: var(--border);

    border-radius: 10px;
}


::-webkit-scrollbar-thumb:hover {
    background: var(--muted);
}


/* =========================
   RESPONSIVE
========================= */

@media (max-width: 1000px) {

    .sidebar {
        width: 220px;
    }

    .main-content {
        margin-left: 220px;
    }

}


@media (max-width: 768px) {

    .sidebar {
        width: 70px;
    }

    .main-content {
        margin-left: 70px;
    }

    .profile-info,
    .profile-arrow,
    .menu-title,
    .logo div:not(.logo-icon) {
        display: none;
    }

}


@media (max-width: 600px) {

    .main-content {
        margin-left: 0;
    }

    .sidebar {
        display: none;
    }

    .topbar {
        padding: 10px;
```

```
    }

    .search-box {
        display: none;
    }

}


/* ========================
   GLOBAL TRANSITION
======================== */

body,
.sidebar,
.topbar,
.card,
.dashboard-card,
.stat-card,
.content-card,
.employee-card,
.form-card,
.section-card,
.panel,
.table-container,
table,
tbody tr,
input,
select,
textarea,
.modal,
.modal-content,
.theme-toggle {
    transition:
        background-color 0.25s ease,
        color 0.25s ease,
        border-color 0.25s ease;
}
```

Frontend/dashboard.html

```html
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport"
        content="width=device-width, initial-scale=1.0">

    <title>HRGenius - Dashboard</title>

    <link rel="stylesheet" href="CSS/style.css">
    <link rel="stylesheet" href="CSS/dashboard.css">

</head>


<body>

    <!-- Page Loader -->

    <div id="pageLoader">

        <div class="loader-box">

            <div class="loader"></div>

            <h3>Loading HRGenius</h3>

            <p>Preparing your dashboard...</p>

        </div>

    </div>


    <!-- Sidebar -->

    <aside class="sidebar">

        <div class="logo">

            <div class="logo-icon">
                H
            </div>

            <div class="logo-text">

                <h2>HRGenius</h2>

                <span>HR Management</span>

            </div>

        </div>


        <div class="sidebar-menu">

            <p class="menu-title">
                MAIN
            </p>


            <a href="dashboard.html"
                class="side-link active">

                <span>🏠</span>

                Dashboard

            </a>


            <p class="menu-title">
                MANAGEMENT
            </p>


            <a href="employees.html"
                class="side-link">

                <span>👥</span>

                Employees

            </a>


            <a href="attendance.html"
                class="side-link">

                <span>🕐</span>

                Attendance

            </a>


            <a href="leaves.html"
```

```
        class="side-link">

        <span>🗓️</span>

        Leaves

    </a>


    <a href="payroll.html"
        class="side-link">

        <span>💰</span>

        Payroll

    </a>


    <a href="performance.html"
        class="side-link">

        <span>⭐</span>

        Performance

    </a>


    <p class="menu-title">
        RECRUITMENT
    </p>


    <a href="jobs.html"
        class="side-link">

        <span>💼</span>

        Jobs

    </a>


    <a href="candidates.html"
        class="side-link">

        <span>👤</span>

        Candidates

    </a>


    <a href="interviews.html"
        class="side-link">

        <span>🎯</span>

        Interviews

    </a>


    <p class="menu-title">
        ORGANIZATION
    </p>


    <a href="departments.html"
        class="side-link">

        <span>🏢</span>

        Departments

    </a>


    <a href="notifications.html"
        class="side-link">

        <span>🔔</span>

        Notifications

        <span class="nav-badge"
            id="notificationBadge">

            0

        </span>

    </a>

</div>


<div class="sidebar-bottom">

    <a href="settings.html"
        class="side-link">

        <span>⚙️</span>
```

```
            Settings

        </a>


        <button
            class="side-link logout-link"
            onclick="logout()">

            <span>🚪</span>

            Logout

        </button>

    </div>

</aside>


<!-- Main -->

<main class="main-content">


    <!-- Topbar -->

    <header class="topbar">


        <div class="mobile-logo">

            <div class="logo-icon">
                H
            </div>

            <strong>
                HRGenius
            </strong>

        </div>


        <div class="search-box">

            <span>🔍</span>

            <input
                id="dashboardSearch"
                type="text"
                placeholder="Search HR modules...">

        </div>


        <div class="topbar-right">


            <button
                id="themeToggle"
                class="theme-toggle"
                onclick="toggleTheme()">

                🌙 Dark Mode

            </button>


            <button
                class="notification-btn"
                onclick="window.location.href='notifications.html'">

                🔔

                <span
                    class="notification-dot"
                    id="notificationDot">
                </span>

            </button>


            <div class="profile">

                <div
                    class="profile-avatar"
                    id="profileAvatar">

                    D

                </div>


                <div class="profile-info">

                    <strong id="profileName">
                        User
                    </strong>

                    <span id="profileRole">
                        Employee
                    </span>

                </div>
```

```
        </div>

    </div>

</header>


<!-- Dashboard -->

<section class="dashboard-container">


    <!-- Welcome -->

    <div class="welcome-section">


        <div class="welcome-content">

            <div class="welcome-badge">

                <span>✨</span>

                HRGenius Dashboard

            </div>


            <h1 id="welcomeTitle">
                Welcome back! 👋
            </h1>


            <p>

                Manage your workforce, recruitment,
                payroll and HR operations from one place.

            </p>

        </div>


        <div class="date-box">

            <span class="date-icon">
                🗓️
            </span>

            <div>

                <small>
                    TODAY
                </small>

                <strong id="currentDate">
                    Loading...
                </strong>

            </div>

        </div>

    </div>


    <!-- Statistics -->

    <div class="stats-grid">


        <div class="stat-card">

            <div class="stat-top">

                <div class="stat-icon employees-icon">
                    👥
                </div>

                <span class="stat-label">
                    Employees
                </span>

            </div>


            <div
                class="stat-number"
                id="totalEmployees">

                0

            </div>


            <div class="stat-bottom">

                <span class="stat-green">
                    ● Workforce
                </span>

                <span class="stat-arrow">
                    →
                </span>
```

```
    </div>

</div>


<div class="stat-card">

    <div class="stat-top">

        <div class="stat-icon jobs-icon">
            💼
        </div>

        <span class="stat-label">
            Open Jobs
        </span>

    </div>


    <div
        class="stat-number"
        id="totalJobs">

        0

    </div>


    <div class="stat-bottom">

        <span class="stat-green">
            ● Recruitment
        </span>

        <span class="stat-arrow">
            →
        </span>

    </div>

</div>


<div class="stat-card">

    <div class="stat-top">

        <div class="stat-icon candidates-icon">
            👤
        </div>

        <span class="stat-label">
            Candidates
        </span>

    </div>


    <div
        class="stat-number"
        id="totalCandidates">

        0

    </div>


    <div class="stat-bottom">

        <span class="stat-green">
            ● Applications
        </span>

        <span class="stat-arrow">
            →
        </span>

    </div>

</div>


<div class="stat-card">

    <div class="stat-top">

        <div class="stat-icon payroll-icon">
            💰
        </div>

        <span class="stat-label">
            Payroll
        </span>

    </div>


    <div
        class="stat-number"
        id="totalPayroll">

        ₹0

    </div>
```

```
        <div class="stat-bottom">

            <span class="stat-green">
                ● Processed
            </span>

            <span class="stat-arrow">
                →
            </span>

        </div>

    </div>

</div>


<!-- Main Grid -->

<div class="content-grid">


    <!-- Employee Overview -->

    <div class="dashboard-card">

        <div class="card-header">

            <div>

                <h2>
                    Employee Overview
                </h2>

                <p>
                    Current workforce summary
                </p>

            </div>


            <a href="employees.html"
                class="view-btn">

                View All →

            </a>

        </div>


        <div class="overview-content">


            <div class="overview-circle">

                <div class="circle-inner">

                    <strong id="employeeCircle">
                        0
                    </strong>

                    <span>
                        Employees
                    </span>

                </div>

            </div>


            <div class="overview-details">


                <div class="overview-item">

                    <span
                        class="overview-dot blue">
                    </span>

                    <div>

                        <strong
                            id="overviewEmployees">
                            0
                        </strong>

                        <small>
                            Total Employees
                        </small>

                    </div>

                </div>


                <div class="overview-item">

                    <span
                        class="overview-dot green">
                    </span>

                    <div>
```

```
                    <strong id="activeEmployees">
                        0
                    </strong>

                    <small>
                        Active Workforce
                    </small>

                </div>

            </div>


            <div class="overview-item">

                <span
                    class="overview-dot orange">
                </span>

                <div>

                    <strong id="overviewLeaves">
                        0
                    </strong>

                    <small>
                        Pending Leaves
                    </small>

                </div>

            </div>

        </div>

    </div>

</div>


<!-- Recruitment -->

<div class="dashboard-card">

    <div class="card-header">

        <div>

            <h2>
                Recruitment
            </h2>

            <p>
                Candidate pipeline
            </p>

        </div>


        <a href="candidates.html"
            class="view-btn">

            View All →

        </a>

    </div>


    <div class="recruitment-list">


        <div class="recruitment-item">

            <div class="recruitment-name">

                <span class="status-dot applied">
                </span>

                Applied

            </div>

            <strong id="appliedCount">
                0
            </strong>

        </div>


        <div class="progress-line">

            <div
                class="progress-bar applied-bar"
                id="appliedBar">
            </div>

        </div>


        <div class="recruitment-item">

            <div class="recruitment-name">
```

```
                    <span class="status-dot shortlisted">
                    </span>

                    Shortlisted

                </div>

                <strong id="shortlistedCount">
                    0
                </strong>

            </div>


            <div class="progress-line">

                <div
                    class="progress-bar shortlisted-bar"
                    id="shortlistedBar">
                </div>

            </div>


            <div class="recruitment-item">

                <div class="recruitment-name">

                    <span class="status-dot interview">
                    </span>

                    Interview

                </div>

                <strong id="interviewCount">
                    0
                </strong>

            </div>


            <div class="progress-line">

                <div
                    class="progress-bar interview-bar"
                    id="interviewBar">
                </div>

            </div>


            <div class="recruitment-item">

                <div class="recruitment-name">

                    <span class="status-dot selected">
                    </span>

                    Selected

                </div>

                <strong id="selectedCount">
                    0
                </strong>

            </div>


            <div class="progress-line">

                <div
                    class="progress-bar selected-bar"
                    id="selectedBar">
                </div>

            </div>

        </div>

    </div>

</div>


<!-- =========================
     HR ANALYTICS
========================== -->

<section class="analytics-section">

    <div class="section-header">

        <div>

            <h2>
                HR Analytics
            </h2>

            <p>
                Quick overview of your HR operations
            </p>

        </div>
```

```
</div>

<div class="analytics-grid">

    <!-- Attendance -->

    <div class="analytics-card">

        <div class="analytics-card-header">

            <span class="analytics-icon">
                🟢
            </span>

            <div>

                <h3>
                    Attendance
                </h3>

                <span>
                    Overall attendance
                </span>

            </div>

        </div>


        <div class="analytics-value">

            <span id="analyticsAttendanceRate">
                0%
            </span>

        </div>


        <div class="analytics-progress">

            <div
                id="analyticsAttendanceBar"
                class="analytics-progress-fill">
            </div>

        </div>


        <div class="analytics-details">

            <span>

                Present:

                <strong id="analyticsPresent">
                    0
                </strong>

            </span>


            <span>

                Absent:

                <strong id="analyticsAbsent">
                    0
                </strong>

            </span>

        </div>

    </div>


    <!-- Payroll -->

    <div class="analytics-card">

        <div class="analytics-card-header">

            <span class="analytics-icon">
                💰
            </span>

            <div>

                <h3>
                    Payroll
                </h3>

                <span>
                    Payment overview
                </span>

            </div>

        </div>


        <div class="analytics-value">
```

```
        <span id="analyticsPayrollAmount">
            ₹0
        </span>

    </div>


    <div class="analytics-details">

        <span>

            Paid:

            <strong id="analyticsPaidPayroll">
                0
            </strong>

        </span>


        <span>

            Pending:

            <strong id="analyticsPendingPayroll">
                0
            </strong>

        </span>

    </div>

</div>


<!-- Leaves -->

<div class="analytics-card">

    <div class="analytics-card-header">

        <span class="analytics-icon">
            📅
        </span>

        <div>

            <h3>
                Leaves
            </h3>

            <span>
                Leave requests
            </span>

        </div>

    </div>


    <div class="analytics-value">

        <span id="analyticsTotalLeaves">
            0
        </span>

    </div>


    <div class="analytics-details">

        <span>

            Pending:

            <strong id="analyticsPendingLeaves">
                0
            </strong>

        </span>


        <span>

            Approved:

            <strong id="analyticsApprovedLeaves">
                0
            </strong>

        </span>


        <span>

            Rejected:

            <strong id="analyticsRejectedLeaves">
                0
            </strong>

        </span>

    </div>
```

```
    </div>


    <!-- Recruitment -->

    <div class="analytics-card">

        <div class="analytics-card-header">

            <span class="analytics-icon">
                👥
            </span>

            <div>

                <h3>
                    Recruitment
                </h3>

                <span>
                    Candidate pipeline
                </span>

            </div>

        </div>


        <div class="analytics-value">

            <span id="analyticsCandidates">
                0
            </span>

        </div>


        <div class="recruitment-mini-list">


            <div>

                <span>
                    Applied
                </span>

                <strong id="analyticsApplied">
                    0
                </strong>

            </div>


            <div>

                <span>
                    Shortlisted
                </span>

                <strong id="analyticsShortlisted">
                    0
                </strong>

            </div>


            <div>

                <span>
                    Interview
                </span>

                <strong id="analyticsInterview">
                    0
                </strong>

            </div>


            <div>

                <span>
                    Selected
                </span>

                <strong id="analyticsSelected">
                    0
                </strong>

            </div>

        </div>

    </div>

</div>


<!-- Department Analytics -->

<div class="department-analytics">

    <div class="section-header">
```

```
            <div>

                <h2>
                    Department Overview
                </h2>

                <p>
                    Employees and salary distribution
                </p>

            </div>

        </div>


        <div
            id="departmentAnalytics"
            class="department-list">

            <div class="analytics-empty">

                Loading department data...

            </div>

        </div>

    </div>

</section>


<!-- Bottom Grid -->

<div class="bottom-grid">


    <!-- Quick Actions -->

    <div class="dashboard-card">

        <div class="card-header">

            <div>

                <h2>
                    Quick Actions
                </h2>

                <p>
                    Frequently used modules
                </p>

            </div>

        </div>


        <div class="quick-actions">


            <a href="employees.html"
                class="quick-action">

                <span class="quick-icon">
                    👥
                </span>

                <div>

                    <strong>
                        Employees
                    </strong>

                    <small>
                        Manage workforce
                    </small>

                </div>

                <span>→</span>

            </a>


            <a href="jobs.html"
                class="quick-action">

                <span class="quick-icon">
                    💼
                </span>

                <div>

                    <strong>
                        Jobs
                    </strong>

                    <small>
                        Manage vacancies
                    </small>

                </div>

                <span>→</span>
```

```
        </a>

        <a href="candidates.html"
            class="quick-action">

            <span class="quick-icon">
                👤
            </span>

            <div>

                <strong>
                    Candidates
                </strong>

                <small>
                    Review applications
                </small>

            </div>

            <span>→</span>

        </a>

        <a href="payroll.html"
            class="quick-action">

            <span class="quick-icon">
                💰
            </span>

            <div>

                <strong>
                    Payroll
                </strong>

                <small>
                    Manage salary
                </small>

            </div>

            <span>→</span>

        </a>

    </div>

</div>


<!-- System Summary -->

<div class="dashboard-card">

    <div class="card-header">

        <div>

            <h2>
                HR Summary
            </h2>

            <p>
                Current system status
            </p>

        </div>

    </div>


    <div class="summary-list">


        <div class="summary-row">

            <div class="summary-left">

                <span class="summary-icon">
                    🎯
                </span>

                <span>
                    Interviews
                </span>

            </div>

            <strong id="totalInterviews">
                0
            </strong>

        </div>


        <div class="summary-row">

            <div class="summary-left">
```

```
                    <span class="summary-icon">
                        📅
                    </span>

                    <span>
                        Pending Leaves
                    </span>

                </div>

                <strong id="summaryLeaves">
                    0
                </strong>

            </div>


            <div class="summary-row">

                <div class="summary-left">

                    <span class="summary-icon">
                        💰
                    </span>

                    <span>
                        Payroll Records
                    </span>

                </div>

                <strong id="summaryPayroll">
                    0
                </strong>

            </div>


            <div class="summary-row">

                <div class="summary-left">

                    <span class="summary-icon">
                        🔔
                    </span>

                    <span>
                        Notifications
                    </span>

                </div>

                <strong id="summaryNotifications">
                    0
                </strong>

            </div>

        </div>

    </div>

</div>


<!-- Recent Activity -->

<div class="dashboard-card activity-card">

    <div class="card-header">

        <div>

            <h2>
                Recent Activity
            </h2>

            <p>
                Latest HR system activities
            </p>

        </div>

    </div>


    <div
        class="activity-list"
        id="activityList">

        <div class="activity-item">

            <div class="activity-icon">
                ✨
            </div>

            <div class="activity-content">

                <strong>
                    HRGenius
                </strong>

                <span>
                    Dashboard loaded successfully
                </span>
```

```
                        </div>

                        <small>
                            Now
                        </small>

                    </div>

                </div>

            </div>

        </section>

    </main>


    <script src="JS/theme.js"></script>

    <script src="JS/auth.js"></script>

    <script src="JS/dashboard.js"></script>


</body>

</html>
```

Frontend/departments.html

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta
        name="viewport"
        content="width=device-width, initial-scale=1.0"
    >

    <title>HRGenius - Departments</title>


    <!-- ================= GLOBAL CSS ================= -->

    <link
        rel="stylesheet"
        href="CSS/style.css"
    >

    <!-- ================= DEPARTMENT CSS ================= -->

    <link
        rel="stylesheet"
        href="CSS/departments.css"
    >

</head>


<body>


    <!-- ======================================================
         NAVBAR
    ====================================================== -->

    <header class="topbar">

        <div class="brand">

            <h2>
                HRGenius
            </h2>

        </div>


        <div class="topbar-actions">

            <button
                type="button"
                class="theme-btn"
                id="themeButton"
                onclick="toggleTheme()"
                aria-label="Toggle theme"
            >
                🌙
            </button>


            <button
                type="button"
                class="logout-btn"
                onclick="logout()"
            >
                Logout
            </button>

        </div>

    </header>



    <!-- ======================================================
         MAIN CONTENT
    ====================================================== -->

    <main class="department-page">


        <!-- ================================================
             PAGE HEADER
        ================================================ -->

        <section class="page-header">

            <div class="page-title">

                <h1>
                    Departments
                </h1>

                <p>
                    Manage departments, employees and department heads
                </p>

            </div>
```

```
    <button
        type="button"
        class="primary-btn"
        onclick="openAddDepartmentModal()"
    >
        + Add Department
    </button>

</section>



<!-- ================================================
     STATISTICS
================================================= -->

<section class="stats-grid">


    <!-- Total Departments -->

    <div class="stat-card">

        <div class="stat-icon">
            🏢
        </div>

        <div class="stat-content">

            <span>
                Total Departments
            </span>

            <strong id="totalDepartments">
                0
            </strong>

        </div>

    </div>



    <!-- Active Departments -->

    <div class="stat-card">

        <div class="stat-icon">
            ✅
        </div>

        <div class="stat-content">

            <span>
                Active Departments
            </span>

            <strong id="activeDepartments">
                0
            </strong>

        </div>

    </div>



    <!-- Total Employees -->

    <div class="stat-card">

        <div class="stat-icon">
            👥
        </div>

        <div class="stat-content">

            <span>
                Total Employees
            </span>

            <strong id="totalEmployees">
                0
            </strong>

        </div>

    </div>



    <!-- Total Salary -->

    <div class="stat-card">

        <div class="stat-icon">
            💰
        </div>

        <div class="stat-content">

            <span>
                Total Salary
            </span>
```

```
            <strong id="totalSalary">
                ₹0
            </strong>

        </div>

    </div>

</section>



<!-- ================================================
     SEARCH + FILTERS
================================================= -->

<section class="filter-section">


    <!-- Search -->

    <div class="search-area">

        <div class="search-box">

            <span class="search-icon">
                🔍
            </span>


            <input
                type="text"
                id="searchInput"
                placeholder="Search department, description or head..."
                autocomplete="off"
            >


            <button
                type="button"
                id="clearSearch"
                class="clear-search-btn"
                onclick="clearDepartmentSearch()"
                style="display: none;"
                aria-label="Clear search"
            >
                ×
            </button>


            <span class="search-shortcut">
                Ctrl K
            </span>

        </div>


        <div
            id="searchResultText"
            class="search-result-text"
        >
            0 departments
        </div>

    </div>



    <!-- Right Filters -->

    <div class="filter-right">


        <!-- Status -->

        <div class="status-filter">

            <label for="statusFilter">
                Status
            </label>


            <select id="statusFilter">

                <option value="all">
                    All Status
                </option>

                <option value="Active">
                    Active
                </option>

                <option value="Inactive">
                    Inactive
                </option>

            </select>

        </div>



        <!-- Reset -->
```

```
        <button
            type="button"
            class="secondary-btn reset-btn"
            onclick="resetFilters()"
        >
            ↻ Reset
        </button>

    </div>

</section>



<!-- ================================================
     DEPARTMENT TABLE
================================================= -->

<section class="table-card">


    <!-- Table Header -->

    <div class="table-header">

        <div>

            <h2>
                Department List
            </h2>

            <p id="departmentCountText">
                0 departments
            </p>

        </div>

    </div>



    <!-- Table -->

    <div class="table-wrapper">

        <table>


            <!-- Table Head -->

            <thead>

                <tr>


                    <!-- Department -->

                    <th>

                        <button
                            type="button"
                            class="sort-btn"
                            onclick="changeDepartmentSort('name')"
                        >

                            Department

                            <span id="sort-name">
                                ↕
                            </span>

                        </button>

                    </th>



                    <!-- Head -->

                    <th>
                        Department Head
                    </th>



                    <!-- Employees -->

                    <th>

                        <button
                            type="button"
                            class="sort-btn"
                            onclick="changeDepartmentSort('employees')"
                        >

                            Employees

                            <span id="sort-employees">
                                ↕
                            </span>

                        </button>

                    </th>
```

```
                        <!-- Salary -->

                        <th>

                            <button
                                type="button"
                                class="sort-btn"
                                onclick="changeDepartmentSort('salary')"
                            >

                                Total Salary

                                <span id="sort-salary">
                                    ↕
                                </span>

                            </button>

                        </th>



                        <!-- Status -->

                        <th>
                            Status
                        </th>



                        <!-- Created -->

                        <th>

                            <button
                                type="button"
                                class="sort-btn"
                                onclick="changeDepartmentSort('createdAt')"
                            >

                                Created

                                <span id="sort-createdAt">
                                    ↕
                                </span>

                            </button>

                        </th>



                        <!-- Actions -->

                        <th>
                            Actions
                        </th>

                    </tr>

                </thead>



                <!-- Table Body -->

                <tbody id="departmentTableBody">

                </tbody>


            </table>

        </div>

    </section>

</main>



<!-- ====================================================
     ADD / EDIT DEPARTMENT MODAL
===================================================== -->

<div
    class="modal-overlay"
    id="departmentModal"
>

    <div class="modal">


        <!-- Modal Header -->

        <div class="modal-header">

            <div>

                <h2 id="departmentModalTitle">
                    Add Department
                </h2>
```

```
        <p>
            Enter department information
        </p>

    </div>


    <button
        type="button"
        class="close-btn"
        onclick="closeDepartmentModal()"
        aria-label="Close"
    >
        ×
    </button>

</div>



<!-- Form -->

<form id="departmentForm">


    <!-- Hidden ID -->

    <input
        type="hidden"
        id="departmentId"
    >



    <!-- Department Name -->

    <div class="form-group">

        <label for="departmentName">
            Department Name
        </label>


        <input
            type="text"
            id="departmentName"
            placeholder="Enter department name"
            required
        >

    </div>



    <!-- Description -->

    <div class="form-group">

        <label for="departmentDescription">
            Description
        </label>


        <textarea
            id="departmentDescription"
            rows="4"
            placeholder="Enter department description"
        ></textarea>

    </div>



    <!-- Status -->

    <div class="form-group">

        <label for="departmentStatus">
            Status
        </label>


        <select id="departmentStatus">

            <option value="Active">
                Active
            </option>

            <option value="Inactive">
                Inactive
            </option>

        </select>

    </div>



    <!-- Buttons -->

    <div class="modal-actions">

        <button
            type="button"
            class="secondary-btn"
```

```
                    onclick="closeDepartmentModal()"
                >
                    Cancel
                </button>


                <button
                    type="submit"
                    class="primary-btn"
                >
                    Save Department
                </button>

            </div>

        </form>

    </div>

</div>



<!-- ====================================================
     DELETE DEPARTMENT MODAL
===================================================== -->

<div
    class="modal-overlay"
    id="deleteModal"
>

    <div class="modal delete-modal">


        <div class="delete-icon">
            
        </div>


        <h2>
            Delete Department
        </h2>


        <p>
            Are you sure you want to delete
            <strong id="deleteDepartmentName">
                -
            </strong>?
        </p>


        <p class="delete-warning">
            This action cannot be undone.
        </p>



        <div class="modal-actions">

            <button
                type="button"
                class="secondary-btn"
                onclick="closeDeleteModal()"
            >
                Cancel
            </button>


            <button
                type="button"
                class="danger-btn"
                onclick="confirmDeleteDepartment()"
            >
                Delete Department
            </button>

        </div>

    </div>

</div>



<!-- ====================================================
     CHANGE DEPARTMENT HEAD MODAL
===================================================== -->

<div
    class="modal-overlay"
    id="headModal"
>

    <div class="modal">


        <!-- Header -->

        <div class="modal-header">

            <div>

                <h2>
```

```
                    Change Department Head
                </h2>

                <p id="headDepartmentName">
                    -
                </p>

            </div>


            <button
                type="button"
                class="close-btn"
                onclick="closeHeadModal()"
                aria-label="Close"
            >
                ×
            </button>

        </div>



        <!-- Hidden Department ID -->

        <input
            type="hidden"
            id="headDepartmentId"
        >



        <!-- Employee Select -->

        <div class="form-group">

            <label for="departmentHeadSelect">
                Select Employee
            </label>


            <select id="departmentHeadSelect">

                <option value="">
                    No Department Head
                </option>

            </select>

        </div>



        <!-- Preview -->

        <div
            class="head-preview"
            id="headPreview"
        >
            Select an employee
        </div>



        <!-- Buttons -->

        <div class="modal-actions">

            <button
                type="button"
                class="secondary-btn"
                onclick="closeHeadModal()"
            >
                Cancel
            </button>


            <button
                type="button"
                class="primary-btn"
                onclick="saveDepartmentHead()"
            >
                Save Head
            </button>

        </div>

    </div>

</div>



<!-- ====================================================
     VIEW EMPLOYEES MODAL
==================================================== -->

<div
    class="modal-overlay"
    id="employeesModal"
>

    <div class="modal large-modal">
```

```
        <!-- Header -->

        <div class="modal-header">

            <div>

                <h2 id="employeesModalTitle">
                    Department Employees
                </h2>

                <p id="employeesModalSubtitle">
                    -
                </p>

            </div>


            <button
                type="button"
                class="close-btn"
                onclick="closeEmployeesModal()"
                aria-label="Close"
            >
                ×
            </button>

        </div>



        <!-- Department Summary -->

        <div
            class="department-summary"
            id="employeesDepartmentSummary"
        >
        </div>



        <!-- Employees Table -->

        <div class="table-wrapper">

            <table>

                <thead>

                    <tr>

                        <th>
                            Employee
                        </th>

                        <th>
                            Email
                        </th>

                        <th>
                            Designation
                        </th>

                        <th>
                            Salary
                        </th>

                    </tr>

                </thead>


                <tbody id="employeesTableBody">

                </tbody>

            </table>

        </div>



        <!-- Close -->

        <div class="modal-actions">

            <button
                type="button"
                class="secondary-btn"
                onclick="closeEmployeesModal()"
            >
                Close
            </button>

        </div>

    </div>

</div>



<!-- ====================================================
     DEPARTMENT DETAILS MODAL
==================================================== -->
```

```
<div
    class="modal-overlay"
    id="departmentDetailsModal"
>

    <div class="modal details-modal">


        <!-- Header -->

        <div class="modal-header">

            <div>

                <h2>
                    Department Details
                </h2>

                <p>
                    Complete department information
                </p>

            </div>


            <button
                type="button"
                class="close-btn"
                onclick="closeDepartmentDetailsModal()"
                aria-label="Close"
            >
                ×
            </button>

        </div>



        <!-- Main Details -->

        <div class="details-main">

            <h3 id="detailsDepartmentName">
                -
            </h3>


            <p id="detailsDepartmentDescription">
                -
            </p>

        </div>



        <!-- Details Grid -->

        <div class="details-grid">


            <!-- Head -->

            <div class="details-card">

                <span>
                    Department Head
                </span>


                <strong id="detailsHeadName">
                    -
                </strong>


                <small id="detailsHeadDesignation">
                    -
                </small>

            </div>



            <!-- Employees -->

            <div class="details-card">

                <span>
                    Employees
                </span>


                <strong id="detailsEmployeeCount">
                    0
                </strong>

            </div>



            <!-- Total Salary -->

            <div class="details-card">

                <span>
                    Total Salary
```

```
                </span>

                <strong id="detailsTotalSalary">
                    ₹0
                </strong>

            </div>


            <!-- Average Salary -->

            <div class="details-card">

                <span>
                    Average Salary
                </span>

                <strong id="detailsAverageSalary">
                    ₹0
                </strong>

            </div>


            <!-- Status -->

            <div class="details-card">

                <span>
                    Status
                </span>

                <strong id="detailsStatus">
                    -
                </strong>

            </div>


            <!-- Created -->

            <div class="details-card">

                <span>
                    Created
                </span>

                <strong id="detailsCreated">
                    -
                </strong>

            </div>

        </div>


        <!-- Close -->

        <div class="modal-actions">

            <button
                type="button"
                class="secondary-btn"
                onclick="closeDepartmentDetailsModal()"
            >
                Close
            </button>

        </div>

    </div>

</div>


<!-- ========================================================
     TOAST CONTAINER
========================================================= -->

<div
    id="toastContainer"
    class="toast-container"
>
</div>


<!-- ========================================================
     JAVASCRIPT
========================================================= -->

<script src="JS/theme.js"></script>

<script src="JS/departments.js"></script>
```

```
</body>
</html>
```

Frontend/employees.html

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta
        name="viewport"
        content="width=device-width, initial-scale=1.0"
    >

    <title>HRGenius - Employees</title>

    <link
        rel="stylesheet"
        href="CSS/style.css"
    >

    <style>

        /* ========================
           NAVBAR
        ======================== */

        .navbar {
            height: 65px;
            background: var(--primary);
            color: #ffffff;

            display: flex;
            align-items: center;
            justify-content: space-between;

            padding: 0 30px;
        }

        .navbar h2 {
            font-size: 24px;
            color: #ffffff;
        }

        .nav-buttons {
            display: flex;
            align-items: center;
            gap: 10px;
        }

        .nav-btn {
            background: #ffffff;
            color: var(--primary);

            border: none;

            padding: 9px 16px;

            border-radius: 6px;

            cursor: pointer;

            font-weight: bold;
        }

        .nav-btn:hover {
            background: #e5e7eb;
        }


        /* ========================
           CONTAINER
        ======================== */

        .container {
            padding: 30px;
        }


        .heading {
            margin-bottom: 25px;
        }

        .heading h1 {
            margin-bottom: 8px;
            color: var(--text);
        }

        .heading p {
            color: var(--muted);
        }


        /* ========================
           FORM
        ======================== */

        .form-box {
            background: var(--card);

            padding: 25px;

            border-radius: 10px;
```

```
    border: 1px solid var(--border);

    box-shadow: var(--shadow);

    margin-bottom: 30px;
}

.form-box h2 {
    margin-bottom: 20px;
    color: var(--text);
}


.form-grid {
    display: grid;

    grid-template-columns: repeat(2, 1fr);

    gap: 15px;
}


.form-group {
    display: flex;
    flex-direction: column;
}


.form-group label {
    margin-bottom: 6px;

    font-weight: bold;

    color: var(--text);
}


.form-group input {
    padding: 11px;

    border: 1px solid var(--border);

    border-radius: 6px;

    font-size: 14px;

    background: var(--input);

    color: var(--text);
}


.form-group input:focus {
    outline: none;

    border-color: var(--primary);
}


.form-actions {
    margin-top: 20px;

    display: flex;

    gap: 10px;
}


.submit-btn {
    background: var(--primary);

    color: #ffffff;

    border: none;

    padding: 11px 22px;

    border-radius: 6px;

    cursor: pointer;

    font-weight: bold;
}


.submit-btn:hover {
    background: var(--primary-hover);
}


.cancel-btn {
    background: #6b7280;

    color: #ffffff;

    border: none;

    padding: 11px 22px;

    border-radius: 6px;

    cursor: pointer;

    display: none;
}
```

```
.cancel-btn:hover {
    background: #4b5563;
}


/* ========================
   SEARCH
======================== */

.search-box {
    margin-bottom: 20px;

    display: flex;

    justify-content: space-between;

    align-items: center;

    gap: 15px;
}


.search-input {
    width: 100%;

    max-width: 400px;

    padding: 11px 14px;

    border: 1px solid var(--border);

    border-radius: 6px;

    background: var(--input);

    color: var(--text);

    font-size: 14px;
}


.search-input:focus {
    outline: none;

    border-color: var(--primary);
}


/* ========================
   MESSAGE
======================== */

#message {
    margin-bottom: 20px;

    padding: 12px;

    border-radius: 6px;

    display: none;
}


/* ========================
   TABLE
======================== */

.table-box {
    background: var(--card);

    padding: 25px;

    border-radius: 10px;

    border: 1px solid var(--border);

    box-shadow: var(--shadow);

    overflow-x: auto;
}


.table-box h2 {
    margin-bottom: 20px;

    color: var(--text);
}


table {
    width: 100%;

    border-collapse: collapse;

    min-width: 1000px;

    background: var(--card);
}


th,
td {
    padding: 12px;
```

```
    border-bottom: 1px solid var(--border);

    text-align: left;
}


th {
    background: var(--hover);

    color: var(--text);
}


td {
    color: var(--text);
}


tr:hover {
    background: var(--hover);
}


/* ========================
   ACTION BUTTONS
======================== */

.edit-btn {
    background: #f59e0b;

    color: #ffffff;

    border: none;

    padding: 7px 12px;

    border-radius: 5px;

    cursor: pointer;

    margin-right: 5px;
}


.delete-btn {
    background: #dc2626;

    color: #ffffff;

    border: none;

    padding: 7px 12px;

    border-radius: 5px;

    cursor: pointer;
}


.edit-btn:hover {
    background: #d97706;
}


.delete-btn:hover {
    background: #b91c1c;
}


.no-data {
    text-align: center;

    padding: 20px;

    color: var(--muted);
}


/* ========================
   RESPONSIVE
======================== */

@media (max-width: 700px) {

    .form-grid {
        grid-template-columns: 1fr;
    }


    .container {
        padding: 15px;
    }


    .navbar {
        padding: 0 15px;
    }


    .nav-buttons {
        gap: 5px;
    }
```

```
            .nav-btn {
                padding: 8px 10px;
            }


            .search-box {
                flex-direction: column;

                align-items: stretch;
            }

        }

    </style>

</head>


<body>


    <!-- =========================
         NAVBAR
    ========================= -->

    <div class="navbar">

        <h2>
            HRGenius
        </h2>


        <div class="nav-buttons">

            <button
                class="nav-btn"
                onclick="window.location.href='dashboard.html'"
            >
                Dashboard
            </button>


            <button
                id="themeToggle"
                class="theme-toggle"
                onclick="toggleTheme()"
            >
                🌙 Dark Mode
            </button>


            <button
                class="nav-btn"
                onclick="logout()"
            >
                Logout
            </button>

        </div>

    </div>


    <!-- =========================
         MAIN
    ========================= -->

    <div class="container">


        <!-- =========================
             HEADING
        ========================= -->

        <div class="heading">

            <h1>
                Employee Management
            </h1>

            <p>
                Manage your organization's employees
            </p>

        </div>


        <!-- =========================
             MESSAGE
        ========================= -->

        <div id="message"></div>


        <!-- =========================
             EMPLOYEE FORM
        ========================= -->

        <div
            class="form-box"
            id="employeeFormBox"
        >

            <h2 id="formTitle">
                Add Employee
```

```
</h2>

<form id="employeeForm">

    <div class="form-grid">

        <!-- NAME -->

        <div class="form-group">

            <label for="name">
                Employee Name
            </label>

            <input
                type="text"
                id="name"
                placeholder="Enter employee name"
                required
            >

        </div>


        <!-- EMAIL -->

        <div class="form-group">

            <label for="email">
                Email
            </label>

            <input
                type="email"
                id="email"
                placeholder="Enter email"
                required
            >

        </div>


        <!-- PHONE -->

        <div class="form-group">

            <label for="phone">
                Phone
            </label>

            <input
                type="text"
                id="phone"
                placeholder="Enter phone number"
                required
            >

        </div>


        <!-- DEPARTMENT -->

        <div class="form-group">

            <label for="department">
                Department
            </label>

            <input
                type="text"
                id="department"
                placeholder="Example: IT"
                required
            >

        </div>


        <!-- DESIGNATION -->

        <div class="form-group">

            <label for="designation">
                Designation
            </label>

            <input
                type="text"
                id="designation"
                placeholder="Example: Software Developer"
                required
            >

        </div>


        <!-- SALARY -->

        <div class="form-group">

            <label for="salary">
                Salary
            </label>
```

```
                <input
                    type="number"
                    id="salary"
                    placeholder="Enter salary"
                    min="0"
                    required
                >

            </div>


            <!-- JOINING DATE -->

            <div class="form-group">

                <label for="joiningDate">
                    Joining Date
                </label>

                <input
                    type="date"
                    id="joiningDate"
                    required
                >

            </div>


            <!-- ADDRESS -->

            <div class="form-group">

                <label for="address">
                    Address
                </label>

                <input
                    type="text"
                    id="address"
                    placeholder="Enter address"
                    required
                >

            </div>

        </div>


        <!-- FORM ACTIONS -->

        <div class="form-actions">

            <button
                type="submit"
                class="submit-btn"
                id="submitBtn"
            >
                Add Employee
            </button>


            <button
                type="button"
                class="cancel-btn"
                id="cancelBtn"
                onclick="cancelEdit()"
            >
                Cancel
            </button>

        </div>

    </form>

</div>


<!-- ========================
     EMPLOYEE TABLE
======================== -->

<div class="table-box">

    <h2>
        Employee Records
    </h2>


    <!-- SEARCH -->

    <div class="search-box">

        <input
            type="text"
            id="searchEmployee"
            class="search-input"
            placeholder="🔍 Search employee by name, email, department..."
        >

    </div>


    <table>
```

```
                <thead>
                    <tr>
                        <th>Name</th>
                        <th>Email</th>
                        <th>Phone</th>
                        <th>Department</th>
                        <th>Designation</th>
                        <th>Salary</th>
                        <th>Joining Date</th>
                        <th>Address</th>
                        <th>Action</th>
                    </tr>
                </thead>

                <tbody id="employeeTableBody">
                    <tr>
                        <td
                            colspan="9"
                            class="no-data"
                        >
                            Loading employees...
                        </td>
                    </tr>
                </tbody>
            </table>
        </div>
    </div>

    <!-- =========================
         JAVASCRIPT
    ========================== -->
    <script src="JS/theme.js"></script>
    <script src="JS/auth.js"></script>
    <script src="JS/employees.js"></script>

</body>
</html>
```

Frontend/forgot-password.html

```
<!DOCTYPE html>

<html lang="en">

<head>


<meta charset="UTF-8">

<meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
>

<title>HRGenius - Forgot Password</title>

<link
    rel="stylesheet"
    href="CSS/style.css"
>

<style>

    body {
        min-height: 100vh;
        display: flex;
        align-items: center;
        justify-content: center;
        background: var(--bg);
    }

    .reset-container {
        width: 100%;
        max-width: 460px;
        padding: 20px;
    }

    .reset-card {
        background: var(--card);
        border: 1px solid var(--border);
        border-radius: 20px;
        padding: 40px;
        box-shadow: var(--shadow);
    }

    .logo {
        text-align: center;
        margin-bottom: 30px;
    }

    .logo h1 {
        margin: 0;
        color: var(--primary);
        font-size: 32px;
    }

    .logo p {
        margin-top: 8px;
        color: var(--muted);
    }

    .step {
        display: none;
    }

    .step.active {
        display: block;
    }

    .step h2 {
        color: var(--text);
        margin-bottom: 10px;
    }

    .step-description {
        color: var(--muted);
        margin-bottom: 25px;
        line-height: 1.6;
    }

    .form-group {
        margin-bottom: 20px;
    }

    .form-group label {
        display: block;
        margin-bottom: 8px;
        color: var(--text);
        font-weight: 600;
    }

    .form-group input {
        width: 100%;
        padding: 14px 15px;
        border: 1px solid var(--border);
        border-radius: 10px;
        background: var(--input);
        color: var(--text);
        font-size: 15px;
        box-sizing: border-box;
    }
```

```
    .form-group input:focus {
        outline: none;
        border-color: var(--primary);
        box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.10);
    }

    .btn {
        width: 100%;
        padding: 14px;
        border: none;
        border-radius: 10px;
        background: var(--primary);
        color: white;
        font-size: 16px;
        font-weight: 600;
        cursor: pointer;
    }

    .btn:hover {
        background: var(--primary-hover);
    }

    .back-login {
        text-align: center;
        margin-top: 22px;
    }

    .back-login a {
        color: var(--primary);
        text-decoration: none;
        font-weight: 600;
    }

    .back-login a:hover {
        text-decoration: underline;
    }

    .message {
        margin-top: 18px;
        padding: 12px;
        border-radius: 8px;
        display: none;
        font-size: 14px;
        line-height: 1.5;
    }

    .message.show {
        display: block;
    }

    .message.success {
        background: rgba(34, 197, 94, 0.12);
        color: #16a34a;
    }

    .message.error {
        background: rgba(239, 68, 68, 0.12);
        color: #dc2626;
    }

    .otp-input {
        text-align: center;
        font-size: 24px !important;
        letter-spacing: 8px;
        font-weight: bold;
    }

    .password-rules {
        color: var(--muted);
        font-size: 13px;
        margin-top: 8px;
    }

    @media (max-width: 500px) {

        .reset-container {
            padding: 15px;
        }

        .reset-card {
            padding: 25px;
        }

        .logo h1 {
            font-size: 28px;
        }

    }

</style>
```

</head>

<body>

```
<div class="reset-container">

    <div class="reset-card">

        <div class="logo">

            <h1>HRGenius</h1>
```

```
    <p>Password Recovery</p>

</div>

<!-- STEP 1 -->

<div
    class="step active"
    id="step1"
>

    <h2>
        Forgot Password?
    </h2>

    <p class="step-description">
        Enter your registered email address and
        we'll send you a 6-digit OTP.
    </p>

    <div class="form-group">

        <label for="email">
            Email Address
        </label>

        <input
            type="email"
            id="email"
            placeholder="Enter your registered email"
        >

    </div>

    <button
        class="btn"
        onclick="sendOTP()"
    >
        Send OTP
    </button>

</div>

<!-- STEP 2 -->

<div
    class="step"
    id="step2"
>

    <h2>
        Verify OTP
    </h2>

    <p class="step-description">
        Enter the 6-digit OTP sent to your email.
    </p>

    <div class="form-group">

        <label for="otp">
            OTP
        </label>

        <input
            type="text"
            id="otp"
            class="otp-input"
            maxlength="6"
            inputmode="numeric"
            placeholder="000000"
        >

    </div>

    <button
        class="btn"
        onclick="verifyOTP()"
    >
        Verify OTP
    </button>

</div>

<!-- STEP 3 -->

<div
    class="step"
    id="step3"
>

    <h2>
        Create New Password
    </h2>

    <p class="step-description">
        Enter your new password below.
    </p>

    <div class="form-group">

        <label for="newPassword">
            New Password
        </label>
```

```
                <input
                    type="password"
                    id="newPassword"
                    placeholder="Enter new password"
                >

                <div class="password-rules">
                    Minimum 6 characters
                </div>

            </div>

            <div class="form-group">

                <label for="confirmPassword">
                    Confirm Password
                </label>

                <input
                    type="password"
                    id="confirmPassword"
                    placeholder="Confirm new password"
                >

            </div>

            <button
                class="btn"
                onclick="resetPassword()"
            >
                Reset Password
            </button>

        </div>

        <div
            id="message"
            class="message"
        ></div>

        <div class="back-login">

            <a href="login.html">
                ← Back to Login
            </a>

        </div>

    </div>

</div>

<script>

    let userEmail = "";
    let userOTP = "";

    function showMessage(message, type) {

        const messageBox =
            document.getElementById("message");

        messageBox.textContent = message;

        messageBox.className =
            "message show " + type;
    }

    function showStep(stepNumber) {

        document
            .querySelectorAll(".step")
            .forEach(function (step) {

                step.classList.remove("active");

            });

        document
            .getElementById("step" + stepNumber)
            .classList.add("active");

        document.getElementById("message")
            .className = "message";
    }

    async function sendOTP() {

        const email =
            document
                .getElementById("email")
                .value
                .trim();

        if (!email) {

            showMessage(
                "Please enter your email address.",
                "error"
            );

            return;
        }

        try {
```

```
        const response =
            await fetch(
                "http://localhost:5000/api/auth/forgot-password",
                {
                    method: "POST",

                    headers: {
                        "Content-Type":
                            "application/json"
                    },

                    body: JSON.stringify({
                        email: email
                    })
                }
            );

        const data =
            await response.json();

        if (!response.ok) {

            showMessage(
                data.message ||
                "Failed to send OTP.",
                "error"
            );

            return;
        }

        userEmail = email;

        showMessage(
            "OTP sent successfully. Check your email.",
            "success"
        );

        setTimeout(
            function () {

                showStep(2);

            },
            1000
        );

    } catch (error) {

        console.error(error);

        showMessage(
            "Server connection failed.",
            "error"
        );
    }
}

async function verifyOTP() {

    const otp =
        document
            .getElementById("otp")
            .value
            .trim();

    if (!otp || otp.length !== 6) {

        showMessage(
            "Please enter a valid 6-digit OTP.",
            "error"
        );

        return;
    }

    try {

        const response =
            await fetch(
                "http://localhost:5000/api/auth/verify-reset-otp",
                {
                    method: "POST",

                    headers: {
                        "Content-Type":
                            "application/json"
                    },

                    body: JSON.stringify({
                        email: userEmail,
                        otp: otp
                    })
                }
            );

        const data =
            await response.json();

        if (!response.ok) {

            showMessage(
                data.message ||
                "Invalid OTP.",
```

```
                "error"
            );

            return;
        }

        userOTP = otp;

        showMessage(
            "OTP verified successfully.",
            "success"
        );

        setTimeout(
            function () {

                showStep(3);

            },
            700
        );

    } catch (error) {

        console.error(error);

        showMessage(
            "Server connection failed.",
            "error"
        );
    }
}

async function resetPassword() {

    const newPassword =
        document
            .getElementById("newPassword")
            .value;

    const confirmPassword =
        document
            .getElementById("confirmPassword")
            .value;

    if (!newPassword || !confirmPassword) {

        showMessage(
            "Please fill both password fields.",
            "error"
        );

        return;
    }

    if (newPassword.length < 6) {

        showMessage(
            "Password must be at least 6 characters.",
            "error"
        );

        return;
    }

    if (newPassword !== confirmPassword) {

        showMessage(
            "Passwords do not match.",
            "error"
        );

        return;
    }

    try {

        const response =
            await fetch(
                "http://localhost:5000/api/auth/reset-password",
                {
                    method: "POST",

                    headers: {
                        "Content-Type":
                            "application/json"
                    },

                    body: JSON.stringify({
                        email: userEmail,
                        otp: userOTP,
                        newPassword: newPassword
                    })
                }
            );

        const data =
            await response.json();

        if (!response.ok) {

            showMessage(
                data.message ||
                "Password reset failed.",
                "error"
```

```
                );

                return;
            }

            showMessage(
                "Password reset successfully! Redirecting to login...",
                "success"
            );

            setTimeout(
                function () {

                    window.location.href =
                        "login.html";

                },
                2000
            );

        } catch (error) {

            console.error(error);

            showMessage(
                "Server connection failed.",
                "error"
            );
        }
    }

</script>


</body>

</html>
```

Frontend/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>HRGenius - HR Management System</title>
    <link rel="stylesheet" href="CSS/style.css">

    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
            font-family: Arial, sans-serif;
        }

        body {
            background: #f4f6f9;
            color: #333;
        }

        nav {
            background: #1f2937;
            color: white;
            padding: 18px 50px;
            display: flex;
            justify-content: space-between;
            align-items: center;
        }

        nav h2 {
            font-size: 24px;
        }

        .nav-buttons {
            display: flex;
            gap: 12px;
        }

        .nav-buttons button {
            padding: 10px 20px;
            border: none;
            border-radius: 6px;
            cursor: pointer;
            font-size: 15px;
        }

        .login-btn {
            background: #2563eb;
            color: white;
        }

        .register-btn {
            background: #16a34a;
            color: white;
        }

        .hero {
            min-height: 500px;
            display: flex;
            align-items: center;
            justify-content: space-between;
            padding: 60px 8%;
            gap: 50px;
        }

        .hero-content {
            max-width: 600px;
        }

        .hero-content h1 {
            font-size: 48px;
            color: #1f2937;
            margin-bottom: 20px;
        }

        .hero-content h1 span {
            color: #2563eb;
        }

        .hero-content p {
            font-size: 18px;
            line-height: 1.7;
            color: #666;
            margin-bottom: 30px;
        }

        .hero-btn {
            background: #2563eb;
            color: white;
            border: none;
            padding: 14px 28px;
            border-radius: 7px;
            font-size: 16px;
            cursor: pointer;
        }

        .hero-btn:hover {
            background: #1d4ed8;
        }
```

```
.hero-box {
    width: 400px;
    height: 300px;
    background: white;
    border-radius: 15px;
    box-shadow: 0 5px 20px rgba(0,0,0,0.1);
    display: flex;
    align-items: center;
    justify-content: center;
    text-align: center;
    padding: 30px;
}

.hero-box h2 {
    color: #2563eb;
    font-size: 32px;
    margin-bottom: 15px;
}

.hero-box p {
    color: #666;
    line-height: 1.6;
}

.features {
    padding: 50px 8%;
    background: white;
}

.features h2 {
    text-align: center;
    font-size: 32px;
    margin-bottom: 40px;
    color: #1f2937;
}

.feature-grid {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 20px;
}

.feature-card {
    background: #f8fafc;
    padding: 25px;
    border-radius: 10px;
    text-align: center;
    border: 1px solid #e5e7eb;
}

.feature-card h3 {
    margin-bottom: 12px;
    color: #2563eb;
}

.feature-card p {
    color: #666;
    line-height: 1.5;
}

.about {
    padding: 60px 8%;
    text-align: center;
}

.about h2 {
    font-size: 32px;
    margin-bottom: 20px;
    color: #1f2937;
}

.about p {
    max-width: 800px;
    margin: auto;
    color: #666;
    line-height: 1.7;
    font-size: 17px;
}

footer {
    background: #1f2937;
    color: white;
    text-align: center;
    padding: 20px;
}

footer p {
    color: #d1d5db;
}

@media (max-width: 900px) {

    .hero {
        flex-direction: column;
        text-align: center;
    }

    .feature-grid {
        grid-template-columns: repeat(2, 1fr);
    }

    .hero-box {
        width: 100%;
        max-width: 400px;
    }
```

```
        }

        @media (max-width: 600px) {

            nav {
                padding: 15px 20px;
            }

            .nav-buttons {
                gap: 5px;
            }

            .nav-buttons button {
                padding: 8px 12px;
            }

            .hero {
                padding: 40px 20px;
            }

            .hero-content h1 {
                font-size: 36px;
            }

            .feature-grid {
                grid-template-columns: 1fr;
            }
        }
    </style>
</head>

<body>

    <!-- Navigation -->
    <nav>

        <h2>HRGenius</h2>

        <div class="nav-buttons">

            <button
                class="login-btn"
                onclick="goLogin()">
                Login
            </button>

            <button
                class="register-btn"
                onclick="goLogin()">
                Get Started
            </button>

        </div>

    </nav>


    <!-- Hero Section -->
    <section class="hero">

        <div class="hero-content">

            <h1>
                Smart <span>HR Management</span> System
            </h1>

            <p>
                HRGenius is a complete Human Resource Management System
                designed to manage employees, recruitment, attendance,
                leaves, payroll, performance and more from one platform.
            </p>

            <button
                class="hero-btn"
                onclick="goLogin()">
                Login to HRGenius
            </button>

        </div>


        <div class="hero-box">

            <div>

                <h2>HRGenius</h2>

                <p>
                    Manage your complete HR operations
                    easily and efficiently.
                </p>

                <br>

                <p>
                    Employee Management • Recruitment • Payroll
                </p>

            </div>

        </div>

    </section>
```

```
<!-- Features -->
<section class="features">

    <h2>HRGenius Features</h2>

    <div class="feature-grid">

        <div class="feature-card">

            <h3>Employees</h3>

            <p>
                Manage employee information,
                departments and designations.
            </p>

        </div>


        <div class="feature-card">

            <h3>Attendance</h3>

            <p>
                Track employee attendance,
                check-in and check-out records.
            </p>

        </div>


        <div class="feature-card">

            <h3>Leave Management</h3>

            <p>
                Manage employee leave requests
                and approval status.
            </p>

        </div>


        <div class="feature-card">

            <h3>Payroll</h3>

            <p>
                Manage salary, allowances,
                deductions and payroll records.
            </p>

        </div>


        <div class="feature-card">

            <h3>Performance</h3>

            <p>
                Track employee performance,
                ratings and feedback.
            </p>

        </div>


        <div class="feature-card">

            <h3>Recruitment</h3>

            <p>
                Manage jobs, candidates,
                resumes and recruitment.
            </p>

        </div>


        <div class="feature-card">

            <h3>Interviews</h3>

            <p>
                Schedule and manage candidate
                interviews and results.
            </p>

        </div>


        <div class="feature-card">

            <h3>Notifications</h3>

            <p>
                Send and manage important
                HR notifications.
            </p>

        </div>

    </div>

</section>
```

```
    <!-- About -->
    <section class="about">

        <h2>About HRGenius</h2>

        <p>
            HRGenius provides a centralized platform for managing
            important HR activities. It helps HR teams maintain
            employee records, manage recruitment, monitor attendance,
            process payroll and evaluate employee performance.
        </p>

    </section>


    <!-- Footer -->
    <footer>

        <p>
            © 2026 HRGenius | HR Management System
        </p>

    </footer>


    <script>

        function goLogin() {
            window.location.href = "login.html";
        }

    </script>
    <script src="JS/auth.js"></script>
    <script src="JS/theme.js"></script>

</body>
</html>
```

Frontend/interviews.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Interview Management - HRGenius</title>

    <link rel="stylesheet" href="CSS/style.css">
    <link rel="stylesheet" href="CSS/interviews.css">
</head>

<body>

    <!-- NAVBAR -->
    <nav class="topbar">

        <div class="nav-left">

            <div class="logo">
                HRGenius
            </div>

            <div class="nav-links">

                <a href="dashboard.html">
                    Dashboard
                </a>

                <a href="jobs.html">
                    Jobs
                </a>

                <a href="candidates.html">
                    Candidates
                </a>

                <a href="interviews.html" class="active">
                    Interviews
                </a>

            </div>

        </div>

        <div class="nav-actions">

            <button
                type="button"
                class="theme-btn"
                onclick="toggleTheme()"
                id="themeButton"
            >
                🌙
            </button>

        </div>

    </nav>


    <!-- MAIN CONTENT -->
    <main class="page-container">

        <!-- PAGE HEADER -->
        <section class="page-header">

            <div>
                <h1>Interview Management</h1>

                <p>
                    Schedule and manage candidate interviews
                </p>
            </div>

        </section>


        <!-- STAT CARDS -->
        <section class="stats-grid">

            <div class="stat-card">

                <div class="stat-title">
                    Total Interviews
                </div>

                <div
                    class="stat-value"
                    id="totalInterviews"
                >
                    0
                </div>

            </div>


            <div class="stat-card">

                <div class="stat-title">
                    Scheduled
                </div>
```

```
        <div
            class="stat-value"
            id="scheduledInterviews"
        >
            0
        </div>

    </div>


    <div class="stat-card">

        <div class="stat-title">
            Completed
        </div>

        <div
            class="stat-value"
            id="completedInterviews"
        >
            0
        </div>

    </div>


    <div class="stat-card">

        <div class="stat-title">
            Selected
        </div>

        <div
            class="stat-value"
            id="selectedInterviews"
        >
            0
        </div>

    </div>

</section>


<!-- SCHEDULE INTERVIEW -->
<section class="form-card">

    <div class="section-header">

        <div>

            <h2>
                Schedule New Interview
            </h2>

            <p>
                Schedule an interview for a shortlisted candidate
            </p>

        </div>

    </div>


    <form id="interviewForm">

        <div class="form-grid">


            <!-- CANDIDATE -->
            <div class="form-group">

                <label for="candidateId">
                    Candidate
                </label>

                <select
                    id="candidateId"
                    required
                >

                    <option value="">
                        Select Candidate
                    </option>

                </select>

            </div>


            <!-- JOB -->
            <div class="form-group">

                <label for="jobId">
                    Job
                </label>

                <select
                    id="jobId"
                    required
                >

                    <option value="">
                        Select Job
```

```
        </option>

    </select>

</div>


<!-- INTERVIEWER -->
<div class="form-group">

    <label for="interviewer">
        Interviewer
    </label>

    <input
        type="text"
        id="interviewer"
        placeholder="Enter interviewer name"
        required
    >

</div>


<!-- DATE -->
<div class="form-group">

    <label for="interviewDate">
        Date & Time
    </label>

    <input
        type="datetime-local"
        id="interviewDate"
        required
    >

</div>


<!-- MODE -->
<div class="form-group">

    <label for="mode">
        Interview Mode
    </label>

    <select
        id="mode"
        required
    >

        <option value="">
            Select Mode
        </option>

        <option value="Online">
            Online
        </option>

        <option value="Offline">
            Offline
        </option>

    </select>

</div>


<!-- MEETING LINK -->
<div
    class="form-group"
    id="meetingLinkGroup"
    style="display: none;"
>

    <label for="meetingLink">
        Meeting Link
    </label>

    <input
        type="url"
        id="meetingLink"
        placeholder="https://meet.google.com/..."
    >

</div>


<!-- LOCATION -->
<div
    class="form-group"
    id="locationGroup"
    style="display: none;"
>

    <label for="location">
        Location
    </label>

    <input
        type="text"
        id="location"
        placeholder="Enter interview location"
    >
```

```
            </div>


        </div>


        <div class="form-actions">

            <button
                type="submit"
                class="primary-btn"
            >
                Schedule Interview
            </button>

            <button
                type="button"
                class="secondary-btn"
                id="resetBtn"
            >
                Reset
            </button>

        </div>

    </form>

</section>


<!-- ALL INTERVIEWS -->
<section class="table-card">

    <div class="table-header">

        <div>

            <h2>
                All Interviews
            </h2>

            <p>
                View and manage scheduled interviews
            </p>

        </div>


        <button
            type="button"
            class="secondary-btn"
            id="refreshBtn"
        >
            ↻ Refresh
        </button>

    </div>


    <div class="table-wrapper">

        <table>

            <thead>

                <tr>

                    <th>
                        CANDIDATE
                    </th>

                    <th>
                        JOB
                    </th>

                    <th>
                        INTERVIEWER
                    </th>

                    <th>
                        DATE & TIME
                    </th>

                    <th>
                        MODE
                    </th>

                    <th>
                        STATUS
                    </th>

                    <th>
                        RESULT
                    </th>

                    <th>
                        ACTIONS
                    </th>

                </tr>

            </thead>
```

```
                    <tbody id="interviewsTableBody">

                        <tr>

                            <td
                                colspan="8"
                                class="empty-state"
                            >
                                Loading interviews...
                            </td>

                        </tr>

                    </tbody>

                </table>

            </div>

        </section>

    </main>


    <script src="JS/theme.js"></script>
    <script src="JS/auth.js"></script>
    <script src="JS/interviews.js"></script>

</body>
</html>
```

Frontend/jobs.html

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta
        name="viewport"
        content="width=device-width, initial-scale=1.0"
    >

    <title>HRGenius - Jobs</title>

    <link
        rel="stylesheet"
        href="CSS/style.css"
    >

    <style>

        .navbar {
            height: 65px;
            background: var(--primary);
            color: #ffffff;
            display: flex;
            align-items: center;
            justify-content: space-between;
            padding: 0 30px;
        }

        .navbar h2 {
            color: #ffffff;
            font-size: 24px;
        }

        .nav-buttons {
            display: flex;
            gap: 10px;
            align-items: center;
        }

        .nav-btn {
            background: #ffffff;
            color: var(--primary);
            border: none;
            padding: 9px 16px;
            border-radius: 6px;
            cursor: pointer;
            font-weight: bold;
        }

        .nav-btn:hover {
            background: #e5e7eb;
        }

        .container {
            padding: 30px;
        }

        .heading {
            margin-bottom: 25px;
        }

        .heading h1 {
            color: var(--text);
            margin-bottom: 8px;
        }

        .heading p {
            color: var(--muted);
        }

        .form-box,
        .table-box {
            background: var(--card);
            padding: 25px;
            border-radius: 10px;
            border: 1px solid var(--border);
            box-shadow: var(--shadow);
            margin-bottom: 30px;
        }

        .form-box h2,
        .table-box h2 {
            color: var(--text);
            margin-bottom: 20px;
        }

        .form-grid {
            display: grid;
            grid-template-columns: repeat(2, 1fr);
            gap: 15px;
        }

        .form-group {
            display: flex;
            flex-direction: column;
        }

        .form-group label {
            color: var(--text);
```

```
    font-weight: bold;
    margin-bottom: 6px;
}

.form-group input,
.form-group select,
.form-group textarea {
    padding: 11px;
    border: 1px solid var(--border);
    border-radius: 6px;
    background: var(--input);
    color: var(--text);
    font-size: 14px;
}

.form-group textarea {
    min-height: 100px;
    resize: vertical;
}

.full-width {
    grid-column: 1 / -1;
}

.form-actions {
    margin-top: 20px;
    display: flex;
    gap: 10px;
}

.submit-btn {
    background: var(--primary);
    color: #ffffff;
    border: none;
    padding: 11px 22px;
    border-radius: 6px;
    cursor: pointer;
    font-weight: bold;
}

.cancel-btn {
    background: #6b7280;
    color: #ffffff;
    border: none;
    padding: 11px 22px;
    border-radius: 6px;
    cursor: pointer;
    display: none;
}

#message {
    display: none;
    padding: 12px;
    border-radius: 6px;
    margin-bottom: 20px;
}

.search-box {
    margin-bottom: 20px;
}

.search-input {
    width: 100%;
    max-width: 450px;
    padding: 11px 14px;
    border: 1px solid var(--border);
    border-radius: 6px;
    background: var(--input);
    color: var(--text);
}

.table-wrapper {
    overflow-x: auto;
}

table {
    width: 100%;
    border-collapse: collapse;
    min-width: 1100px;
}

th,
td {
    padding: 12px;
    border-bottom: 1px solid var(--border);
    text-align: left;
}

th {
    background: var(--hover);
    color: var(--text);
}

td {
    color: var(--text);
}

.edit-btn {
    background: #f59e0b;
    color: #ffffff;
    border: none;
    padding: 7px 12px;
    border-radius: 5px;
    cursor: pointer;
    margin-right: 5px;
```

```
        }

        .delete-btn {
            background: #dc2626;
            color: #ffffff;
            border: none;
            padding: 7px 12px;
            border-radius: 5px;
            cursor: pointer;
        }

        .status-open {
            color: #16a34a;
            font-weight: bold;
        }

        .status-closed {
            color: #dc2626;
            font-weight: bold;
        }

        .no-data {
            text-align: center;
            padding: 20px;
            color: var(--muted);
        }

        @media (max-width: 700px) {

            .container {
                padding: 15px;
            }

            .form-grid {
                grid-template-columns: 1fr;
            }

            .full-width {
                grid-column: auto;
            }

            .navbar {
                padding: 0 15px;
            }

        }

    </style>

</head>


<body>


    <div class="navbar">

        <h2>HRGenius</h2>

        <div class="nav-buttons">

            <button
                class="nav-btn"
                onclick="window.location.href='dashboard.html'"
            >
                Dashboard
            </button>

            <button
                id="themeToggle"
                class="nav-btn"
                onclick="toggleTheme()"
            >
                🌙 Dark Mode
            </button>

            <button
                class="nav-btn"
                onclick="logout()"
            >
                Logout
            </button>

        </div>

    </div>


    <div class="container">

        <div class="heading">

            <h1>Job Management</h1>

            <p>
                Manage job openings and recruitment positions
            </p>

        </div>


        <div id="message"></div>


        <div
```

```
    class="form-box"
    id="jobFormBox"
>

    <h2 id="formTitle">
        Add Job
    </h2>


    <form id="jobForm">

        <div class="form-grid">


            <div class="form-group">

                <label>
                    Job Title
                </label>

                <input
                    type="text"
                    id="title"
                    placeholder="Software Developer"
                    required
                >

            </div>


            <div class="form-group">

                <label>
                    Department
                </label>

                <input
                    type="text"
                    id="department"
                    placeholder="IT"
                    required
                >

            </div>


            <div class="form-group">

                <label>
                    Location
                </label>

                <input
                    type="text"
                    id="location"
                    placeholder="Ghaziabad"
                    required
                >

            </div>


            <div class="form-group">

                <label>
                    Job Type
                </label>

                <select
                    id="jobType"
                    required
                >

                    <option value="Full Time">
                        Full Time
                    </option>

                    <option value="Part Time">
                        Part Time
                    </option>

                    <option value="Internship">
                        Internship
                    </option>

                    <option value="Contract">
                        Contract
                    </option>

                </select>

            </div>


            <div class="form-group">

                <label>
                    Salary
                </label>

                <input
                    type="number"
                    id="salary"
                    placeholder="55000"
                    min="0"
```

```
                    required
                >

            </div>


            <div class="form-group">

                <label>
                    Status
                </label>

                <select id="status">

                    <option value="Open">
                        Open
                    </option>

                    <option value="Closed">
                        Closed
                    </option>

                </select>

            </div>


            <div class="form-group full-width">

                <label>
                    Job Description
                </label>

                <textarea
                    id="description"
                    placeholder="Enter job description"
                    required
                ></textarea>

            </div>


            <div class="form-group full-width">

                <label>
                    Requirements
                </label>

                <textarea
                    id="requirements"
                    placeholder="Enter required skills and qualifications"
                    required
                ></textarea>

            </div>

        </div>


        <div class="form-actions">

            <button
                type="submit"
                class="submit-btn"
                id="submitBtn"
            >
                Add Job
            </button>

            <button
                type="button"
                class="cancel-btn"
                id="cancelBtn"
                onclick="cancelEdit()"
            >
                Cancel
            </button>

        </div>

    </form>

</div>


<div class="table-box">

    <h2>
        Job Records
    </h2>


    <div class="search-box">

        <input
            type="text"
            id="searchJob"
            class="search-input"
            placeholder="🔍 Search jobs..."
        >

    </div>


    <div class="table-wrapper">
```

```
                <table>

                    <thead>

                        <tr>

                            <th>Job Title</th>
                            <th>Department</th>
                            <th>Location</th>
                            <th>Type</th>
                            <th>Salary</th>
                            <th>Status</th>
                            <th>Description</th>
                            <th>Action</th>

                        </tr>

                    </thead>


                    <tbody id="jobTableBody">

                        <tr>

                            <td
                                colspan="8"
                                class="no-data"
                            >
                                Loading jobs...

                            </td>

                        </tr>

                    </tbody>

                </table>

            </div>

        </div>

    </div>


    <!-- JAVASCRIPT -->

    <script src="./JS/theme.js"></script>

    <script src="./JS/auth.js"></script>

    <script src="./JS/jobs.js"></script>


</body>

</html>
```

Frontend/JS/attendance.js

```
(function () {

    const token = localStorage.getItem("token");

    if (!token) {
        window.location.href = "login.html";
        return;
    }

    const API_URL = "http://localhost:5000/api/attendance";

    function getUserFromToken() {

        try {

            const payload = JSON.parse(
                atob(
                    token
                        .split(".")[1]
                        .replace(/-/g, "+")
                        .replace(/_/g, "/")
                )
            );

            return payload;

        } catch (error) {

            console.log(
                "Token Decode Error:",
                error
            );

            return null;
        }
    }

    const currentUser = getUserFromToken();

    const userRole =
        currentUser
            ? currentUser.role
            : "";

    function showMessage(message, type) {

        const messageBox =
            document.getElementById("message");

        if (!messageBox) {
            return;
        }

        messageBox.textContent = message;

        messageBox.style.color =
            type === "success"
                ? "var(--success)"
                : "var(--danger)";

        setTimeout(function () {

            messageBox.textContent = "";

        }, 3000);
    }

    function formatDate(dateValue) {

        if (!dateValue) {
            return "-";
        }

        const date =
            new Date(dateValue);

        return date.toLocaleDateString(
            "en-US"
        );
    }

    function formatTime(dateValue) {

        if (!dateValue) {
            return "-";
        }

        const date =
            new Date(dateValue);

        return date.toLocaleTimeString(
            "en-US"
        );
    }

    function displayAttendance(records) {

        const table =
            document.getElementById(
                "attendanceTableBody"
            );
```

```
    if (!table) {
        return;
    }

    table.innerHTML = "";

    if (
        !records ||
        records.length === 0
    ) {

        table.innerHTML = `
            <tr>
                <td colspan="7" class="no-data">
                    No attendance records found
                </td>
            </tr>
        `;

        return;
    }

    records.forEach(function (record) {

        const employee =
            record.employee;

        const row =
            document.createElement("tr");

        row.innerHTML = `
            <td>
                ${employee?.name || "-"}
            </td>

            <td>
                ${employee?.email || "-"}
            </td>

            <td>
                ${employee?.department || "-"}
            </td>

            <td>
                ${formatDate(record.date)}
            </td>

            <td>
                <span class="status present">
                    ${record.status || "-"}
                </span>
            </td>

            <td>
                ${formatTime(record.checkIn)}
            </td>

            <td>
                ${formatTime(record.checkOut)}
            </td>
        `;

        table.appendChild(row);

    });
}

function updateSummary(records) {

    const totalAttendance =
        document.getElementById(
            "totalAttendance"
        );

    const attendanceRecords =
        document.getElementById(
            "attendanceRecords"
        );

    const presentToday =
        document.getElementById(
            "presentToday"
        );

    if (totalAttendance) {

        totalAttendance.textContent =
            records.length;
    }

    if (attendanceRecords) {

        attendanceRecords.textContent =
            records.length;
    }

    const today =
        new Date();

    const todayString =
        today.toLocaleDateString(
            "en-US"
        );

    const presentCount =
```

```
        records.filter(function (record) {

            const recordDate =
                new Date(
                    record.date
                ).toLocaleDateString(
                    "en-US"
                );

            return (
                recordDate === todayString &&
                record.status === "Present"
            );

        }).length;

    if (presentToday) {

        presentToday.textContent =
            presentCount;
    }
}

async function loadAttendance() {

    console.log(
        "Loading attendance..."
    );

    try {

        let endpoint =
            API_URL;

        if (userRole === "employee") {

            endpoint =
                `${API_URL}/my`;

        }

        console.log(
            "Attendance endpoint:",
            endpoint
        );

        const response =
            await fetch(
                endpoint,
                {
                    method: "GET",

                    headers: {
                        "Authorization":
                            `Bearer ${token}`
                    }
                }
            );

        console.log(
            "Attendance API Status:",
            response.status
        );

        const data =
            await response.json();

        if (!response.ok) {

            const table =
                document.getElementById(
                    "attendanceTableBody"
                );

            if (table) {

                table.innerHTML = `
                    <tr>
                        <td colspan="7"
                            class="no-data">
                            ${data.message ||
                            "Unable to load attendance"}
                        </td>
                    </tr>
                `;
            }

            showMessage(
                data.message ||
                "Unable to load attendance",
                "error"
            );

            return;
        }

        const records =
            data.attendance || [];

        updateSummary(records);

        displayAttendance(records);

    } catch (error) {
```

```
        console.log(
            "Attendance Load Error:",
            error
        );

        const table =
            document.getElementById(
                "attendanceTableBody"
            );

        if (table) {

            table.innerHTML = `
                <tr>
                    <td colspan="7"
                        class="no-data">
                        Server connection failed
                    </td>
                </tr>
            `;
        }

        showMessage(
            "Server connection failed",
            "error"
        );
    }
}

window.checkIn = async function () {

    const employeeInput =
        document.getElementById(
            "employeeId"
        );

    const employeeId =
        employeeInput
            ? employeeInput.value.trim()
            : "";

    if (
        userRole !== "employee" &&
        !employeeId
    ) {

        showMessage(
            "Employee ID is required",
            "error"
        );

        return;
    }

    try {

        const body = {};

        if (userRole !== "employee") {

            body.employeeId =
                employeeId;
        }

        const response =
            await fetch(
                `${API_URL}/check-in`,
                {
                    method: "POST",

                    headers: {
                        "Content-Type":
                            "application/json",

                        "Authorization":
                            `Bearer ${token}`
                    },

                    body:
                        JSON.stringify(body)
                }
            );

        const data =
            await response.json();

        if (!response.ok) {

            showMessage(
                data.message ||
                "Check-in failed",
                "error"
            );

            return;
        }

        showMessage(
            "Check-in successful",
            "success"
        );

        if (employeeInput) {
            employeeInput.value = "";
        }
```

```
        loadAttendance();

    } catch (error) {

        console.log(
            "Check-In Error:",
            error
        );

        showMessage(
            "Server connection failed",
            "error"
        );
    }
};

window.checkOut = async function () {

    const employeeInput =
        document.getElementById(
            "employeeId"
        );

    const employeeId =
        employeeInput
            ? employeeInput.value.trim()
            : "";

    if (
        userRole !== "employee" &&
        !employeeId
    ) {

        showMessage(
            "Employee ID is required",
            "error"
        );

        return;
    }

    try {

        const body = {};

        if (userRole !== "employee") {

            body.employeeId =
                employeeId;
        }

        const response =
            await fetch(
                `${API_URL}/check-out`,
                {
                    method: "POST",

                    headers: {
                        "Content-Type":
                            "application/json",

                        "Authorization":
                            `Bearer ${token}`
                    },

                    body:
                        JSON.stringify(body)
                }
            );

        const data =
            await response.json();

        if (!response.ok) {

            showMessage(
                data.message ||
                "Check-out failed",
                "error"
            );

            return;
        }

        showMessage(
            "Check-out successful",
            "success"
        );

        if (employeeInput) {
            employeeInput.value = "";
        }

        loadAttendance();

    } catch (error) {

        console.log(
            "Check-Out Error:",
            error
        );

        showMessage(
            "Server connection failed",
```

```
                "error"
            );
        }
    };

    window.logout = function () {

        localStorage.removeItem(
            "token"
        );

        window.location.href =
            "login.html";
    };

    console.log(
        "attendance.js loaded successfully"
    );

    console.log(
        "Current User Role:",
        userRole
    );

    loadAttendance();

})();
```

Frontend/JS/auth.js

```
const API = "http://localhost:5000/api";

async function loginUser() {

    const email = document.getElementById("email").value.trim();
    const password = document.getElementById("password").value;

    if (!email || !password) {
        alert("Please enter email and password");
        return;
    }

    try {

        const response = await fetch(`${API}/auth/login`, {
            method: "POST",

            headers: {
                "Content-Type": "application/json"
            },

            body: JSON.stringify({
                email: email,
                password: password
            })
        });

        const data = await response.json();

        console.log("Login Response:", data);

        if (!response.ok) {
            alert(data.message || "Invalid email or password");
            return;
        }

        if (!data.token || !data.user) {
            alert("Login response is incomplete");
            return;
        }

        localStorage.setItem("token", data.token);
        localStorage.setItem("user", JSON.stringify(data.user));

        window.location.href = "dashboard.html";

    } catch (error) {

        console.error("Login Error:", error);

        alert("Server connection error");
    }
}
```

Frontend/JS/dashboard.js

```
(function () {

    const DASHBOARD_API = "http://localhost:5000/api";

    let token = localStorage.getItem("token");
    let userData = localStorage.getItem("user");

    if (!token || !userData) {
        window.location.href = "login.html";
        return;
    }

    let user;

    try {
        user = JSON.parse(userData);
    } catch (error) {
        console.error("User data error:", error);

        localStorage.removeItem("token");
        localStorage.removeItem("user");

        window.location.href = "login.html";
        return;
    }


    function loadUserInfo() {

        const name = user?.name || "User";
        const role = user?.role || "employee";

        const profileName = document.getElementById("profileName");
        const profileRole = document.getElementById("profileRole");
        const profileAvatar = document.getElementById("profileAvatar");
        const welcomeTitle = document.getElementById("welcomeTitle");

        if (profileName) {
            profileName.textContent = name;
        }

        if (profileRole) {
            profileRole.textContent =
                role.charAt(0).toUpperCase() +
                role.slice(1);
        }

        if (profileAvatar) {
            profileAvatar.textContent =
                name.charAt(0).toUpperCase();
        }

        if (welcomeTitle) {
            const firstName = name.split(" ")[0];

            welcomeTitle.textContent =
                "Welcome back, " +
                firstName +
                "! 👋";
        }
    }


    function loadDate() {

        const dateElement =
            document.getElementById("currentDate");

        if (!dateElement) {
            return;
        }

        const today = new Date();

        dateElement.textContent =
            today.toLocaleDateString(
                "en-IN",
                {
                    weekday: "long",
                    day: "numeric",
                    month: "long",
                    year: "numeric"
                }
            );
    }


    function formatNumber(value) {

        const number = Number(value);

        if (!Number.isFinite(number)) {
            return "0";
        }

        return number.toLocaleString("en-IN");
    }


    function formatCurrency(value) {

        const number = Number(value);
```

```
    if (!Number.isFinite(number)) {
        return "₹0";
    }

    return "₹" +
        number.toLocaleString("en-IN");
}


function setText(id, value) {

    const element =
        document.getElementById(id);

    if (element) {
        element.textContent = value;
    }
}


function setWidth(id, percentage) {

    const element =
        document.getElementById(id);

    if (!element) {
        return;
    }

    let value = Number(percentage);

    if (!Number.isFinite(value)) {
        value = 0;
    }

    value = Math.min(
        Math.max(value, 0),
        100
    );

    element.style.width =
        value + "%";
}


async function loadDashboard() {

    const apiUrl =
        DASHBOARD_API +
        "/dashboard/stats";

    console.log(
        "Dashboard API:",
        apiUrl
    );

    console.log(
        "Dashboard Token Exists:",
        !!token
    );

    try {

        const response =
            await fetch(
                apiUrl,
                {
                    method: "GET",

                    headers: {
                        "Authorization":
                            "Bearer " + token,

                        "Content-Type":
                            "application/json"
                    }
                }
            );

        console.log(
            "Dashboard HTTP Status:",
            response.status
        );

        const responseText =
            await response.text();

        let data = {};

        if (responseText) {

            try {

                data =
                    JSON.parse(
                        responseText
                    );

            } catch (error) {

                console.error(
                    "Dashboard JSON Parse Error:",
                    error
                );
```

```
                console.error(
                    "Raw API Response:",
                    responseText
                );

                data = {};
            }
        }

        console.log(
            "Dashboard Response:",
            data
        );

        if (response.status === 401) {

            console.error(
                "Dashboard authentication failed"
            );

            localStorage.removeItem("token");
            localStorage.removeItem("user");

            window.location.href =
                "login.html";

            return;
        }

        if (!response.ok) {

            console.error(
                "Dashboard API Error:",
                data
            );

            clearDashboard();

            return;
        }

        if (
            !data ||
            typeof data !== "object"
        ) {

            console.error(
                "Invalid dashboard data"
            );

            clearDashboard();

            return;
        }

        updateDashboard(data);

    } catch (error) {

        console.error(
            "Dashboard Fetch Error:",
            error
        );

        clearDashboard();

    }
}


function updateDashboard(data) {

    console.log(
        "Updating Dashboard with:",
        data
    );


    /*
    ========================
    MAIN STATISTICS
    ========================
    */

    setText(
        "totalEmployees",
        formatNumber(
            data.totalEmployees
        )
    );

    setText(
        "totalJobs",
        formatNumber(
            data.totalJobs
        )
    );

    setText(
        "totalCandidates",
        formatNumber(
            data.totalCandidates
        )
    );
```

```
setText(
    "totalPayroll",
    formatCurrency(
        data.totalPayroll
    )
);


/*
=========================
EMPLOYEE OVERVIEW
=========================
*/

setText(
    "employeeCircle",
    formatNumber(
        data.totalEmployees
    )
);

setText(
    "overviewEmployees",
    formatNumber(
        data.totalEmployees
    )
);

const activeEmployees =
    data.activeEmployees !== undefined
        ? data.activeEmployees
        : data.totalEmployees;

setText(
    "activeEmployees",
    formatNumber(
        activeEmployees
    )
);

setText(
    "overviewLeaves",
    formatNumber(
        data.pendingLeaves
    )
);


/*
=========================
HR SUMMARY
=========================
*/

setText(
    "totalInterviews",
    formatNumber(
        data.totalInterviews
    )
);

setText(
    "summaryLeaves",
    formatNumber(
        data.pendingLeaves
    )
);

setText(
    "summaryPayroll",
    formatNumber(
        data.totalPayrollRecords
    )
);

setText(
    "summaryNotifications",
    formatNumber(
        data.unreadNotifications
    )
);


/*
=========================
RECRUITMENT
=========================
*/

updateRecruitment(
    data.candidateStatus,
    data.analytics
);


/*
=========================
HR ANALYTICS
=========================
*/

updateAnalytics(
    data.analytics,
    data.candidateStatus
```

```
    );

    /*
    ========================
    NOTIFICATIONS
    ========================
    */

    updateNotificationBadge(
        data.unreadNotifications
    );


    /*
    ========================
    RECENT ACTIVITY
    ========================
    */

    loadActivities(
        data.recentActivity
    );
}


/*
=======================================
RECRUITMENT FIX
=======================================
*/

function updateRecruitment(
    candidateStatus,
    analytics
) {

    let applied = 0;
    let shortlisted = 0;
    let interview = 0;
    let selected = 0;


    /*
    ------------------------------------
    CASE 1:
    candidateStatus is an array
    ------------------------------------
    */

    if (Array.isArray(candidateStatus)) {

        candidateStatus.forEach(
            function (item) {

                const status =
                    String(
                        item?._id ??
                        item?.status ??
                        ""
                    )
                        .toLowerCase()
                        .trim();

                const count =
                    Number(
                        item?.count ??
                        item?.total ??
                        0
                    );


                if (
                    status === "applied"
                ) {
                    applied += count;
                }


                if (
                    status === "shortlisted"
                ) {
                    shortlisted += count;
                }


                if (
                    status === "interview" ||
                    status === "interview scheduled"
                ) {
                    interview += count;
                }


                if (
                    status === "selected" ||
                    status === "hired"
                ) {
                    selected += count;
                }
            }
        );
    }
```

```
/*
-------------------------------------
CASE 2:
candidateStatus is an object
-------------------------------------
*/

else if (
    candidateStatus &&
    typeof candidateStatus === "object"
) {

    applied =
        Number(
            candidateStatus.applied ??
            candidateStatus.Applied ??
            0
        );

    shortlisted =
        Number(
            candidateStatus.shortlisted ??
            candidateStatus.Shortlisted ??
            0
        );

    interview =
        Number(
            candidateStatus.interview ??
            candidateStatus.Interview ??
            candidateStatus.interviewScheduled ??
            0
        );

    selected =
        Number(
            candidateStatus.selected ??
            candidateStatus.Selected ??
            candidateStatus.hired ??
            0
        );
}


/*
-------------------------------------
CASE 3:
Use analytics recruitment as fallback
-------------------------------------
*/

const recruitment =
    analytics?.recruitment || {};


if (applied === 0) {

    applied =
        Number(
            recruitment.applied ??
            recruitment.totalApplied ??
            0
        );
}


if (shortlisted === 0) {

    shortlisted =
        Number(
            recruitment.shortlisted ??
            recruitment.totalShortlisted ??
            0
        );
}


if (interview === 0) {

    interview =
        Number(
            recruitment.interview ??
            recruitment.totalInterview ??
            recruitment.interviews ??
            0
        );
}


if (selected === 0) {

    selected =
        Number(
            recruitment.selected ??
            recruitment.totalSelected ??
            0
        );
}


/*
-------------------------------------
UPDATE RECRUITMENT CARD
-------------------------------------
*/
```

```
    setText(
        "appliedCount",
        formatNumber(applied)
    );

    setText(
        "shortlistedCount",
        formatNumber(shortlisted)
    );

    setText(
        "interviewCount",
        formatNumber(interview)
    );

    setText(
        "selectedCount",
        formatNumber(selected)
    );


    updateRecruitmentProgress(
        applied,
        shortlisted,
        interview,
        selected
    );
}


function updateRecruitmentProgress(
    applied,
    shortlisted,
    interview,
    selected
) {

    const values = [
        applied,
        shortlisted,
        interview,
        selected
    ];

    const maximum =
        Math.max(
            ...values,
            1
        );


    setWidth(
        "appliedBar",
        (applied / maximum) * 100
    );

    setWidth(
        "shortlistedBar",
        (shortlisted / maximum) * 100
    );

    setWidth(
        "interviewBar",
        (interview / maximum) * 100
    );

    setWidth(
        "selectedBar",
        (selected / maximum) * 100
    );
}


/*
========================================
HR ANALYTICS
========================================
*/

function updateAnalytics(
    analytics,
    candidateStatus
) {

    console.log(
        "Analytics Data:",
        analytics
    );


    if (
        !analytics ||
        typeof analytics !== "object"
    ) {

        console.warn(
            "No analytics data received"
        );

        clearAnalytics();

        return;
    }
```

```
/*
=========================
ATTENDANCE
=========================
*/

const attendance =
    analytics.attendance || {};


const attendanceRate =
    Number(
        attendance.rate ??
        attendance.attendanceRate ??
        0
    );


/*
IMPORTANT:
Actual dashboard HTML IDs
*/

setText(
    "analyticsAttendanceRate",
    attendanceRate.toFixed(1) +
    "%"
);


setWidth(
    "analyticsAttendanceBar",
    attendanceRate
);


const present =
    Number(
        attendance.presentThisMonth ??
        attendance.present ??
        attendance.presentToday ??
        0
    );


const absent =
    Number(
        attendance.absentThisMonth ??
        attendance.absent ??
        attendance.absentToday ??
        0
    );


setText(
    "analyticsPresent",
    formatNumber(present)
);


setText(
    "analyticsAbsent",
    formatNumber(absent)
);


/*
=========================
PAYROLL
=========================
*/

const payroll =
    analytics.payroll || {};


const payrollAmount =
    payroll.totalAmount ??
    payroll.totalPayroll ??
    0;


const paidPayroll =
    payroll.paidCount ??
    payroll.paid ??
    0;


const pendingPayroll =
    payroll.pendingCount ??
    payroll.pending ??
    0;


setText(
    "analyticsPayrollAmount",
    formatCurrency(
        payrollAmount
    )
);


setText(
    "analyticsPaidPayroll",
```

```
    formatNumber(
        paidPayroll
    )
);


setText(
    "analyticsPendingPayroll",
    formatNumber(
        pendingPayroll
    )
);


/*
=========================
LEAVES
=========================
*/

const leaves =
    analytics.leaves || {};


const totalLeaves =
    Number(
        leaves.total ??
        0
    );


const approvedLeaves =
    Number(
        leaves.approved ??
        0
    );


const pendingLeaves =
    Number(
        leaves.pending ??
        0
    );


const rejectedLeaves =
    Number(
        leaves.rejected ??
        0
    );


setText(
    "analyticsTotalLeaves",
    formatNumber(
        totalLeaves
    )
);


setText(
    "analyticsApprovedLeaves",
    formatNumber(
        approvedLeaves
    )
);


setText(
    "analyticsPendingLeaves",
    formatNumber(
        pendingLeaves
    )
);


setText(
    "analyticsRejectedLeaves",
    formatNumber(
        rejectedLeaves
    )
);


const leavePercentage =
    totalLeaves > 0
        ? (
            approvedLeaves /
            totalLeaves
        ) * 100
        : 0;


setWidth(
    "leaveProgress",
    leavePercentage
);


/*
=========================
RECRUITMENT ANALYTICS
=========================
*/
```

```
const recruitment =
    analytics.recruitment || {};


const totalCandidates =
    Number(
        recruitment.totalCandidates ??
        recruitment.total ??
        0
    );


const applied =
    Number(
        recruitment.applied ??
        recruitment.totalApplied ??
        0
    );


const shortlisted =
    Number(
        recruitment.shortlisted ??
        recruitment.totalShortlisted ??
        0
    );


const interview =
    Number(
        recruitment.interview ??
        recruitment.totalInterview ??
        recruitment.interviews ??
        0
    );


const selected =
    Number(
        recruitment.selected ??
        recruitment.totalSelected ??
        0
    );


/*
If backend doesn't provide recruitment
counts, calculate from candidateStatus.
*/

let finalApplied = applied;
let finalShortlisted = shortlisted;
let finalInterview = interview;
let finalSelected = selected;


if (
    Array.isArray(candidateStatus)
) {

    let calculatedApplied = 0;
    let calculatedShortlisted = 0;
    let calculatedInterview = 0;
    let calculatedSelected = 0;


    candidateStatus.forEach(
        function (item) {

            const status =
                String(
                    item?._id ??
                    item?.status ??
                    ""
                )
                    .toLowerCase()
                    .trim();

            const count =
                Number(
                    item?.count ??
                    item?.total ??
                    0
                );


            if (
                status === "applied"
            ) {
                calculatedApplied += count;
            }


            if (
                status === "shortlisted"
            ) {
                calculatedShortlisted += count;
            }


            if (
                status === "interview" ||
                status === "interview scheduled"
            ) {
                calculatedInterview += count;
```

```
                }

                if (
                    status === "selected" ||
                    status === "hired"
                ) {
                    calculatedSelected += count;
                }
            }
        );

        if (finalApplied === 0) {
            finalApplied =
                calculatedApplied;
        }

        if (finalShortlisted === 0) {
            finalShortlisted =
                calculatedShortlisted;
        }

        if (finalInterview === 0) {
            finalInterview =
                calculatedInterview;
        }

        if (finalSelected === 0) {
            finalSelected =
                calculatedSelected;
        }
    }

    setText(
        "analyticsCandidates",
        formatNumber(
            totalCandidates
        )
    );

    setText(
        "analyticsApplied",
        formatNumber(
            finalApplied
        )
    );

    setText(
        "analyticsShortlisted",
        formatNumber(
            finalShortlisted
        )
    );

    setText(
        "analyticsInterview",
        formatNumber(
            finalInterview
        )
    );

    setText(
        "analyticsSelected",
        formatNumber(
            finalSelected
        )
    );

    /*
    ==========================
    DEPARTMENTS
    ==========================
    */

    updateDepartments(
        analytics.departments,
        analytics.departmentDetails
    );
}

/*
========================================
DEPARTMENT FIX
========================================
*/

function updateDepartments(
    departments,
    departmentDetails
) {

    /*
    IMPORTANT:
    Actual dashboard HTML uses:
```

```
departmentAnalytics
*/

const container =
    document.getElementById(
        "departmentAnalytics"
    );


if (!container) {

    console.warn(
        "departmentAnalytics element not found"
    );

    return;
}


let departmentData = [];


/*
-------------------------------------
CASE 1:
analytics.departments
-------------------------------------
*/

if (
    Array.isArray(departments)
) {

    departmentData =
        departments;
}


/*
-------------------------------------
CASE 2:
departmentDetails
-------------------------------------
*/

if (
    departmentData.length === 0 &&
    Array.isArray(departmentDetails)
) {

    departmentData =
        departmentDetails;
}


if (
    departmentData.length === 0
) {

    container.innerHTML =
        '<div class="analytics-empty">' +
        'No department analytics available' +
        '</div>';

    return;
}


/*
-------------------------------------
FIND EMPLOYEE COUNTS
-------------------------------------
*/

const normalizedDepartments =
    departmentData.map(
        function (department) {

            const employees =
                Array.isArray(
                    department?.employees
                )
                    ? department.employees
                    : [];


            let employeeCount =
                Number(
                    department?.employeeCount ??
                    department?.employeesCount ??
                    department?.count ??
                    0
                );


            /*
            If employeeCount is missing,
            calculate from employees array.
            */

            if (
                employeeCount === 0 &&
                employees.length > 0
            ) {
```

```
                employeeCount =
                    employees.length;
            }


            const totalSalary =
                Number(
                    department?.totalSalary ??
                    department?.salary ??
                    department?.totalPayroll ??
                    0
                );


            return {
                name:
                    department?.name ??
                    department?._id ??
                    "Department",

                employeeCount:
                    employeeCount,

                totalSalary:
                    totalSalary
            };
        }
    );


/*
-------------------------------------
MAX EMPLOYEE COUNT
-------------------------------------
*/

const employeeCounts =
    normalizedDepartments.map(
        function (department) {

            return department.employeeCount;
        }
    );


const maximumEmployees =
    Math.max(
        ...employeeCounts,
        1
    );


/*
-------------------------------------
RENDER DEPARTMENTS
-------------------------------------
*/

container.innerHTML = "";


normalizedDepartments.forEach(
    function (department) {

        const percentage =
            (
                department.employeeCount /
                maximumEmployees
            ) * 100;


        const item =
            document.createElement(
                "div"
            );

        item.className =
            "department-item";


        /*
        Department information
        */

        const nameBox =
            document.createElement(
                "div"
            );

        nameBox.className =
            "department-info";


        const departmentName =
            document.createElement(
                "div"
            );

        departmentName.className =
            "department-name";

        departmentName.textContent =
            department.name;
```

```
const departmentCount =
    document.createElement(
        "div"
    );

departmentCount.className =
    "department-count";

departmentCount.textContent =
    formatNumber(
        department.employeeCount
    ) +
    " employee" +
    (
        department.employeeCount === 1
            ? ""
            : "s"
    );


nameBox.appendChild(
    departmentName
);

nameBox.appendChild(
    departmentCount
);


/*
Employee progress
*/

const progressBox =
    document.createElement(
        "div"
    );

progressBox.className =
    "department-progress";


const progressFill =
    document.createElement(
        "div"
    );

progressFill.className =
    "department-progress-fill";

progressFill.style.width =
    percentage + "%";


progressBox.appendChild(
    progressFill
);


/*
Salary
*/

const salaryBox =
    document.createElement(
        "div"
    );

salaryBox.className =
    "department-salary";


const salaryAmount =
    document.createElement(
        "strong"
    );

salaryAmount.textContent =
    formatCurrency(
        department.totalSalary
    );


const salaryLabel =
    document.createElement(
        "span"
    );

salaryLabel.textContent =
    "Total Salary";


salaryBox.appendChild(
    salaryAmount
);

salaryBox.appendChild(
    salaryLabel
);


/*
Final department item
*/
```

```
            item.appendChild(
                nameBox
            );

            item.appendChild(
                progressBox
            );

            item.appendChild(
                salaryBox
            );


            container.appendChild(
                item
            );
        }
    );
}


/*
========================================
NOTIFICATION BADGE
========================================
*/

function updateNotificationBadge(
    unreadNotifications
) {

    const badge =
        document.getElementById(
            "notificationBadge"
        );

    if (!badge) {
        return;
    }

    const count =
        Number(
            unreadNotifications || 0
        );


    if (count > 0) {

        badge.textContent =
            count > 99
                ? "99+"
                : count;

        badge.style.display =
            "inline-block";

    } else {

        badge.style.display =
            "none";
    }
}


/*
========================================
RECENT ACTIVITY
========================================
*/

function loadActivities(
    activities
) {

    const activityList =
        document.getElementById(
            "activityList"
        );

    if (!activityList) {
        return;
    }


    if (
        !Array.isArray(
            activities
        ) ||
        activities.length === 0
    ) {

        activityList.innerHTML =
            '<div class="activity-item">' +
                '<div class="activity-icon">✨</div>' +
                '<div class="activity-content">' +
                    '<strong>HRGenius</strong>' +
                    '<span>Dashboard loaded successfully</span>' +
                '</div>' +
                '<small>Recently</small>' +
            '</div>';

        return;
    }
```

```
    activityList.innerHTML = "";


    activities.forEach(
        function (activity) {

            const item =
                document.createElement(
                    "div"
                );

            item.className =
                "activity-item";


            const icon =
                document.createElement(
                    "div"
                );

            icon.className =
                "activity-icon";

            icon.textContent =
                getActivityIcon(
                    activity?.type
                );


            const content =
                document.createElement(
                    "div"
                );

            content.className =
                "activity-content";


            const title =
                document.createElement(
                    "strong"
                );

            title.textContent =
                activity?.title ||
                "HRGenius";


            const description =
                document.createElement(
                    "span"
                );

            description.textContent =
                activity?.description ||
                "System activity";


            content.appendChild(
                title
            );

            content.appendChild(
                description
            );


            const time =
                document.createElement(
                    "small"
                );

            time.textContent =
                formatActivityDate(
                    activity?.date
                );


            item.appendChild(
                icon
            );

            item.appendChild(
                content
            );

            item.appendChild(
                time
            );


            activityList.appendChild(
                item
            );
        }
    );
}


function getActivityIcon(
    type
) {

    const value =
        String(type || "")
```

```
            .toLowerCase();

    if (
        value.includes("employee")
    ) {
        return "👥";
    }

    if (
        value.includes("candidate")
    ) {
        return "👤";
    }

    if (
        value.includes("payroll")
    ) {
        return "💰";
    }

    if (
        value.includes("leave")
    ) {
        return "🗓️";
    }

    if (
        value.includes("interview")
    ) {
        return "🎯";
    }

    if (
        value.includes("job")
    ) {
        return "💼";
    }

    if (
        value.includes("attendance")
    ) {
        return "🕒";
    }

    if (
        value.includes("performance")
    ) {
        return "📈";
    }

    return "✨";
}

function formatActivityDate(
    date
) {
    if (!date) {
        return "Recently";
    }

    const activityDate =
        new Date(date);

    if (
        Number.isNaN(
            activityDate.getTime()
        )
    ) {
        return "Recently";
    }

    const now =
        new Date();

    const difference =
        now.getTime() -
        activityDate.getTime();

    if (difference < 0) {
        return "Recently";
    }

    const minutes =
        Math.floor(
            difference /
            (1000 * 60)
        );
```

```
    if (minutes < 1) {
        return "Just now";
    }


    if (minutes < 60) {

        return (
            minutes +
            " min ago"
        );
    }


    const hours =
        Math.floor(
            minutes / 60
        );


    if (hours < 24) {

        return (
            hours +
            " hr ago"
        );
    }


    const days =
        Math.floor(
            hours / 24
        );


    if (days < 7) {

        return (
            days +
            " day" +
            (
                days === 1
                    ? ""
                    : "s"
            ) +
            " ago"
        );
    }


    return activityDate.toLocaleDateString(
        "en-IN",
        {
            day: "numeric",
            month: "short",
            year: "numeric"
        }
    );
}


/*
========================================
SEARCH
========================================
*/

function setupSearch() {

    const searchInput =
        document.getElementById(
            "dashboardSearch"
        );

    if (!searchInput) {
        return;
    }


    searchInput.addEventListener(
        "input",
        function () {

            const value =
                this.value
                    .toLowerCase()
                    .trim();


            const links =
                document.querySelectorAll(
                    ".sidebar .side-link"
                );


            links.forEach(
                function (link) {

                    const text =
                        link.textContent
                            .toLowerCase();


                    if (!value) {
```

```
                        link.style.display =
                            "flex";

                    } else if (
                        text.includes(value)
                    ) {

                        link.style.display =
                            "flex";

                    } else {

                        link.style.display =
                            "none";
                    }
                }
            );
        }
    );
}


/*
========================================
LOGOUT
========================================
*/

function logout() {

    const confirmLogout =
        confirm(
            "Are you sure you want to logout?"
        );

    if (!confirmLogout) {
        return;
    }


    localStorage.removeItem(
        "token"
    );

    localStorage.removeItem(
        "user"
    );


    window.location.href =
        "login.html";
}


window.logout =
    logout;


/*
========================================
LOADER
========================================
*/

function hideLoader() {

    const loader =
        document.getElementById(
            "pageLoader"
        );

    if (!loader) {
        return;
    }


    loader.style.opacity =
        "0";

    loader.style.transition =
        "opacity 0.25s ease";


    setTimeout(
        function () {

            loader.style.display =
                "none";

        },
        250
    );
}


/*
========================================
CLEAR ANALYTICS
========================================
*/

function clearAnalytics() {

    setText(
```

```
        "analyticsAttendanceRate",
        "0.0%"
    );

    setText(
        "analyticsPresent",
        "0"
    );

    setText(
        "analyticsAbsent",
        "0"
    );

    setText(
        "analyticsPayrollAmount",
        "₹0"
    );

    setText(
        "analyticsPaidPayroll",
        "0"
    );

    setText(
        "analyticsPendingPayroll",
        "0"
    );

    setText(
        "analyticsTotalLeaves",
        "0"
    );

    setText(
        "analyticsApprovedLeaves",
        "0"
    );

    setText(
        "analyticsPendingLeaves",
        "0"
    );

    setText(
        "analyticsRejectedLeaves",
        "0"
    );

    setText(
        "analyticsCandidates",
        "0"
    );

    setText(
        "analyticsApplied",
        "0"
    );

    setText(
        "analyticsShortlisted",
        "0"
    );

    setText(
        "analyticsInterview",
        "0"
    );

    setText(
        "analyticsSelected",
        "0"
    );


    setWidth(
        "analyticsAttendanceBar",
        0
    );

    setWidth(
        "leaveProgress",
        0
    );


    updateDepartments([]);
}


/*
========================================
CLEAR DASHBOARD
========================================
*/

function clearDashboard() {

    setText(
        "totalEmployees",
        "0"
    );

    setText(
        "totalJobs",
```

```
        "0"
    );

    setText(
        "totalCandidates",
        "0"
    );

    setText(
        "totalPayroll",
        "₹0"
    );

    setText(
        "employeeCircle",
        "0"
    );

    setText(
        "overviewEmployees",
        "0"
    );

    setText(
        "activeEmployees",
        "0"
    );

    setText(
        "overviewLeaves",
        "0"
    );

    setText(
        "totalInterviews",
        "0"
    );

    setText(
        "summaryLeaves",
        "0"
    );

    setText(
        "summaryPayroll",
        "0"
    );

    setText(
        "summaryNotifications",
        "0"
    );


    updateRecruitment(
        [],
        {}
    );


    clearAnalytics();


    updateNotificationBadge(
        0
    );


    loadActivities([]);
}


/*
========================================
PAGE INITIALIZATION
========================================
*/

function initializeDashboard() {

    console.log(
        "Initializing HRGenius Dashboard..."
    );

    console.log(
        "HRGenius Dashboard Started"
    );

    console.log(
        "Dashboard API:",
        DASHBOARD_API
    );

    console.log(
        "Dashboard Token Exists:",
        !!token
    );


    loadUserInfo();

    loadDate();

    setupSearch();
```

```
        loadDashboard()
            .finally(
                function () {

                    hideLoader();

                }
            );
    }


    /*
    ========================================
    DOM READY
    ========================================
    */

    if (
        document.readyState ===
        "loading"
    ) {

        document.addEventListener(
            "DOMContentLoaded",
            initializeDashboard
        );

    } else {

        initializeDashboard();
    }


    /*
    ========================================
    LOADER SAFETY
    ========================================
    */

    setTimeout(
        function () {

            hideLoader();

        },
        5000
    );


    window.addEventListener(
        "load",
        function () {

            setTimeout(
                function () {

                    hideLoader();

                },
                1500
            );
        }
    );

})();
```

Frontend/JS/departments.js

```
const API_URL =
    "http://localhost:5000/api";


let departments = [];
let employees = [];

let currentSort = {
    field: null,
    direction: "asc"
};

let departmentToDelete = null;


/* ======================================================
   AUTH
====================================================== */

function getToken() {

    return (
        localStorage.getItem("token") ||
        localStorage.getItem("authToken")
    );

}


function getAuthHeaders() {

    const token = getToken();

    return {
        "Content-Type": "application/json",
        Authorization: `Bearer ${token}`
    };

}


function logout() {

    localStorage.removeItem("token");
    localStorage.removeItem("authToken");
    localStorage.removeItem("user");

    window.location.href = "login.html";

}


/* ======================================================
   INITIAL LOAD
====================================================== */

document.addEventListener(
    "DOMContentLoaded",
    async () => {

        setupEvents();

        setupSearch();

        await loadEmployees();
        await loadDepartments();

    }
);


/* ======================================================
   EVENTS
====================================================== */

function setupEvents() {

    const statusFilter =
        document.getElementById(
            "statusFilter"
        );

    if (statusFilter) {

        statusFilter.addEventListener(
            "change",
            renderDepartments
        );

    }


    const departmentForm =
        document.getElementById(
            "departmentForm"
        );

    if (departmentForm) {

        departmentForm.addEventListener(
            "submit",
            handleDepartmentSubmit
```

```
        );

    }


    const headSelect =
        document.getElementById(
            "departmentHeadSelect"
        );

    if (headSelect) {

        headSelect.addEventListener(
            "change",
            updateHeadPreview
        );

    }


    document.addEventListener(
        "keydown",
        event => {

            if (
                event.key === "Escape"
            ) {

                closeDepartmentModal();
                closeDeleteModal();
                closeHeadModal();
                closeEmployeesModal();
                closeDepartmentDetailsModal();

            }


            /*
                Ctrl + K
                Search box focus
            */

            if (
                (event.ctrlKey ||
                    event.metaKey) &&
                event.key.toLowerCase() === "k"
            ) {

                event.preventDefault();

                const searchInput =
                    document.getElementById(
                        "searchInput"
                    );

                if (searchInput) {

                    searchInput.focus();

                    searchInput.select();

                }

            }

        }
    );

}


/* ====================================================
   SEARCH SETUP
==================================================== */

function setupSearch() {

    const searchInput =
        document.getElementById(
            "searchInput"
        );


    if (!searchInput) {

        return;

    }


    searchInput.addEventListener(
        "input",
        () => {

            updateSearchUI();

            renderDepartments();

        }
    );


    searchInput.addEventListener(
        "keydown",
        event => {
```

```
            if (
                event.key === "Escape"
            ) {

                searchInput.value = "";

                updateSearchUI();

                renderDepartments();

                searchInput.blur();

            }

        }
    );


    updateSearchUI();

}


/* ================================================
   SEARCH UI
================================================ */

function updateSearchUI() {

    const searchInput =
        document.getElementById(
            "searchInput"
        );


    if (!searchInput) {

        return;

    }


    const clearButton =
        document.getElementById(
            "clearSearch"
        );


    const searchResult =
        document.getElementById(
            "searchResultText"
        );


    const search =
        searchInput.value.trim();


    if (clearButton) {

        clearButton.style.display =
            search
                ? "flex"
                : "none";

    }


    if (searchResult) {

        if (search) {

            const filtered =
                getFilteredDepartments();

            searchResult.textContent =
                `${filtered.length} result${
                    filtered.length === 1
                        ? ""
                        : "s"
                } found`;

        }
        else {

            searchResult.textContent =
                `${departments.length} department${
                    departments.length === 1
                        ? ""
                        : "s"
                }`;

        }

    }

}


/* ================================================
   CLEAR SEARCH
================================================ */

function clearDepartmentSearch() {
```

```
    const searchInput =
        document.getElementById(
            "searchInput"
        );


    if (!searchInput) {

        return;

    }


    searchInput.value = "";


    updateSearchUI();

    renderDepartments();


    searchInput.focus();

}


/* ================================================
   LOAD EMPLOYEES
================================================ */

async function loadEmployees() {

    try {

        const response =
            await fetch(
                `${API_URL}/employees`,
                {
                    method: "GET",
                    headers:
                        getAuthHeaders()
                }
            );


        if (
            response.status === 401 ||
            response.status === 403
        ) {

            logout();

            return;

        }


        const data =
            await response.json();


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to load employees"
            );

        }


        if (
            Array.isArray(data)
        ) {

            employees = data;

        }
        else if (
            Array.isArray(
                data.employees
            )
        ) {

            employees =
                data.employees;

        }
        else {

            employees = [];

        }

    }
    catch (error) {

        console.error(
            "Load Employees Error:",
            error
        );

        employees = [];
```

```
        showToast(
            "error",
            "Error",
            "Unable to load employees."
        );

    }

}


/* ====================================================
   LOAD DEPARTMENTS
==================================================== */

async function loadDepartments() {

    try {

        const response =
            await fetch(
                `${API_URL}/departments`,
                {
                    method: "GET",
                    headers:
                        getAuthHeaders()
                }
            );


        if (
            response.status === 401 ||
            response.status === 403
        ) {

            logout();

            return;

        }


        const data =
            await response.json();


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to load departments"
            );

        }


        if (
            Array.isArray(
                data.departments
            )
        ) {

            departments =
                data.departments;

        }
        else if (
            Array.isArray(data)
        ) {

            departments = data;

        }
        else {

            departments = [];

        }


        updateStats();

        updateSearchUI();

        renderDepartments();

    }
    catch (error) {

        console.error(
            "Load Departments Error:",
            error
        );

        departments = [];

        updateStats();

        updateSearchUI();

        renderDepartments();

        showToast(
            "error",
            "Error",
```

```
            "Unable to load departments."
        );

    }

}


/* ====================================================
   STATS
==================================================== */

function updateStats() {

    const total =
        departments.length;


    const active =
        departments.filter(
            department =>
                department.status ===
                "Active"
        ).length;


    const totalEmployees =
        departments.reduce(
            (
                total,
                department
            ) => {

                return (
                    total +
                    Number(
                        department.employeeCount ||
                        0
                    )
                );

            },
            0
        );


    const totalSalary =
        departments.reduce(
            (
                total,
                department
            ) => {

                return (
                    total +
                    Number(
                        department.totalSalary ||
                        0
                    )
                );

            },
            0
        );


    setText(
        "totalDepartments",
        total
    );

    setText(
        "activeDepartments",
        active
    );

    setText(
        "totalEmployees",
        totalEmployees
    );

    setText(
        "totalSalary",
        formatCurrency(
            totalSalary
        )
    );

}


/* ====================================================
   FILTER / SEARCH
==================================================== */

function getFilteredDepartments() {

    const searchInput =
        document.getElementById(
            "searchInput"
        );


    const statusFilter =
        document.getElementById(
```

```
            "statusFilter"
        );

    const search =
        (
            searchInput?.value ||
            ""
        )
            .trim()
            .toLowerCase();

    const status =
        statusFilter?.value ||
        "all";

    return departments.filter(
        department => {

            const name =
                String(
                    department.name ||
                    ""
                ).toLowerCase();

            const description =
                String(
                    department.description ||
                    ""
                ).toLowerCase();

            const headName =
                String(
                    department.head?.name ||
                    ""
                ).toLowerCase();

            const headEmail =
                String(
                    department.head?.email ||
                    ""
                ).toLowerCase();

            const matchesSearch =
                !search ||
                name.includes(search) ||
                description.includes(search) ||
                headName.includes(search) ||
                headEmail.includes(search);

            const matchesStatus =
                status === "all" ||
                department.status ===
                    status;

            return (
                matchesSearch &&
                matchesStatus
            );

        }
    );

}

/* ================================================
   SORT
================================================ */

function changeDepartmentSort(
    field
) {

    if (
        currentSort.field ===
        field
    ) {

        currentSort.direction =
            currentSort.direction ===
            "asc"
                ? "desc"
                : "asc";

    }
    else {

        currentSort.field =
            field;

        currentSort.direction =
            "asc";

    }
```

```
    updateSortArrow();

    renderDepartments();

}


function updateSortArrow() {

    const fields = [
        "name",
        "employees",
        "salary",
        "createdAt"
    ];


    fields.forEach(
        field => {

            const element =
                document.getElementById(
                    `sort-${field}`
                );


            if (!element) {

                return;

            }


            if (
                currentSort.field !==
                field
            ) {

                element.textContent =
                    "$";

            }
            else if (
                currentSort.direction ===
                "asc"
            ) {

                element.textContent =
                    "↑";

            }
            else {

                element.textContent =
                    "↓";

            }

        }
    );

}


function sortDepartments(
    list
) {

    if (!currentSort.field) {

        return list;

    }


    return [...list].sort(
        (
            first,
            second
        ) => {

            let valueA;
            let valueB;


            if (
                currentSort.field ===
                "name"
            ) {

                valueA =
                    String(
                        first.name ||
                        ""
                    ).toLowerCase();

                valueB =
                    String(
                        second.name ||
                        ""
                    ).toLowerCase();

            }
```

```
            else if (
                currentSort.field ===
                "employees"
            ) {

                valueA =
                    Number(
                        first.employeeCount ||
                        0
                    );

                valueB =
                    Number(
                        second.employeeCount ||
                        0
                    );

            }


            else if (
                currentSort.field ===
                "salary"
            ) {

                valueA =
                    Number(
                        first.totalSalary ||
                        0
                    );

                valueB =
                    Number(
                        second.totalSalary ||
                        0
                    );

            }


            else if (
                currentSort.field ===
                "createdAt"
            ) {

                valueA =
                    new Date(
                        first.createdAt ||
                        0
                    ).getTime();

                valueB =
                    new Date(
                        second.createdAt ||
                        0
                    ).getTime();

            }


            if (
                valueA <
                valueB
            ) {

                return (
                    currentSort.direction ===
                    "asc"
                        ? -1
                        : 1
                );

            }


            if (
                valueA >
                valueB
            ) {

                return (
                    currentSort.direction ===
                    "asc"
                        ? 1
                        : -1
                );

            }


            return 0;

        }
    );

}


/* ====================================================
   RENDER DEPARTMENTS
==================================================== */

function renderDepartments() {

    const tableBody =
```

```
    document.getElementById(
        "departmentTableBody"
    );


if (!tableBody) {

    return;

}


let filtered =
    getFilteredDepartments();


filtered =
    sortDepartments(
        filtered
    );


const searchInput =
    document.getElementById(
        "searchInput"
    );


const search =
    searchInput?.value.trim() ||
    "";


const statusFilter =
    document.getElementById(
        "statusFilter"
    );


const status =
    statusFilter?.value ||
    "all";


setText(
    "departmentCountText",
    `${filtered.length} department${
        filtered.length === 1
            ? ""
            : "s"
    }`
);


updateSearchUI();


if (
    filtered.length === 0
) {

    let message =
        "No departments found.";


    let subMessage =
        "Try changing your search or filter.";


    if (search) {

        message =
            `No results for "${search}"`;

        subMessage =
            "Try another department name, description or head name.";

    }
    else if (
        status !== "all"
    ) {

        message =
            `No ${status.toLowerCase()} departments`;

        subMessage =
            "Try selecting another status.";

    }


    tableBody.innerHTML = `
        <tr>

            <td
                colspan="7"
                class="empty-state"
            >

                <div class="empty-state-icon">
                    🔍
                </div>

                <h3>
                    ${escapeHTML(
```

```
                            message
                        )}
                    </h3>

                    <p>
                        ${escapeHTML(
                            subMessage
                        )}
                    </p>

                </td>

            </tr>
        `;

        return;

    }


    tableBody.innerHTML =
        filtered
            .map(
                department =>
                    createDepartmentRow(
                        department
                    )
            )
            .join("");

}


/* ==================================================
   DEPARTMENT ROW
================================================== */

function createDepartmentRow(
    department
) {

    const id =
        department._id;


    const head =
        department.head;


    const headHTML =
        head
            ? `
                <div class="head-info">

                    <div class="avatar">
                        ${getInitials(
                            head.name
                        )}
                    </div>

                    <div>

                        <div class="head-name">
                            ${escapeHTML(
                                head.name
                            )}
                        </div>

                        <div class="head-designation">
                            ${escapeHTML(
                                head.designation ||
                                "Department Head"
                            )}
                        </div>

                    </div>

                </div>
            `
            : `
                <span class="muted-text">
                    No Head Assigned
                </span>
            `;


    const statusClass =
        department.status ===
        "Active"
            ? "status-active"
            : "status-inactive";


    return `
        <tr>

            <td>

                <div class="department-name">
                    ${escapeHTML(
                        department.name ||
                        "-"
                    )}
                </div>
```

```
    <div class="department-description">
        ${escapeHTML(
            department.description ||
            "No description"
        )}
    </div>

</td>


<td>
    ${headHTML}
</td>


<td>

    <button
        type="button"
        class="employee-count-btn"
        onclick="openEmployeesModal('${id}')"
    >

        ${Number(
            department.employeeCount ||
            0
        )}

        <span>
            View Employees
        </span>

    </button>

</td>


<td>
    ${formatCurrency(
        department.totalSalary ||
        0
    )}
</td>


<td>

    <span
        class="status-badge ${statusClass}"
    >

        ${escapeHTML(
            department.status ||
            "Active"
        )}

    </span>

</td>


<td>
    ${formatDate(
        department.createdAt
    )}
</td>


<td>

    <div class="action-buttons">

        <button
            type="button"
            class="action-btn view"
            onclick="openDepartmentDetails('${id}')"
        >
            👁️ View
        </button>


        <button
            type="button"
            class="action-btn edit"
            onclick="openEditDepartmentModal('${id}')"
        >
            🖍️ Edit
        </button>


        <button
            type="button"
            class="action-btn head"
            onclick="openHeadModal('${id}')"
        >
            👤 Head
        </button>


        <button
            type="button"
            class="action-btn delete"
            onclick="openDeleteModal('${id}')"
        >
            🗑️ Delete
```

```
                    </button>

                </div>

            </td>

        </tr>
    `;

}


/* ====================================================
   ADD DEPARTMENT
==================================================== */

function openAddDepartmentModal() {

    setText(
        "departmentModalTitle",
        "Add Department"
    );


    setValue(
        "departmentId",
        ""
    );


    setValue(
        "departmentName",
        ""
    );


    setValue(
        "departmentDescription",
        ""
    );


    setValue(
        "departmentStatus",
        "Active"
    );


    showModal(
        "departmentModal"
    );

}


/* ====================================================
   EDIT DEPARTMENT
==================================================== */

function openEditDepartmentModal(
    id
) {

    const department =
        departments.find(
            item =>
                item._id === id
        );


    if (!department) {

        showToast(
            "error",
            "Error",
            "Department not found."
        );

        return;

    }


    setText(
        "departmentModalTitle",
        "Edit Department"
    );


    setValue(
        "departmentId",
        department._id
    );


    setValue(
        "departmentName",
        department.name ||
        ""
    );


    setValue(
        "departmentDescription",
        department.description ||
```

```
        ""
    );

    setValue(
        "departmentStatus",
        department.status ||
        "Active"
    );

    showModal(
        "departmentModal"
    );

}

/* ====================================================
   SAVE DEPARTMENT
==================================================== */

async function handleDepartmentSubmit(
    event
) {

    event.preventDefault();

    const id =
        document.getElementById(
            "departmentId"
        )?.value.trim();

    const name =
        document.getElementById(
            "departmentName"
        )?.value.trim();

    const description =
        document.getElementById(
            "departmentDescription"
        )?.value.trim();

    const status =
        document.getElementById(
            "departmentStatus"
        )?.value;

    if (!name) {

        showToast(
            "warning",
            "Validation",
            "Department name is required."
        );

        return;

    }

    const body = {
        name,
        description,
        status
    };

    try {

        const isEdit =
            Boolean(id);

        const url =
            isEdit
                ? `${API_URL}/departments/${id}`
                : `${API_URL}/departments`;

        const response =
            await fetch(
                url,
                {
                    method:
                        isEdit
                            ? "PUT"
                            : "POST",

                    headers:
                        getAuthHeaders(),

                    body:
                        JSON.stringify(
                            body
                        )
                }
            );
```

```
        const data =
            await response.json();


        if (
            response.status === 401 ||
            response.status === 403
        ) {

            logout();

            return;

        }


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to save department"
            );

        }


        closeDepartmentModal();


        await loadDepartments();


        showToast(
            "success",
            "Success",
            isEdit
                ? "Department updated successfully."
                : "Department created successfully."
        );

    }
    catch (error) {

        console.error(
            "Save Department Error:",
            error
        );


        showToast(
            "error",
            "Error",
            error.message ||
            "Unable to save department."
        );

    }

}


/* ====================================================
   DELETE
==================================================== */

function openDeleteModal(
    id
) {

    const department =
        departments.find(
            item =>
                item._id === id
        );


    if (!department) {

        showToast(
            "error",
            "Error",
            "Department not found."
        );

        return;

    }


    departmentToDelete =
        department._id;


    setText(
        "deleteDepartmentName",
        department.name
    );


    showModal(
        "deleteModal"
    );

}
```

```
function closeDeleteModal() {

    departmentToDelete =
        null;


    hideModal(
        "deleteModal"
    );

}


async function confirmDeleteDepartment() {

    if (!departmentToDelete) {

        return;

    }


    const id =
        departmentToDelete;


    try {

        const response =
            await fetch(
                `${API_URL}/departments/${id}`,
                {
                    method: "DELETE",
                    headers:
                        getAuthHeaders()
                }
            );


        const data =
            await response.json();


        if (
            response.status === 401 ||
            response.status === 403
        ) {

            logout();

            return;

        }


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to delete department"
            );

        }


        closeDeleteModal();


        await loadDepartments();


        showToast(
            "success",
            "Deleted",
            "Department deleted successfully."
        );

    }
    catch (error) {

        console.error(
            "Delete Department Error:",
            error
        );


        showToast(
            "error",
            "Delete Failed",
            error.message ||
            "Unable to delete department."
        );

    }

}


/* ====================================================
   HEAD MODAL
==================================================== */

function openHeadModal(
    id
```

```
) {

    const department =
        departments.find(
            item =>
                item._id === id
        );


    if (!department) {

        showToast(
            "error",
            "Error",
            "Department not found."
        );

        return;

    }


    setValue(
        "headDepartmentId",
        department._id
    );


    setText(
        "headDepartmentName",
        department.name
    );


    const select =
        document.getElementById(
            "departmentHeadSelect"
        );


    if (!select) {

        return;

    }


    const departmentEmployees =
        employees.filter(
            employee =>
                String(
                    employee.department ||
                    ""
                ).toLowerCase() ===
                String(
                    department.name ||
                    ""
                ).toLowerCase()
        );


    select.innerHTML = `
        <option value="">
            No Department Head
        </option>
    `;


    departmentEmployees.forEach(
        employee => {

            const option =
                document.createElement(
                    "option"
                );


            option.value =
                employee._id;


            option.textContent =
                `${employee.name} - ${
                    employee.designation ||
                    "Employee"
                }`;


            select.appendChild(
                option
            );

        }
    );


    if (
        department.head &&
        department.head._id
    ) {

        select.value =
            department.head._id;

    }
```

```
    else {

        select.value =
            "";

    }


    updateHeadPreview();


    showModal(
        "headModal"
    );

}


function updateHeadPreview() {

    const select =
        document.getElementById(
            "departmentHeadSelect"
        );


    const preview =
        document.getElementById(
            "headPreview"
        );


    if (
        !select ||
        !preview
    ) {

        return;

    }


    const selectedId =
        select.value;


    if (!selectedId) {

        preview.textContent =
            "No department head selected.";


        preview.classList.remove(
            "active"
        );


        return;

    }


    const employee =
        employees.find(
            item =>
                item._id ===
                selectedId
        );


    if (!employee) {

        preview.textContent =
            "Employee not found.";


        preview.classList.remove(
            "active"
        );


        return;

    }


    preview.innerHTML = `
        <strong>
            ${escapeHTML(
                employee.name
            )}
        </strong>

        <br>

        <span>
            ${escapeHTML(
                employee.designation ||
                "Employee"
            )}
        </span>

        <br>

        <span>
```

```
            ${escapeHTML(
                employee.email ||
                ""
            )}
        </span>
    `;


    preview.classList.add(
        "active"
    );

}


async function saveDepartmentHead() {

    const departmentId =
        document.getElementById(
            "headDepartmentId"
        )?.value;


    const head =
        document.getElementById(
            "departmentHeadSelect"
        )?.value;


    if (!departmentId) {

        return;

    }


    try {

        const response =
            await fetch(
                `${API_URL}/departments/${departmentId}/head`,
                {
                    method: "PUT",

                    headers:
                        getAuthHeaders(),

                    body:
                        JSON.stringify({
                            head:
                                head || null
                        })
                }
            );


        const data =
            await response.json();


        if (
            response.status === 401 ||
            response.status === 403
        ) {

            logout();

            return;

        }


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to update department head"
            );

        }


        closeHeadModal();


        await loadDepartments();


        showToast(
            "success",
            "Success",
            "Department head updated successfully."
        );

    }
    catch (error) {

        console.error(
            "Save Head Error:",
            error
        );


        showToast(
            "error",
```

```
            "Error",
            error.message ||
            "Unable to update department head."
        );

    }

}


/* ================================================
   VIEW EMPLOYEES
================================================ */

async function openEmployeesModal(
    id
) {

    try {

        showModal(
            "employeesModal"
        );


        setText(
            "employeesModalTitle",
            "Department Employees"
        );


        setText(
            "employeesModalSubtitle",
            "Loading..."
        );


        const response =
            await fetch(
                `${API_URL}/departments/${id}`,
                {
                    method: "GET",
                    headers:
                        getAuthHeaders()
                }
            );


        const data =
            await response.json();


        if (
            response.status === 401 ||
            response.status === 403
        ) {

            logout();

            return;

        }


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to load department"
            );

        }


        const department =
            data.department;


        const departmentEmployees =
            Array.isArray(
                data.employees
            )
                ? data.employees
                : [];


        setText(
            "employeesModalTitle",
            department?.name ||
            "Department Employees"
        );


        setText(
            "employeesModalSubtitle",
            department?.description ||
            "Employees in this department"
        );


        renderEmployeesSummary(
            department,
            departmentEmployees
        );
```

```
        renderEmployeeList(
            departmentEmployees
        );

    }
    catch (error) {

        console.error(
            "Open Employees Error:",
            error
        );


        showToast(
            "error",
            "Error",
            error.message ||
            "Unable to load employees."
        );


        closeEmployeesModal();

    }

}


function renderEmployeesSummary(
    department,
    departmentEmployees
) {

    const container =
        document.getElementById(
            "employeesDepartmentSummary"
        );


    if (!container) {

        return;

    }


    const totalSalary =
        departmentEmployees.reduce(
            (
                total,
                employee
            ) => {

                return (
                    total +
                    Number(
                        employee.salary ||
                        0
                    )
                );

            },
            0
        );


    const headName =
        department?.head?.name ||
        "Not Assigned";


    container.innerHTML = `

        <div class="summary-item">

            <span>
                Department Head
            </span>

            <strong>
                ${escapeHTML(
                    headName
                )}
            </strong>

        </div>


        <div class="summary-item">

            <span>
                Employees
            </span>

            <strong>
                ${departmentEmployees.length}
            </strong>

        </div>


        <div class="summary-item">

            <span>
```

```
                Total Salary
            </span>

            <strong>
                ${formatCurrency(
                    totalSalary
                )}
            </strong>

        </div>

    `;

}


function renderEmployeeList(
    list
) {

    const tableBody =
        document.getElementById(
            "employeesTableBody"
        );


    if (!tableBody) {

        return;

    }


    if (
        list.length === 0
    ) {

        tableBody.innerHTML = `
            <tr>

                <td
                    colspan="4"
                    class="empty-state"
                >

                    <div class="empty-state-icon">
                        👥
                    </div>

                    <h3>
                        No employees
                    </h3>

                    <p>
                        No employees are assigned to this department.
                    </p>

                </td>

            </tr>
        `;

        return;

    }


    tableBody.innerHTML =
        list
            .map(
                employee => {

                    return `
                        <tr>

                            <td>

                                <div class="head-info">

                                    <div class="avatar">
                                        ${getInitials(
                                            employee.name
                                        )}
                                    </div>

                                    <div>

                                        <div class="head-name">
                                            ${escapeHTML(
                                                employee.name ||
                                                "-"
                                            )}
                                        </div>

                                    </div>

                                </div>

                            </td>


                            <td>
                                ${escapeHTML(
                                    employee.email ||
                                    "-"
```

```
                            )}
                        </td>


                        <td>
                            ${escapeHTML(
                                employee.designation ||
                                "-"
                            )}
                        </td>


                        <td>
                            ${formatCurrency(
                                employee.salary ||
                                0
                            )}
                        </td>

                    </tr>
                `;

            }
        )
        .join("");

}


/* ================================================
   DEPARTMENT DETAILS
================================================ */

async function openDepartmentDetails(
    id
) {

    try {

        showModal(
            "departmentDetailsModal"
        );


        setText(
            "detailsDepartmentName",
            "Loading..."
        );


        setText(
            "detailsDepartmentDescription",
            "Loading department information..."
        );


        const response =
            await fetch(
                `${API_URL}/departments/${id}`,
                {
                    method: "GET",
                    headers:
                        getAuthHeaders()
                }
            );


        const data =
            await response.json();


        if (
            response.status === 401 ||
            response.status === 403
        ) {

            logout();

            return;

        }


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to load department details"
            );

        }


        const department =
            data.department;


        const departmentEmployees =
            Array.isArray(
                data.employees
            )
                ? data.employees
                : [];
```

```
    const employeeCount =
        Number(
            data.employeeCount ||
            departmentEmployees.length ||
            0
        );


    const totalSalary =
        Number(
            data.totalSalary ||
            departmentEmployees.reduce(
                (
                    total,
                    employee
                ) => {

                    return (
                        total +
                        Number(
                            employee.salary ||
                            0
                        )
                    );

                },
                0
            )
        );


    const averageSalary =
        employeeCount > 0
            ? totalSalary /
                employeeCount
            : 0;


    setText(
        "detailsDepartmentName",
        department?.name ||
        "-"
    );


    setText(
        "detailsDepartmentDescription",
        department?.description ||
        "No description available."
    );


    setText(
        "detailsHeadName",
        department?.head?.name ||
        "Not Assigned"
    );


    setText(
        "detailsHeadDesignation",
        department?.head?.designation ||
        ""
    );


    setText(
        "detailsEmployeeCount",
        employeeCount
    );


    setText(
        "detailsTotalSalary",
        formatCurrency(
            totalSalary
        )
    );


    setText(
        "detailsAverageSalary",
        formatCurrency(
            averageSalary
        )
    );


    setText(
        "detailsStatus",
        department?.status ||
        "-"
    );


    setText(
        "detailsCreated",
        formatDate(
            department?.createdAt
        )
    );

}
catch (error) {
```

```
        console.error(
            "Department Details Error:",
            error
        );


        showToast(
            "error",
            "Error",
            error.message ||
            "Unable to load department details."
        );


        closeDepartmentDetailsModal();

    }

}


/* ======================================================
   RESET FILTERS
====================================================== */

function resetFilters() {

    const searchInput =
        document.getElementById(
            "searchInput"
        );


    const statusFilter =
        document.getElementById(
            "statusFilter"
        );


    if (searchInput) {

        searchInput.value = "";

    }


    if (statusFilter) {

        statusFilter.value =
            "all";

    }


    currentSort = {
        field: null,
        direction: "asc"
    };


    updateSortArrow();

    updateSearchUI();

    renderDepartments();

}


/* ======================================================
   MODALS
====================================================== */

function showModal(
    id
) {

    const modal =
        document.getElementById(
            id
        );


    if (!modal) {

        return;

    }


    modal.classList.add(
        "show"
    );


    document.body.style.overflow =
        "hidden";

}


function hideModal(
    id
) {
```

```
    const modal =
        document.getElementById(
            id
        );


    if (!modal) {

        return;

    }


    modal.classList.remove(
        "show"
    );


    const anyOpenModal =
        document.querySelector(
            ".modal-overlay.show"
        );


    if (!anyOpenModal) {

        document.body.style.overflow =
            "";

    }

}


function closeDepartmentModal() {

    hideModal(
        "departmentModal"
    );

}


function closeHeadModal() {

    hideModal(
        "headModal"
    );

}


function closeEmployeesModal() {

    hideModal(
        "employeesModal"
    );

}


function closeDepartmentDetailsModal() {

    hideModal(
        "departmentDetailsModal"
    );

}


document.addEventListener(
    "click",
    event => {

        if (
            event.target.classList.contains(
                "modal-overlay"
            )
        ) {

            hideModal(
                event.target.id
            );

        }

    }
);


/* ==================================================
   TOAST
================================================== */

function showToast(
    type,
    title,
    message
) {

    let container =
        document.getElementById(
            "toastContainer"
        );
```

```
if (!container) {

    container =
        document.createElement(
            "div"
        );


    container.id =
        "toastContainer";


    container.className =
        "toast-container";


    document.body.appendChild(
        container
    );

}


const toast =
    document.createElement(
        "div"
    );


toast.className =
    `toast ${type || ""}`;


let icon = "ℹ️";


if (
    type === "success"
) {

    icon = "✅";

}
else if (
    type === "error"
) {

    icon = "❌";

}
else if (
    type === "warning"
) {

    icon = "⚠️";

}


toast.innerHTML = `

    <div class="toast-icon">
        ${icon}
    </div>

    <div class="toast-content">

        <div class="toast-title">
            ${escapeHTML(
                title ||
                "Notification"
            )}
        </div>

        <div class="toast-message">
            ${escapeHTML(
                message ||
                ""
            )}
        </div>

    </div>

    <button
        type="button"
        class="toast-close"
    >
        ×
    </button>

`;


const closeButton =
    toast.querySelector(
        ".toast-close"
    );


closeButton.addEventListener(
    "click",
    () => {

        removeToast(
```

```
                toast
            );

        }
    );


    container.appendChild(
        toast
    );


    setTimeout(
        () => {

            removeToast(
                toast
            );

        },
        4000
    );

}


function removeToast(
    toast
) {

    if (
        !toast ||
        !toast.parentNode
    ) {

        return;

    }


    toast.classList.add(
        "toast-hide"
    );


    setTimeout(
        () => {

            if (
                toast.parentNode
            ) {

                toast.parentNode.removeChild(
                    toast
                );

            }

        },
        250
    );

}


/* ==================================================
   HELPERS
================================================== */

function setText(
    id,
    value
) {

    const element =
        document.getElementById(
            id
        );


    if (element) {

        element.textContent =
            value ?? "";

    }

}


function setValue(
    id,
    value
) {

    const element =
        document.getElementById(
            id
        );


    if (element) {

        element.value =
            value ?? "";
```

```
    }

}


function formatCurrency(
    value
) {

    const number =
        Number(value) || 0;


    return number.toLocaleString(
        "en-IN",
        {
            style: "currency",
            currency: "INR",
            maximumFractionDigits: 0
        }
    );

}


function formatDate(
    value
) {

    if (!value) {

        return "-";

    }


    const date =
        new Date(value);


    if (
        Number.isNaN(
            date.getTime()
        )
    ) {

        return "-";

    }


    return date.toLocaleDateString(
        "en-IN",
        {
            day: "2-digit",
            month: "short",
            year: "numeric"
        }
    );

}


function getInitials(
    name
) {

    if (!name) {

        return "?";

    }


    const parts =
        String(name)
            .trim()
            .split(/\s+/)
            .filter(Boolean);


    if (
        parts.length === 1
    ) {

        return parts[0]
            .substring(0, 2)
            .toUpperCase();

    }


    return (
        parts[0][0] +
        parts[
            parts.length - 1
        ][0]
    ).toUpperCase();

}


function escapeHTML(
    value
```

```
) {

    return String(
        value ?? ""
    )
        .replace(
            /&/g,
            "&amp;"
        )
        .replace(
            /</g,
            "&lt;"
        )
        .replace(
            />/g,
            "&gt;"
        )
        .replace(
            /"/g,
            "&quot;"
        )
        .replace(
            /'/g,
            "&#039;"
        );

}
```

Frontend/JS/employees.js

```
const EMPLOYEE_API =
    "http://localhost:5000/api/employees";


/* ========================
   AUTH
======================== */

const employeeToken =
    localStorage.getItem("token");


const employeeUserData =
    localStorage.getItem("user");


if (
    !employeeToken ||
    !employeeUserData
) {

    window.location.href =
        "login.html";

}


let employeeUser = null;


try {

    employeeUser =
        JSON.parse(
            employeeUserData
        );

} catch (error) {

    console.error(
        "User data error:",
        error
    );

    localStorage.removeItem(
        "token"
    );

    localStorage.removeItem(
        "user"
    );

    window.location.href =
        "login.html";

}


/* ========================
   VARIABLES
======================== */

let employees = [];

let editEmployeeId = null;


/* ========================
   MESSAGE
======================== */

function showMessage(
    message,
    type
) {

    const messageBox =
        document.getElementById(
            "message"
        );


    if (!messageBox) return;


    messageBox.textContent =
        message;


    messageBox.style.display =
        "block";


    if (type === "success") {

        messageBox.style.background =
            "rgba(22, 163, 74, 0.12)";

        messageBox.style.color =
            "var(--success)";

    } else {
```

```
        messageBox.style.background =
            "rgba(220, 38, 38, 0.12)";

        messageBox.style.color =
            "var(--danger)";

    }


    setTimeout(
        function () {

            messageBox.style.display =
                "none";

        },
        3000
    );

}


/* ========================
   CHECK ADMIN
======================== */

function isAdmin() {

    return (
        employeeUser &&
        employeeUser.role === "admin"
    );

}


/* ========================
   ADMIN UI
======================== */

function setupRoleAccess() {

    const formBox =
        document.getElementById(
            "employeeFormBox"
        );


    if (
        !isAdmin()
    ) {

        if (formBox) {

            formBox.style.display =
                "none";

        }

    }

}


/* ========================
   LOAD EMPLOYEES
======================== */

async function loadEmployees() {

    const tableBody =
        document.getElementById(
            "employeeTableBody"
        );


    if (tableBody) {

        tableBody.innerHTML = `

            <tr>

                <td
                    colspan="9"
                    class="no-data"
                >
                    Loading employees...
                </td>

            </tr>

        `;

    }


    try {

        const response =
            await fetch(
                EMPLOYEE_API,
                {
                    method: "GET",
```

```
                headers: {

                    "Authorization":
                        `Bearer ${employeeToken}`

                }

            }
        );


        const data =
            await response.json();


        console.log(
            "Employees Response:",
            data
        );


        if (!response.ok) {

            showMessage(
                data.message ||
                "Unable to load employees",
                "error"
            );

            return;

        }


        employees =
            Array.isArray(
                data.employees
            )
                ? data.employees
                : [];


        displayEmployees(
            employees
        );


    } catch (error) {

        console.error(
            "Load Employees Error:",
            error
        );


        showMessage(
            "Server connection failed",
            "error"
        );

    }

}


/* ========================
   DISPLAY EMPLOYEES
======================== */

function displayEmployees(
    employeeList
) {

    const tableBody =
        document.getElementById(
            "employeeTableBody"
        );


    if (!tableBody) return;


    tableBody.innerHTML =
        "";


    if (
        !employeeList ||
        employeeList.length === 0
    ) {

        tableBody.innerHTML = `

            <tr>

                <td
                    colspan="9"
                    class="no-data"
                >
                    No employees found
                </td>

            </tr>

        `;
```

```
    return;

}


employeeList.forEach(
    function (employee) {

        const row =
            document.createElement(
                "tr"
            );


        const joiningDate =
            employee.joiningDate
                ? new Date(
                    employee.joiningDate
                  ).toLocaleDateString(
                    "en-IN"
                  )
                : "-";


        const salary =
            Number(
                employee.salary || 0
            ).toLocaleString(
                "en-IN"
            );


        row.innerHTML = `

            <td>
                ${escapeHTML(
                    employee.name
                )}
            </td>

            <td>
                ${escapeHTML(
                    employee.email
                )}
            </td>

            <td>
                ${escapeHTML(
                    employee.phone
                )}
            </td>

            <td>
                ${escapeHTML(
                    employee.department
                )}
            </td>

            <td>
                ${escapeHTML(
                    employee.designation
                )}
            </td>

            <td>
                ₹${salary}
            </td>

            <td>
                ${joiningDate}
            </td>

            <td>
                ${escapeHTML(
                    employee.address
                )}
            </td>

            <td>

                ${
                    isAdmin()
                    ?

                    `
                    <button
                        class="edit-btn"
                        onclick="editEmployee('${employee._id}')"
                    >
                        Edit
                    </button>

                    <button
                        class="delete-btn"
                        onclick="deleteEmployee('${employee._id}')"
                    >
                        Delete
                    </button>
                    `

                    :

                    `<span style="color: var(--muted);">
                        View Only
```

```
                        </span>`

                    }

                </td>

            `;


            tableBody.appendChild(
                row
            );

        }
    );

}


/* ========================
   ESCAPE HTML
======================== */

function escapeHTML(
    value
) {

    if (
        value === null ||
        value === undefined
    ) {

        return "";

    }


    return String(value)
        .replace(
            /&/g,
            "&amp;"
        )
        .replace(
            /</g,
            "&lt;"
        )
        .replace(
            />/g,
            "&gt;"
        )
        .replace(
            /"/g,
            "&quot;"
        )
        .replace(
            /'/g,
            "&#039;"
        );

}


/* ========================
   ADD / UPDATE
======================== */

const employeeForm =
    document.getElementById(
        "employeeForm"
    );


if (employeeForm) {

    employeeForm.addEventListener(
        "submit",
        async function (event) {

            event.preventDefault();


            if (!isAdmin()) {

                showMessage(
                    "Only admin can manage employees",
                    "error"
                );

                return;

            }


            const employeeData = {

                name:
                    document.getElementById(
                        "name"
                    ).value.trim(),

                email:
                    document.getElementById(
                        "email"
                    ).value.trim(),
```

```
        phone:
            document.getElementById(
                "phone"
            ).value.trim(),

        department:
            document.getElementById(
                "department"
            ).value.trim(),

        designation:
            document.getElementById(
                "designation"
            ).value.trim(),

        salary:
            Number(
                document.getElementById(
                    "salary"
                ).value
            ),

        joiningDate:
            document.getElementById(
                "joiningDate"
            ).value,

        address:
            document.getElementById(
                "address"
            ).value.trim()

    };


    try {

        let url =
            EMPLOYEE_API;


        let method =
            "POST";


        if (
            editEmployeeId
        ) {

            url =
                `${EMPLOYEE_API}/${editEmployeeId}`;

            method =
                "PUT";

        }


        const response =
            await fetch(
                url,
                {

                    method:

                        method,

                    headers: {

                        "Content-Type":
                            "application/json",

                        "Authorization":
                            `Bearer ${employeeToken}`

                    },

                    body:
                        JSON.stringify(
                            employeeData
                        )

                }
            );


        const data =
            await response.json();


        console.log(
            "Employee Save Response:",
            data
        );


        if (!response.ok) {

            showMessage(
                data.message ||
                "Operation failed",
                "error"
            );

            return;
```

```
                }

                if (
                    editEmployeeId
                ) {

                    showMessage(
                        "Employee updated successfully",
                        "success"
                    );

                } else {

                    showMessage(
                        "Employee added successfully",
                        "success"
                    );

                }

                resetForm();

                await loadEmployees();

            } catch (error) {

                console.error(
                    "Save Employee Error:",
                    error
                );

                showMessage(
                    "Server connection failed",
                    "error"
                );

            }

        }
    );

}


/* ========================
   EDIT EMPLOYEE
======================== */

async function editEmployee(
    id
) {

    if (!isAdmin()) {

        showMessage(
            "Only admin can edit employees",
            "error"
        );

        return;

    }

    try {

        const response =
            await fetch(
                `${EMPLOYEE_API}/${id}`,
                {

                    method: "GET",

                    headers: {

                        "Authorization":
                            `Bearer ${employeeToken}`

                    }

                }
            );

        const data =
            await response.json();

        if (!response.ok) {

            showMessage(
                data.message ||
                "Unable to get employee",
                "error"
            );

            return;

        }
```

```
        const employee =
            data.employee;


        document.getElementById(
            "name"
        ).value =
            employee.name || "";


        document.getElementById(
            "email"
        ).value =
            employee.email || "";


        document.getElementById(
            "phone"
        ).value =
            employee.phone || "";


        document.getElementById(
            "department"
        ).value =
            employee.department || "";


        document.getElementById(
            "designation"
        ).value =
            employee.designation || "";


        document.getElementById(
            "salary"
        ).value =
            employee.salary || "";


        document.getElementById(
            "joiningDate"
        ).value =
            employee.joiningDate
                ? employee.joiningDate.substring(
                    0,
                    10
                  )
                : "";


        document.getElementById(
            "address"
        ).value =
            employee.address || "";


        editEmployeeId =
            employee._id;


        document.getElementById(
            "formTitle"
        ).textContent =
            "Edit Employee";


        document.getElementById(
            "submitBtn"
        ).textContent =
            "Update Employee";


        document.getElementById(
            "cancelBtn"
        ).style.display =
            "inline-block";


        window.scrollTo({

            top: 0,

            behavior: "smooth"

        });


    } catch (error) {

        console.error(
            "Edit Employee Error:",
            error
        );


        showMessage(
            "Server connection failed",
            "error"
        );

    }
```

```
}


/* ========================
   DELETE EMPLOYEE
======================== */

async function deleteEmployee(
    id
) {

    if (!isAdmin()) {

        showMessage(
            "Only admin can delete employees",
            "error"
        );

        return;

    }


    const confirmDelete =
        confirm(
            "Are you sure you want to delete this employee?"
        );


    if (!confirmDelete) {

        return;

    }


    try {

        const response =
            await fetch(
                `${EMPLOYEE_API}/${id}`,
                {

                    method: "DELETE",

                    headers: {

                        "Authorization":
                            `Bearer ${employeeToken}`

                    }

                }
            );


        const data =
            await response.json();


        if (!response.ok) {

            showMessage(
                data.message ||
                "Unable to delete employee",
                "error"
            );

            return;

        }


        showMessage(
            "Employee deleted successfully",
            "success"
        );


        await loadEmployees();


    } catch (error) {

        console.error(
            "Delete Employee Error:",
            error
        );


        showMessage(
            "Server connection failed",
            "error"
        );

    }

}


/* ========================
   CANCEL EDIT
======================== */

function cancelEdit() {
```

```
    resetForm();

}


/* ========================
   RESET FORM
======================== */

function resetForm() {

    const form =
        document.getElementById(
            "employeeForm"
        );


    if (form) {

        form.reset();

    }


    editEmployeeId =
        null;


    const formTitle =
        document.getElementById(
            "formTitle"
        );


    const submitBtn =
        document.getElementById(
            "submitBtn"
        );


    const cancelBtn =
        document.getElementById(
            "cancelBtn"
        );


    if (formTitle) {

        formTitle.textContent =
            "Add Employee";

    }


    if (submitBtn) {

        submitBtn.textContent =
            "Add Employee";

    }


    if (cancelBtn) {

        cancelBtn.style.display =
            "none";

    }

}


/* ========================
   SEARCH
======================== */

const searchEmployee =
    document.getElementById(
        "searchEmployee"
    );


if (searchEmployee) {

    searchEmployee.addEventListener(
        "input",
        function () {

            const searchValue =
                this.value
                    .toLowerCase()
                    .trim();


            if (!searchValue) {

                displayEmployees(
                    employees
                );

                return;

            }
```

```
            const filteredEmployees =
                employees.filter(
                    function (employee) {

                        return (

                            String(
                                employee.name ||
                                ""
                            )
                            .toLowerCase()
                            .includes(
                                searchValue
                            )

                            ||

                            String(
                                employee.email ||
                                ""
                            )
                            .toLowerCase()
                            .includes(
                                searchValue
                            )

                            ||

                            String(
                                employee.department ||
                                ""
                            )
                            .toLowerCase()
                            .includes(
                                searchValue
                            )

                            ||

                            String(
                                employee.designation ||
                                ""
                            )
                            .toLowerCase()
                            .includes(
                                searchValue
                            )

                            ||

                            String(
                                employee.phone ||
                                ""
                            )
                            .toLowerCase()
                            .includes(
                                searchValue
                            )

                        );

                    }
                );


            displayEmployees(
                filteredEmployees
            );

        }
    );

}


/* ========================
   LOGOUT
======================== */

function logout() {

    const confirmLogout =
        confirm(
            "Are you sure you want to logout?"
        );


    if (!confirmLogout) {

        return;

    }


    localStorage.removeItem(
        "token"
    );


    localStorage.removeItem(
        "user"
    );
```

```
    window.location.href =
        "login.html";

}


/* =========================
   START
========================= */

document.addEventListener(
    "DOMContentLoaded",
    function () {

        setupRoleAccess();

        loadEmployees();

    }
);
```

Frontend/JS/interviews.js

```
const INTERVIEW_API = "http://localhost:5000/api";


/* =========================
   AUTH
========================= */

const token = localStorage.getItem("token");

if (!token) {
    window.location.href = "login.html";
}


/* =========================
   ELEMENTS
========================= */

const interviewForm =
    document.getElementById("interviewForm");

const candidateSelect =
    document.getElementById("candidateId");

const jobSelect =
    document.getElementById("jobId");

const modeSelect =
    document.getElementById("mode");

const meetingLinkGroup =
    document.getElementById("meetingLinkGroup");

const locationGroup =
    document.getElementById("locationGroup");

const meetingLinkInput =
    document.getElementById("meetingLink");

const locationInput =
    document.getElementById("location");

const tableBody =
    document.getElementById("interviewsTableBody");

const refreshBtn =
    document.getElementById("refreshBtn");

const resetBtn =
    document.getElementById("resetBtn");


/* =========================
   HTML ESCAPE
========================= */

function escapeHTML(value) {

    if (value === null || value === undefined) {
        return "";
    }

    return String(value)
        .replace(/&/g, "&amp;")
        .replace(/</g, "&lt;")
        .replace(/>/g, "&gt;")
        .replace(/"/g, "&quot;")
        .replace(/'/g, "&#039;");
}


/* =========================
   AUTH HEADER
========================= */

function getHeaders() {

    return {
        "Content-Type": "application/json",

        "Authorization":
            `Bearer ${token}`
    };
}


/* =========================
   LOAD CANDIDATES
========================= */

async function loadCandidates() {

    try {

        const response = await fetch(
            `${INTERVIEW_API}/candidates`,
            {
                method: "GET",

                headers: getHeaders()
            }
        );
```

```
        if (!response.ok) {

            throw new Error(
                "Unable to load candidates"
            );
        }


        const data =
            await response.json();


        candidateSelect.innerHTML = `
            <option value="">
                Select Candidate
            </option>
        `;


        const candidates =
            (data.candidates || [])
                .filter(
                    candidate =>
                        candidate.status === "Interview"
                );


        candidates.forEach(candidate => {

            const option =
                document.createElement("option");


            option.value =
                candidate._id;


            option.textContent =
                `${candidate.name} - ${candidate.email}`;


            candidateSelect.appendChild(
                option
            );

        });


    } catch (error) {

        console.error(
            "Load Candidates Error:",
            error
        );

    }
}


/* ========================
   LOAD JOBS
======================== */

async function loadJobs() {

    try {

        const response = await fetch(
            `${INTERVIEW_API}/jobs`,
            {
                method: "GET",

                headers: getHeaders()
            }
        );


        if (!response.ok) {

            throw new Error(
                "Unable to load jobs"
            );
        }


        const data =
            await response.json();


        jobSelect.innerHTML = `
            <option value="">
                Select Job
            </option>
        `;


        const jobs =
            data.jobs || [];


        jobs.forEach(job => {

            const option =
```

```
                document.createElement("option");

            option.value =
                job._id;

            option.textContent =
                job.title;

            jobSelect.appendChild(
                option
            );

        });

    } catch (error) {

        console.error(
            "Load Jobs Error:",
            error
        );

    }
}

/* ========================
   MODE CHANGE
======================== */

modeSelect.addEventListener(
    "change",
    function () {

        const mode =
            modeSelect.value;

        if (mode === "Online") {

            meetingLinkGroup.style.display =
                "flex";

            locationGroup.style.display =
                "none";

            meetingLinkInput.required =
                true;

            locationInput.required =
                false;

            locationInput.value =
                "";

        }

        else if (mode === "Offline") {

            meetingLinkGroup.style.display =
                "none";

            locationGroup.style.display =
                "flex";

            meetingLinkInput.required =
                false;

            locationInput.required =
                true;

            meetingLinkInput.value =
                "";

        }

        else {

            meetingLinkGroup.style.display =
                "none";

            locationGroup.style.display =
                "none";

            meetingLinkInput.required =
                false;

            locationInput.required =
                false;

        }

    }
);

/* ========================
   SCHEDULE INTERVIEW
======================== */
```

```
interviewForm.addEventListener(
    "submit",
    async function (event) {

        event.preventDefault();


        const candidateId =
            candidateSelect.value;


        const jobId =
            jobSelect.value;


        const interviewer =
            document
                .getElementById("interviewer")
                .value
                .trim();


        const interviewDate =
            document
                .getElementById("interviewDate")
                .value;


        const mode =
            modeSelect.value;


        const meetingLink =
            meetingLinkInput.value
                .trim();


        const location =
            locationInput.value
                .trim();


        if (
            !candidateId ||
            !jobId ||
            !interviewer ||
            !interviewDate ||
            !mode
        ) {

            alert(
                "Please fill all required fields"
            );

            return;
        }


        if (
            mode === "Online" &&
            !meetingLink
        ) {

            alert(
                "Meeting link is required for online interview"
            );

            return;
        }


        if (
            mode === "Offline" &&
            !location
        ) {

            alert(
                "Location is required for offline interview"
            );

            return;
        }


        const requestBody = {

            candidateId,

            jobId,

            interviewer,

            interviewDate,

            mode,

            meetingLink:
                mode === "Online"
                    ? meetingLink
                    : "",

            location:
                mode === "Offline"
                    ? location
```

```
                : ""
        };

        try {

            const response =
                await fetch(
                    `${INTERVIEW_API}/interviews`,
                    {
                        method: "POST",

                        headers:
                            getHeaders(),

                        body:
                            JSON.stringify(
                                requestBody
                            )
                    }
                );


            const data =
                await response.json();


            if (!response.ok) {

                alert(
                    data.message ||
                    "Unable to schedule interview"
                );

                return;
            }


            alert(
                "Interview scheduled successfully"
            );


            interviewForm.reset();


            meetingLinkGroup.style.display =
                "none";

            locationGroup.style.display =
                "none";


            meetingLinkInput.required =
                false;

            locationInput.required =
                false;


            await loadInterviews();


        } catch (error) {

            console.error(
                "Schedule Interview Error:",
                error
            );


            alert(
                "Server connection error"
            );

        }

    }
);


/* =========================
   LOAD INTERVIEWS
========================= */

async function loadInterviews() {

    try {

        tableBody.innerHTML = `
            <tr>
                <td
                    colspan="8"
                    class="empty-state"
                >
                    Loading interviews...
                </td>
            </tr>
        `;


        const response =
            await fetch(
                `${INTERVIEW_API}/interviews`,
```

```
        {
            method: "GET",

            headers:
                getHeaders()
        }
    );


if (!response.ok) {

    throw new Error(
        "Unable to load interviews"
    );
}


const data =
    await response.json();


const interviews =
    data.interviews || [];


updateStats(interviews);


if (interviews.length === 0) {

    tableBody.innerHTML = `
        <tr>
            <td
                colspan="8"
                class="empty-state"
            >
                No interviews found
            </td>
        </tr>
    `;

    return;
}


tableBody.innerHTML = "";


interviews.forEach(
    interview => {

        const row =
            document.createElement("tr");


        const candidate =
            interview.candidateId;


        const job =
            interview.jobId;


        const candidateName =
            candidate?.name ||
            "Unknown Candidate";


        const jobTitle =
            job?.title ||
            "Unknown Job";


        const interviewer =
            interview.interviewer ||
            "-";


        const date =
            formatDate(
                interview.interviewDate
            );


        const mode =
            interview.mode ||
            "-";


        const status =
            interview.status ||
            "Scheduled";


        const result =
            interview.result ||
            "Pending";


        const meetingLink =
            interview.meetingLink ||
            "";


        const location =
```

```
    interview.location ||
    "";


let meetingButton =
    "";


if (
    mode === "Online" &&
    meetingLink
) {

    meetingButton = `
        <a
            href="${escapeHTML(meetingLink)}"
            target="_blank"
            rel="noopener noreferrer"
            class="action-btn join-btn"
        >
            Join Meeting
        </a>
    `;

}


let modeDetails =
    "";


if (
    mode === "Offline" &&
    location
) {

    modeDetails = `
        <div class="location-text">
            ${escapeHTML(location)}
        </div>
    `;

}


row.innerHTML = `

    <td>
        <strong>
            ${escapeHTML(candidateName)}
        </strong>
    </td>


    <td>
        ${escapeHTML(jobTitle)}
    </td>


    <td>
        ${escapeHTML(interviewer)}
    </td>


    <td>
        ${escapeHTML(date)}
    </td>


    <td>

        <div
            class="${
                mode === "Online"
                    ? "mode-online"
                    : "mode-offline"
            }"
        >
            ${escapeHTML(mode)}
        </div>

        ${modeDetails}

    </td>


    <td>

        <span
            class="status-badge
            ${
                status === "Scheduled"
                    ? "status-scheduled"
                    : status === "Completed"
                        ? "status-completed"
                        : "status-cancelled"
            }"
        >
            ${escapeHTML(status)}
        </span>

    </td>


    <td>
```

```
    <span
        class="result-badge
        ${
            result === "Pending"
                ? "result-pending"
                : result === "Selected"
                    ? "result-selected"
                    : "result-rejected"
        }"
    >
        ${escapeHTML(result)}
    </span>

</td>


<td>

    <div class="actions-container">

        ${
            meetingButton
        }


        <div class="action-row">

            <select
                class="action-select"
                onchange="
                    updateInterviewStatus(
                        '${interview._id}',
                        this.value
                    )
                "
            >

                <option value="">
                    Status
                </option>

                <option
                    value="Scheduled"
                    ${
                        status === "Scheduled"
                            ? "selected"
                            : ""
                    }
                >
                    Scheduled
                </option>

                <option
                    value="Completed"
                    ${
                        status === "Completed"
                            ? "selected"
                            : ""
                    }
                >
                    Completed
                </option>

                <option
                    value="Cancelled"
                    ${
                        status === "Cancelled"
                            ? "selected"
                            : ""
                    }
                >
                    Cancelled
                </option>

            </select>


            <select
                class="action-select"
                onchange="
                    updateInterviewResult(
                        '${interview._id}',
                        this.value
                    )
                "
            >

                <option value="">
                    Result
                </option>

                <option
                    value="Pending"
                    ${
                        result === "Pending"
                            ? "selected"
                            : ""
                    }
                >
                    Pending
                </option>

                <option
                    value="Selected"
```

```
                                    ${
                                        result === "Selected"
                                            ? "selected"
                                            : ""
                                    }
                                >
                                    Selected
                                </option>

                                <option
                                    value="Rejected"
                                    ${
                                        result === "Rejected"
                                            ? "selected"
                                            : ""
                                    }
                                >
                                    Rejected
                                </option>

                            </select>

                        </div>


                        <div class="action-row">

                            <button
                                type="button"
                                class="action-btn feedback-btn"
                                onclick="
                                    addFeedback(
                                        '${interview._id}'
                                    )
                                "
                            >
                                Feedback
                            </button>


                            <button
                                type="button"
                                class="action-btn delete-btn"
                                onclick="
                                    deleteInterview(
                                        '${interview._id}'
                                    )
                                "
                            >
                                Delete
                            </button>

                        </div>

                    </div>

                </td>

            `;


            tableBody.appendChild(
                row
            );

        }
    );


} catch (error) {

    console.error(
        "Load Interviews Error:",
        error
    );


    tableBody.innerHTML = `
        <tr>
            <td
                colspan="8"
                class="empty-state"
            >
                Unable to load interviews
            </td>
        </tr>
    `;

}
}


/* ========================
   DATE FORMAT
======================== */

function formatDate(dateValue) {

    if (!dateValue) {
        return "-";
    }


    const date =
        new Date(dateValue);
```

```
    if (isNaN(date.getTime())) {
        return "-";
    }


    return date.toLocaleString(
        "en-IN",
        {
            day: "2-digit",
            month: "short",
            year: "numeric",

            hour: "2-digit",
            minute: "2-digit",

            hour12: true
        }
    );

}


/* ========================
   UPDATE STATUS
======================== */

async function updateInterviewStatus(
    interviewId,
    status
) {

    if (!status) {
        return;
    }


    try {

        const response =
            await fetch(
                `${INTERVIEW_API}/interviews/${interviewId}/status`,
                {
                    method: "PUT",

                    headers:
                        getHeaders(),

                    body:
                        JSON.stringify({
                            status
                        })
                }
            );


        const data =
            await response.json();


        if (!response.ok) {

            alert(
                data.message ||
                "Unable to update status"
            );

            return;
        }


        await loadInterviews();


    } catch (error) {

        console.error(
            "Update Status Error:",
            error
        );


        alert(
            "Server connection error"
        );

    }
}


/* ========================
   UPDATE RESULT
======================== */

async function updateInterviewResult(
    interviewId,
    result
) {

    if (!result) {
        return;
    }
```

```
    try {

        const response =
            await fetch(
                `${INTERVIEW_API}/interviews/${interviewId}/result`,
                {
                    method: "PUT",

                    headers:
                        getHeaders(),

                    body:
                        JSON.stringify({
                            result
                        })
                }
            );


        const data =
            await response.json();


        if (!response.ok) {

            alert(
                data.message ||
                "Unable to update result"
            );

            return;
        }


        await loadInterviews();


    } catch (error) {

        console.error(
            "Update Result Error:",
            error
        );


        alert(
            "Server connection error"
        );

    }
}


/* ========================
   FEEDBACK
======================== */

async function addFeedback(
    interviewId
) {

    const feedback =
        prompt(
            "Enter interview feedback:"
        );


    if (
        feedback === null
    ) {
        return;
    }


    const cleanFeedback =
        feedback.trim();


    if (!cleanFeedback) {

        alert(
            "Feedback cannot be empty"
        );

        return;
    }


    try {

        const response =
            await fetch(
                `${INTERVIEW_API}/interviews/${interviewId}/result`,
                {
                    method: "PUT",

                    headers:
                        getHeaders(),

                    body:
                        JSON.stringify({
                            feedback:
                                cleanFeedback
                        })
                }
```

```
            );

        const data =
            await response.json();

        if (!response.ok) {

            alert(
                data.message ||
                "Unable to save feedback"
            );

            return;
        }

        alert(
            "Feedback saved successfully"
        );

        await loadInterviews();

    } catch (error) {

        console.error(
            "Feedback Error:",
            error
        );

        alert(
            "Server connection error"
        );

    }
}

/* ========================
   DELETE INTERVIEW
======================== */

async function deleteInterview(
    interviewId
) {

    const confirmDelete =
        confirm(
            "Are you sure you want to delete this interview?"
        );

    if (!confirmDelete) {
        return;
    }

    try {

        const response =
            await fetch(
                `${INTERVIEW_API}/interviews/${interviewId}`,
                {
                    method: "DELETE",

                    headers:
                        getHeaders()
                }
            );

        const data =
            await response.json();

        if (!response.ok) {

            alert(
                data.message ||
                "Unable to delete interview"
            );

            return;
        }

        alert(
            "Interview deleted successfully"
        );

        await loadInterviews();

    } catch (error) {

        console.error(
            "Delete Interview Error:",
            error
        );
```

```
        alert(
            "Server connection error"
        );

    }
}


/* ========================
   UPDATE STATS
======================== */

function updateStats(
    interviews
) {

    const total =
        interviews.length;


    const scheduled =
        interviews.filter(
            interview =>
                interview.status === "Scheduled"
        ).length;


    const completed =
        interviews.filter(
            interview =>
                interview.status === "Completed"
        ).length;


    const selected =
        interviews.filter(
            interview =>
                interview.result === "Selected"
        ).length;


    document.getElementById(
        "totalInterviews"
    ).textContent =
        total;


    document.getElementById(
        "scheduledInterviews"
    ).textContent =
        scheduled;


    document.getElementById(
        "completedInterviews"
    ).textContent =
        completed;


    document.getElementById(
        "selectedInterviews"
    ).textContent =
        selected;

}


/* ========================
   RESET FORM
======================== */

resetBtn.addEventListener(
    "click",
    function () {

        interviewForm.reset();


        meetingLinkGroup.style.display =
            "none";


        locationGroup.style.display =
            "none";


        meetingLinkInput.required =
            false;


        locationInput.required =
            false;

    }
);


/* ========================
   REFRESH
======================== */

refreshBtn.addEventListener(
    "click",
    async function () {
```

```
        await loadCandidates();

        await loadJobs();

        await loadInterviews();

    }
);


/* =========================
   INITIAL LOAD
========================= */

async function initializePage() {

    await loadCandidates();

    await loadJobs();

    await loadInterviews();

}


initializePage();
```

Frontend/JS/jobs.js

```
console.log("JOBS JS LOADED");

const JOB_API = "http://localhost:5000/api/jobs";

const jobToken = localStorage.getItem("token");
const jobUserData = localStorage.getItem("user");

let jobUser = null;
let jobs = [];
let editJobId = null;

try {
    jobUser = JSON.parse(jobUserData);
} catch (error) {
    console.error("User data error:", error);
}

function escapeHTML(value) {
    if (value === null || value === undefined) {
        return "";
    }

    return String(value)
        .replace(/&/g, "&amp;")
        .replace(/</g, "&lt;")
        .replace(/>/g, "&gt;")
        .replace(/"/g, "&quot;")
        .replace(/'/g, "&#039;");
}

function canManageJobs() {
    return (
        jobUser &&
        (
            jobUser.role === "admin" ||
            jobUser.role === "hr"
        )
    );
}

function setupRoleAccess() {
    const addJobButton = document.querySelector(
        'button[type="submit"]'
    );

    if (!canManageJobs() && addJobButton) {
        addJobButton.style.display = "none";
    }
}

async function loadJobs() {
    console.log("Loading jobs...");

    const table = document.getElementById("jobTableBody");

    if (!table) {
        console.error("jobTableBody not found");
        return;
    }

    try {
        table.innerHTML = `
            <tr>
                <td colspan="8" class="no-data">
                    Loading jobs...
                </td>
            </tr>
        `;

        const response = await fetch(JOB_API, {
            method: "GET",
            headers: {
                "Authorization": `Bearer ${jobToken}`,
                "Content-Type": "application/json"
            }
        });

        const data = await response.json();

        console.log("Jobs API Status:", response.status);
        console.log("Jobs API Data:", data);

        if (!response.ok) {
            table.innerHTML = `
                <tr>
                    <td colspan="8" class="no-data">
                        ${escapeHTML(
                            data.message ||
                            "Unable to load jobs"
                        )}
                    </td>
                </tr>
            `;
            return;
        }

        jobs = Array.isArray(data.jobs)
            ? data.jobs
            : [];

        displayJobs(jobs);
```

```
    } catch (error) {
        console.error("Load Jobs Error:", error);

        table.innerHTML = `
            <tr>
                <td colspan="8" class="no-data">
                    Server connection failed
                </td>
            </tr>
        `;
    }
}

function displayJobs(jobList) {
    const table = document.getElementById("jobTableBody");

    if (!table) {
        return;
    }

    if (!jobList.length) {
        table.innerHTML = `
            <tr>
                <td colspan="8" class="no-data">
                    No jobs found
                </td>
            </tr>
        `;
        return;
    }

    table.innerHTML = jobList.map(job => {

        const salary = Number(job.salary || 0);

        const description =
            job.description || "";

        return `
            <tr>
                <td>
                    ${escapeHTML(job.title)}
                </td>

                <td>
                    ${escapeHTML(job.department)}
                </td>

                <td>
                    ${escapeHTML(job.location)}
                </td>

                <td>
                    ${escapeHTML(job.jobType)}
                </td>

                <td>
                    ₹${salary.toLocaleString("en-IN")}
                </td>

                <td>
                    <span class="status-badge">
                        ${escapeHTML(job.status)}
                    </span>
                </td>

                <td>
                    ${escapeHTML(description)}
                </td>

                <td>
                    ${
                        canManageJobs()
                        ? `
                            <button
                                type="button"
                                onclick="editJob('${job._id}')"
                            >
                                Edit
                            </button>

                            <button
                                type="button"
                                onclick="deleteJob('${job._id}')"
                            >
                                Delete
                            </button>
                        `
                        : "View Only"
                    }
                </td>
            </tr>
        `;
    }).join("");
}

async function addOrUpdateJob(event) {
    event.preventDefault();

    if (!canManageJobs()) {
        alert(
            "You do not have permission to manage jobs."
        );
        return;
    }
```

```
const title =
    document.getElementById("title").value.trim();

const department =
    document.getElementById("department").value.trim();

const location =
    document.getElementById("location").value.trim();

const jobType =
    document.getElementById("jobType").value;

const status =
    document.getElementById("status").value;

const salary =
    document.getElementById("salary").value;

const description =
    document.getElementById("description")
        .value.trim();

const requirements =
    document.getElementById("requirements")
        .value.trim();

if (
    !title ||
    !department ||
    !location ||
    !jobType ||
    !salary ||
    !description ||
    !requirements
) {
    alert("Please fill all job fields.");
    return;
}

const jobData = {
    title,
    department,
    location,
    jobType,
    status,
    salary: Number(salary),
    description,
    requirements
};

console.log("Sending Job Data:", jobData);

try {
    const url = editJobId
        ? `${JOB_API}/${editJobId}`
        : JOB_API;

    const method = editJobId
        ? "PUT"
        : "POST";

    const response = await fetch(url, {
        method,
        headers: {
            "Authorization": `Bearer ${jobToken}`,
            "Content-Type": "application/json"
        },
        body: JSON.stringify(jobData)
    });

    const data = await response.json();

    console.log(
        "Save Job Status:",
        response.status
    );

    console.log(
        "Save Job Response:",
        data
    );

    if (!response.ok) {
        alert(
            data.message ||
            "Operation failed"
        );
        return;
    }

    alert(
        editJobId
            ? "Job updated successfully"
            : "Job added successfully"
    );

    resetJobForm();

    await loadJobs();

} catch (error) {
    console.error(
        "Save Job Error:",
        error
```

```
        );

        alert("Server connection failed");
    }
}

function editJob(id) {
    const job = jobs.find(
        item => item._id === id
    );

    if (!job) {
        alert("Job not found");
        return;
    }

    document.getElementById("title").value =
        job.title || "";

    document.getElementById("department").value =
        job.department || "";

    document.getElementById("location").value =
        job.location || "";

    document.getElementById("jobType").value =
        job.jobType || "Full Time";

    document.getElementById("status").value =
        job.status || "Open";

    document.getElementById("salary").value =
        job.salary || "";

    document.getElementById("description").value =
        job.description || "";

    document.getElementById("requirements").value =
        job.requirements || "";

    editJobId = id;

    const submitButton =
        document.getElementById("submitBtn");

    if (submitButton) {
        submitButton.textContent =
            "Update Job";
    }

    const cancelButton =
        document.getElementById("cancelBtn");

    if (cancelButton) {
        cancelButton.style.display =
            "inline-block";
    }

    window.scrollTo({
        top: 0,
        behavior: "smooth"
    });
}

async function deleteJob(id) {
    if (!canManageJobs()) {
        alert(
            "You do not have permission to delete jobs."
        );
        return;
    }

    const confirmDelete = confirm(
        "Are you sure you want to delete this job?"
    );

    if (!confirmDelete) {
        return;
    }

    try {
        const response = await fetch(
            `${JOB_API}/${id}`,
            {
                method: "DELETE",
                headers: {
                    "Authorization":
                        `Bearer ${jobToken}`
                }
            }
        );

        const data = await response.json();

        if (!response.ok) {
            alert(
                data.message ||
                "Unable to delete job"
            );
            return;
        }

        alert(
            "Job deleted successfully"
        );
```

```
        await loadJobs();

    } catch (error) {
        console.error(
            "Delete Job Error:",
            error
        );

        alert(
            "Server connection failed"
        );
    }
}

function resetJobForm() {
    const form =
        document.getElementById("jobForm");

    if (form) {
        form.reset();
    }

    editJobId = null;

    const submitButton =
        document.getElementById("submitBtn");

    if (submitButton) {
        submitButton.textContent =
            "Add Job";
    }

    const cancelButton =
        document.getElementById("cancelBtn");

    if (cancelButton) {
        cancelButton.style.display =
            "none";
    }
}

function cancelEdit() {
    resetJobForm();
}

function searchJobs() {
    const searchInput =
        document.getElementById("searchJob");

    if (!searchInput) {
        return;
    }

    const searchText =
        searchInput.value
            .toLowerCase()
            .trim();

    const filteredJobs =
        jobs.filter(job =>

            String(job.title || "")
                .toLowerCase()
                .includes(searchText)

            ||

            String(job.department || "")
                .toLowerCase()
                .includes(searchText)

            ||

            String(job.location || "")
                .toLowerCase()
                .includes(searchText)

            ||

            String(job.jobType || "")
                .toLowerCase()
                .includes(searchText)

            ||

            String(job.status || "")
                .toLowerCase()
                .includes(searchText)
        );

    displayJobs(filteredJobs);
}

function logout() {
    localStorage.removeItem("token");
    localStorage.removeItem("user");

    window.location.href =
        "login.html";
}

document.addEventListener(
    "DOMContentLoaded",
    function () {
```

```
        console.log(
            "Jobs page loaded"
        );

        setupRoleAccess();

        const form =
            document.getElementById("jobForm");

        if (form) {
            form.addEventListener(
                "submit",
                addOrUpdateJob
            );
        }

        const searchInput =
            document.getElementById("searchJob");

        if (searchInput) {
            searchInput.addEventListener(
                "input",
                searchJobs
            );
        }

        loadJobs();
    }
);
```

Frontend/JS/notifications.js

```
const NOTIFICATION_API =
    "http://localhost:5000/api/notifications";


let allNotifications = [];

let filteredNotifications = [];

let currentPage = 1;

const pageSize = 5;


const notificationToken =
    localStorage.getItem("token");


if (!notificationToken) {

    window.location.href =
        "login.html";

}


/* ========================
   HEADERS
======================== */

function notificationHeaders() {

    return {

        "Authorization":
            `Bearer ${notificationToken}`,

        "Content-Type":
            "application/json"

    };

}


/* ========================
   READ STATUS
======================== */

function isNotificationRead(
    notification
) {

    return (

        notification.isRead === true ||

        notification.isRead === "true" ||

        notification.isRead === 1 ||

        notification.isRead === "1"

    );

}


/* ========================
   MESSAGE
======================== */

function showNotificationMessage(
    message,
    type = "success"
) {

    const box =
        document.getElementById(
            "message"
        );


    if (!box) return;


    box.textContent =
        message;


    box.style.display =
        "block";


    if (type === "error") {

        box.style.background =
            "rgba(220,38,38,0.12)";

        box.style.color =
            "var(--danger)";

    } else {
```

```
        box.style.background =
            "rgba(22,163,74,0.12)";

        box.style.color =
            "var(--success)";

    }


    setTimeout(() => {

        box.style.display =
            "none";

    }, 3000);

}


/* ========================
   LOAD
======================== */

async function loadNotifications(
    page = 1
) {

    currentPage = page;


    const loading =
        document.getElementById(
            "loadingState"
        );


    const empty =
        document.getElementById(
            "emptyState"
        );


    const list =
        document.getElementById(
            "notificationList"
        );


    if (loading) {

        loading.style.display =
            "block";

    }


    if (empty) {

        empty.style.display =
            "none";

    }


    if (list) {

        list.innerHTML =
            "";

    }


    try {

        const response =
            await fetch(
                `${NOTIFICATION_API}?page=1&limit=100`,
                {
                    method: "GET",

                    headers:
                        notificationHeaders()
                }
            );


        const data =
            await response.json();


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to load notifications"
            );

        }


        allNotifications =
            Array.isArray(
                data
            )
```

```
                ? data
                : (
                    data.notifications ||
                    data.data ||
                    []
                );


        applyFilters();


    } catch (error) {

        if (loading) {

            loading.style.display =
                "none";

        }


        showNotificationMessage(

            error.message ||
            "Server connection error",

            "error"

        );

    }

}


/* ========================
   FILTER
======================== */

function applyFilters() {

    const searchInput =
        document.getElementById(
            "searchInput"
        );


    const typeFilter =
        document.getElementById(
            "typeFilter"
        );


    const statusFilter =
        document.getElementById(
            "statusFilter"
        );


    const search =
        searchInput
            ? searchInput.value
                .trim()
                .toLowerCase()
            : "";


    const type =
        typeFilter
            ? typeFilter.value
            : "all";


    const status =
        statusFilter
            ? statusFilter.value
            : "all";


    filteredNotifications =
        allNotifications.filter(
            notification => {

                const title =
                    String(
                        notification.title ||
                        ""
                    )
                    .toLowerCase();


                const message =
                    String(
                        notification.message ||
                        ""
                    )
                    .toLowerCase();


                const notificationType =
                    String(
                        notification.type ||
                        ""
                    );
```

```
                const matchesSearch =
                    !search ||

                    title.includes(
                        search
                    ) ||

                    message.includes(
                        search
                    ) ||

                    notificationType
                        .toLowerCase()
                        .includes(search);


                const matchesType =
                    type === "all" ||
                    notificationType === type;


                let matchesStatus =
                    true;


                if (
                    status === "unread"
                ) {

                    matchesStatus =
                        !isNotificationRead(
                            notification
                        );

                }


                if (
                    status === "read"
                ) {

                    matchesStatus =
                        isNotificationRead(
                            notification
                        );

                }


                return (

                    matchesSearch &&

                    matchesType &&

                    matchesStatus

                );

            }
        );


    filteredNotifications.sort(
        (a, b) =>

            new Date(
                b.createdAt
            ) -

            new Date(
                a.createdAt
            )
    );


    currentPage = 1;


    updateStats();

    renderNotifications();

}


/* ========================
   STATS
======================== */

function updateStats() {

    const total =
        allNotifications.length;


    const unread =
        allNotifications.filter(
            notification =>
                !isNotificationRead(
                    notification
                )
        ).length;
```

```
    const read =
        total - unread;


    const totalElement =
        document.getElementById(
            "totalNotifications"
        );


    const unreadElement =
        document.getElementById(
            "unreadNotifications"
        );


    const readElement =
        document.getElementById(
            "readNotifications"
        );


    if (totalElement) {

        totalElement.textContent =
            total;

    }


    if (unreadElement) {

        unreadElement.textContent =
            unread;

    }


    if (readElement) {

        readElement.textContent =
            read;

    }


    const resultText =
        document.getElementById(
            "resultText"
        );


    if (resultText) {

        resultText.textContent =
            `${filteredNotifications.length} notification${
                filteredNotifications.length !== 1
                    ? "s"
                    : ""
            }`;

    }

}


/* =========================
   RENDER
========================= */

function renderNotifications() {

    const loading =
        document.getElementById(
            "loadingState"
        );


    const empty =
        document.getElementById(
            "emptyState"
        );


    const list =
        document.getElementById(
            "notificationList"
        );


    if (loading) {

        loading.style.display =
            "none";

    }


    if (!list) return;


    if (
        filteredNotifications.length === 0
    ) {
```

```
        list.innerHTML =
            "";


        if (empty) {

            empty.style.display =
                "block";

        }


        renderPagination();

        return;

    }


    if (empty) {

        empty.style.display =
            "none";

    }


    const totalPages =
        Math.ceil(
            filteredNotifications.length /
            pageSize
        );


    if (
        currentPage > totalPages
    ) {

        currentPage =
            totalPages;

    }


    const start =
        (currentPage - 1) *
        pageSize;


    const pageNotifications =
        filteredNotifications.slice(
            start,
            start + pageSize
        );


    list.innerHTML =
        pageNotifications
            .map(
                notification =>
                    createNotificationHTML(
                        notification
                    )
            )
            .join("");


    renderPagination();

}


/* ========================
   CARD
======================== */

function createNotificationHTML(
    notification
) {

    const id =
        notification._id;


    const isRead =
        isNotificationRead(
            notification
        );


    const type =
        notification.type ||
        "System";


    const priority =
        notification.priority ||
        "Normal";


    const title =
        escapeHTML(
            notification.title ||
            "Notification"
```

```
    );

const message =
    escapeHTML(
        notification.message ||
        ""
    );

const date =
    notification.createdAt
        ? new Date(
            notification.createdAt
        ).toLocaleString()
        : "-";

const timeAgo =
    getTimeAgo(
        notification.createdAt
    );

return `

    <div
        class="notification-item ${
            isRead
                ? "read"
                : "unread"
        }"
        onclick="
            openNotificationFromCard(
                event,
                '${id}'
            )
        "
    >

        <div class="notification-icon">

            ${getNotificationIcon(type)}

        </div>

        <div class="notification-content">

            <div class="notification-title-row">

                <h3 class="notification-title">

                    ${title}

                </h3>

                <span
                    class="notification-time"
                    title="${date}"
                >

                    ${timeAgo}

                </span>

            </div>

            <div class="notification-message">

                ${message}

            </div>

            <div class="notification-meta">

                <span class="notification-type">

                    ${escapeHTML(type)}

                </span>

                <span class="notification-priority">

                    ${escapeHTML(priority)}

                </span>

                <span
                    class="notification-status ${
                        isRead
                            ? "read"
                            : "unread"
                    }"
                >

                    ${
                        isRead
                            ? "Read"
```

```
                            : "Unread"
                    }

                </span>

            </div>

        </div>


        <div class="notification-actions">


            <button
                type="button"
                class="notification-action-btn"
                title="Open notification"
                onclick="
                    viewNotification(
                        event,
                        '${id}'
                    )
                "
            >
                👁
            </button>


            ${
                isRead
                    ? `
                        <button
                            type="button"
                            class="notification-action-btn"
                            title="Mark as unread"
                            onclick="
                                markAsUnread(
                                    event,
                                    '${id}'
                                )
                            "
                        >
                            🔵
                        </button>
                    `
                    : ""
            }


            <button
                type="button"
                class="notification-action-btn delete"
                title="Delete"
                onclick="
                    deleteNotification(
                        event,
                        '${id}'
                    )
                "
            >
                🗑
            </button>


        </div>

    </div>

`;

}


/* ========================
   CARD OPEN
======================== */

function openNotificationFromCard(
    event,
    id
) {

    if (
        event &&
        event.target.closest("button")
    ) {

        return;

    }


    viewNotification(
        event,
        id
    );

}


/* ========================
   ICON
======================== */

function getNotificationIcon(
```

```
    type
) {

    const icons = {

        Recruitment: "👤",

        Interview: "🗓️",

        Leave: "🏖️",

        Payroll: "💰",

        Employee: "👨‍💼",

        Department: "🏬",

        Attendance: "🕘",

        Performance: "📈",

        System: "🔔"

    };


    return icons[type] ||
        "🔔";

}


/* ========================
   VIEW
   AUTO READ
======================== */

async function viewNotification(
    event,
    id
) {

    if (event) {

        event.stopPropagation();

    }


    const notification =
        allNotifications.find(
            item =>
                String(item._id) ===
                String(id)
        );


    if (!notification) {

        return;

    }


    /* ========================
       AUTO READ
    ======================== */

    if (
        !isNotificationRead(
            notification
        )
    ) {

        try {

            const response =
                await fetch(
                    `${NOTIFICATION_API}/${id}/read`,
                    {
                        method: "PUT",

                        headers:
                            notificationHeaders()
                    }
                );


            const data =
                await response.json();


            if (!response.ok) {

                throw new Error(
                    data.message ||
                    "Unable to mark as read"
                );

            }


            /*
             * IMPORTANT:
             * Use backend response
```

```
         */

        if (
            data.notification
        ) {

            Object.assign(
                notification,
                data.notification
            );

        } else {

            notification.isRead =
                true;

            notification.readAt =
                new Date().toISOString();

        }


        updateStats();


        /*
         * Re-render card without
         * closing modal
         */

        renderNotifications();


    } catch (error) {

        console.error(
            "Auto Read Error:",
            error
        );


        showNotificationMessage(

            error.message ||
            "Unable to mark as read",

            "error"

        );


        return;

    }

}


/* =========================
   MODAL
========================= */

const modalType =
    document.getElementById(
        "modalType"
    );


const modalTitle =
    document.getElementById(
        "modalTitle"
    );


const modalMessage =
    document.getElementById(
        "modalMessage"
    );


const modalPriority =
    document.getElementById(
        "modalPriority"
    );


const modalStatus =
    document.getElementById(
        "modalStatus"
    );


const modalDate =
    document.getElementById(
        "modalDate"
    );


if (modalType) {

    modalType.textContent =
        notification.type ||
        "System";

}
```

```
    if (modalTitle) {

        modalTitle.textContent =
            notification.title ||
            "-";

    }


    if (modalMessage) {

        modalMessage.textContent =
            notification.message ||
            "-";

    }


    if (modalPriority) {

        modalPriority.textContent =
            notification.priority ||
            "Normal";

    }


    if (modalStatus) {

        modalStatus.textContent =
            "Read";

    }


    if (modalDate) {

        modalDate.textContent =
            notification.createdAt
                ? new Date(
                    notification.createdAt
                ).toLocaleString()
                : "-";

    }


    const modal =
        document.getElementById(
            "notificationModal"
        );


    if (modal) {

        modal.style.display =
            "flex";

    }

}


/* ========================
   MARK UNREAD
======================== */

async function markAsUnread(
    event,
    id
) {

    if (event) {

        event.stopPropagation();

    }


    try {

        const response =
            await fetch(
                `${NOTIFICATION_API}/${id}/unread`,
                {
                    method: "PUT",

                    headers:
                        notificationHeaders()
                }
            );


        const data =
            await response.json();


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to mark as unread"
            );
```

```
        }


        const notification =
            allNotifications.find(
                item =>
                    String(item._id) ===
                    String(id)
            );


        if (notification) {

            if (
                data.notification
            ) {

                Object.assign(
                    notification,
                    data.notification
                );

            } else {

                notification.isRead =
                    false;

                notification.readAt =
                    null;

            }

        }


        applyFilters();


        showNotificationMessage(
            "Notification marked as unread"
        );


    } catch (error) {

        showNotificationMessage(

            error.message ||
            "Unable to mark as unread",

            "error"

        );

    }

}


/* ========================
   MARK ALL READ
======================== */

async function markAllAsRead() {

    try {

        const response =
            await fetch(
                `${NOTIFICATION_API}/read-all`,
                {
                    method: "PUT",

                    headers:
                        notificationHeaders()
                }
            );


        const data =
            await response.json();


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to mark all as read"
            );

        }


        /*
         * Update local data
         */

        allNotifications =
            allNotifications.map(
                notification => ({

                    ...notification,

                    isRead: true,
```

```
                    readAt:
                        new Date().toISOString()

                })
            );


        applyFilters();


        showNotificationMessage(
            "All notifications marked as read"
        );


    } catch (error) {

        showNotificationMessage(

            error.message ||
            "Unable to mark all as read",

            "error"

        );

    }

}


/* ========================
   DELETE
======================== */

async function deleteNotification(
    event,
    id
) {

    if (event) {

        event.stopPropagation();

    }


    if (
        !confirm(
            "Delete this notification?"
        )
    ) {

        return;

    }


    try {

        const response =
            await fetch(
                `${NOTIFICATION_API}/${id}`,
                {
                    method: "DELETE",

                    headers:
                        notificationHeaders()
                }
            );


        const data =
            await response.json();


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to delete notification"
            );

        }


        allNotifications =
            allNotifications.filter(
                notification =>
                    String(
                        notification._id
                    ) !==
                    String(id)
            );


        applyFilters();


        showNotificationMessage(
            "Notification deleted"
        );
```

```
    } catch (error) {

        showNotificationMessage(

            error.message ||
            "Unable to delete notification",

            "error"

        );

    }

}


/* ========================
   DELETE READ
======================== */

async function deleteReadNotifications() {

    if (
        !confirm(
            "Delete all read notifications?"
        )
    ) {

        return;

    }


    try {

        const response =
            await fetch(
                `${NOTIFICATION_API}/read`,
                {
                    method: "DELETE",

                    headers:
                        notificationHeaders()
                }
            );


        const data =
            await response.json();


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Unable to delete read notifications"
            );

        }


        allNotifications =
            allNotifications.filter(
                notification =>
                    !isNotificationRead(
                        notification
                    )
            );


        applyFilters();


        showNotificationMessage(
            "Read notifications deleted"
        );


    } catch (error) {

        showNotificationMessage(

            error.message ||
            "Unable to delete read notifications",

            "error"

        );

    }

}


/* ========================
   CLOSE MODAL
======================== */

function closeNotificationModal() {

    const modal =
        document.getElementById(
            "notificationModal"
        );
```

```
    if (modal) {

        modal.style.display =
            "none";

    }

}


/* =========================
   SEARCH
========================= */

function clearSearch() {

    const input =
        document.getElementById(
            "searchInput"
        );


    if (!input) return;


    input.value =
        "";


    const button =
        document.getElementById(
            "clearSearch"
        );


    if (button) {

        button.style.display =
            "none";

    }


    applyFilters();

}


/* =========================
   RESET
========================= */

function resetFilters() {

    const search =
        document.getElementById(
            "searchInput"
        );


    const type =
        document.getElementById(
            "typeFilter"
        );


    const status =
        document.getElementById(
            "statusFilter"
        );


    if (search) {

        search.value =
            "";

    }


    if (type) {

        type.value =
            "all";

    }


    if (status) {

        status.value =
            "all";

    }


    const button =
        document.getElementById(
            "clearSearch"
        );


    if (button) {
```

```
        button.style.display =
            "none";

    }


    applyFilters();

}


/* ========================
   PAGINATION
======================== */

function renderPagination() {

    const pagination =
        document.getElementById(
            "pagination"
        );


    if (!pagination) return;


    const totalPages =
        Math.ceil(
            filteredNotifications.length /
            pageSize
        );


    if (totalPages <= 1) {

        pagination.innerHTML =
            "";

        return;

    }


    let html =
        "";


    html += `

        <button
            class="page-btn"
            ${
                currentPage === 1
                    ? "disabled"
                    : ""
            }
            onclick="
                changePage(
                    ${currentPage - 1}
                )
            "
        >
            ‹
        </button>

    `;


    for (
        let i = 1;
        i <= totalPages;
        i++
    ) {

        html += `

            <button
                class="page-btn ${
                    i === currentPage
                        ? "active"
                        : ""
                }"
                onclick="
                    changePage(${i})
                "
            >
                ${i}
            </button>

        `;

    }


    html += `

        <button
            class="page-btn"
            ${
                currentPage === totalPages
                    ? "disabled"
                    : ""
            }
            onclick="
                changePage(
```

```
                    ${currentPage + 1}
                )
            "
        >
            ›
        </button>

    `;


    pagination.innerHTML =
        html;

}


/* ========================
   CHANGE PAGE
======================== */

function changePage(
    page
) {

    const totalPages =
        Math.ceil(
            filteredNotifications.length /
            pageSize
        );


    if (
        page < 1 ||
        page > totalPages
    ) {

        return;

    }


    currentPage =
        page;


    renderNotifications();

}


/* ========================
   TIME AGO
======================== */

function getTimeAgo(
    dateValue
) {

    if (!dateValue) {

        return "";

    }


    const date =
        new Date(
            dateValue
        );


    const now =
        new Date();


    const seconds =
        Math.floor(
            (now - date) /
            1000
        );


    if (seconds < 60) {

        return "Just now";

    }


    const minutes =
        Math.floor(
            seconds / 60
        );


    if (minutes < 60) {

        return `${minutes}m ago`;

    }


    const hours =
        Math.floor(
            minutes / 60
```

```
        );

    if (hours < 24) {

        return `${hours}h ago`;

    }

    const days =
        Math.floor(
            hours / 24
        );

    if (days < 30) {

        return `${days}d ago`;

    }

    return date.toLocaleDateString();

}

/* ========================
   ESCAPE HTML
======================== */

function escapeHTML(
    value
) {

    return String(value)

        .replace(
            /&/g,
            "&amp;"
        )

        .replace(
            /</g,
            "&lt;"
        )

        .replace(
            />/g,
            "&gt;"
        )

        .replace(
            /"/g,
            "&quot;"
        )

        .replace(
            /'/g,
            "&#039;"
        );

}

/* ========================
   DOM READY
======================== */

document.addEventListener(
    "DOMContentLoaded",
    () => {

        const searchInput =
            document.getElementById(
                "searchInput"
            );

        const typeFilter =
            document.getElementById(
                "typeFilter"
            );

        const statusFilter =
            document.getElementById(
                "statusFilter"
            );

        const clearButton =
            document.getElementById(
                "clearSearch"
            );

        if (searchInput) {

            searchInput.addEventListener(
                "input",
                () => {
```

```
                if (clearButton) {

                    clearButton.style.display =
                        searchInput.value
                            ? "block"
                            : "none";

                }


                applyFilters();

            }
        );

    }


    if (typeFilter) {

        typeFilter.addEventListener(
            "change",
            applyFilters
        );

    }


    if (statusFilter) {

        statusFilter.addEventListener(
            "change",
            applyFilters
        );

    }


    const modal =
        document.getElementById(
            "notificationModal"
        );


    if (modal) {

        modal.addEventListener(
            "click",
            event => {

                if (
                    event.target ===
                    modal
                ) {

                    closeNotificationModal();

                }

            }
        );

    }


    loadNotifications(1);

}
);
```

Frontend/JS/payroll.js

```
// ========================
// HRGENIUS PAYROLL
// ========================

(function () {

    // ========================
    // TOKEN
    // ========================

    const payrollToken =
        localStorage.getItem("token");

    // ========================
    // LOGIN CHECK
    // ========================

    if (!payrollToken) {
        window.location.href = "login.html";
        return;
    }

    // ========================
    // PAYROLL API
    // ========================

    const PAYROLL_API =
        "http://localhost:5000/api/payroll";

    // ========================
    // DASHBOARD
    // ========================

    window.goDashboard = function () {
        window.location.href =
            "dashboard.html";
    };

    // ========================
    // LOGOUT
    // ========================

    window.logout = function () {

        localStorage.removeItem("token");

        window.location.href =
            "login.html";
    };

    // ========================
    // SHOW MESSAGE
    // ========================

    function showMessage(message, type) {

        const messageBox =
            document.getElementById("message");

        if (!messageBox) {
            return;
        }

        messageBox.textContent =
            message;

        if (type === "success") {

            messageBox.style.color =
                "var(--success)";

        } else {

            messageBox.style.color =
                "var(--danger)";
        }

        setTimeout(function () {

            messageBox.textContent = "";

        }, 3000);
    }

    // ========================
    // FORMAT PAYROLL MONTH
    // ========================

    function formatPayrollMonth(monthValue) {

        if (!monthValue) {
            return "";
        }

        const parts =
            monthValue.split("-");

        const year =
            Number(parts[0]);

        const monthNumber =
            Number(parts[1]);
```

```
    if (
        !year ||
        !monthNumber ||
        monthNumber < 1 ||
        monthNumber > 12
    ) {
        return "";
    }

    const monthDate =
        new Date(
            year,
            monthNumber - 1,
            1
        );

    return monthDate.toLocaleString(
        "en-US",
        {
            month: "long",
            year: "numeric"
        }
    );
}

// ========================
// LOAD PAYROLL
// ========================

async function loadPayroll() {

    const table =
        document.getElementById(
            "payrollTableBody"
        );

    try {

        const response =
            await fetch(
                PAYROLL_API,
                {
                    method: "GET",

                    headers: {
                        "Authorization":
                            `Bearer ${payrollToken}`
                    }
                }
            );

        const data =
            await response.json();

        if (!response.ok) {

            if (table) {

                table.innerHTML = `
                    <tr>
                        <td
                            colspan="10"
                            class="no-data"
                        >
                            ${
                                data.message ||
                                "Unable to load payroll"
                            }
                        </td>
                    </tr>
                `;
            }

            showMessage(
                data.message ||
                "Unable to load payroll",
                "error"
            );

            return;
        }

        const payrolls =
            data.payrolls || [];

        // ========================
        // SUMMARY
        // ========================

        const totalPayroll =
            document.getElementById(
                "totalPayroll"
            );

        const paidPayroll =
            document.getElementById(
                "paidPayroll"
            );

        const pendingPayroll =
            document.getElementById(
                "pendingPayroll"
            );
```

```
if (totalPayroll) {

    totalPayroll.textContent =
        payrolls.length;
}

if (paidPayroll) {

    paidPayroll.textContent =
        payrolls.filter(
            function (payroll) {

                return (
                    payroll.paymentStatus ===
                    "Paid"
                );
            }
        ).length;
}

if (pendingPayroll) {

    pendingPayroll.textContent =
        payrolls.filter(
            function (payroll) {

                return (
                    payroll.paymentStatus ===
                    "Pending"
                );
            }
        ).length;
}

// ========================
// TABLE
// ========================

if (!table) {
    return;
}

table.innerHTML = "";

if (payrolls.length === 0) {

    table.innerHTML = `
        <tr>
            <td
                colspan="10"
                class="no-data"
            >
                No payroll records found
            </td>
        </tr>
    `;

    return;
}

// ========================
// PAYROLL ROWS
// ========================

payrolls.forEach(
    function (payroll) {

        const employee =
            payroll.employeeId;

        const employeeName =
            employee &&
            typeof employee === "object"
                ? employee.name
                : "Unknown";

        const employeeEmail =
            employee &&
            typeof employee === "object"
                ? employee.email
                : "-";

        const department =
            employee &&
            typeof employee === "object"
                ? employee.department
                : "-";

        const status =
            payroll.paymentStatus ||
            "Pending";

        const statusClass =
            status.toLowerCase();

        let action = "-";

        if (status === "Pending") {

            action = `
                <button
                    class="action-btn pay-btn"
                    onclick="markAsPaid('${payroll._id}')"
                >
                    Mark Paid
```

```
                        </button>
                    `;
                }

                const row =
                    document.createElement("tr");

                row.innerHTML = `
                    <td>
                        ${employeeName}
                    </td>

                    <td>
                        ${employeeEmail}
                    </td>

                    <td>
                        ${department}
                    </td>

                    <td>
                        ${payroll.month}
                    </td>

                    <td>
                        ₹${payroll.basicSalary}
                    </td>

                    <td>
                        ₹${payroll.allowances}
                    </td>

                    <td>
                        ₹${payroll.deductions}
                    </td>

                    <td>
                        ₹${payroll.netSalary}
                    </td>

                    <td>
                        <span
                            class="status ${statusClass}"
                        >
                            ${status}
                        </span>
                    </td>

                    <td>
                        ${action}
                    </td>
                `;

                table.appendChild(row);
            }
        );

    } catch (error) {

        console.log(
            "Error loading payroll:",
            error
        );

        if (table) {

            table.innerHTML = `
                <tr>
                    <td
                        colspan="10"
                        class="no-data"
                    >
                        Server connection failed
                    </td>
                </tr>
            `;
        }

        showMessage(
            "Server connection failed",
            "error"
        );
    }
}

// ========================
// GENERATE PAYROLL
// ========================

const payrollForm =
    document.getElementById(
        "payrollForm"
    );

if (payrollForm) {

    payrollForm.addEventListener(
        "submit",
        async function (event) {

            event.preventDefault();

            const employeeId =
                document
                    .getElementById(
```

```
            "employeeId"
        )
        .value
        .trim();

const monthValue =
    document
        .getElementById(
            "month"
        )
        .value;

const basicSalary =
    Number(
        document
            .getElementById(
                "basicSalary"
            )
            .value
    );

const allowances =
    Number(
        document
            .getElementById(
                "allowances"
            )
            .value
    );

const deductions =
    Number(
        document
            .getElementById(
                "deductions"
            )
            .value
    );

// ========================
// VALIDATION
// ========================

if (!employeeId) {

    showMessage(
        "Employee ID is required",
        "error"
    );

    return;
}

if (!monthValue) {

    showMessage(
        "Please select payroll month",
        "error"
    );

    return;
}

const month =
    formatPayrollMonth(
        monthValue
    );

if (!month) {

    showMessage(
        "Invalid payroll month",
        "error"
    );

    return;
}

if (
    Number.isNaN(
        basicSalary
    ) ||
    basicSalary < 0
) {

    showMessage(
        "Basic salary cannot be negative",
        "error"
    );

    return;
}

if (
    Number.isNaN(
        allowances
    ) ||
    allowances < 0
) {

    showMessage(
        "Allowances cannot be negative",
        "error"
    );
```

```
    return;
}

if (
    Number.isNaN(
        deductions
    ) ||
    deductions < 0
) {

    showMessage(
        "Deductions cannot be negative",
        "error"
    );

    return;
}

// ========================
// NET SALARY
// ========================

const netSalary =
    basicSalary +
    allowances -
    deductions;

if (netSalary < 0) {

    showMessage(
        "Deductions cannot be greater than salary",
        "error"
    );

    return;
}

// ========================
// BUTTON
// ========================

const generateButton =
    document.getElementById(
        "generatePayrollBtn"
    );

if (generateButton) {

    generateButton.disabled =
        true;

    generateButton.textContent =
        "Generating...";
}

// ========================
// CREATE PAYROLL
// ========================

try {

    const response =
        await fetch(
            `${PAYROLL_API}/create`,
            {
                method: "POST",

                headers: {
                    "Content-Type":
                        "application/json",

                    "Authorization":
                        `Bearer ${payrollToken}`
                },

                body:
                    JSON.stringify({
                        employeeId:
                            employeeId,

                        month:
                            month,

                        basicSalary:
                            basicSalary,

                        allowances:
                            allowances,

                        deductions:
                            deductions
                    })
            }
        );

    const data =
        await response.json();

    if (!response.ok) {

        showMessage(
            data.message ||
            "Failed to generate payroll",
            "error"
        );
```

```
                return;
            }

            showMessage(
                "Payroll generated successfully!",
                "success"
            );

            // =========================
            // RESET FORM
            // =========================

            payrollForm.reset();

            const allowancesInput =
                document.getElementById(
                    "allowances"
                );

            const deductionsInput =
                document.getElementById(
                    "deductions"
                );

            if (allowancesInput) {
                allowancesInput.value =
                    "0";
            }

            if (deductionsInput) {
                deductionsInput.value =
                    "0";
            }

            // =========================
            // RELOAD
            // =========================

            loadPayroll();

        } catch (error) {

            console.log(
                "Generate Payroll Error:",
                error
            );

            showMessage(
                "Server connection failed",
                "error"
            );

        } finally {

            if (generateButton) {

                generateButton.disabled =
                    false;

                generateButton.textContent =
                    "Generate Payroll";
            }
        }
    }
    );
}

// =========================
// MARK AS PAID
// =========================

window.markAsPaid =
    async function (id) {

        try {

            const response =
                await fetch(
                    `${PAYROLL_API}/${id}/status`,
                    {
                        method: "PUT",

                        headers: {
                            "Content-Type":
                                "application/json",

                            "Authorization":
                                `Bearer ${payrollToken}`
                        },

                        body:
                            JSON.stringify({
                                paymentStatus:
                                    "Paid"
                            })
                    }
                );

            const data =
                await response.json();

            if (!response.ok) {

                showMessage(
```

```
                        data.message ||
                        "Failed to update payment status",
                        "error"
                    );

                    return;
                }

                showMessage(
                    "Payroll marked as Paid",
                    "success"
                );

                loadPayroll();

            } catch (error) {

                console.log(
                    "Mark Paid Error:",
                    error
                );

                showMessage(
                    "Server connection failed",
                    "error"
                );
            }
        };

    // ========================
    // INITIAL LOAD
    // ========================

    loadPayroll();

})();
```

Frontend/JS/theme.js

```
(function () {
    const savedTheme = localStorage.getItem("hrgeniusTheme");

    document.documentElement.setAttribute(
        "data-theme",
        savedTheme === "dark" ? "dark" : "light"
    );

    window.toggleTheme = function () {
        const currentTheme =
            document.documentElement.getAttribute("data-theme");

        const newTheme =
            currentTheme === "dark" ? "light" : "dark";

        document.documentElement.setAttribute("data-theme", newTheme);

        localStorage.setItem("hrgeniusTheme", newTheme);

        updateThemeButton();
    };

    window.updateThemeButton = function () {
        const button = document.getElementById("themeToggle");

        if (!button) return;

        const currentTheme =
            document.documentElement.getAttribute("data-theme");

        if (currentTheme === "dark") {
            button.textContent = "🌞 Light Mode";
        } else {
            button.textContent = "🌙 Dark Mode";
        }
    };

    document.addEventListener("DOMContentLoaded", function () {
        updateThemeButton();
    });
})();
```

Frontend/leaves.html

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Leave Management - HRGenius</title>

    <link rel="stylesheet" href="CSS/style.css">

    <style>

        /* ========================
           NAVBAR
        ======================== */

        nav {
            background: var(--sidebar);

            color: var(--text);

            padding: 15px 30px;

            display: flex;

            justify-content: space-between;

            align-items: center;

            border-bottom: 1px solid var(--border);

            box-shadow: var(--shadow);
        }

        nav h2 {
            color: var(--primary);
        }

        nav div {
            display: flex;

            align-items: center;

            gap: 10px;
        }

        nav button {
            padding: 8px 14px;

            border: none;

            border-radius: 5px;

            cursor: pointer;

            font-weight: 600;
        }

        .dashboard-btn {
            background: var(--primary);

            color: white;
        }

        .dashboard-btn:hover {
            background: var(--primary-hover);
        }

        .logout-btn {
            background: var(--danger);

            color: white;
        }

        .logout-btn:hover {
            opacity: 0.9;
        }


        /* ========================
           CONTAINER
        ======================== */

        .container {
            width: 90%;

            margin: 30px auto;
        }

        h1 {
            margin-bottom: 25px;

            color: var(--text);
        }


        /* ========================
           SUMMARY CARDS
        ======================== */
```

```
.cards {
    display: grid;

    grid-template-columns: repeat(4, 1fr);

    gap: 20px;

    margin-bottom: 30px;
}

.card {
    background: var(--card);

    color: var(--text);

    padding: 20px;

    border-radius: 10px;

    border: 1px solid var(--border);

    box-shadow: var(--shadow);
}

.card h3 {
    margin-bottom: 10px;

    color: var(--muted);
}

.card p {
    font-size: 28px;

    font-weight: bold;

    color: var(--primary);
}


/* ========================
   FORM
======================== */

.form-container {
    background: var(--card);

    color: var(--text);

    padding: 25px;

    border-radius: 10px;

    margin-bottom: 30px;

    border: 1px solid var(--border);

    box-shadow: var(--shadow);
}

.form-container h2 {
    margin-bottom: 20px;

    color: var(--text);
}

form {
    display: grid;

    grid-template-columns: repeat(2, 1fr);

    gap: 15px;
}

input,
select,
textarea {
    width: 100%;

    padding: 11px;

    border: 1px solid var(--border);

    border-radius: 5px;

    font-size: 14px;

    background: var(--input);

    color: var(--text);
}

input::placeholder,
textarea::placeholder {
    color: var(--muted);
}

input:focus,
select:focus,
textarea:focus {
    outline: none;

    border-color: var(--primary);
}
```

```
textarea {
    resize: vertical;

    min-height: 90px;
}

.full {
    grid-column: span 2;
}


/* ========================
   SUBMIT BUTTON
======================== */

.submit-btn {
    background: var(--primary);

    color: white;

    border: none;

    padding: 12px;

    border-radius: 5px;

    cursor: pointer;

    font-size: 15px;

    font-weight: 600;
}

.submit-btn:hover {
    background: var(--primary-hover);
}


/* ========================
   MESSAGE
======================== */

.message {
    margin-top: 15px;

    font-weight: bold;
}


/* ========================
   TABLE
======================== */

.table-container {
    background: var(--card);

    color: var(--text);

    padding: 25px;

    border-radius: 10px;

    border: 1px solid var(--border);

    box-shadow: var(--shadow);

    overflow-x: auto;
}

.table-container h2 {
    color: var(--text);
}

table {
    width: 100%;

    border-collapse: collapse;

    margin-top: 15px;

    background: var(--card);
}

th,
td {
    padding: 12px;

    border-bottom: 1px solid var(--border);

    text-align: left;

    white-space: nowrap;
}

th {
    background: var(--hover);

    color: var(--text);
}

td {
    color: var(--text);
}

tbody tr:hover {
```

```
    background: var(--hover);
}


/* ========================
   STATUS
======================== */

.status {
    padding: 5px 10px;

    border-radius: 15px;

    font-size: 12px;

    font-weight: bold;

    display: inline-block;
}

.pending {
    background: rgba(217, 119, 6, 0.12);

    color: var(--warning);
}

.approved {
    background: rgba(22, 163, 74, 0.12);

    color: var(--success);
}

.rejected {
    background: rgba(220, 38, 38, 0.12);

    color: var(--danger);
}


/* ========================
   ACTION BUTTONS
======================== */

.action-btn {
    padding: 6px 10px;

    border: none;

    border-radius: 4px;

    cursor: pointer;

    margin-right: 5px;

    font-weight: 600;
}

.approve-btn {
    background: #22c55e;

    color: white;
}

.approve-btn:hover {
    background: #16a34a;
}

.reject-btn {
    background: #ef4444;

    color: white;
}

.reject-btn:hover {
    background: #dc2626;
}


/* ========================
   RESPONSIVE
======================== */

@media (max-width: 900px) {

    .cards {
        grid-template-columns: repeat(2, 1fr);
    }

    form {
        grid-template-columns: 1fr;
    }

    .full {
        grid-column: span 1;
    }

}


@media (max-width: 600px) {

    .cards {
        grid-template-columns: 1fr;
    }
```

```
            nav {
                flex-direction: column;

                gap: 10px;
            }

            nav div {
                flex-wrap: wrap;

                justify-content: center;
            }

            .container {
                width: 95%;
            }

        }

    </style>

</head>


<body>


    <!-- ========================
         NAVBAR
    ======================== -->

    <nav>

        <h2>
            HRGenius
        </h2>


        <div>

            <button
                class="dashboard-btn"
                onclick="goDashboard()">

                Dashboard

            </button>


            <button
                id="themeToggle"
                class="theme-toggle"
                onclick="toggleTheme()">

                🌙 Dark Mode

            </button>


            <button
                class="logout-btn"
                onclick="logout()">

                Logout

            </button>

        </div>

    </nav>


    <!-- ========================
         MAIN CONTAINER
    ======================== -->

    <div class="container">


        <h1>
            Leave Management
        </h1>


        <!-- ========================
             SUMMARY CARDS
        ======================== -->

        <div class="cards">


            <div class="card">

                <h3>
                    Total Leaves
                </h3>

                <p id="totalLeaves">
                    0
                </p>

            </div>


            <div class="card">
```

```
        <h3>
            Pending
        </h3>

        <p id="pendingLeaves">
            0
        </p>

    </div>


    <div class="card">

        <h3>
            Approved
        </h3>

        <p id="approvedLeaves">
            0
        </p>

    </div>


    <div class="card">

        <h3>
            Rejected
        </h3>

        <p id="rejectedLeaves">
            0
        </p>

    </div>


</div>


<!-- ========================
     APPLY LEAVE
======================== -->

<div class="form-container">

    <h2>
        Apply Leave
    </h2>


    <form id="leaveForm">


        <input
            type="text"
            id="employeeId"
            placeholder="Employee ID"
            required
        >


        <select
            id="leaveType"
            required>

            <option value="">
                Select Leave Type
            </option>

            <option value="Sick">
                Sick
            </option>

            <option value="Casual">
                Casual
            </option>

            <option value="Annual">
                Annual
            </option>

            <option value="Emergency">
                Emergency
            </option>

            <option value="Other">
                Other
            </option>

        </select>


        <input
            type="date"
            id="startDate"
            required
        >


        <input
            type="date"
            id="endDate"
            required
        >
```

```
            <textarea
                id="reason"
                class="full"
                placeholder="Reason for leave"
                required
            ></textarea>


            <button
                type="submit"
                class="submit-btn full">

                Apply Leave

            </button>


        </form>


        <div
            id="message"
            class="message">

        </div>

    </div>


    <!-- ========================
         LEAVE RECORDS
    ======================== -->

    <div class="table-container">

        <h2>
            Leave Requests
        </h2>


        <table>

            <thead>

                <tr>

                    <th>
                        Employee
                    </th>

                    <th>
                        Email
                    </th>

                    <th>
                        Department
                    </th>

                    <th>
                        Leave Type
                    </th>

                    <th>
                        Start Date
                    </th>

                    <th>
                        End Date
                    </th>

                    <th>
                        Reason
                    </th>

                    <th>
                        Status
                    </th>

                    <th>
                        Action
                    </th>

                </tr>

            </thead>


            <tbody id="leaveTableBody">

            </tbody>

        </table>

    </div>

</div>


<!-- ========================
     JAVASCRIPT
======================== -->
```

```
<script>

    const token =
        localStorage.getItem("token");


    // ========================
    // CHECK LOGIN
    // ========================

    if (!token) {

        window.location.href =
            "login.html";

    }


    const API =
        "http://localhost:5000/api/leaves";


    // ========================
    // DASHBOARD
    // ========================

    function goDashboard() {

        window.location.href =
            "dashboard.html";

    }


    // ========================
    // LOGOUT
    // ========================

    function logout() {

        localStorage.removeItem(
            "token"
        );

        window.location.href =
            "login.html";

    }


    // ========================
    // APPLY LEAVE
    // ========================

    document
        .getElementById("leaveForm")
        .addEventListener(
            "submit",
            async function(event) {

                event.preventDefault();


                const employeeId =
                    document
                        .getElementById(
                            "employeeId"
                        )
                        .value
                        .trim();


                const leaveType =
                    document
                        .getElementById(
                            "leaveType"
                        )
                        .value;


                const startDate =
                    document
                        .getElementById(
                            "startDate"
                        )
                        .value;


                const endDate =
                    document
                        .getElementById(
                            "endDate"
                        )
                        .value;


                const reason =
                    document
                        .getElementById(
                            "reason"
                        )
                        .value
                        .trim();
```

```
    const message =
        document.getElementById(
            "message"
        );


    try {


        const response =
            await fetch(

                `${API}/create`,

                {

                    method: "POST",

                    headers: {

                        "Content-Type":
                            "application/json",

                        "Authorization":
                            `Bearer ${token}`

                    },

                    body:
                        JSON.stringify({

                            employeeId,

                            leaveType,

                            startDate,

                            endDate,

                            reason

                        })

                }

            );


        const data =
            await response.json();


        if (!response.ok) {

            message.style.color =
                "var(--danger)";

            message.textContent =
                data.message ||
                "Failed to apply leave";

            return;

        }


        message.style.color =
            "var(--success)";


        message.textContent =
            "Leave applied successfully!";


        document
            .getElementById(
                "leaveForm"
            )
            .reset();


        loadLeaves();


    }

    catch (error) {

        console.log(
            "Apply Leave Error:",
            error
        );


        message.style.color =
            "var(--danger)";


        message.textContent =
            "Server connection failed";

    }

}
```

```
    );

// ========================
// LOAD LEAVES
// ========================

async function loadLeaves() {

    try {


        const response =
            await fetch(

                API,

                {

                    headers: {

                        "Authorization":
                            `Bearer ${token}`

                    }

                }

            );


        const data =
            await response.json();


        if (!response.ok) {

            console.log(data);

            return;

        }


        const leaves =
            data.leaves || [];


        // ========================
        // SUMMARY COUNTS
        // ========================

        document.getElementById(
            "totalLeaves"
        ).textContent =
            leaves.length;


        document.getElementById(
            "pendingLeaves"
        ).textContent =
            leaves.filter(

                leave =>
                    leave.status === "Pending"

            ).length;


        document.getElementById(
            "approvedLeaves"
        ).textContent =
            leaves.filter(

                leave =>
                    leave.status === "Approved"

            ).length;


        document.getElementById(
            "rejectedLeaves"
        ).textContent =
            leaves.filter(

                leave =>
                    leave.status === "Rejected"

            ).length;


        // ========================
        // TABLE
        // ========================

        const table =
            document.getElementById(
                "leaveTableBody"
            );


        table.innerHTML = "";


        if (leaves.length === 0) {
```

```
    table.innerHTML = `

        <tr>

            <td
                colspan="9"
                style="
                    text-align:center;
                    padding:20px;
                    color:var(--muted);
                ">

                No leave requests found

            </td>

        </tr>

    `;

    return;

}


leaves.forEach(
    leave => {


        const employee =
            leave.employeeId;


        const employeeName =
            employee &&
            typeof employee === "object"
                ? employee.name
                : "Unknown";


        const employeeEmail =
            employee &&
            typeof employee === "object"
                ? employee.email
                : "-";


        const department =
            employee &&
            typeof employee === "object"
                ? employee.department
                : "-";


        const statusClass =
            leave.status
                ? leave.status.toLowerCase()
                : "pending";


        const startDate =
            leave.startDate
                ? new Date(
                    leave.startDate
                  )
                    .toLocaleDateString()
                : "-";


        const endDate =
            leave.endDate
                ? new Date(
                    leave.endDate
                  )
                    .toLocaleDateString()
                : "-";


        let actions =
            "-";


        // =========================
        // APPROVE / REJECT
        // =========================

        if (
            leave.status === "Pending"
        ) {

            actions = `

                <button
                    class="action-btn approve-btn"
                    onclick="updateLeave('${leave._id}', 'Approved')">

                    Approve

                </button>


                <button
                    class="action-btn reject-btn"
                    onclick="updateLeave('${leave._id}', 'Rejected')">
```

```
                            Reject

                        </button>

                    `;

                }


                // ========================
                // TABLE ROW
                // ========================

                table.innerHTML += `

                    <tr>

                        <td>
                            ${employeeName}
                        </td>

                        <td>
                            ${employeeEmail}
                        </td>

                        <td>
                            ${department}
                        </td>

                        <td>
                            ${leave.leaveType || "-"}
                        </td>

                        <td>
                            ${startDate}
                        </td>

                        <td>
                            ${endDate}
                        </td>

                        <td>
                            ${leave.reason || "-"}
                        </td>

                        <td>

                            <span
                                class="status ${statusClass}">

                                ${leave.status || "Pending"}

                            </span>

                        </td>

                        <td>
                            ${actions}
                        </td>

                    </tr>

                `;

            }
        );


    }

    catch (error) {

        console.log(
            "Error loading leaves:",
            error
        );

    }

}


// ========================
// UPDATE LEAVE STATUS
// ========================

async function updateLeave(
    id,
    status
) {


    try {


        let endpoint;


        if (status === "Approved") {

            endpoint =
                `${API}/${id}/approve`;

        }
```

```
                else {

                    endpoint =
                        `${API}/${id}/reject`;

                }


                const response =
                    await fetch(

                        endpoint,

                        {

                            method: "PUT",

                            headers: {

                                "Authorization":
                                    `Bearer ${token}`

                            }

                        }

                    );


                const data =
                    await response.json();


                if (!response.ok) {

                    alert(
                        data.message ||
                        "Failed to update leave"
                    );

                    return;

                }


                alert(
                    `Leave ${status.toLowerCase()} successfully`
                );


                loadLeaves();


            }

            catch (error) {

                console.log(
                    "Update Leave Error:",
                    error
                );


                alert(
                    "Server connection failed"
                );

            }

        }


        // ========================
        // INITIAL LOAD
        // ========================

        loadLeaves();

    </script>


    <!-- Authentication -->

    <script src="JS/auth.js"></script>


    <!-- Theme -->

    <script src="JS/theme.js"></script>


</body>

</html>
```

Frontend/login.html

```
<!DOCTYPE html>

<html lang="en">

<head>


<meta charset="UTF-8">

<meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
>

<title>HRGenius - Login</title>

<link
    rel="stylesheet"
    href="CSS/style.css"
>

<style>

    * {
        margin: 0;
        padding: 0;
        box-sizing: border-box;
    }

    body {
        font-family: Arial, Helvetica, sans-serif;
        background: var(--bg);
        color: var(--text);
        min-height: 100vh;
        overflow-x: hidden;
    }

    .login-page {
        min-height: 100vh;
        display: grid;
        grid-template-columns: 1.15fr 0.85fr;
    }

    .login-left {
        position: relative;
        overflow: hidden;
        padding: 70px;
        display: flex;
        align-items: center;
        background:
            linear-gradient(
                135deg,
                #eef4ff 0%,
                #dce9ff 50%,
                #f4f7ff 100%
            );
        color: #172554;
    }

    .orb {
        position: absolute;
        border-radius: 50%;
        pointer-events: none;
    }

    .orb-one {
        width: 420px;
        height: 420px;
        top: -180px;
        right: -120px;
        background: rgba(37, 99, 235, 0.10);
        filter: blur(2px);
    }

    .orb-two {
        width: 320px;
        height: 320px;
        bottom: -160px;
        left: -130px;
        background: rgba(59, 130, 246, 0.08);
    }

    .orb-three {
        width: 150px;
        height: 150px;
        bottom: 20%;
        right: 12%;
        background: rgba(96, 165, 250, 0.10);
    }

    .grid-bg {
        position: absolute;
        inset: 0;
        background-image:
            linear-gradient(
                rgba(37, 99, 235, 0.035) 1px,
                transparent 1px
            ),
            linear-gradient(
                90deg,
                rgba(37, 99, 235, 0.035) 1px,
                transparent 1px
```

```
        );
    background-size: 40px 40px;
    pointer-events: none;
}

.brand-content {
    position: relative;
    z-index: 5;
    max-width: 650px;
    animation: slideInLeft 0.8s ease;
}

@keyframes slideInLeft {

    from {
        opacity: 0;
        transform: translateX(-25px);
    }

    to {
        opacity: 1;
        transform: translateX(0);
    }

}

.brand-logo {
    display: flex;
    align-items: center;
    gap: 14px;
    margin-bottom: 28px;
}

.logo-icon {
    width: 52px;
    height: 52px;
    border-radius: 14px;
    display: flex;
    align-items: center;
    justify-content: center;
    background:
        linear-gradient(
            135deg,
            #2563eb,
            #4f46e5
        );
    color: white;
    font-size: 25px;
    box-shadow:
        0 12px 30px
        rgba(37, 99, 235, 0.25);
}

.logo-name {
    font-size: 28px;
    font-weight: 800;
    color: #1d4ed8;
}

.brand-content h1 {
    font-size: clamp(38px, 4vw, 58px);
    line-height: 1.08;
    letter-spacing: -2px;
    color: #172554;
    margin-bottom: 20px;
}

.brand-content h1 span {
    color: #2563eb;
}

.brand-description {
    max-width: 560px;
    font-size: 16px;
    line-height: 1.75;
    color: #475569;
    margin-bottom: 35px;
}

.features {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 14px;
    max-width: 570px;
}

.feature {
    display: flex;
    align-items: center;
    gap: 13px;
    padding: 15px 16px;
    border-radius: 13px;
    background: rgba(255, 255, 255, 0.72);
    border: 1px solid rgba(37, 99, 235, 0.12);
    backdrop-filter: blur(10px);
    transition:
        transform 0.25s ease,
        background 0.25s ease,
        box-shadow 0.25s ease;
}

.feature:hover {
    transform: translateY(-4px);
    background: rgba(255, 255, 255, 0.95);
    box-shadow:
```

```
        0 12px 28px
        rgba(30, 64, 175, 0.10);
}

.feature-icon {
    width: 40px;
    height: 40px;
    flex-shrink: 0;
    border-radius: 10px;
    display: flex;
    align-items: center;
    justify-content: center;
    background: #eff6ff;
    font-size: 19px;
}

.feature-text {
    font-size: 13px;
    font-weight: 700;
    color: #1e293b;
}

.feature-small {
    font-size: 11px;
    color: #64748b;
    margin-top: 3px;
}

.secure-info {
    display: flex;
    align-items: center;
    gap: 10px;
    margin-top: 30px;
    color: #64748b;
    font-size: 12px;
}

.secure-dot {
    width: 8px;
    height: 8px;
    border-radius: 50%;
    background: #22c55e;
    box-shadow:
        0 0 0 5px
        rgba(34, 197, 94, 0.10);
}

.login-right {
    position: relative;
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 50px 45px;
    background: var(--bg);
}

.theme-toggle-login {
    position: absolute;
    top: 25px;
    right: 30px;
    width: 44px;
    height: 44px;
    border-radius: 12px;
    border: 1px solid var(--border);
    background: var(--card);
    color: var(--text);
    display: flex;
    align-items: center;
    justify-content: center;
    cursor: pointer;
    font-size: 18px;
    transition:
        transform 0.2s ease,
        background 0.2s ease,
        border-color 0.2s ease;
}

.theme-toggle-login:hover {
    transform: translateY(-2px);
    background: var(--hover);
}

.login-wrapper {
    width: 100%;
    max-width: 410px;
    animation: fadeUp 0.8s ease;
}

@keyframes fadeUp {

    from {
        opacity: 0;
        transform: translateY(20px);
    }

    to {
        opacity: 1;
        transform: translateY(0);
    }

}

.mobile-logo {
    display: none;
}
```

```
.login-heading {
    margin-bottom: 30px;
}

.welcome-label {
    display: inline-flex;
    align-items: center;
    gap: 7px;
    padding: 6px 10px;
    border-radius: 20px;
    background: rgba(37, 99, 235, 0.08);
    color: var(--primary);
    font-size: 11px;
    font-weight: 700;
    margin-bottom: 14px;
}

.welcome-label span {
    width: 6px;
    height: 6px;
    border-radius: 50%;
    background: #22c55e;
}

.login-heading h2 {
    font-size: 34px;
    line-height: 1.2;
    color: var(--text);
    margin-bottom: 8px;
    letter-spacing: -0.8px;
}

.login-heading p {
    color: var(--muted);
    font-size: 14px;
    line-height: 1.6;
}

.login-card {
    padding: 30px;
    border-radius: 18px;
    background: var(--card);
    border: 1px solid var(--border);
    box-shadow:
        0 18px 50px
        rgba(15, 23, 42, 0.08);
}

.form-group {
    margin-bottom: 20px;
}

.form-label-row {
    display: flex;
    align-items: center;
    justify-content: space-between;
    margin-bottom: 8px;
}

.form-group label {
    font-size: 13px;
    font-weight: 700;
    color: var(--text);
}

.input-wrapper {
    position: relative;
}

.input-icon {
    position: absolute;
    left: 14px;
    top: 50%;
    transform: translateY(-50%);
    font-size: 17px;
    opacity: 0.65;
    pointer-events: none;
}

.form-group input {
    width: 100%;
    height: 50px;
    padding: 0 45px;
    border: 1px solid var(--border);
    border-radius: 10px;
    background: var(--input);
    color: var(--text);
    font-size: 14px;
    outline: none;
    transition:
        border-color 0.2s ease,
        box-shadow 0.2s ease,
        background 0.2s ease;
}

.form-group input::placeholder {
    color: var(--muted);
}

.form-group input:focus {
    border-color: var(--primary);
    box-shadow:
        0 0 0 4px
        rgba(37, 99, 235, 0.10);
```

```
}

.password-toggle {
    position: absolute;
    right: 12px;
    top: 50%;
    transform: translateY(-50%);
    border: none;
    background: transparent;
    color: var(--muted);
    cursor: pointer;
    width: 32px;
    height: 32px;
    border-radius: 7px;
    font-size: 16px;
}

.password-toggle:hover {
    background: var(--hover);
}

.login-options {
    display: flex;
    align-items: center;
    justify-content: space-between;
    margin-bottom: 22px;
}

.remember-me {
    display: flex;
    align-items: center;
    gap: 8px;
    color: var(--muted);
    font-size: 12px;
    cursor: pointer;
}

.remember-me input {
    width: 15px;
    height: 15px;
    accent-color: var(--primary);
    cursor: pointer;
}

.forgot-password {
    color: var(--primary);
    font-size: 12px;
    font-weight: 600;
    text-decoration: none;
}

.forgot-password:hover {
    text-decoration: underline;
}

.login-btn {
    position: relative;
    width: 100%;
    height: 50px;
    border: none;
    border-radius: 10px;
    background:
        linear-gradient(
            135deg,
            #2563eb,
            #4f46e5
        );
    color: white;
    font-size: 14px;
    font-weight: 700;
    cursor: pointer;
    overflow: hidden;
    box-shadow:
        0 10px 24px
        rgba(37, 99, 235, 0.22);
    transition:
        transform 0.2s ease,
        box-shadow 0.2s ease,
        opacity 0.2s ease;
}

.login-btn:hover {
    transform: translateY(-2px);
    box-shadow:
        0 14px 30px
        rgba(37, 99, 235, 0.30);
}

.login-btn:active {
    transform: translateY(0);
}

.login-btn:disabled {
    opacity: 0.65;
    cursor: not-allowed;
    transform: none;
}

#message {
    min-height: 22px;
    margin-top: 15px;
    text-align: center;
    font-size: 12px;
    font-weight: 600;
    line-height: 1.5;
```

```
}

.login-footer {
    margin-top: 25px;
    text-align: center;
    color: var(--muted);
    font-size: 11px;
}

.login-footer strong {
    color: var(--text);
}

html[data-theme="dark"] .login-left {
    background:
        linear-gradient(
            135deg,
            #0f1f45 0%,
            #172554 50%,
            #111827 100%
        );
    color: #ffffff;
}

html[data-theme="dark"] .brand-content h1 {
    color: #ffffff;
}

html[data-theme="dark"] .brand-content h1 span {
    color: #60a5fa;
}

html[data-theme="dark"] .logo-name {
    color: #ffffff;
}

html[data-theme="dark"] .brand-description {
    color: #cbd5e1;
}

html[data-theme="dark"] .feature {
    background: rgba(255, 255, 255, 0.07);
    border-color: rgba(255, 255, 255, 0.12);
}

html[data-theme="dark"] .feature:hover {
    background: rgba(255, 255, 255, 0.11);
}

html[data-theme="dark"] .feature-icon {
    background: rgba(255, 255, 255, 0.10);
}

html[data-theme="dark"] .feature-text {
    color: #f8fafc;
}

html[data-theme="dark"] .feature-small {
    color: #94a3b8;
}

html[data-theme="dark"] .secure-info {
    color: #94a3b8;
}

@media (max-width: 1000px) {

    .login-page {
        grid-template-columns: 1fr;
    }

    .login-left {
        display: none;
    }

    .login-right {
        min-height: 100vh;
        padding: 80px 25px 40px;
    }

    .mobile-logo {
        display: flex;
        align-items: center;
        justify-content: center;
        gap: 10px;
        margin-bottom: 35px;
    }

    .mobile-logo-icon {
        width: 40px;
        height: 40px;
        border-radius: 10px;
        display: flex;
        align-items: center;
        justify-content: center;
        background:
            linear-gradient(
                135deg,
                #2563eb,
                #4f46e5
            );
        color: white;
    }

    .mobile-logo-name {
```

```
            font-size: 22px;
            font-weight: 800;
            color: var(--text);
        }

    }

    @media (max-width: 500px) {

        .login-right {
            padding: 75px 16px 30px;
        }

        .login-card {
            padding: 22px;
        }

        .login-heading h2 {
            font-size: 28px;
        }

        .theme-toggle-login {
            top: 18px;
            right: 18px;
        }

    }

    @media (prefers-reduced-motion: reduce) {

        *,
        *::before,
        *::after {
            animation-duration: 0.01ms !important;
            transition-duration: 0.01ms !important;
        }

    }

</style>
```

</head>

<body>

```
<div class="login-page">

    <section class="login-left">

        <div class="grid-bg"></div>

        <div class="orb orb-one"></div>

        <div class="orb orb-two"></div>

        <div class="orb orb-three"></div>

        <div class="brand-content">

            <div class="brand-logo">

                <div class="logo-icon">
                    ✦
                </div>

                <div class="logo-name">
                    HRGenius
                </div>

            </div>

            <h1>
                Smarter HR.
                <br>
                <span>Better People.</span>
            </h1>

            <p class="brand-description">
                A modern HR management platform designed
                to simplify employee management, attendance,
                payroll, recruitment and performance —
                all in one place.
            </p>

            <div class="features">

                <div class="feature">

                    <div class="feature-icon">
                        👥
                    </div>

                    <div>

                        <div class="feature-text">
                            Employee Management
                        </div>

                        <div class="feature-small">
                            Manage employee records
                        </div>

                    </div>
```

```
            </div>

            <div class="feature">

                <div class="feature-icon">
                    🕒
                </div>

                <div>

                    <div class="feature-text">
                        Attendance Tracking
                    </div>

                    <div class="feature-small">
                        Track daily attendance
                    </div>

                </div>

            </div>

            <div class="feature">

                <div class="feature-icon">
                    💰
                </div>

                <div>

                    <div class="feature-text">
                        Payroll Management
                    </div>

                    <div class="feature-small">
                        Simplify salary processing
                    </div>

                </div>

            </div>

            <div class="feature">

                <div class="feature-icon">
                    📋
                </div>

                <div>

                    <div class="feature-text">
                        Recruitment
                    </div>

                    <div class="feature-small">
                        Manage hiring workflows
                    </div>

                </div>

            </div>

        </div>

        <div class="secure-info">

            <div class="secure-dot"></div>

            Secure & reliable HR management

        </div>

    </div>

</section>

<section class="login-right">

    <button
        type="button"
        id="themeToggle"
        class="theme-toggle-login"
        onclick="toggleLoginTheme()"
        title="Switch to Dark Mode"
    >
        🌙
    </button>

    <div class="login-wrapper">

        <div class="mobile-logo">

            <div class="mobile-logo-icon">
                ✦
            </div>

            <div class="mobile-logo-name">
                HRGenius
            </div>

        </div>

        <div class="login-heading">
```

```
    <div class="welcome-label">

        <span></span>

        Secure Login

    </div>

    <h2>
        Welcome back
    </h2>

    <p>
        Sign in to access your HRGenius dashboard.
    </p>

</div>

<div class="login-card">

    <form
        id="loginForm"
        onsubmit="event.preventDefault(); loginUser();"
    >

        <div class="form-group">

            <div class="form-label-row">

                <label for="email">
                    Email Address
                </label>

            </div>

            <div class="input-wrapper">

                <span class="input-icon">
                    ✉
                </span>

                <input
                    type="email"
                    id="email"
                    placeholder="Enter your email"
                    autocomplete="email"
                    required
                >

            </div>

        </div>

        <div class="form-group">

            <div class="form-label-row">

                <label for="password">
                    Password
                </label>

            </div>

            <div class="input-wrapper">

                <span class="input-icon">
                    🔒
                </span>

                <input
                    type="password"
                    id="password"
                    placeholder="Enter your password"
                    autocomplete="current-password"
                    required
                >

                <button
                    type="button"
                    class="password-toggle"
                    id="passwordToggle"
                    onclick="togglePassword()"
                    title="Show password"
                >
                    👁
                </button>

            </div>

        </div>

        <div class="login-options">

            <label class="remember-me">

                <input
                    type="checkbox"
                    id="rememberMe"
                >

                Remember me

            </label>

            <a
```

```
                        href="forgot-password.html"
                        class="forgot-password"
                    >
                        Forgot password?
                    </a>

                </div>

                <button
                    type="submit"
                    class="login-btn"
                    id="loginButton"
                >
                    Sign In
                </button>

            </form>

            <div id="message"></div>

        </div>

        <div class="login-footer">

            <strong>HRGenius</strong>

             • 

            HR Management System

        </div>

    </div>

</section>

</div>

<script src="JS/auth.js"></script>

<script src="JS/theme.js"></script>

<script>

    function updateLoginThemeIcon() {

        const button =
            document.getElementById("themeToggle");

        if (!button) {
            return;
        }

        const theme =
            document.documentElement
                .getAttribute("data-theme");

        if (theme === "dark") {

            button.textContent = "🌞";

            button.title =
                "Switch to Light Mode";

        } else {

            button.textContent = "🌙";

            button.title =
                "Switch to Dark Mode";

        }

    }

    function toggleLoginTheme() {

        toggleTheme();

        updateLoginThemeIcon();

    }

    function togglePassword() {

        const password =
            document.getElementById("password");

        const button =
            document.getElementById("passwordToggle");

        if (password.type === "password") {

            password.type = "text";

            button.textContent = "🙈";

            button.title = "Hide password";

        } else {

            password.type = "password";

            button.textContent = "👁";
```

```
            button.title = "Show password";

        }

    }

    document.addEventListener(
        "DOMContentLoaded",
        function () {

            updateLoginThemeIcon();

            const savedEmail =
                localStorage.getItem(
                    "hrgeniusRememberEmail"
                );

            if (savedEmail) {

                document.getElementById("email").value =
                    savedEmail;

                document.getElementById("rememberMe").checked =
                    true;

            }

        }
    );

</script>


</body>

</html>
```

Frontend/notifications.html

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta
        name="viewport"
        content="width=device-width, initial-scale=1.0"
    >

    <title>HRGenius - Notifications</title>

    <link
        rel="stylesheet"
        href="CSS/style.css"
    >

    <link
        rel="stylesheet"
        href="CSS/notifications.css"
    >

</head>


<body>

    <!-- ========================
         TOPBAR
    ======================== -->

    <header class="notifications-topbar">

        <div class="notification-brand">

            <h2>
                HRGenius
            </h2>

        </div>


        <nav class="notification-nav">

            <a href="dashboard.html">
                Dashboard
            </a>

            <a href="employees.html">
                Employees
            </a>

            <a href="departments.html">
                Departments
            </a>

            <a
                href="notifications.html"
                class="active"
            >
                Notifications
            </a>


            <button
                id="themeToggle"
                class="theme-toggle"
                onclick="toggleTheme()"
            >
                🌙 Dark Mode
            </button>


            <button
                class="notification-logout"
                onclick="logout()"
            >
                Logout
            </button>

        </nav>

    </header>


    <!-- ========================
         MAIN
    ======================== -->

    <main class="notifications-page">


        <!-- PAGE HEADER -->

        <section class="notifications-header">

            <div>

                <h1>
                    Notifications
```

```
        </h1>

        <p>
            Stay updated with your HR activities
        </p>

    </div>


    <div class="header-actions">

        <button
            type="button"
            class="notification-btn"
            onclick="loadNotifications(1)"
        >
            ↻ Refresh
        </button>


        <button
            type="button"
            class="notification-btn primary"
            onclick="markAllAsRead()"
        >
            ✓ Mark All as Read
        </button>

    </div>

</section>


<!-- MESSAGE -->

<div
    id="message"
    class="message"
></div>


<!-- ==========================
     STAT CARDS
========================== -->

<section class="notification-stats">


    <div class="notification-stat-card">

        <div class="stat-icon">
            🔔
        </div>

        <div>

            <span>
                Total Notifications
            </span>

            <strong id="totalNotifications">
                0
            </strong>

        </div>

    </div>


    <div class="notification-stat-card unread-card">

        <div class="stat-icon">
            🔵
        </div>

        <div>

            <span>
                Unread
            </span>

            <strong id="unreadNotifications">
                0
            </strong>

        </div>

    </div>


    <div class="notification-stat-card">

        <div class="stat-icon">
            ✓
        </div>

        <div>

            <span>
                Read
            </span>

            <strong id="readNotifications">
                0
            </strong>
```

```
        </div>

    </div>

</section>


<!-- ========================
     MAIN PANEL
======================== -->

<section class="notifications-panel">


    <!-- FILTERS -->

    <div class="notification-filters">


        <div class="search-wrapper">

            <span class="search-icon">
                🔍
            </span>


            <input
                type="text"
                id="searchInput"
                placeholder="Search notifications..."
                autocomplete="off"
            >


            <button
                type="button"
                id="clearSearch"
                class="clear-search"
                onclick="clearSearch()"
                style="display: none;"
            >
                ×
            </button>

        </div>


        <select
            id="typeFilter"
            class="notification-select"
        >

            <option value="all">
                All Types
            </option>

            <option value="Recruitment">
                Recruitment
            </option>

            <option value="Interview">
                Interview
            </option>

            <option value="Leave">
                Leave
            </option>

            <option value="Payroll">
                Payroll
            </option>

            <option value="Employee">
                Employee
            </option>

            <option value="Department">
                Department
            </option>

            <option value="Attendance">
                Attendance
            </option>

            <option value="Performance">
                Performance
            </option>

            <option value="System">
                System
            </option>

        </select>


        <select
            id="statusFilter"
            class="notification-select"
        >

            <option value="all">
                All Status
            </option>
```

```
                <option value="unread">
                    Unread
                </option>

                <option value="read">
                    Read
                </option>

            </select>


            <button
                type="button"
                class="reset-btn"
                onclick="resetFilters()"
            >
                ↻ Reset
            </button>

        </div>


        <!-- LIST HEADER -->

        <div class="notification-list-header">

            <div>

                <h2>
                    My Notifications
                </h2>

                <p id="resultText">
                    0 notifications
                </p>

            </div>


            <button
                type="button"
                class="delete-read-btn"
                onclick="deleteReadNotifications()"
            >
                🗑 Delete Read
            </button>

        </div>


        <!-- LOADING -->

        <div
            id="loadingState"
            class="loading-state"
        >

            <div class="spinner"></div>

            <p>
                Loading notifications...
            </p>

        </div>


        <!-- NOTIFICATIONS -->

        <div id="notificationList"></div>


        <!-- EMPTY -->

        <div
            id="emptyState"
            class="empty-state"
        >

            <div class="empty-icon">
                🔔
            </div>

            <h3>
                No notifications found
            </h3>

            <p>
                You're all caught up!
            </p>

        </div>


        <!-- PAGINATION -->

        <div
            id="pagination"
            class="pagination"
        ></div>


    </section>


</main>
```

```
<!-- ========================
     DETAILS MODAL
========================= -->

<div
    id="notificationModal"
    class="notification-modal"
>

    <div class="notification-modal-box">


        <!-- MODAL HEADER -->

        <div class="modal-header">

            <h2>
                Notification Details
            </h2>


            <button
                type="button"
                class="modal-close"
                onclick="closeNotificationModal()"
            >
                ×
            </button>

        </div>


        <!-- MODAL BODY -->

        <div class="modal-body">


            <div class="detail-item">

                <span>
                    Type
                </span>

                <strong id="modalType">
                    -
                </strong>

            </div>


            <div class="detail-item">

                <span>
                    Title
                </span>

                <strong id="modalTitle">
                    -
                </strong>

            </div>


            <div class="detail-item">

                <span>
                    Message
                </span>

                <p id="modalMessage">
                    -
                </p>

            </div>


            <div class="detail-grid">


                <div class="detail-item">

                    <span>
                        Priority
                    </span>

                    <strong id="modalPriority">
                        -
                    </strong>

                </div>


                <div class="detail-item">

                    <span>
                        Status
                    </span>

                    <strong id="modalStatus">
                        -
                    </strong>

                </div>
```

```
                </div>


                <div class="detail-item">

                    <span>
                        Date
                    </span>

                    <strong id="modalDate">
                        -
                    </strong>

                </div>

            </div>


            <!-- MODAL FOOTER -->

            <div class="modal-footer">

                <button
                    type="button"
                    class="notification-btn"
                    onclick="closeNotificationModal()"
                >
                    Close
                </button>

            </div>

        </div>

    </div>


    <!-- =========================
         SCRIPTS
    ========================== -->

    <script src="JS/auth.js"></script>

    <script src="JS/theme.js"></script>

    <script src="JS/notifications.js"></script>


</body>

</html>
```

Frontend/payroll.html

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Payroll Management - HRGenius</title>

    <link rel="stylesheet" href="CSS/style.css">

    <style>

        /* =========================
           NAVBAR
        ========================= */

        nav {
            background: var(--sidebar);
            color: var(--text);
            padding: 15px 30px;
            display: flex;
            justify-content: space-between;
            align-items: center;
            border-bottom: 1px solid var(--border);
            box-shadow: var(--shadow);
        }

        nav h2 {
            color: var(--primary);
        }

        nav div {
            display: flex;
            align-items: center;
            gap: 10px;
        }

        nav button {
            padding: 8px 14px;
            border: none;
            border-radius: 5px;
            cursor: pointer;
            font-weight: 600;
        }

        .dashboard-btn {
            background: var(--primary);
            color: white;
        }

        .dashboard-btn:hover {
            background: var(--primary-hover);
        }

        .logout-btn {
            background: var(--danger);
            color: white;
        }

        .logout-btn:hover {
            opacity: 0.9;
        }


        /* =========================
           MAIN CONTAINER
        ========================= */

        .container {
            width: 90%;
            margin: 30px auto;
        }

        h1 {
            margin-bottom: 25px;
            color: var(--text);
        }


        /* =========================
           SUMMARY CARDS
        ========================= */

        .cards {
            display: grid;
            grid-template-columns: repeat(3, 1fr);
            gap: 20px;
            margin-bottom: 30px;
        }

        .card {
            background: var(--card);
            color: var(--text);
            padding: 20px;
            border-radius: 10px;
            border: 1px solid var(--border);
            box-shadow: var(--shadow);
        }
```

```
.card h3 {
    margin-bottom: 10px;
    color: var(--muted);
}

.card p {
    font-size: 28px;
    font-weight: bold;
    color: var(--primary);
}


/* ========================
   FORM
======================== */

.form-container {
    background: var(--card);
    color: var(--text);
    padding: 25px;
    border-radius: 10px;
    margin-bottom: 30px;
    border: 1px solid var(--border);
    box-shadow: var(--shadow);
}

.form-container h2 {
    margin-bottom: 20px;
    color: var(--text);
}

form {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 15px;
}

.form-group {
    display: flex;
    flex-direction: column;
    gap: 7px;
}

.form-group label {
    color: var(--text);
    font-weight: 600;
    font-size: 14px;
}

input {
    width: 100%;
    padding: 11px;
    border: 1px solid var(--border);
    border-radius: 5px;
    font-size: 14px;
    background: var(--input);
    color: var(--text);
    box-sizing: border-box;
}

input::placeholder {
    color: var(--muted);
}

input:focus {
    outline: none;
    border-color: var(--primary);
}

input[type="month"] {
    cursor: pointer;
}

.full {
    grid-column: span 2;
}


/* ========================
   SUBMIT BUTTON
======================== */

.submit-btn {
    background: var(--primary);
    color: white;
    border: none;
    padding: 12px;
    border-radius: 5px;
    cursor: pointer;
    font-size: 15px;
    font-weight: bold;
}

.submit-btn:hover {
    background: var(--primary-hover);
}

.submit-btn:disabled {
    opacity: 0.6;
    cursor: not-allowed;
}


/* ========================
   MESSAGE
```

```
========================= */

.message {
    margin-top: 15px;
    font-weight: bold;
    min-height: 20px;
}


/* =========================
   TABLE
========================= */

.table-container {
    background: var(--card);
    color: var(--text);
    padding: 25px;
    border-radius: 10px;
    border: 1px solid var(--border);
    box-shadow: var(--shadow);
    overflow-x: auto;
}

.table-container h2 {
    color: var(--text);
}

table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 15px;
    background: var(--card);
}

th,
td {
    padding: 12px;
    border-bottom: 1px solid var(--border);
    text-align: left;
    white-space: nowrap;
}

th {
    background: var(--hover);
    color: var(--text);
}

td {
    color: var(--text);
}

tbody tr:hover {
    background: var(--hover);
}


/* =========================
   STATUS
========================= */

.status {
    padding: 5px 10px;
    border-radius: 15px;
    font-size: 12px;
    font-weight: bold;
    display: inline-block;
}

.paid {
    background: rgba(22, 163, 74, 0.12);
    color: var(--success);
}

.pending {
    background: rgba(217, 119, 6, 0.12);
    color: var(--warning);
}


/* =========================
   ACTION BUTTON
========================= */

.action-btn {
    padding: 6px 10px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    color: white;
    font-weight: 600;
}

.pay-btn {
    background: #22c55e;
}

.pay-btn:hover {
    background: #16a34a;
}


/* =========================
   NO DATA
========================= */
```

```
        .no-data {
            text-align: center;
            padding: 25px;
            color: var(--muted);
        }


        /* ========================
           RESPONSIVE
        ======================== */

        @media (max-width: 900px) {

            .cards {
                grid-template-columns: 1fr;
            }

            form {
                grid-template-columns: 1fr;
            }

            .full {
                grid-column: span 1;
            }

        }


        @media (max-width: 600px) {

            nav {
                flex-direction: column;
                gap: 10px;
            }

            nav div {
                flex-wrap: wrap;
                justify-content: center;
            }

            .container {
                width: 95%;
            }

        }

    </style>

</head>


<body>


    <!-- ========================
         NAVBAR
    ======================== -->

    <nav>

        <h2>
            HRGenius
        </h2>

        <div>

            <button
                class="dashboard-btn"
                onclick="goDashboard()">

                Dashboard

            </button>


            <button
                id="themeToggle"
                class="theme-toggle"
                onclick="toggleTheme()">

                🌙 Dark Mode

            </button>


            <button
                class="logout-btn"
                onclick="logout()">

                Logout

            </button>

        </div>

    </nav>


    <!-- ========================
         MAIN CONTAINER
    ======================== -->

    <div class="container">
```

```
<h1>
    Payroll Management
</h1>


<!-- ========================
     SUMMARY CARDS
======================== -->

<div class="cards">


    <div class="card">

        <h3>
            Total Payroll
        </h3>

        <p id="totalPayroll">
            0
        </p>

    </div>


    <div class="card">

        <h3>
            Paid Payroll
        </h3>

        <p id="paidPayroll">
            0
        </p>

    </div>


    <div class="card">

        <h3>
            Pending Payroll
        </h3>

        <p id="pendingPayroll">
            0
        </p>

    </div>


</div>


<!-- ========================
     GENERATE PAYROLL
======================== -->

<div class="form-container">


    <h2>
        Generate Payroll
    </h2>


    <form id="payrollForm">


        <!-- EMPLOYEE ID -->

        <div class="form-group">

            <label for="employeeId">
                Employee ID
            </label>

            <input
                type="text"
                id="employeeId"
                placeholder="Enter Employee ID"
                required
            >

        </div>


        <!-- PAYROLL MONTH -->

        <div class="form-group">

            <label for="month">
                Payroll Month
            </label>

            <input
                type="month"
                id="month"
                required
            >

        </div>


        <!-- BASIC SALARY -->
```

```
        <div class="form-group">

            <label for="basicSalary">
                Basic Salary
            </label>

            <input
                type="number"
                id="basicSalary"
                placeholder="Basic Salary"
                min="0"
                required
            >

        </div>


        <!-- ALLOWANCES -->

        <div class="form-group">

            <label for="allowances">
                Allowances
            </label>

            <input
                type="number"
                id="allowances"
                placeholder="Allowances"
                value="0"
                min="0"
                required
            >

        </div>


        <!-- DEDUCTIONS -->

        <div class="form-group">

            <label for="deductions">
                Deductions
            </label>

            <input
                type="number"
                id="deductions"
                placeholder="Deductions"
                value="0"
                min="0"
                required
            >

        </div>


        <!-- SUBMIT -->

        <button
            type="submit"
            id="generatePayrollBtn"
            class="submit-btn full">

            Generate Payroll

        </button>


    </form>


    <div
        id="message"
        class="message">

    </div>


</div>


<!-- ========================
     PAYROLL RECORDS
======================== -->

<div class="table-container">


    <h2>
        Payroll Records
    </h2>


    <table>


        <thead>

            <tr>

                <th>
                    Employee
                </th>
```

```
                        <th>
                            Email
                        </th>

                        <th>
                            Department
                        </th>

                        <th>
                            Month
                        </th>

                        <th>
                            Basic Salary
                        </th>

                        <th>
                            Allowances
                        </th>

                        <th>
                            Deductions
                        </th>

                        <th>
                            Net Salary
                        </th>

                        <th>
                            Status
                        </th>

                        <th>
                            Action
                        </th>

                    </tr>

                </thead>


                <tbody id="payrollTableBody">

                    <tr>

                        <td
                            colspan="10"
                            class="no-data">

                            Loading payroll records...

                        </td>

                    </tr>

                </tbody>


            </table>


        </div>


    </div>


    <!-- =========================
         JAVASCRIPT
    ========================== -->

    <script src="JS/auth.js"></script>

    <script src="JS/theme.js"></script>

    <script src="JS/payroll.js"></script>


</body>

</html>
```

Frontend/performance.html

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Performance Management - HRGenius</title>

    <link rel="stylesheet" href="CSS/style.css">

    <style>

        /* ========================
           NAVBAR
        ======================== */

        nav {
            background: var(--sidebar);

            color: var(--text);

            padding: 15px 30px;

            display: flex;

            justify-content: space-between;

            align-items: center;

            border-bottom: 1px solid var(--border);

            box-shadow: var(--shadow);
        }

        nav h2 {
            color: var(--primary);
        }

        nav div {
            display: flex;

            gap: 15px;

            align-items: center;

            flex-wrap: wrap;
        }

        nav a {
            color: var(--text);

            text-decoration: none;

            font-weight: 600;
        }

        nav a:hover {
            color: var(--primary);

            text-decoration: underline;
        }

        .logout-link {
            cursor: pointer;
        }


        /* ========================
           MAIN CONTAINER
        ======================== */

        .container {
            width: 90%;

            max-width: 1200px;

            margin: 30px auto;
        }

        .header {
            margin-bottom: 25px;
        }

        .header h1 {
            color: var(--text);
        }


        /* ========================
           SUMMARY CARDS
        ======================== */

        .cards {
            display: grid;

            grid-template-columns: repeat(4, 1fr);

            gap: 20px;
```

```
    margin-bottom: 30px;
}

.card {
    background: var(--card);

    color: var(--text);

    padding: 25px;

    border-radius: 10px;

    border: 1px solid var(--border);

    box-shadow: var(--shadow);
}

.card h3 {
    color: var(--muted);

    margin-bottom: 10px;

    font-size: 16px;
}

.card p {
    font-size: 28px;

    font-weight: bold;

    color: var(--primary);
}


/* ========================
   FORM BOX
======================== */

.form-box {
    background: var(--card);

    color: var(--text);

    padding: 25px;

    border-radius: 10px;

    border: 1px solid var(--border);

    box-shadow: var(--shadow);

    margin-bottom: 30px;
}

.form-box h2 {
    margin-bottom: 20px;

    color: var(--text);
}

.form-grid {
    display: grid;

    grid-template-columns: repeat(2, 1fr);

    gap: 15px;
}

.form-group {
    display: flex;

    flex-direction: column;
}

.form-group.full {
    grid-column: span 2;
}

label {
    margin-bottom: 6px;

    font-weight: bold;

    color: var(--text);
}

input,
textarea,
select {
    padding: 11px;

    border: 1px solid var(--border);

    border-radius: 6px;

    font-size: 14px;

    background: var(--input);

    color: var(--text);
}

input::placeholder,
textarea::placeholder {
    color: var(--muted);
```

```
}

input:focus,
textarea:focus,
select:focus {
    outline: none;

    border-color: var(--primary);
}

textarea {
    resize: vertical;

    min-height: 90px;
}


/* ========================
   SUBMIT BUTTON
======================== */

.submit-btn {
    margin-top: 20px;

    padding: 11px 18px;

    border: none;

    border-radius: 6px;

    cursor: pointer;

    font-weight: bold;

    background: var(--primary);

    color: white;
}

.submit-btn:hover {
    background: var(--primary-hover);
}


/* ========================
   MESSAGE
======================== */

.message {
    margin-top: 15px;

    padding: 10px;

    border-radius: 6px;

    display: none;
}

.success {
    background: rgba(22, 163, 74, 0.12);

    color: var(--success);
}

.error {
    background: rgba(220, 38, 38, 0.12);

    color: var(--danger);
}


/* ========================
   TABLE
======================== */

.table-box {
    background: var(--card);

    color: var(--text);

    padding: 25px;

    border-radius: 10px;

    border: 1px solid var(--border);

    box-shadow: var(--shadow);

    overflow-x: auto;
}

.table-box h2 {
    margin-bottom: 20px;

    color: var(--text);
}

table {
    width: 100%;

    border-collapse: collapse;

    min-width: 1000px;

    background: var(--card);
```

```
}

th,
td {
    padding: 12px;

    border-bottom: 1px solid var(--border);

    text-align: left;

    vertical-align: top;

    color: var(--text);
}

th {
    background: var(--hover);

    color: var(--text);
}

tbody tr:hover {
    background: var(--hover);
}

td small {
    color: var(--muted);
}


/* ========================
   STATUS
======================== */

.status {
    padding: 5px 10px;

    border-radius: 20px;

    font-size: 12px;

    font-weight: bold;

    display: inline-block;
}

.pending {
    background: rgba(217, 119, 6, 0.12);

    color: var(--warning);
}

.reviewed {
    background: rgba(22, 163, 74, 0.12);

    color: var(--success);
}


/* ========================
   RATING
======================== */

.rating {
    font-weight: bold;

    color: var(--primary);
}


/* ========================
   REVIEW BUTTON
======================== */

.review-btn {
    padding: 7px 12px;

    background: #16a34a;

    color: white;

    border: none;

    border-radius: 5px;

    cursor: pointer;

    font-weight: bold;
}

.review-btn:hover {
    background: #15803d;
}


/* ========================
   NO DATA
======================== */

.no-data {
    text-align: center;

    padding: 25px;

    color: var(--muted);
```

```
        }

        /* ========================
           RESPONSIVE
        ======================== */

        @media (max-width: 900px) {

            .cards {
                grid-template-columns: repeat(2, 1fr);
            }

            .form-grid {
                grid-template-columns: 1fr;
            }

            .form-group.full {
                grid-column: span 1;
            }

        }


        @media (max-width: 600px) {

            .cards {
                grid-template-columns: 1fr;
            }

            nav {
                flex-direction: column;

                gap: 10px;
            }

            nav div {
                gap: 10px;

                justify-content: center;
            }

            .container {
                width: 95%;
            }

        }

    </style>

</head>


<body>


    <!-- ========================
         NAVBAR
    ======================== -->

    <nav>

        <h2>
            HRGenius
        </h2>


        <div>

            <a href="dashboard.html">
                Dashboard
            </a>

            <a href="employees.html">
                Employees
            </a>

            <a href="performance.html">
                Performance
            </a>


            <button
                id="themeToggle"
                class="theme-toggle"
                onclick="toggleTheme()">

                🌙 Dark Mode

            </button>


            <a
                class="logout-link"
                onclick="logout()">

                Logout

            </a>

        </div>

    </nav>
```

```
<!-- ========================
     MAIN CONTAINER
======================== -->

<div class="container">


    <!-- HEADER -->

    <div class="header">

        <h1>
            Performance Management
        </h1>

    </div>


    <!-- ========================
         SUMMARY CARDS
    ======================== -->

    <div class="cards">


        <div class="card">

            <h3>
                Total Reviews
            </h3>

            <p id="totalReviews">
                0
            </p>

        </div>


        <div class="card">

            <h3>
                Reviewed
            </h3>

            <p id="reviewedReviews">
                0
            </p>

        </div>


        <div class="card">

            <h3>
                Pending
            </h3>

            <p id="pendingReviews">
                0
            </p>

        </div>


        <div class="card">

            <h3>
                Average Rating
            </h3>

            <p id="averageRating">
                0
            </p>

        </div>


    </div>


    <!-- ========================
         ADD PERFORMANCE REVIEW
    ======================== -->

    <div class="form-box">

        <h2>
            Add Performance Review
        </h2>


        <form id="performanceForm">


            <div class="form-grid">


                <!-- EMPLOYEE ID -->

                <div class="form-group">

                    <label>
                        Employee ID
                    </label>
```

```
    <input
        type="text"
        id="employeeId"
        placeholder="Enter Employee ID"
        required
    >

</div>


<!-- REVIEW PERIOD -->

<div class="form-group">

    <label>
        Review Period
    </label>

    <input
        type="text"
        id="reviewPeriod"
        placeholder="Example: October 2026"
        required
    >

</div>


<!-- GOALS -->

<div class="form-group full">

    <label>
        Goals
    </label>

    <textarea
        id="goals"
        placeholder="Enter employee goals"
        required
    ></textarea>

</div>


<!-- ACHIEVEMENTS -->

<div class="form-group full">

    <label>
        Achievements
    </label>

    <textarea
        id="achievements"
        placeholder="Enter employee achievements"
        required
    ></textarea>

</div>


<!-- RATING -->

<div class="form-group">

    <label>
        Rating
    </label>

    <select
        id="rating"
        required>

        <option value="">
            Select Rating
        </option>

        <option value="1">
            1 - Poor
        </option>

        <option value="2">
            2 - Below Average
        </option>

        <option value="3">
            3 - Average
        </option>

        <option value="4">
            4 - Good
        </option>

        <option value="5">
            5 - Excellent
        </option>

    </select>

</div>


<!-- FEEDBACK -->
```

```
            <div class="form-group">

                <label>
                    Feedback
                </label>

                <input
                    type="text"
                    id="feedback"
                    placeholder="Enter feedback"
                    required
                >

            </div>


            <!-- STATUS -->

            <div class="form-group">

                <label>
                    Status
                </label>

                <select id="status">

                    <option value="Pending">
                        Pending
                    </option>

                    <option value="Reviewed">
                        Reviewed
                    </option>

                </select>

            </div>


        </div>


        <button
            type="submit"
            class="submit-btn">

            Add Performance Review

        </button>


        <div
            id="message"
            class="message">

        </div>


    </form>

</div>


<!-- ========================
     PERFORMANCE TABLE
======================== -->

<div class="table-box">

    <h2>
        Performance Reviews
    </h2>


    <table>

        <thead>

            <tr>

                <th>
                    Employee
                </th>

                <th>
                    Review Period
                </th>

                <th>
                    Goals
                </th>

                <th>
                    Achievements
                </th>

                <th>
                    Rating
                </th>

                <th>
                    Feedback
                </th>

                <th>
```

```
                        Status
                    </th>

                    <th>
                        Action
                    </th>

                </tr>

            </thead>


            <tbody id="performanceTable">

                <tr>

                    <td
                        colspan="8"
                        class="no-data">

                        Loading...

                    </td>

                </tr>

            </tbody>

        </table>

    </div>


</div>


<!-- =========================
     JAVASCRIPT
========================= -->

<script>


    const API_URL =
        "http://localhost:5000/api/performance";


    const token =
        localStorage.getItem("token");


    // =========================
    // LOGIN CHECK
    // =========================

    if (!token) {

        window.location.href =
            "login.html";

    }


    // =========================
    // SHOW MESSAGE
    // =========================

    function showMessage(
        message,
        type
    ) {

        const messageBox =
            document.getElementById(
                "message"
            );


        messageBox.className =
            "message " + type;


        messageBox.style.display =
            "block";


        messageBox.textContent =
            message;


        setTimeout(
            function() {

                messageBox.style.display =
                    "none";

            },
            3000
        );

    }


    // =========================
    // LOAD PERFORMANCE REVIEWS
```

```
// ========================

async function loadPerformances() {

    try {

        const response =
            await fetch(
                API_URL,
                {
                    method: "GET",

                    headers: {

                        "Authorization":
                            `Bearer ${token}`

                    }

                }
            );


        const data =
            await response.json();


        if (!response.ok) {

            throw new Error(
                data.message ||
                "Failed to load performance records"
            );

        }


        const performances =
            data.performances || [];


        displayPerformances(
            performances
        );


        updateCards(
            performances
        );


    }

    catch (error) {

        console.error(error);


        document.getElementById(
            "performanceTable"
        ).innerHTML = `

            <tr>

                <td
                    colspan="8"
                    class="no-data">

                    Error loading performance records

                </td>

            </tr>

        `;

    }

}


// ========================
// DISPLAY PERFORMANCE REVIEWS
// ========================

function displayPerformances(
    performances
) {

    const table =
        document.getElementById(
            "performanceTable"
        );


    table.innerHTML = "";


    if (
        performances.length === 0
    ) {

        table.innerHTML = `

            <tr>
```

```
            <td
                colspan="8"
                class="no-data">

                No performance records found

            </td>

        </tr>

    `;

    return;

}


performances.forEach(
    function(performance) {


        const employee =
            performance.employeeId;


        const employeeName =
            employee &&
            typeof employee ===
            "object"
                ? employee.name
                : "Unknown";


        const employeeEmail =
            employee &&
            typeof employee ===
            "object"
                ? employee.email
                : "-";


        const status =
            performance.status ||
            "Pending";


        const statusClass =
            status === "Reviewed"
                ? "reviewed"
                : "pending";


        let action = "-";


        if (
            status ===
            "Pending"
        ) {

            action = `

                <button
                    class="review-btn"
                    onclick="updateStatus(
                        '${performance._id}',
                        'Reviewed'
                    )">

                    Review

                </button>

            `;

        }


        const row =
            document.createElement(
                "tr"
            );


        row.innerHTML = `

            <td>

                <strong>
                    ${employeeName}
                </strong>

                <br>

                <small>
                    ${employeeEmail}
                </small>

            </td>


            <td>
                ${performance.reviewPeriod}
            </td>
```

```
                <td>
                    ${performance.goals}
                </td>


                <td>
                    ${performance.achievements}
                </td>


                <td class="rating">

                    ${performance.rating}
                    / 5

                </td>


                <td>
                    ${performance.feedback}
                </td>


                <td>

                    <span
                        class="status ${statusClass}">

                        ${status}

                    </span>

                </td>


                <td>

                    ${action}

                </td>

            `;


            table.appendChild(row);

        }
    );

}


// ========================
// UPDATE SUMMARY CARDS
// ========================

function updateCards(
    performances
) {


    const total =
        performances.length;


    const reviewed =
        performances.filter(
            function(performance) {

                return (
                    performance.status ===
                    "Reviewed"
                );

            }
        ).length;


    const pending =
        performances.filter(
            function(performance) {

                return (
                    performance.status ===
                    "Pending"
                );

            }
        ).length;


    let average = 0;


    if (total > 0) {

        const totalRating =
            performances.reduce(
                function(
                    sum,
                    performance
                ) {
```

```
                    return (
                        sum +
                        Number(
                            performance.rating
                        )
                    );

                },
                0
            );


        average =
            (
                totalRating /
                total
            ).toFixed(1);

    }


    document.getElementById(
        "totalReviews"
    ).textContent =
        total;


    document.getElementById(
        "reviewedReviews"
    ).textContent =
        reviewed;


    document.getElementById(
        "pendingReviews"
    ).textContent =
        pending;


    document.getElementById(
        "averageRating"
    ).textContent =
        average;

}


// ========================
// ADD PERFORMANCE REVIEW
// ========================

document
    .getElementById(
        "performanceForm"
    )
    .addEventListener(
        "submit",
        async function(event) {

            event.preventDefault();


            const performanceData = {

                employeeId:
                    document
                        .getElementById(
                            "employeeId"
                        )
                        .value
                        .trim(),

                reviewPeriod:
                    document
                        .getElementById(
                            "reviewPeriod"
                        )
                        .value
                        .trim(),

                goals:
                    document
                        .getElementById(
                            "goals"
                        )
                        .value
                        .trim(),

                achievements:
                    document
                        .getElementById(
                            "achievements"
                        )
                        .value
                        .trim(),

                rating:
                    Number(
                        document
                            .getElementById(
                                "rating"
                            )
                            .value
                    ),
```

```
                feedback:
                    document
                        .getElementById(
                            "feedback"
                        )
                        .value
                        .trim()

            };


            try {


                const response =
                    await fetch(
                        `${API_URL}/create`,
                        {

                            method:
                                "POST",

                            headers: {

                                "Content-Type":
                                    "application/json",

                                "Authorization":
                                    `Bearer ${token}`

                            },

                            body:
                                JSON.stringify(
                                    performanceData
                                )

                        }
                    );


                const data =
                    await response.json();


                if (!response.ok) {

                    throw new Error(
                        data.message ||
                        data.error ||
                        "Failed to create performance review"
                    );

                }


                showMessage(
                    "Performance review created successfully",
                    "success"
                );


                document
                    .getElementById(
                        "performanceForm"
                    )
                    .reset();


                loadPerformances();


            }

            catch (error) {

                console.error(error);


                showMessage(
                    error.message,
                    "error"
                );

            }

        }
    );


// ========================
// UPDATE PERFORMANCE STATUS
// ========================

async function updateStatus(
    id,
    status
) {


    try {


        const response =
            await fetch(
```

```
                    `${API_URL}/${id}/status`,
                    {

                        method:
                            "PUT",

                        headers: {

                            "Content-Type":
                                "application/json",

                            "Authorization":
                                `Bearer ${token}`

                        },

                        body:
                            JSON.stringify({

                                status:
                                    status

                            })

                    }
                );


            const data =
                await response.json();


            if (!response.ok) {

                throw new Error(
                    data.message ||
                    data.error ||
                    "Failed to update status"
                );

            }


            showMessage(
                "Performance status updated successfully",
                "success"
            );


            loadPerformances();


        }

        catch (error) {

            console.error(error);


            showMessage(
                error.message,
                "error"
            );

        }

    }


    // =========================
    // LOGOUT
    // =========================

    function logout() {

        localStorage.removeItem(
            "token"
        );


        window.location.href =
            "login.html";

    }


    // =========================
    // INITIAL LOAD
    // =========================

    loadPerformances();


</script>


<script src="JS/auth.js"></script>

<script src="JS/theme.js"></script>


</body>

</html>
```

Frontend/settings.html

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta
        name="viewport"
        content="width=device-width, initial-scale=1.0">

    <title>HRGenius - Settings</title>

    <link rel="stylesheet" href="CSS/style.css">

    <style>

        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        html,
        body {
            width: 100%;
            min-height: 100%;
            font-family: Arial, Helvetica, sans-serif;
            background: var(--bg);
            color: var(--text);
        }

        /* ================= SIDEBAR ================= */

        .sidebar {
            position: fixed;
            left: 0;
            top: 0;
            width: 240px;
            height: 100vh;
            background: var(--sidebar);
            color: white;
            display: flex;
            flex-direction: column;
            z-index: 1000;
        }

        .logo {
            height: 80px;
            padding: 0 20px;
            display: flex;
            align-items: center;
            gap: 12px;
            border-bottom: 1px solid rgba(255,255,255,0.1);
        }

        .logo-icon {
            width: 42px;
            height: 42px;
            border-radius: 10px;
            background: var(--primary);
            color: white;
            display: flex;
            align-items: center;
            justify-content: center;
            font-size: 22px;
            font-weight: bold;
        }

        .logo h2 {
            font-size: 19px;
            color: white;
            margin-bottom: 3px;
        }

        .logo span {
            font-size: 11px;
            color: #9ca3af;
        }

        .sidebar-menu {
            flex: 1;
            padding: 15px 12px;
            overflow-y: auto;
        }

        .menu-title {
            margin: 18px 10px 8px;
            color: #6b7280;
            font-size: 10px;
            font-weight: bold;
            letter-spacing: 1px;
        }

        .side-link {
            width: 100%;
            border: none;
            background: transparent;
            color: #d1d5db;
            display: flex;
            align-items: center;
            gap: 12px;
```

```
    padding: 11px 12px;
    margin-bottom: 4px;
    border-radius: 8px;
    font-size: 14px;
    text-align: left;
    cursor: pointer;
    transition: 0.2s;
}

.side-link:hover {
    background: var(--hover);
    color: white;
}

.side-link.active {
    background: var(--primary);
    color: white;
}

.side-link span {
    width: 20px;
    min-width: 20px;
    text-align: center;
    font-size: 16px;
}

.sidebar-bottom {
    padding: 12px;
    border-top: 1px solid rgba(255,255,255,0.1);
}

.logout-link:hover {
    background: #7f1d1d;
}

/* ================ MAIN ================ */

.main-content {
    margin-left: 240px;
    min-height: 100vh;
    background: var(--bg);
}

/* ================ TOPBAR ================ */

.topbar {
    height: 70px;
    background: var(--topbar);
    border-bottom: 1px solid var(--border);
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 0 30px;
}

.search-box {
    width: 300px;
    height: 40px;
    background: var(--input);
    border-radius: 8px;
    display: flex;
    align-items: center;
    gap: 10px;
    padding: 0 12px;
    border: 1px solid var(--border);
}

.search-box span {
    color: var(--muted);
}

.search-box input {
    width: 100%;
    border: none;
    outline: none;
    background: transparent;
    font-size: 14px;
    color: var(--text);
}

.search-box input::placeholder {
    color: var(--muted);
}

.topbar-right {
    display: flex;
    align-items: center;
    gap: 20px;
}

.notification-btn {
    position: relative;
    border: none;
    background: transparent;
    font-size: 20px;
    cursor: pointer;
    color: var(--text);
}

.notification-count {
    position: absolute;
    top: -7px;
    right: -8px;
    min-width: 17px;
    height: 17px;
```

```
    padding: 0 4px;
    background: var(--danger);
    color: white;
    border-radius: 20px;
    font-size: 10px;
    display: flex;
    align-items: center;
    justify-content: center;
}

.profile {
    display: flex;
    align-items: center;
    gap: 10px;
}

.profile-avatar {
    width: 38px;
    height: 38px;
    border-radius: 50%;
    background: var(--primary);
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    font-weight: bold;
}

.profile-info {
    display: flex;
    flex-direction: column;
    gap: 3px;
}

.profile-info strong {
    font-size: 14px;
    color: var(--text);
}

.profile-info span {
    font-size: 11px;
    color: var(--muted);
}

.profile-arrow {
    font-size: 10px;
    color: var(--muted);
}

/* ================= SETTINGS ================= */

.settings-container {
    padding: 30px;
    min-height: calc(100vh - 70px);
}

.settings-header {
    margin-bottom: 25px;
}

.settings-header h1 {
    margin-bottom: 8px;
    font-size: 28px;
    font-weight: 700;
    color: var(--text);
}

.settings-header p {
    color: var(--muted);
    font-size: 14px;
}

.settings-grid {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 22px;
}

.settings-card {
    background: var(--card);
    padding: 25px;
    border-radius: 14px;
    border: 1px solid var(--border);
    box-shadow: var(--shadow);
}

.settings-card.full-width {
    grid-column: 1 / -1;
}

.settings-card h2 {
    margin-bottom: 8px;
    font-size: 19px;
    color: var(--text);
}

.settings-card > p {
    margin-bottom: 22px;
    color: var(--muted);
    font-size: 14px;
}

/* ================= FORM ================= */

.form-group {
```

```
    margin-bottom: 18px;
}

.form-group label {
    display: block;
    margin-bottom: 7px;
    font-size: 14px;
    font-weight: 600;
    color: var(--text);
}

.form-group input,
.form-group select {
    width: 100%;
    height: 43px;
    padding: 0 14px;
    border: 1px solid var(--border);
    border-radius: 8px;
    background: var(--input);
    color: var(--text);
    font-size: 14px;
    outline: none;
}

.form-group input::placeholder {
    color: var(--muted);
}

.form-group input:focus,
.form-group select:focus {
    border-color: var(--primary);
    box-shadow: 0 0 0 3px rgba(79,70,229,0.1);
}

.form-group input[readonly] {
    background: var(--hover);
    color: var(--muted);
}

/* ================= BUTTON ================= */

.btn {
    border: none;
    border-radius: 8px;
    padding: 11px 18px;
    font-size: 14px;
    font-weight: 600;
    cursor: pointer;
}

.btn-primary {
    background: var(--primary);
    color: white;
}

.btn-primary:hover {
    background: var(--primary-hover);
}

/* ================= PREFERENCES ================= */

.preference-row {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 20px;
    padding: 16px 0;
    border-bottom: 1px solid var(--border);
}

.preference-row:last-child {
    border-bottom: none;
}

.preference-info {
    display: flex;
    flex-direction: column;
    gap: 5px;
}

.preference-info strong {
    font-size: 14px;
    color: var(--text);
}

.preference-info span {
    font-size: 13px;
    color: var(--muted);
}

.preference-row input[type="checkbox"] {
    width: 18px;
    height: 18px;
    cursor: pointer;
}

/* ================= SYSTEM ================= */

.system-info {
    display: grid;
    grid-template-columns: repeat(5, 1fr);
    gap: 15px;
}

.system-item {
```

```
            background: var(--hover);
            border: 1px solid var(--border);
            border-radius: 10px;
            padding: 16px;
        }

        .system-item span {
            display: block;
            margin-bottom: 7px;
            font-size: 12px;
            color: var(--muted);
        }

        .system-item strong {
            display: block;
            font-size: 14px;
            color: var(--text);
        }

        /* ================ RESPONSIVE ================ */

        @media (max-width: 1100px) {

            .settings-grid {
                grid-template-columns: 1fr;
            }

            .settings-card.full-width {
                grid-column: auto;
            }

            .system-info {
                grid-template-columns: repeat(2, 1fr);
            }

        }

        @media (max-width: 700px) {

            .sidebar {
                width: 210px;
            }

            .main-content {
                margin-left: 210px;
            }

            .topbar {
                padding: 0 15px;
            }

            .search-box {
                width: 200px;
            }

            .settings-container {
                padding: 18px;
            }

            .system-info {
                grid-template-columns: 1fr;
            }

            .profile-info,
            .profile-arrow {
                display: none;
            }

        }

    </style>

</head>


<body>

    <!-- ================ SIDEBAR ================ -->

    <aside class="sidebar">

        <div class="logo">

            <div class="logo-icon">
                H
            </div>

            <div>

                <h2>
                    HRGenius
                </h2>

                <span>
                    HR Management
                </span>

            </div>

        </div>


        <div class="sidebar-menu">

            <p class="menu-title">
```

```
    MAIN
</p>

<button
    class="side-link"
    onclick="window.location.href='dashboard.html'">

    <span>🏠</span>
    Dashboard

</button>


<p class="menu-title">
    MANAGEMENT
</p>

<button
    class="side-link"
    onclick="window.location.href='employees.html'">

    <span>👥</span>
    Employees

</button>


<button
    class="side-link"
    onclick="window.location.href='attendance.html'">

    <span>🕐</span>
    Attendance

</button>


<button
    class="side-link"
    onclick="window.location.href='leaves.html'">

    <span>🗓️</span>
    Leaves

</button>


<button
    class="side-link"
    onclick="window.location.href='payroll.html'">

    <span>💰</span>
    Payroll

</button>


<button
    class="side-link"
    onclick="window.location.href='performance.html'">

    <span>📊</span>
    Performance

</button>


<p class="menu-title">
    RECRUITMENT
</p>


<button
    class="side-link"
    onclick="window.location.href='jobs.html'">

    <span>💼</span>
    Jobs

</button>


<button
    class="side-link"
    onclick="window.location.href='candidates.html'">

    <span>👤</span>
    Candidates

</button>


<button
    class="side-link"
    onclick="window.location.href='interviews.html'">

    <span>🎯</span>
    Interviews

</button>


<p class="menu-title">
    ORGANIZATION
</p>
```

```
        <button
            class="side-link"
            onclick="window.location.href='departments.html'">

            <span>🏢</span>
            Departments

        </button>


        <button
            class="side-link"
            onclick="window.location.href='notifications.html'">

            <span>🔔</span>
            Notifications

        </button>

    </div>


    <div class="sidebar-bottom">

        <button
            class="side-link active"
            onclick="window.location.href='settings.html'">

            <span>⚙️</span>
            Settings

        </button>


        <button
            class="side-link logout-link"
            onclick="logout()">

            <span>🚪</span>
            Logout

        </button>

    </div>

</aside>


<!-- ================ MAIN ================ -->

<main class="main-content">


    <!-- TOPBAR -->

    <header class="topbar">

        <div class="search-box">

            <span>
                🔍
            </span>

            <input
                type="text"
                placeholder="Search...">

        </div>


        <div class="topbar-right">

            <button
                id="themeToggle"
                class="theme-toggle"
                onclick="toggleTheme()">

                🌙 Dark Mode

            </button>


            <button
                class="notification-btn"
                onclick="window.location.href='notifications.html'">

                🔔

                <span
                    class="notification-count"
                    id="notificationCount">

                    0

                </span>

            </button>


            <div class="profile">

                <div
                    class="profile-avatar"
```

```
                id="profileAvatar">

                A

            </div>


            <div class="profile-info">

                <strong id="topProfileName">
                    Admin User
                </strong>

                <span>
                    Administrator
                </span>

            </div>


            <span class="profile-arrow">
                ▼
            </span>

        </div>

    </div>

</header>


<!-- ================= SETTINGS ================= -->

<section class="settings-container">


    <div class="settings-header">

        <h1>
            Settings
        </h1>

        <p>
            Manage your HRGenius account and preferences.
        </p>

    </div>


    <div class="settings-grid">


        <!-- PROFILE SETTINGS -->

        <div class="settings-card">

            <h2>
                👤 Profile Settings
            </h2>

            <p>
                Manage your account information.
            </p>


            <div class="form-group">

                <label>
                    Full Name
                </label>

                <input
                    type="text"
                    id="profileName"
                    placeholder="Enter full name">

            </div>


            <div class="form-group">

                <label>
                    Email Address
                </label>

                <input
                    type="email"
                    id="profileEmail"
                    placeholder="Enter email">

            </div>


            <div class="form-group">

                <label>
                    Role
                </label>

                <input
                    type="text"
                    id="profileRole"
                    readonly>

            </div>
```

```
    <button
        class="btn btn-primary"
        onclick="saveProfile()">

        Save Changes

    </button>

</div>


<!-- CHANGE PASSWORD -->

<div class="settings-card">

    <h2>
        🔐 Change Password
    </h2>

    <p>
        Update your account password.
    </p>


    <div class="form-group">

        <label>
            Current Password
        </label>

        <input
            type="password"
            id="currentPassword"
            placeholder="Enter current password">

    </div>


    <div class="form-group">

        <label>
            New Password
        </label>

        <input
            type="password"
            id="newPassword"
            placeholder="Enter new password">

    </div>


    <div class="form-group">

        <label>
            Confirm Password
        </label>

        <input
            type="password"
            id="confirmPassword"
            placeholder="Confirm new password">

    </div>


    <button
        class="btn btn-primary"
        onclick="changePassword()">

        Update Password

    </button>

</div>


<!-- NOTIFICATION PREFERENCES -->

<div class="settings-card">

    <h2>
        🔔 Notification Preferences
    </h2>

    <p>
        Choose which notifications you want to receive.
    </p>


    <div class="preference-row">

        <div class="preference-info">

            <strong>
                Leave Notifications
            </strong>

            <span>
                Receive notifications about leave requests.
            </span>

        </div>
```

```
        <input
            type="checkbox"
            id="leaveNotifications"
            checked>

    </div>


    <div class="preference-row">

        <div class="preference-info">

            <strong>
                Interview Notifications
            </strong>

            <span>
                Receive interview scheduling notifications.
            </span>

        </div>

        <input
            type="checkbox"
            id="interviewNotifications"
            checked>

    </div>


    <div class="preference-row">

        <div class="preference-info">

            <strong>
                Payroll Notifications
            </strong>

            <span>
                Receive payroll related notifications.
            </span>

        </div>

        <input
            type="checkbox"
            id="payrollNotifications"
            checked>

    </div>

</div>


<!-- APPEARANCE -->

<div class="settings-card">

    <h2>
        🎨 Appearance
    </h2>

    <p>
        Customize the appearance of HRGenius.
    </p>


    <div class="form-group">

        <label>
            Theme
        </label>

        <select id="themeSelect">

            <option value="light">
                Light
            </option>

            <option value="dark">
                Dark
            </option>

        </select>

    </div>


    <button
        class="btn btn-primary"
        onclick="savePreferences()">

        Save Preferences

    </button>

</div>


<!-- SYSTEM INFORMATION -->

<div class="settings-card full-width">

    <h2>
```

```
                    ⚙️ System Information
                </h2>

                <p>
                    HRGenius application information.
                </p>


                <div class="system-info">


                    <div class="system-item">

                        <span>
                            Application
                        </span>

                        <strong>
                            HRGenius
                        </strong>

                    </div>


                    <div class="system-item">

                        <span>
                            Version
                        </span>

                        <strong>
                            1.0.0
                        </strong>

                    </div>


                    <div class="system-item">

                        <span>
                            Environment
                        </span>

                        <strong>
                            Local Development
                        </strong>

                    </div>


                    <div class="system-item">

                        <span>
                            Backend
                        </span>

                        <strong>
                            Node.js + Express
                        </strong>

                    </div>


                    <div class="system-item">

                        <span>
                            Database
                        </span>

                        <strong>
                            MongoDB
                        </strong>

                    </div>

                </div>

            </div>

        </div>

    </section>

</main>


<script>

    /* ================ LOAD PROFILE ================ */

    async function loadProfile() {

        const token =
            localStorage.getItem("token");


        if (!token) {

            alert("Please login again.");

            window.location.href =
                "login.html";

            return;
```

```
    }

    try {

        const response =
            await fetch(
                "http://localhost:5000/api/users/profile",
                {
                    method: "GET",

                    headers: {
                        "Authorization":
                            `Bearer ${token}`
                    }
                }
            );


        const data =
            await response.json();


        if (!response.ok) {

            alert(
                data.message ||
                "Unable to load profile."
            );

            return;

        }


        document.getElementById(
            "profileName"
        ).value =
            data.user.name || "";


        document.getElementById(
            "profileEmail"
        ).value =
            data.user.email || "";


        document.getElementById(
            "profileRole"
        ).value =
            data.user.role || "";


        document.getElementById(
            "topProfileName"
        ).textContent =
            data.user.name ||
            "Admin User";


        const firstLetter =
            (data.user.name || "A")
                .charAt(0)
                .toUpperCase();


        document.getElementById(
            "profileAvatar"
        ).textContent =
            firstLetter;


    } catch (error) {

        console.log(
            "Load Profile Error:",
            error
        );

        alert(
            "Unable to connect to backend."
        );

    }

}


/* ================ SAVE PROFILE ================ */

async function saveProfile() {

    const name =
        document.getElementById(
            "profileName"
        ).value.trim();


    const email =
        document.getElementById(
            "profileEmail"
        ).value.trim();


    if (!name || !email) {
```

```
    alert(
        "Name and email are required."
    );

    return;

}


const token =
    localStorage.getItem("token");


if (!token) {

    alert("Please login again.");

    window.location.href =
        "login.html";

    return;

}


try {

    const response =
        await fetch(
            "http://localhost:5000/api/users/profile",
            {
                method: "PUT",

                headers: {

                    "Content-Type":
                        "application/json",

                    "Authorization":
                        `Bearer ${token}`

                },

                body:
                    JSON.stringify({
                        name: name,
                        email: email
                    })

            }
        );


    const data =
        await response.json();


    if (!response.ok) {

        alert(
            data.message ||
            "Failed to update profile."
        );

        return;

    }


    document.getElementById(
        "profileName"
    ).value =
        data.user.name;


    document.getElementById(
        "profileEmail"
    ).value =
        data.user.email;


    document.getElementById(
        "profileRole"
    ).value =
        data.user.role;


    document.getElementById(
        "topProfileName"
    ).textContent =
        data.user.name;


    document.getElementById(
        "profileAvatar"
    ).textContent =
        data.user.name
            .charAt(0)
            .toUpperCase();


    alert(
        "Profile updated successfully!"
    );
```

```
    } catch (error) {

        console.log(
            "Profile Update Error:",
            error
        );

        alert(
            "Unable to connect to backend."
        );

    }

}


/* ================ CHANGE PASSWORD ================ */

async function changePassword() {

    const currentPassword =
        document.getElementById(
            "currentPassword"
        ).value;


    const newPassword =
        document.getElementById(
            "newPassword"
        ).value;


    const confirmPassword =
        document.getElementById(
            "confirmPassword"
        ).value;


    if (
        !currentPassword ||
        !newPassword ||
        !confirmPassword
    ) {

        alert(
            "Please fill all password fields."
        );

        return;

    }


    if (
        newPassword !==
        confirmPassword
    ) {

        alert(
            "New password and confirm password do not match."
        );

        return;

    }


    if (newPassword.length < 6) {

        alert(
            "New password must be at least 6 characters."
        );

        return;

    }


    const token =
        localStorage.getItem("token");


    if (!token) {

        alert("Please login again.");

        window.location.href =
            "login.html";

        return;

    }


    try {

        const response =
            await fetch(
                "http://localhost:5000/api/users/change-password",
                {
                    method: "PUT",

                    headers: {
```

```
                        "Content-Type":
                            "application/json",

                        "Authorization":
                            `Bearer ${token}`

                    },

                    body:
                        JSON.stringify({

                            currentPassword:
                                currentPassword,

                            newPassword:
                                newPassword

                        })

                }
            );


        const data =
            await response.json();


        if (!response.ok) {

            alert(
                data.message ||
                "Failed to change password."
            );

            return;

        }


        alert(
            "Password changed successfully!"
        );


        document.getElementById(
            "currentPassword"
        ).value = "";


        document.getElementById(
            "newPassword"
        ).value = "";


        document.getElementById(
            "confirmPassword"
        ).value = "";


    } catch (error) {

        console.log(
            "Change Password Error:",
            error
        );

        alert(
            "Unable to connect to backend."
        );

    }

}


/* ================= SAVE PREFERENCES ================= */

function savePreferences() {

    const theme =
        document.getElementById(
            "themeSelect"
        ).value;


    const leave =
        document.getElementById(
            "leaveNotifications"
        ).checked;


    const interview =
        document.getElementById(
            "interviewNotifications"
        ).checked;


    const payroll =
        document.getElementById(
            "payrollNotifications"
        ).checked;


    localStorage.setItem(
```

```
        "hrgeniusTheme",
        theme
    );


    localStorage.setItem(
        "leaveNotifications",
        leave
    );


    localStorage.setItem(
        "interviewNotifications",
        interview
    );


    localStorage.setItem(
        "payrollNotifications",
        payroll
    );


    document.documentElement.setAttribute(
        "data-theme",
        theme === "dark"
            ? "dark"
            : "light"
    );


    if (typeof updateThemeButton === "function") {
        updateThemeButton();
    }


    alert(
        "Preferences saved successfully!"
    );

}


/* ================= LOAD PREFERENCES ================= */

function loadPreferences() {

    const theme =
        localStorage.getItem(
            "hrgeniusTheme"
        ) || "light";


    document.getElementById(
        "themeSelect"
    ).value =
        theme;


    document.documentElement.setAttribute(
        "data-theme",
        theme === "dark"
            ? "dark"
            : "light"
    );


    const leave =
        localStorage.getItem(
            "leaveNotifications"
        );


    const interview =
        localStorage.getItem(
            "interviewNotifications"
        );


    const payroll =
        localStorage.getItem(
            "payrollNotifications"
        );


    if (leave !== null) {

        document.getElementById(
            "leaveNotifications"
        ).checked =
            leave === "true";

    }


    if (interview !== null) {

        document.getElementById(
            "interviewNotifications"
        ).checked =
            interview === "true";

    }
```

```
    if (payroll !== null) {

        document.getElementById(
            "payrollNotifications"
        ).checked =
            payroll === "true";

    }


    if (typeof updateThemeButton === "function") {
        updateThemeButton();
    }

}


/* ================ LOAD NOTIFICATIONS ================ */

async function loadNotificationCount() {

    const token =
        localStorage.getItem("token");


    if (!token) {
        return;
    }


    try {

        const response =
            await fetch(
                "http://localhost:5000/api/notifications",
                {
                    method: "GET",

                    headers: {
                        "Authorization":
                            `Bearer ${token}`
                    }
                }
            );


        if (!response.ok) {
            return;
        }


        const data =
            await response.json();


        let notifications = [];


        if (Array.isArray(data)) {

            notifications =
                data;

        } else if (
            Array.isArray(
                data.notifications
            )
        ) {

            notifications =
                data.notifications;

        }


        const unread =
            notifications.filter(
                notification =>
                    notification.isRead === false
            ).length;


        document.getElementById(
            "notificationCount"
        ).textContent =
            unread;


    } catch (error) {

        console.log(
            "Notification Error:",
            error
        );

    }

}


/* ================ LOGOUT ================ */

function logout() {

    localStorage.removeItem(
```

```
                "token"
            );

            window.location.href =
                "login.html";

        }


        /* ================= PAGE LOAD ================= */

        document.addEventListener(
            "DOMContentLoaded",
            function () {

                loadProfile();

                loadPreferences();

                loadNotificationCount();

            }
        );

    </script>


    <script src="JS/theme.js"></script>

</body>

</html>
```